

Técnicas Computacionales en Biología

Del entorno de trabajo al aprendizaje automático

Álvaro Serrano Navarro

Edición 2

@toc spanish

Índice general

1. Entorno Linux, Shell y reproducibilidad	1
1.1. Antes de la primera orden	1
1.2. Unix y sus paradigmas	2
1.3. El modelo terminal, Shell y procesos	4
1.4. El sistema de archivos: el mapa de navegación	6
1.5. Navegación e inspección sin alterar	8
1.6. Enlaces duros y enlaces simbólicos	10
1.7. Propiedad y permisos	10
1.8. Ejecutar un archivo y variables de entorno	12
1.9. Ergonomía y productividad en la terminal	13
1.10. Espacio de almacenamiento y organización del proyecto	14
1.11. Antes de una operación que sobrescribe o elimina	14
1.12. Reproducibilidad, empaquetado e integridad	15
1.13. Método de verificación y errores frecuentes	15
1.14. Síntesis de referencia	16
1.15. Glosario esencial	17
1.16. Fuentes y límites de uso	17
Anexo práctico · Ambientarse, inspeccionar y entregar de forma reproducible	17
2. Bash, automatización y procesamiento de texto	20
2.1. De la orden repetida al procedimiento automático	21
2.2. Fundamentos de Bash: órdenes, opciones y variables	22
2.3. Caracteres especiales y comillas	24
2.4. Flujos estándar: stdin, stdout, stderr	24
2.5. Redirección: escribir, añadir, leer y separar errores	25
2.6. Tuberías: componer programas sin que se conozcan entre sí	25
2.7. De la orden interactiva al script	26
2.8. Variables, comillas y argumentos posicionales	27
2.9. Aplicación: proteger una operación destructiva	28
2.10. Condicionales	29
2.11. Bucles	30
2.12. Funciones	31
2.13. Arrays y expansión de llaves	32
2.14. Operadores nativos de cadena	33
2.15. Hito de consolidación: del programa robusto al texto biológico	33

2.16.	El formato FASTA como primer registro multilínea	34
2.17.	Buscar con <code>grep</code>	35
2.18.	Expresiones regulares	36
2.19.	Utilidades clásicas de texto: <code>cut</code> , <code>sort</code> , <code>uniq</code> y <code>tr</code>	38
2.20.	Transformar con <code>sed</code>	39
2.21.	Procesar texto estructurado con <code>awk</code>	40
2.22.	Lo que viene después: de procesar texto a razonar sobre su coste	42
2.23.	Método y errores frecuentes	42
2.24.	Síntesis de referencia	43
2.25.	Glosario esencial	43
2.26.	Fuentes y límites de uso	44
Anexo práctico · Cinco programas sobre un corpus real		44
3. Algoritmos, complejidad y estrategias de resolución		47
3.1.	La receta, la despensa y la escala biológica	48
3.2.	El coste de un algoritmo y el modelo de computación	49
3.3.	Notación Asintótica: el lenguaje de la eficiencia	51
3.4.	Las cinco grandes familias de diseño algorítmico	54
3.5.	La operación peligrosa: la explosión combinatoria oculta	59
3.6.	Actividad de depuración: localizar fallos en algoritmos ajenos	59
3.7.	Síntesis y diagnóstico de problemas algorítmicos	60
3.8.	Glosario esencial	61
3.9.	Conexión con el curso y lo que viene después	61
3.10.	Fuentes y límites de uso	62
Anexo práctico · Auditar el coste y sentir la diferencia entre exponencial y lineal		62
4. Representación en memoria y estructuras lineales		65
4.1.	Por qué una estructura de datos decide el éxito de un programa	66
4.2.	Representar un valor antes de organizar una colección	66
4.3.	Modelos de almacenamiento en memoria: contiguo frente a enlazado	68
4.4.	El array y los arrays dinámicos	70
4.5.	La lista enlazada	71
4.6.	La pila (<i>Stack</i>) — Disciplina LIFO	72
4.7.	La cola (<i>Queue</i>) — Disciplina FIFO	75
4.8.	La operación peligrosa: mutación concurrente e índices inválidos	76
4.9.	Actividad de depuración: diagnóstico de errores en punteros	76
4.10.	Síntesis operativa y matriz comparativa de costes	77
4.11.	Glosario conceptual	78

4.12.	Conexión con el curso y siguientes pasos	78
4.13.	Fuentes y reproducibilidad	78
Anexo práctico · Pila y cola con arrays Bash sobre estructura secundaria de ARN		79
5.	Tablas hash, árboles y grafos	82
5.1.	El límite de lo lineal	83
5.2.	Árboles y estructuras jerárquicas	83
5.3.	Árboles Binarios de Búsqueda (BST)	85
5.4.	Montículos binarios (<i>Heaps</i>) y colas de prioridad	87
5.5.	Tablas Hash y resolución de colisiones	88
5.6.	Grafos y redes biológicas	90
5.7.	La operación peligrosa: borrado en direccionamiento abierto y ciclos	93
5.8.	Actividad de depuración: errores en BST y grafos	94
5.9.	Síntesis operativa y tablas de diagnóstico	94
5.10.	Glosario conceptual	95
5.11.	Conexión con el curso y siguientes pasos	95
5.12.	Fuentes y reproducibilidad	96
Anexo práctico · Tabla hash de codones y recorrido de un grafo de interacción		96
6.	Búsqueda, ordenación y evaluación experimental	99
6.1.	Las dos operaciones más fundamentales	99
6.2.	Algoritmos de Búsqueda	100
6.3.	Algoritmos de Ordenación	102
6.4.	Límite inferior teórico de la ordenación por comparación	105
6.5.	Criterios de Selección y Evaluación Experimental	107
6.6.	La operación peligrosa: recursión profunda en QuickSort	107
6.7.	Actividad de depuración: fallos en partición y búsqueda	108
6.8.	Síntesis operativa y tablas de diagnóstico	108
6.9.	Glosario conceptual	109
6.10.	Conexión con el curso y siguientes pasos	109
6.11.	Fuentes y reproducibilidad	109
Anexo práctico · Medir el escalado real de dos algoritmos de ordenación		109
7.	Medir sorpresa e incertidumbre	112
7.1.	Una pregunta cuantitativa antes de empezar	113
7.2.	Del suceso a la sorpresa	113
7.3.	De muchas sorpresas a una incertidumbre media	114
7.4.	Ejemplos que pueden comprobarse	116

7.5.	De una secuencia a una distribución empírica	116
7.6.	Lo que la composición no puede mostrar	119
7.7.	Cuando hay datos y también hay un modelo	120
7.8.	Divergencia KL: cuantificar el desacuerdo	121
7.9.	Información mutua: pasar de diversidad a dependencia	123
7.10.	Una lectura unificada de las cinco cantidades	125
7.11.	La escala total y el supuesto iid	125
7.12.	Implementar sin ocultar el razonamiento	125
7.13.	Límites de inferencia biológica	127
7.14.	Mapa de decisión para resolver problemas	128
7.15.	Síntesis y transición a modelos probabilísticos	128
Anexo práctico · Auditar información en secuencias		129
8. Bayes, cadenas de Markov y modelos ocultos		134
8.1.	De medir a decidir	135
8.2.	Probabilidad condicional: la pieza que faltaba	136
8.3.	Clasificar con Bayes	136
8.4.	Cadenas de Markov: recuperar el orden	138
8.5.	Del estado observado al oculto (HMM)	141
8.6.	Trazar Viterbi	142
8.7.	La operación peligrosa: matrices no normalizadas y ceros	145
8.8.	Actividad de depuración: errores en Viterbi	145
8.9.	Síntesis operativa y tablas de diagnóstico	145
8.10.	Glosario conceptual	146
8.11.	Conexión con el curso y siguientes pasos	146
8.12.	Fuentes y reproducibilidad	146
Anexo práctico · Clasificar, modelar y decodificar con probabilidad		146
9. Alineamiento, similitud y búsqueda en bases de datos		150
9.1.	De decodificar una secuencia a comparar dos	151
9.2.	Alinear es optimizar	151
9.3.	Needleman–Wunsch: el óptimo global	152
9.4.	Smith–Waterman: el óptimo local	155
9.5.	Huecos y matrices de sustitución	156
9.6.	Bases de datos: identificadores, versiones y reproducibilidad	157
9.7.	BLAST: una heurística, no una garantía	158
9.8.	La operación peligrosa: parámetros invertidos y números sin versión	160
9.9.	Actividad de depuración: errores en Needleman–Wunsch	160

9.10. Síntesis operativa y tablas de diagnóstico	160
9.11. Glosario conceptual	161
9.12. Conexión con el curso y siguientes pasos	161
9.13. Fuentes y reproducibilidad	161
Anexo práctico • Alinear, validar huecos y decidir con un E-value	162
10. Redes neuronales para bioinformática	166
10.1. De modelos explícitos a representaciones aprendidas	166
10.2. El Perceptrón y la no linealidad	167
10.3. Entrenamiento, pérdida y retropropagación	168
10.4. Caso de estudio resuelto: predicción de enhancers	169
10.5. Sobreajuste y alta dimensionalidad en ciencias ómicas	173
10.6. Responsabilidad, ética y límites del aprendizaje automático	173
10.7. La operación peligrosa: fugas de datos y saturación	174
10.8. Actividad de depuración: errores en la retropropagación	174
10.9. Síntesis operativa y tablas de diagnóstico	175
10.10. Glosario conceptual	175
10.11. Cierre del libro: la caja de herramientas computacionales	175
10.12. Fuentes y reproducibilidad	176
Anexo práctico • Entrenar, propagar gradientes y auditar un modelo	176

Entorno Linux, Shell y reproducibilidad

TC-CH01 – Técnicas Computacionales en Biología

Edición 2

Pregunta del tema. ¿Qué ocurre exactamente entre que teclea una orden y obtiene un resultado, y cómo consigue que otra persona pueda reproducir ese resultado en su propia máquina?

Producto final. Un espacio de trabajo Linux explorado sin alterarlo, con el entorno registrado, las operaciones peligrosas comprobadas antes de ejecutarlas y una entrega empaquetada cuya integridad se verifica con sha256sum.

Mapa del tema

1.1. Antes de la primera orden	1
1.2. Unix y sus paradigmas	2
1.3. El modelo terminal, Shell y procesos	4
1.4. El sistema de archivos: el mapa de navegación	6
1.5. Navegación e inspección sin alterar	8
1.6. Enlaces duros y enlaces simbólicos	10
1.7. Propiedad y permisos	10
1.8. Ejecutar un archivo y variables de entorno	12
1.9. Ergonomía y productividad en la terminal	13
1.10. Espacio de almacenamiento y organización del proyecto	14
1.11. Antes de una operación que sobrescribe o elimina	14
1.12. Reproducibilidad, empaquetado e integridad	15
1.13. Método de verificación y errores frecuentes	15
1.14. Síntesis de referencia	16
1.15. Glosario esencial	17
1.16. Fuentes y límites de uso	17

Cómo usar este tema

Este capítulo se lee antes de la primera sesión de laboratorio y se trabaja en tres tiempos: primero el modelo (qué es un proceso, qué es una ruta, qué es un permiso), después las órdenes que lo hacen visible, y por último el anexo, donde todo se ejecuta sobre un corpus pequeño. Las cuatro predicciones de la sección siguiente conviene responderlas por escrito antes de leer: el capítulo vuelve sobre cada una.

Al terminar sabrá: orientarse en el sistema de archivos con rutas absolutas y relativas; inspeccionar un fichero grande sin cargarlo entero; leer y cambiar permisos; explicar por qué una orden se encuentra o no se encuentra; registrar su entorno de trabajo; y empaquetar y verificar una entrega.

Las marcas **ESENCIAL**, **ESTUDIO** y **REFERENCIA** indican qué bloques forman la ruta esencial (asignable antes de clase), cuáles amplían y cuáles son de consulta. La ruta esencial comprende la filosofía Unix, el modelo de procesos, el mapa del sistema de archivos, la navegación e inspección segura, los permisos, los enlaces y la ejecución de programas. La ruta de estudio añade variables de entorno, personalización, ergonomía, almacenamiento, registro de entorno, protocolo de seguridad y empaquetado. Los bloques de terminal se escriben sin el símbolo del *prompt*; ejecute cada uno en una carpeta de trabajo, prediga la salida y compárela.

1.1 Antes de la primera orden

El análisis masivo de datos biológicos rara vez cabe en una hoja de cálculo. Cuando un secuenciador entrega decenas de gigabytes con millones de lecturas, o un modelo estructural produce miles de conformaciones, una interfaz gráfica deja de ser práctica: cargar el fichero entero satura la memoria, y repetir la misma operación sobre cientos de muestras a mano es inviable. La terminal procesa los datos por partes y permite automatizar la operación de forma reproducible.

La terminal no es un entorno hostil diseñado para programadores expertos; es una conversación directa con el sistema operativo. En vez de hacer clic sobre botones que ocultan lo que ocurre por debajo, usted escribe

instrucciones precisas que transforman flujos de información biológica de forma transparente y automatizable.

Sin embargo, en biomedicina y bioinformática, obtener un número o una tabla no basta. La ciencia exige que cualquier análisis ejecutado hoy pueda ser repetido por otra persona, o por usted mismo seis meses más tarde, con el mismo procedimiento y un criterio declarado para comparar el resultado. A esta exigencia la llamamos **reproducibilidad computacional**.

Este capítulo establece los cimientos de todo su trabajo técnico posterior. Aprenderá cómo está construido el ecosistema Unix y Linux; qué ocurre en la memoria del ordenador cuando se ejecuta un programa; cómo orientarse en el sistema de archivos sin perderse; cómo inspeccionar ficheros biológicos masivos sin alterarlos; cómo gestionar permisos y enlaces; cómo personalizar su entorno de trabajo; y cómo registrar un experimento computacional para que sea verificable.

Antes de empezar: cuatro predicciones formativas.

ESENCIAL Sin ejecutar ninguna orden en la terminal, anote su predicción razonada ante estas cuatro situaciones:

1. ¿Designa siempre la expresión `datos/muestra.fasta` exactamente el mismo fichero?
2. ¿Un fichero con permiso de lectura es obligatoriamente ejecutable por el usuario?
3. ¿Un estado de salida 0 (`$? = 0`) certifica que el resultado científico del cálculo es correcto?
4. ¿Una variable creada en la sesión actual de Bash llega automáticamente a un subprocesso hijo?

Guarde sus respuestas. Las fundamentaremos y resolveremos progresivamente en las secciones 1.4 (rutas), 1.7 (permisos), 1.3 (estado de salida) y 1.8 (variables de entorno).

1.2 Unix y sus paradigmas

1.2.1 Un poco de historia: el origen de la computación moderna

En 1969, en los laboratorios Bell de AT&T, Ken Thompson y Dennis Ritchie comenzaron a diseñar un sistema operativo interactivo, multitarea y multiusuario al que bautizaron como Unix. Poco después, Ritchie diseñó el lenguaje C para programación de sistemas, y en 1973 Unix se reescribió en C. Esa decisión lo hizo portable a hardware muy diverso: hasta entonces, los sistemas operativos estaban atados a la máquina para la que se programaban.

A partir de 1974, AT&T licenció Unix a universidades y centros de investigación a precios simbólicos. En la Universidad de California en Berkeley nació una influyente variante académica conocida como BSD (*Berkeley Software Distribution*). En 1983, Richard Stallman fundó el proyecto GNU con el objetivo de construir un sistema operativo completamente libre y compatible con Unix. En 1991, un estudiante finlandés llamado Linus Torvalds desarrolló el núcleo (*kernel*) que faltaba para completar ese sistema: nacía Linux. Por su parte, la familia macOS de Apple deriva de NeXTSTEP, creado en 1988 tras la salida de Steve Jobs de Apple; Apple adquirió NeXT en 1996 y, sobre esa base, Mac OS X (2001) incorporó el núcleo Darwin, de raíz BSD y Mach.

Unix se desarrolló en los laboratorios Bell de AT&T en 1969 (Ken Thompson y Dennis Ritchie); Linux y macOS son sistemas derivados o basados en Unix, no el propio Unix original.

«Unix» y «tipo Unix» (*Unix-like*) no son lo mismo: Linux y macOS clonan o derivan el funcionamiento central de Unix sin ser Unix en sentido estricto.

Esta genealogía no es una anécdota histórica: es la razón de que los conocimientos de línea de órdenes y Bash que aprenderá en este libro se transfieran en gran medida a un portátil con Linux, a un MacBook o al nodo de un superordenador. El detalle cambia: macOS abre `zsh` por defecto y muchas de sus utilidades provienen de BSD, no de GNU, por lo que algunas opciones difieren. Cuando una orden se comporte distinto, la explicación suele estar ahí.

1.2.2 Por qué importa: presencia profesional e infraestructura científica

En la actualidad, los sistemas basados en Linux constituyen el estándar predominante de la computación científica y de la infraestructura informática mundial. No se trata de una preferencia estética, sino de una

realidad operativa.

Los sistemas basados en Linux constituyen el estándar predominante en servidores, infraestructuras de cálculo científico en la nube y supercomputación, proporcionando el entorno operativo homogéneo donde se ejecutan los análisis bioinformáticos modernos.

La conclusión didáctica no depende de una cuota estadística puntual: cuando descargue datos del NCBI o de Ensembl, ejecute un modelo estructural, alinee millones de lecturas en el clúster de un hospital o lance un contenedor reproducible, casi siempre estará trabajando sobre un sistema Linux o compatible. Por eso lo que aprende aquí se reutiliza en todos esos contextos.

1.2.3 Distribuciones: el mismo núcleo, distintas maneras de empaquetarlo

ESTUDIO Linux es el núcleo; una **distribución** añade el gestor de paquetes, las utilidades y la política de actualizaciones. Debian y sus derivadas (Ubuntu, Mint) usan paquetes `.deb` y la orden `apt`; Fedora, RHEL y sus derivadas (Rocky, Alma) usan `.rpm` y `dnf`. En un clúster científico encontrará casi siempre una de estas dos familias, y las diferencias que notará son de gestor de paquetes y de versiones, no de Shell. Cómo probar o instalar Ubuntu en su portátil se explica en la guía de laboratorio y en la documentación de Canonical [6].

1.2.4 Filosofía de código abierto

El software de código abierto (*open source*) permite que cualquiera vea, modifique y redistribuya el código; su desarrollo suele ser descentralizado y colaborativo, con comunidades que revisan el código públicamente en repositorios.

En biomedicina, la ciencia abierta es un imperativo ético y metodológico. Si un algoritmo calcula una dosis terapéutica o predice la patogenicidad de una variante genética, la comunidad debe poder auditar el código fuente para entender su mecanismo y descartar artefactos. Conviene distinguir tres cosas que a menudo se confunden: que el código sea *visible* (se puede leer), que sea *libre* (se puede modificar y usar sin restricción) y que sea *redistribuable* (se puede compartir la copia modificada). Linux, Bash y las utilidades que empleamos en este curso cumplen las tres.

1.2.5 Primer paradigma: todo es un fichero

En el diseño clásico de Unix existe un principio unificador de enorme elegancia: *todo es un fichero*.

En los sistemas Unix, la mayoría de los recursos se exponen como ficheros con una interfaz común de lectura y escritura: discos, dispositivos, tuberías, incluso información del núcleo en `/proc`.

Es un principio de diseño de la interfaz, no una identidad literal: la memoria o la pantalla no se editan como un texto. Lo que significa es que, para el sistema operativo, no hay una diferencia conceptual abismal entre leer una línea de un fichero FASTA guardado en el disco, recibir bytes de un secuenciador a través de la red o leer los datos que otro programa le envía en tiempo real: todos se presentan como una corriente de bytes que puede abrirse, leerse, transformarse y cerrarse con las mismas órdenes básicas.

1.2.6 Segundo paradigma: herramientas pequeñas que se combinan

Frente a la tentación de construir programas gigantescos que intenten resolver todas las tareas posibles, y que acaban siendo frágiles y difíciles de mantener, Unix apostó por una arquitectura modular: herramientas pequeñas y robustas, cada una dedicada a una sola cosa, con mecanismos estándar para conectarse entre sí.

Un ejemplo clásico es `tar`: su única función es empaquetar múltiples ficheros dentro de un único contenedor sin comprimir. Cuando se necesita comprimir, Unix no reescribió `tar` para convertirlo en un compresor; utilizó `gzip`, un programa especializado en comprimir flujos de bytes [20].

El diseño de `tar -z`, que integra la compresión `gzip` en el proceso de archivado sin fusionar ambos programas en uno, ejemplifica la filosofía Unix de herramientas pequeñas diseñadas para trabajar juntas, no de un programa único con todas las funciones.

En el capítulo siguiente aprenderá a encadenar estas herramientas mediante tuberías (`|`), construyendo auténticas cadenas de montaje de datos biológicos.

1.3 El modelo terminal, Shell y procesos

Para diagnosticar con éxito cualquier situación en la línea de órdenes, es imprescindible distinguir tres elementos que los principiantes suelen confundir:

Una terminal es una interfaz de texto que envía órdenes a una Shell, la cual las interpreta y, cuando procede, lanza procesos; el resultado observable de un proceso incluye su salida y un estado de salida entero.

- **La terminal:** es la ventana (el emulador) que captura lo que usted teclea y dibuja los caracteres que el sistema devuelve.
- **La Shell:** es el programa que lee las líneas que usted escribe, analiza su sintaxis, busca los ejecutables correspondientes y pide al sistema operativo su ejecución. En este libro utilizamos **Bash** (*Bourne Again Shell*) [16].
- **El proceso:** es la instancia viva de un programa bajo control del sistema operativo: ocupa memoria propia y, según su estado, se ejecuta en la CPU o espera (un dato del disco, una señal, un turno de planificación).

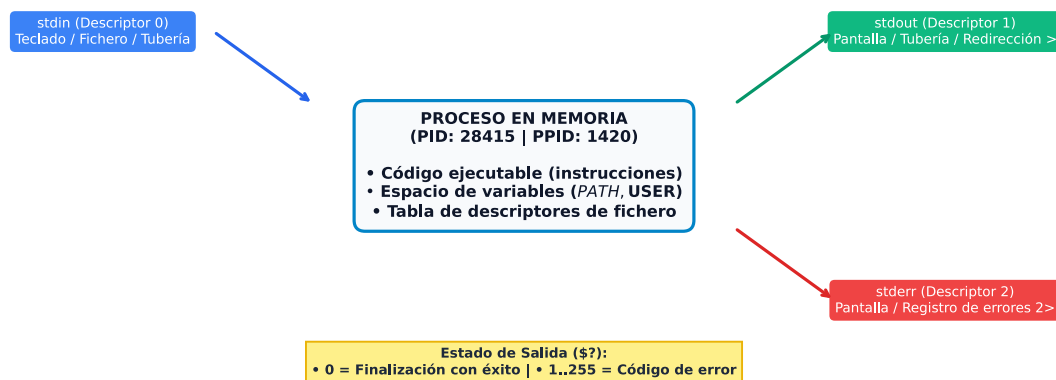


Figura 1.1: Modelo de proceso en memoria y descriptores estándar de entrada y salida (stdin, stdout, stderr) bajo el sistema operativo. Figura original adaptada de las presentaciones docentes de la asignatura; las redirecciones se estudian en el capítulo siguiente.

1.3.1 Qué escribe usted: la anatomía de una orden

ESENCIAL Cada línea que teclea tiene una estructura fija que la Shell analiza de izquierda a derecha. Tome como ejemplo esta línea tal como aparece en pantalla:

```
alumno@servidor:~/TC01$ ls -l -h datos/ > listado.txt
```

- **Prompt** (alumno@servidor:~/TC01\$): lo dibuja la Shell; no se teclea. Suele indicar usuario, máquina y directorio de trabajo.
- **Orden** (ls): el programa que se ejecuta; el verbo de la frase.
- **Opciones** (-l, -h): modifican el comportamiento; empiezan por - en su forma corta o por -- en la larga.
- **Argumentos** (datos/): los datos sobre los que actúa la orden, normalmente rutas.
- **Redirección** (> listado.txt): a dónde va la salida; se estudia en el capítulo siguiente.

Separar estas piezas es lo que permite leer una orden ajena, o una que le proponga un asistente, y predecir qué hará antes de ejecutarla.

1.3.2 De un texto a un proceso

ESENCIAL El programa que usted escribe, o que trae el sistema, es en el disco un fichero: texto legible o código ya traducido. Para que la CPU lo ejecute, su contenido debe convertirse en instrucciones de **código máquina**, números que la CPU interpreta como sumar, guardar o saltar. Esa traducción la hace un compilador, una vez y de antemano, o un intérprete, sobre la marcha, como hace Bash con un script. Cuando lanza la orden, el sistema operativo carga esas instrucciones en la memoria, les asigna un identificador (PID) y un turno de CPU, y nace el **proceso**: el programa deja de ser un fichero inerte y pasa a consumir recursos y a transitar por los estados que siguen.

1.3.3 El ciclo de vida de un proceso

Un proceso no es un bloque inerte. Desde que nace hasta que termina, transita por diversos estados:

Un proceso Unix transita durante su vida por estados bien definidos, en ejecución, en espera, detenido y, en casos patológicos, zombi; no es una entidad estática mientras dura.

- **En ejecución** (*running*): el proceso está ejecutando instrucciones en la CPU o esperando su turno inmediato en la cola del planificador.
- **En espera** (*sleeping*): el proceso aguarda que ocurra un evento externo, por ejemplo que el disco termine de leer un fichero de secuencias o que llegue un paquete de red. Una espera puede ser interrumpible por una señal o no serlo, según lo que espere.
- **Detenido** (*stopped*): el proceso ha sido pausado temporalmente, por ejemplo mediante una señal del usuario para depuración.
- **Zombi**: el proceso ha finalizado pero su padre aún no ha leído su estado de terminación. Apenas consume recursos, pero conserva su entrada en la tabla de procesos y ocupa un PID.

Los cambios de estado provocados desde fuera se hacen con **señales**: SIGINT es la que envía Ctrl+C y pide interrumpir; SIGTERM pide terminar de forma ordenada; SIGKILL obliga al sistema a eliminar el proceso sin que este pueda atenderla.

1.3.4 Jerarquía y aislamiento de memoria

En Linux, todos los procesos forman un árbol cuya raíz es el proceso primordial del sistema (*init* o *systemd*, con PID 1).

Los procesos de un sistema Linux se organizan en un árbol con *init* como raíz; cada proceso hijo obtenido con *fork()* recibe un PID distinto del de su padre pero hereda sus valores iniciales.

Para crear un nuevo proceso, el sistema operativo utiliza la llamada al sistema *fork()*, que duplica el proceso actual generando un hijo con su propio identificador. El hijo suele invocar a continuación *exec()* para sustituir su imagen por el programa pedido: así lanza la Shell cada orden que usted teclea.

Cada proceso Unix tiene un espacio de memoria aislado del de los demás; la comunicación entre procesos requiere un mecanismo explícito (señales, tuberías o memoria compartida gestionada por el núcleo), no ocurre por defecto.

Este aislamiento sostiene la estabilidad de los servidores científicos: si un alineador de secuencias comete un error de segmentación o agota su memoria, normalmente solo muere ese proceso, sin corromper la memoria de la Shell ni de otros procesos. Agotar la memoria o el disco del nodo entero sí puede afectar a los demás: por eso los clústeres imponen límites por trabajo.

1.3.5 Privilegios: usuarios y superusuario

El superusuario de un sistema Linux se llama *root* y tiene UID 0; un usuario sin privilegios puede escalar temporalmente su nivel de privilegio solo si pertenece a un grupo autorizado para ello, y debe hacerlo de forma explícita, no por defecto.

En un entorno de cálculo compartido, los usuarios ordinarios tienen permiso para crear, modificar y borrar

exclusivamente sus propios ficheros. Solo el administrador (root) o usuarios autorizados mediante sudo pueden alterar ficheros del sistema o instalar software global. El hardware colabora en esta separación: la CPU distingue un modo privilegiado, reservado al núcleo, del modo en que corren los programas de usuario. Un usuario normal solo escala a root mediante sudo, y únicamente si pertenece a un grupo autorizado (sudo, wheel o admin, según la distribución).

1.3.6 El estado de salida de un proceso

Cuando un proceso finaliza, devuelve al sistema operativo un número entero entre 0 y 255 conocido como **estado de salida** (*exit status*). En Bash, este valor queda registrado en la variable especial \$?.

El estado de salida de un proceso en Bash queda disponible en \$? inmediatamente después de su ejecución, y vale 0 por convención cuando el proceso no detectó error; no certifica que el resultado sea correcto, solo que el proceso terminó de cierta manera.

Un estado de salida 0 indica que el programa terminó sin detectar errores internos. Un valor distinto de 0 indica una condición que el propio programa define: a menudo es un error (1, 2, 127 son valores frecuentes), pero puede ser un resultado esperado. grep devuelve 1 cuando no encuentra coincidencias, y eso no es un fallo. Esta convención es la base que permite a los flujos automatizados decidir si continúan con el siguiente paso o se detienen.

Se reproduce con `verification/chapter_results.sh exit_status`.

Resolución de la predicción 3.

El estado de salida 0 solo certifica que el proceso finalizó conforme a las convenciones del sistema operativo sin abortar por error técnico. No valida la veracidad, calidad ni adecuación del resultado biológico; la comprobación científica requiere siempre la inspección explícita de lo que se ha producido.

1.4 El sistema de archivos: el mapa de navegación

A diferencia de otros sistemas operativos que asignan letras independientes a cada disco o partición (C:, D:), Linux organiza todo su almacenamiento en un **único árbol jerárquico** que parte de una raíz común representada por una barra: /. Todo lo que existe en el sistema, discos locales, discos de red, memorias USB, ficheros de configuración y datos, se encuentra en alguna rama de ese único árbol.

1.4.1 Directorio de trabajo: el suelo donde pisamos

Cada sesión de Shell y cada proceso tiene en todo momento un **directorio de trabajo** (*working directory*): es la posición actual dentro del árbol desde la cual se interpretan todas las rutas relativas.

- **Ruta absoluta:** la dirección completa de un fichero desde la raíz. Comienza siempre por /, por ejemplo /home/alumno/TC01/datos/secuencias.fasta. Es válida sin importar en qué carpeta se encuentre usted.
- **Ruta relativa:** una dirección calculada a partir del directorio de trabajo. No empieza por /, por ejemplo datos/secuencias.fasta.

Una ruta relativa se interpreta respecto al directorio de trabajo del proceso que la evalúa; la misma cadena de texto puede referirse a ficheros distintos según ese directorio.

Resolución de la predicción 1.

La expresión datos/muestra.fasta es una ruta relativa: su significado cambia según cuál sea el directorio de trabajo del proceso en cada momento. Solo una ruta absoluta identifica la misma ubicación desde cualquier sitio.

La mayoría de los errores de las primeras semanas (el mensaje No such file or directory) ocurren porque quien trabaja asume que está en una carpeta distinta de la real y usa una ruta relativa que no coincide con su posición en el árbol.

Para el árbol de ejemplo, la ruta relativa datos/tc01_mini.fasta solo resuelve al fichero real cuando el

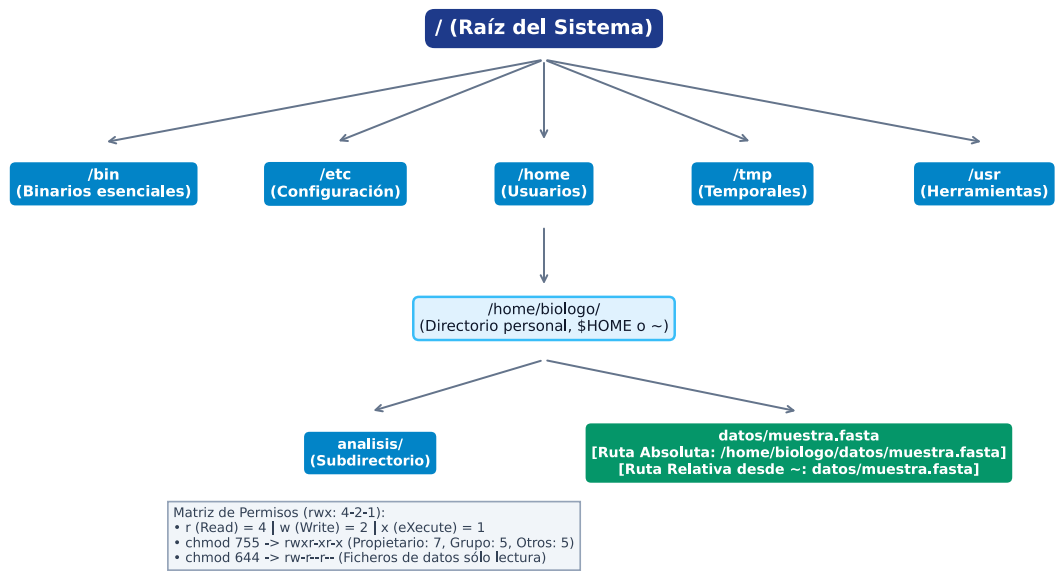


Figura 1.2: Jerarquía en árbol de raíz única (/), resolución de rutas absolutas frente a relativas y matriz octal de permisos (rwx). Figura original adaptada de las presentaciones docentes de la asignatura; la parte derecha se usa en la sección de permisos.

	/
	home/
	alumno/
	TC01/
	datos/tc01_mini.fasta
Directorio de trabajo	datos/tc01_mini.fasta señala a
/home/alumno/TC01	/home/alumno/TC01/datos/tc01_mini.fasta (existe)
/home/alumno	/home/alumno/datos/tc01_mini.fasta (no existe)

Figura 1.3: La misma ruta relativa apunta a ficheros distintos, o a ninguno, según el directorio de trabajo. Figura original.

directorio de trabajo es `/home/alumno/TC01`; desde `/home/alumno` señala a una ruta inexistente.

1.4.2 La brújula de la terminal

Para moverse con agilidad y escribir rutas sin redundancia, la Shell dispone de referencias estándar:

- `.` (punto): el directorio actual.
- `..` (dos puntos): el directorio padre, un nivel por encima.
- `~` (virgulilla): el directorio personal del usuario, habitualmente `/home/nombre_usuario`.
- `pwd`: imprime la ruta absoluta del directorio de trabajo [5, pp. 51–54].

Tras `mkdir -p a/b/c && cd a/b/c`, la ruta relativa `../..` conduce al directorio `a`; `pwd` tras ambas operaciones lo confirma.

Se reproduce con `verification/chapter_results.sh relative_path`.

1.5 Navegación e inspección sin alterar

El trabajo cotidiano en la línea de órdenes sigue una regla de oro: **inspeccionar antes de modificar**. Modificar o borrar a ciegas es la causa principal de pérdidas irreparables de datos experimentales. Las órdenes de esta tabla están documentadas en el manual de GNU Coreutils [18] y, de forma compacta, en la guía de bolsillo de Barrett [3].

Orden	Efecto	Nota de control
<code>pwd</code>	Imprime el directorio de trabajo absoluto	La primera orden ante cualquier duda de ubicación.
<code>ls</code>	Lista los ficheros del directorio indicado o actual	<code>ls -l</code> añade permisos, propietario y tamaño; <code>ls -la</code> muestra ocultos; <code>ls -lh</code> muestra tamaños legibles.
<code>cd RUTA</code>	Cambia el directorio de trabajo	<code>cd</code> solo vuelve al directorio personal; <code>cd -</code> regresa al anterior; <code>cd ..</code> sube un nivel.
<code>mkdir -p RUTA</code>	Crea directorios, intermedios incluidos	<code>-p</code> evita errores si los padres no existen o la carpeta ya existía.
<code>cp ORIGEN DESTINO</code>	Copia ficheros o carpetas (<code>cp -r</code>)	Conserva el original; confirme el destino antes de sobrescribir.
<code>mv ORIGEN DESTINO</code>	Mueve o renombra	No deja copia en el origen; en el mismo disco es instantáneo.
<code>touch FICHERO</code>	Crea un fichero vacío o actualiza su fecha	No altera el contenido si ya existía.
<code>file FICHERO</code>	Determina la naturaleza real del fichero	Analiza los bytes de cabecera, no la extensión.
<code>head / tail</code>	Muestra las primeras o últimas líneas	Por defecto 10; <code>-n NUM</code> fija la cantidad.
<code>cat / tac / more</code>	Vuelca un fichero entero, del revés, o por páginas	<code>cat</code> solo para ficheros pequeños; <code>tac</code> invierte el orden de líneas; <code>more</code> es un paginador anterior a <code>less</code> .
<code>less FICHERO</code>	Visor interactivo paginado	Explora ficheros de gigabytes sin cargarlos enteros; se sale con <code>q</code> .
<code>wc FICHERO</code>	Cuenta líneas (<code>-l</code>), palabras (<code>-w</code>) y bytes (<code>-c</code>)	<code>wc -l</code> cuenta saltos de línea, no entidades biológicas.

Orden	Efecto	Nota de control
<code>basename / dirname</code>	Separan el nombre del fichero y su directorio en una ruta	Útiles en scripts para construir nombres de salida.

`pwd`, `ls`, `cd`, `mkdir -p`, `cp`, `mv`, `touch` y las órdenes de inspección (`file`, `head`, `tail`, `less`, `wc`) son las órdenes mínimas de navegación, organización e inspección documentadas en GNU Coreutils.

La comparación de ficheros con `diff` y la búsqueda con `find` se estudian en el capítulo siguiente, junto con las herramientas de texto.

1.5.1 Inspección segura de datos biológicos masivos

Uno de los errores más destructivos al comenzar es hacer doble clic sobre un fichero de secuenciación de 15 GB para «abrirlo» con un editor gráfico. El programa intentará cargar los 15 gigabytes íntegros en la memoria y el sistema dejará de responder. Las utilidades de Unix están diseñadas para manejar flujos de datos sin importar su tamaño:

- `head -n 20 lecturas.fastq` lee únicamente las primeras 20 líneas y se detiene, consumiendo unos pocos kilobytes.
- `tail -n 20 lecturas.fastq` muestra las últimas 20 líneas; útil para comprobar si una descarga o una simulación terminó.
- `less lecturas.fastq` abre un visor que lee el fichero por bloques a medida que usted avanza. Se desplaza con la barra espaciadora, retrocede con `b`, busca con `/patrón` y navega entre coincidencias con `n` y `N`, y sale con `q`. La misma tecla `q` cierra las páginas de `man`.

1.5.2 La trampa de contar líneas frente a contar secuencias

`file` identifica el tipo de un fichero a partir de su contenido, no de su extensión; `data/tc01_sample.fasta` tiene 19 líneas, verificable con `wc -l` y a mano.

La orden `wc -l` cuenta estrictamente caracteres de fin de línea. En un fichero FASTA donde cada secuencia puede ocupar varias líneas, devuelve el número de líneas del fichero, no el número de secuencias. Para contar secuencias hay que filtrar las cabeceras, como aprenderá en el capítulo siguiente con `grep -c '^>'`.

Se reproduce con `verification/chapter_results.sh file_type`.

Se reproduce con `verification/chapter_results.sh line_count_sample`.

1.5.3 El problema silencioso de los saltos de línea de Windows

Un problema frecuente ocurre cuando un fichero de texto se exporta desde una hoja de cálculo en Windows y luego se procesa en un servidor Linux. Linux y Unix representan el fin de línea con un único carácter, *line feed* (LF); Windows utiliza dos, *carriage return* seguido de *line feed* (CRLF). La orden `file` analiza los primeros bytes y detecta la discrepancia.

`file` detecta terminadores de línea de DOS y Windows (CRLF) distinguiéndolos de los nativos de Unix (LF), lo que evita fallos sutiles en scripts al importar datos.

Se reproduce con `verification/chapter_results.sh crlf_detection`.

1.5.4 Ficheros ocultos y configuración

Cualquier fichero o carpeta cuyo nombre comience por un punto (`.bashrc`, `.gitignore`, la carpeta `.git`) se considera **oculto**. Suelen contener configuraciones o metadatos, y `ls` los omite por defecto.

Los ficheros y directorios ocultos que comienzan por un punto se omiten en la salida estándar de `ls` y se listan explícitamente con `ls -a` o `ls -la`.

Se reproduce con `verification/chapter_results.sh hidden_files_listing`.

1.5.5 La verdad de su sistema: `--help` y `man`

Nunca confíe ciegamente en ejemplos copiados de internet sin contrastar las opciones de la herramienta en su propia instalación. Distintas distribuciones y versiones tienen parámetros ligeramente distintos.

`--help` y `man` ORDEN documentan la versión de la orden realmente instalada; un ejemplo copiado de otra fuente puede corresponder a otra versión o a otra utilidad de nombre igual.

1.6 Enlaces duros y enlaces simbólicos

En el sistema de archivos de Linux, la información de un fichero se divide en dos partes: los bloques de datos en el disco y una estructura de metadatos denominada **inodo**. El inodo guarda el tamaño, los permisos, las fechas y los punteros a los bloques, pero **no** guarda el nombre. El nombre es una entrada en una tabla de directorio que apunta a un número de inodo. Esta arquitectura permite dos tipos de enlace:

- **Enlace duro** (`ln origen enlace`): una nueva entrada de directorio que apunta al **mismo inodo**. Ambos nombres tienen la misma categoría; los datos permanecen mientras quede al menos un enlace. No pueden crearse entre sistemas de archivos distintos ni sobre directorios.
- **Enlace simbólico** (`ln -s origen enlace`): un fichero especial con inodo propio cuyo contenido es la ruta del destino. Borrar el enlace no afecta al original; borrar o mover el original deja el enlace «roto».

Un enlace simbólico `ln -s` crea un puntero de ruta independiente cuya eliminación deja el fichero original intacto, mientras que un enlace duro `ln` comparte el mismo inodo y el mismo contador de enlaces.

Se reproduce con `verification/chapter_results.sh symlink_creates_pointer`.

1.6.1 El valor de los enlaces simbólicos en bioinformática

En proyectos de genómica es habitual disponer de un genoma de referencia (como `GRCh38.fa`, que supera los 3 GB) o colecciones de lecturas almacenadas en un disco central del laboratorio.

En bioinformática, los enlaces simbólicos permiten que múltiples proyectos independientes referencien conjuntos de datos masivos compartidos sin duplicar almacenamiento físico.

En lugar de copiar 3 GB en la carpeta de cada estudiante, lo que multiplicaría el consumo de disco, se crea un enlace simbólico `ln -s /datos/referencias/GRCh38.fa ./genoma_ref.fa`. El software procesa el enlace igual que el fichero original, y el almacenamiento adicional es de unos pocos bytes.

1.7 Propiedad y permisos

La seguridad y la colaboración en un entorno multiusuario se basan en el modelo de permisos de Unix. Cada fichero y carpeta tiene un **usuario propietario** (`u`) y un **grupo propietario** (`g`), y define permisos independientes para tres categorías: el propietario, los miembros del grupo y todos los demás (`o`).

Para cada categoría existen tres permisos: lectura (`r`), escritura (`w`) y ejecución (`x`). La orden `ls -l` los muestra en una cadena de diez caracteres, por ejemplo `-rwxr-xr--`: el primer carácter indica el tipo de fichero y los nueve restantes son los permisos, de tres en tres, para usuario, grupo y otros.

Primer carácter	Tipo de fichero
-	fichero regular
d	directorio
l	enlace simbólico
c	dispositivo de caracteres (una terminal, <code>/dev/null</code>)
b	dispositivo de bloques (un disco)
p	tubería con nombre
s	socket

Los cuatro últimos tipos son la forma concreta en que «todo es un fichero»: un disco o una terminal aparecen en

el árbol con un nombre y unos permisos, aunque no contengan texto.

Cada fichero Unix tiene un permiso de lectura, escritura y ejecución independiente para su propietario, su grupo y el resto de usuarios; `ls -l` muestra los nueve bits resultantes en ese orden.

1.7.1 El significado especial de la ejecución en directorios

En un fichero ordinario, el bit `x` otorga permiso para ejecutarlo como programa. En un **directorio**, `x` significa **permiso de paso**: entrar con `cd`, atravesarlo para alcanzar subcarpetas o acceder a los inodos de los ficheros que contiene. Un directorio con `r` pero sin `x` deja ver la lista de nombres, pero no leer su contenido ni sus tamaños.

1.7.2 Modificación de permisos con `chmod`

La orden `chmod` (*change mode*) admite dos notaciones:

Forma	Ejemplo	Efecto
Simbólica	<code>chmod u+x script.sh</code>	Añade ejecución solo al propietario.
Simbólica	<code>chmod g-w,o-r datos/</code>	Retira escritura al grupo y lectura a los demás.
Octal	<code>chmod 751 script.sh</code>	<code>r=4, w=2, x=1</code> sumados por categoría: usuario 7 (<code>rwX</code>), grupo 5 (<code>r-x</code>), otros 1 (<code>--x</code>).
Octal	<code>chmod 644 tabla.tsv</code>	Permisos habituales de datos: usuario 6, grupo 4, otros 4.
Octal	<code>chmod 755 proyecto/</code>	Permisos habituales de carpetas y programas públicos.
Propiedad	<code>chown usuario:grupo fichero</code>	Cambia propietario y grupo. Cambiar el propietario requiere privilegios de <code>root</code> : un usuario normal no puede ceder un fichero a otro.

`chmod` admite forma simbólica (`u+x, g-w`) y forma numérica (suma de `r=4, w=2, x=1` por categoría, por ejemplo 751); `chown` cambia propietario y grupo.

Se reproduce con `verification/chapter_results.sh chmod_numeric_matches_symbolic`.

Diagnóstico de acceso en dos fases.

REFERENCIA Ante un fallo de acceso, el sistema resuelve primero la ruta y después los permisos, en este orden. La tabla resume qué comprobar en cada fase.

Mensaje	Fase y causa	Comprobación
No such file or directory	la ruta no resuelve: directorio de trabajo distinto del supuesto, errata o fichero inexistente	<code>pwd; ls -ld -- RUTA</code> sobre cada tramo
Permission denied al entrar	falta <code>x</code> en algún directorio del camino	<code>ls -ld</code> sobre cada directorio ancestro
Permission denied al leer o escribir	falta <code>r</code> o <code>w</code> en el fichero, o el sistema de archivos es de solo lectura	<code>ls -l FICHERO; df</code>
Permission denied al ejecutar	falta <code>x</code> en el fichero	<code>ls -l; chmod u+x</code> solo sobre scripts propios revisados

Resolución de la predicción 2.

Tener permiso de lectura permite examinar los bytes de un script o de un binario, pero no ejecutarlo como programa si carece del bit `x`. Los tres permisos son independientes.

1.7.3 Un cuarto grupo de bits: SUID, SGID y sticky

ESTUDIO Además de los nueve bits `rwx`, un fichero puede llevar tres bits especiales que cambian *quién* actúa cuando se usa.

- **SUID** (`chmod u+s, octal 4xxx`): al ejecutar el programa, el proceso adopta el usuario *propietario* del fichero, no el que lo lanzó. Así `passwd` puede modificar `/etc/shadow` aunque lo invoque un usuario normal. `ls -l` muestra una `s` en la posición de la `x` del propietario: `-rwsr-xr-x`.
- **SGID** (`chmod g+s, 2xxx`): en un ejecutable, el proceso adopta el *grupo* del fichero; en un directorio, los ficheros creados dentro heredan su grupo, lo que resuelve las carpetas de proyecto compartidas.
- **Sticky bit** (`chmod +t, 1xxx`): en un directorio de escritura común como `/tmp`, impide que un usuario borre ficheros de otro aunque tenga permiso de escritura en la carpeta. `ls -l` lo muestra como `t` final: `drwxrwxrwt`.

Atención. Un ejecutable con SUID de `root` y un fallo de programación es una vía clásica de escalado de privilegios. Nunca active SUID en un script propio.

1.8 Ejecutar un archivo y variables de entorno

Para que un fichero pueda ejecutarse como programa deben cumplirse dos condiciones a la vez: la Shell debe saber dónde encontrarlo, y el fichero debe tener activo el bit `x` para el usuario que lo invoca.

Ejecutar un archivo exige que la Shell pueda localizarlo (por `$PATH` o ruta explícita) y que tenga permiso de ejecución para el usuario que lo invoca; ninguna de las dos condiciones es opcional.

Si intenta ejecutar un fichero que carece del bit `x`, Bash responderá `Permission denied`.

1.8.1 La variable `$PATH` frente a la ruta explícita `./`

Cuando escribe el nombre de una orden sin barras (`ls`, `fastqc`), Bash no inspecciona todo el disco. Consulta la variable de entorno `$PATH`, una lista ordenada de directorios separados por dos puntos, por ejemplo `/usr/local/bin:/usr/bin:/bin`, y ejecuta el primer fichero coincidente que encuentre recorriéndola de izquierda a derecha.

Por seguridad, el directorio actual (`.`) no suele estar en `$PATH`. Si escribe un script `analisis.sh` en su carpeta y teclea `analisis.sh`, Bash dirá `command not found`. Para ejecutar el fichero de la carpeta actual sin buscar en `$PATH`, se antepone una ruta explícita: `./analisis.sh`.

Cuando se invoca un ejecutable por nombre, Bash lo busca en los directorios listados en `$PATH`, en orden; una ruta explícita omite esa búsqueda.

Se reproduce con `verification/chapter_results.sh path_resolution`.

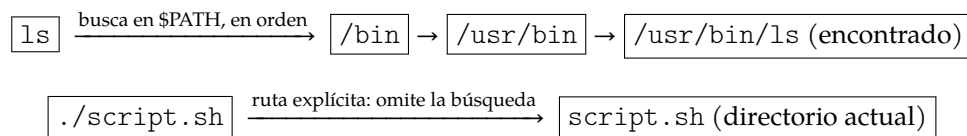


Figura 1.4: Arriba, el nombre `ls` se busca en cada directorio de `$PATH` hasta encontrar `/usr/bin/ls`; abajo, la ruta explícita `./script.sh` evita esa búsqueda y usa el fichero del directorio actual. Figura original.

Localizar un ejecutable por nombre recorre `$PATH` en orden hasta encontrarlo; una ruta explícita, por ejemplo `./script.sh`, omite esa búsqueda y usa directamente el fichero indicado.

1.8.2 El shebang: el contrato de ejecución

La primera línea de un script ejecutable contiene el **shebang**:

```
#!/usr/bin/env bash
```

Cuando el núcleo intenta ejecutar un fichero de texto con permiso `x`, lee los dos primeros bytes (`#!`). Al detectarlos sabe que no es código máquina, y usa la ruta que sigue para invocar el intérprete correspondiente, en este caso Bash, pasándole el fichero como argumento [16]. Hay cuatro formas de ejecutar un script y no son equivalentes: `./script.sh` (necesita `x` y el shebang), `bash script.sh` (no necesita `x`), `source script.sh` (se ejecuta en la Shell actual, sin proceso nuevo) y por nombre si está en `$PATH`. El capítulo siguiente vuelve sobre ellas.

1.8.3 Variables locales frente a variables de entorno exportadas

En Bash se crea una variable asignando sin espacios alrededor del igual (`MUESTRA="paciente_01"`). Hay una diferencia crucial de visibilidad:

- **Variable de Shell:** solo existe en la sesión que la creó. Un script o programa lanzado desde esa sesión no la verá.
- **Variable de entorno:** se declara con `export VARIABLE=valor` o, de forma equivalente, `declare -x VARIABLE`. El sistema la copia en el entorno de todos los procesos hijos lanzados a partir de ese momento.

Una variable de Bash ordinaria (`NOMBRE=valor`) es local a la sesión que la crea; solo se convierte en variable de entorno disponible para los procesos que esa sesión lance si se exporta con `export NOMBRE` o `export NOMBRE=valor`.

Se reproduce con `verification/chapter_results.sh export_makes_visible_to_subprocess`.

Resolución de la predicción 4.

Una variable asignada en Bash solo existe en la memoria del intérprete actual; los subprocesos no la reciben a menos que se exporte explícitamente.

1.8.4 Variables predefinidas y configuración permanente

El sistema y la Shell definen de forma automática variables fundamentales: `$HOME`, el directorio personal; `$PWD` y `$OLDPWD`, el directorio de trabajo actual y el anterior al último `cd`; `$PATH`, la lista de búsqueda; `$HOSTNAME`, el nombre de la máquina, útil para saber en qué nodo se ejecutó un análisis.

`$HOME`, `$PWD`, `$OLDPWD` y `$PATH` son variables de entorno ya definidas por el sistema: el directorio personal, el directorio de trabajo actual, el directorio de trabajo anterior tras el último `cd`, y la lista de búsqueda de ejecutables.

Se reproduce con `verification/chapter_results.sh oldpwd_tracks_previous_cd`.

Si desea que una variable esté disponible en las terminales que abra (por ejemplo, añadir una carpeta personal de herramientas con `export PATH="$HOME/bin:$PATH"`), añada esa línea a `~/.bashrc`, que leen las shells interactivas de Bash. Una shell de inicio de sesión puede leer en su lugar `~/.bash_profile`, y otras shells como `zsh` usan otros ficheros. Para recargar los cambios sin cerrar la terminal, `source ~/.bashrc`.

1.9 Ergonomía y productividad en la terminal

ESTUDIO Teclear letra por letra cada ruta larga genera fatiga y erratas. Bash incorpora la biblioteca GNU Readline, con atajos que conviene automatizar:

- **Tab:** autocompleta nombres. Si escribe `cd dat` y pulsa Tab, Bash completa `datos/`; dos pulsaciones muestran todas las alternativas. Elimina las erratas de tecleo en nombres largos; sigue siendo tarea suya comprobar que el nombre completado es el fichero que quería.
- **Ctrl+C:** envía SIGINT al proceso en primer plano. La mayoría de los programas terminan al recibirla, pero un programa puede atenderla de otra forma, retrasarla o ignorarla.
- **Ctrl+L:** limpia la pantalla, como `clear`.
- **Ctrl+A** y **Ctrl+E:** llevan el cursor al principio o al final de la línea.
- **Ctrl+U** y **Ctrl+K:** borran desde el cursor hasta el principio o hasta el final.
- **Alt+.**: inserta el último argumento de la orden anterior; útil para entrar en una carpeta recién creada.

Los atajos de GNU Readline en Bash, Tab para autocompletar, Ctrl+C para interrumpir, Ctrl+L para limpiar y Ctrl+A y Ctrl+E para saltar a los extremos de la línea, constituyen la ergonomía interactiva fundamental.

1.9.1 El historial y sus límites

Bash registra las órdenes en un historial que se guarda en `~/.bash_history`. Con Ctrl+R se busca en él de forma interactiva.

`history` refleja únicamente la sesión interactiva donde se ejecuta, tiene un límite de líneas conservadas y puede omitir órdenes ejecutadas en otra terminal; no es, por sí solo, un procedimiento documentado.

Un volcado del historial no sustituye a un protocolo: mezcla erratas, intentos fallidos y órdenes irrelevantes. Un procedimiento reproducible queda plasmado en un script limpio y un documento explicativo.

1.10 Espacio de almacenamiento y organización del proyecto

ESTUDIO Un proyecto de bioinformática genera con facilidad terabytes de ficheros temporales. Monitorizar el disco evita fallos durante ejecuciones que duran días. `df -h` muestra el espacio total, usado y disponible en cada sistema de archivos montado, en unidades legibles; `du -sh RUTA` calcula lo que ocupa un directorio con todo su contenido. Los discos externos y de red se incorporan al árbol con `mount` y se retiran con `umount`; en un clúster lo hace el administrador, pero `df` le mostrará qué está montado y dónde.

`df` inspecciona el espacio libre y ocupado en los sistemas de archivos montados, mientras que `du` calcula el espacio consumido por ficheros y directorios concretos.

Se reproduce con `verification/chapter_results.sh disk_usage_check`.

REFERENCIA En un sistema Linux moderno, los programas que corren de fondo, servicios como el planificador de trabajos del clúster o el servidor de ficheros, los gestiona `systemd`. Como usuario no los administra, pero dos órdenes le ahorran preguntas: `systemctl status NOMBRE` dice si un servicio está activo, y `journalctl -u NOMBRE` muestra su registro. Si un análisis falla porque «el disco de red no responde», es ahí donde está la explicación.

1.10.1 Estructura canónica de un proyecto bioinformático

Siguiendo las recomendaciones de buenas prácticas [5, 1], un proyecto mantiene una estructura de carpetas predecible: `data/raw/` con los datos originales, de solo lectura, que jamás se modifican; `data/processed/` con los datos limpios y transformados; `scripts/` con el código; `results/` con gráficos, tablas y resultados finales; y un `README.md` que explica el objetivo, las dependencias y cómo ejecutar el análisis.

1.11 Antes de una operación que sobrescribe o elimina

ESTUDIO La línea de órdenes no tiene papelera. Cuando se ejecuta `rm fichero`, el sistema desvincula el nombre de su inodo y los bloques quedan libres para reutilizarse. En la práctica, recuperar datos borrados no es fiable: actúe como si fuera irreversible. Para evitar accidentes, aplique siempre el protocolo de dos pasos:

1. **Verificar el directorio actual con `pwd`:** asegúrese de que está en la carpeta de trabajo correcta y no en la raíz o en su carpeta personal.
2. **Inspeccionar el objetivo exacto con `ls -ld -- RUTA`:** compruebe que la ruta señala al fichero o directorio que desea afectar antes de sustituir `ls -ld` por la orden destructiva.

Antes de cualquier operación que pueda sobrescribir o eliminar datos conviene confirmar el directorio con `pwd` e inspeccionar el objetivo exacto con `ls -ld -- RUTA` antes de ejecutar nada; solo se actúa sobre objetivos explícitos ya comprobados.

Un caso especialmente dañino aparece al usar variables no definidas dentro de órdenes recursivas: `rm -rf "$CARPETA_TEMPORAL"/*`, si la variable está vacía, se expande a `rm -rf "/"`, que intentará borrar todo aquello sobre lo que su usuario tenga permiso de escritura; en una cuenta de trabajo, todos sus datos, y con privilegios elevados, el sistema. El mecanismo de expansión de variables que lo explica se estudia en el capítulo

siguiente.

1.12 Reproducibilidad, empaquetado e integridad

ESTUDIO La reproducibilidad de un análisis exige documentar con precisión el contexto en el que se obtuvieron los resultados. Un registro reproducible recoge, como mínimo, siete campos:

Un registro mínimo de entorno reproducible incluye: sistema operativo, versión de Bash, origen y versión de los datos, fecha, las órdenes ejecutadas, el resultado esperado y las limitaciones conocidas.

Delimitación conceptual: reproducibilidad y replicabilidad.

En este libro, **reproducibilidad computacional** significa reejecutar el mismo procedimiento sobre los mismos datos brutos y el mismo entorno registrado y obtener el mismo resultado. El caso ideal es la coincidencia bit a bit; en flujos con paralelismo o aleatoriedad, una reproducción válida puede diferir en detalles no significativos, y lo que se exige es que el procedimiento y el criterio de comparación estén declarados. Obtener conclusiones convergentes con una cohorte independiente, nuevos datos o una implementación distinta es otra dimensión, la **replicabilidad**.

- *Reproducibilidad*: una investigadora ejecuta el script publicado sobre los datos originales y comprueba que la suma SHA-256 de la tabla de salida coincide con la del artículo.
- *Replicabilidad*: otro laboratorio recoge una nueva muestra de pacientes, aplica un protocolo análogo y confirma que el biomarcador mantiene significación.

En el anexo, `practice_assets/registrar_entorno.sh` genera un registro con estos siete campos a partir de `uname -a`, `bash --version` y `date`, y lo valida [5, pp. 67–70].

1.12.1 Empaquetado portable con tar

Para compartir un proyecto preservando su estructura y permisos se utiliza `tar` con compresión `gzip` [20]: `tar -czf entrega.tar.gz carpeta/` crea el paquete (c crear, z comprimir, f nombre del fichero); `tar -tzf entrega.tar.gz` lista el contenido sin extraerlo; `tar -xzf entrega.tar.gz` lo extrae.

Empaquetar el directorio `data/` con `tar -czf` y volver a extraerlo en un directorio distinto reproduce el mismo número de ficheros y las mismas sumas `sha256sum` que el original.

Se reproduce con `verification/check_package_roundtrip.sh`.

1.12.2 Integridad con sha256sum

Un fichero puede corromperse durante una descarga o por un fallo del soporte. Una función de resumen criptográfico como SHA-256 calcula una huella de 64 caracteres hexadecimales a partir de los bytes exactos del fichero. Si cambia un solo bit de un fichero de 50 GB, la huella cambia de forma impredecible, con altísima probabilidad en aproximadamente la mitad de sus bits.

`sha256sum -c` detecta, con probabilidad extremadamente alta, cualquier cambio de bytes respecto a un valor de referencia fiable; no distingue un cambio intencionado de una corrupción accidental, ni valida el significado del contenido, ni acredita autoría.

Se reproduce con `verification/check_manifest_integrity.sh`.

1.13 Método de verificación y errores frecuentes

ESENCIAL Ante cualquier instrucción propuesta por usted o por un asistente de inteligencia artificial, aplique el ciclo metodológico del curso:

El ciclo predecir, probar en pequeño, inspeccionar cada etapa, contrastar con la estructura esperada y registrar reduce la aceptación de una salida plausible pero incorrecta; este capítulo lo introduce y el siguiente lo recupera sin volver a enseñarlo.

1. **Predecir:** formule qué espera que haga la orden antes de pulsar Enter.
2. **Probar en pequeño:** ejecute primero sobre un subconjunto diminuto cuyo resultado conozca.
3. **Inspeccionar cada etapa:** compruebe el estado de salida (`echo $?`) y la salida intermedia.
4. **Contrastar con la estructura:** si esperaba 4 muestras, un resultado con 0 o con 400 es señal de alarma.
5. **Registrar:** guarde la orden validada en su script documentado.
6. **Validar antes de ejecutar:** un script se comprueba sin efectos con `bash -n script.sh`, que detecta errores de estructura, como comillas sin cerrar o un `if` sin `fi`, sin ejecutar ninguna orden.

Síntoma observable	Causa probable	Comprobación y corrección
No such file or directory command not found	directorio de trabajo incorrecto o errata en la ruta relativa orden no instalada o script local invocado sin <code>./</code>	<code>pwd</code> ; Tab para completar la ruta real <code>command -v ORDEN</code> ; anteponer <code>./</code>
Permission denied	falta <code>x</code> o <code>r</code>	<code>ls -l</code> ; <code>chmod u+x</code> solo sobre scripts propios revisados entrecorillar la ruta
Un fichero con espacios «desaparece» o parte la orden en dos	el espacio separa argumentos	
Una variable no llega a un script	definida como local y no exportada	<code>export NOMBRE</code> antes de lanzar el subproceso

Los cinco síntomas de esta tabla corresponden a los errores de principiante más frecuentes documentados en la presentación oficial de fundamentos de Linux del curso.

Compruebe antes de corregir: diagnóstico en dos fases.

Un guion de resumen devuelve `No such file or directory` en una terminal y `Permission denied` en otra. No aplique `chmod +x` de entrada. Primero `pwd` en ambas ventanas, para saber si están en el mismo directorio; después `ls -ld -- ./resultados/resumir.sh`, para confirmar que el fichero existe en esa ruta; y solo entonces `ls -l` para mirar sus permisos. Nunca corrija permisos sobre una ruta que no ha comprobado que apunta al fichero correcto.

1.14 Síntesis de referencia

Necesidad operativa	Construcción principal	Pregunta de control
Entender el diseño	todo es un fichero; herramientas modulares	¿Por qué combinamos herramientas pequeñas en vez de usar una sola?
Orientarse	<code>pwd</code> , <code>ls -la</code> , <code>cd</code>	¿Desde qué directorio de trabajo se interpreta esta ruta?
Organizar	<code>mkdir -p</code> , <code>cp</code> , <code>mv</code> , <code>touch</code>	¿He comprobado el destino antes de escribir?
Inspeccionar sin alterar	<code>file</code> , <code>head</code> , <code>tail</code> , <code>less</code> , <code>wc -l</code>	¿Qué unidad estoy contando y cómo salgo (q)?
Gestionar enlaces	<code>ln -s ORIGEN ENLACE</code>	¿Estoy evitando duplicar gigabytes en el disco?
Proteger y compartir	<code>rxw</code> , <code>chmod</code> , <code>chown</code>	¿Quién puede leer, escribir o ejecutar este fichero?
Ejecutar programas	<code>\$PATH</code> , <code>./</code> , <code>\$?</code>	¿Qué significa el estado de salida para este programa concreto?

Necesidad operativa	Construcción principal	Pregunta de control
Configurar el entorno	variables locales frente a <code>export</code> , <code>~/.bashrc</code>	¿Esta variable llega a los subprocesos que lance?
Ergonomía	<code>Tab</code> , <code>Ctrl+C</code> , <code>Ctrl+L</code> , <code>Ctrl+R</code>	¿Estoy usando el autocompletado para evitar erratas?
Monitorizar disco	<code>df -h</code> , <code>du -sh</code>	¿Hay espacio suficiente antes de lanzar el análisis?
Seguridad destructiva	<code>pwd</code> y <code>ls -ld -- RUTA</code>	¿Es exactamente el objetivo previsto?
Reproducir y entregar	<code>tar -czf</code> , <code>tar -tzf</code> , <code>sha256sum</code> <code>-c</code>	¿El paquete conserva la estructura y los bytes?

La tabla de síntesis enlaza cada necesidad operativa del capítulo con su construcción principal y una pregunta de control verificable.

1.15 Glosario esencial

- **Unix / tipo Unix:** sistema operativo original de 1969 frente a sus descendientes y clones (Linux, macOS, BSD).
- **Proceso:** programa en ejecución con un identificador único (PID) y recursos propios.
- **Terminal:** interfaz de texto que conecta al usuario con el intérprete de órdenes.
- **Shell:** intérprete de órdenes; en este texto, Bash.
- **Estado de salida** (\$?): entero que devuelve un proceso al terminar; 0 cuando no detectó error, otro valor para una condición que el programa define, a menudo un error.
- **\$PATH:** lista ordenada de directorios donde la Shell busca ejecutables por nombre.
- **Directorio de trabajo:** posición actual en el árbol desde donde se resuelven las rutas relativas.
- **Enlace simbólico:** fichero especial que apunta a otra ruta sin duplicar los datos.
- **Permisos rwx:** lectura, escritura y ejecución para usuario, grupo y otros.
- **Variable de entorno:** variable exportada, disponible para los subprocesos de la Shell.
- **Reproducibilidad:** capacidad de repetir un análisis obteniendo el mismo resultado según el criterio de comparación declarado, gracias al registro explícito del entorno.
- **Suma SHA-256:** huella criptográfica que detecta con altísima probabilidad cualquier cambio de bytes en un fichero.

1.16 Fuentes y límites de uso

Este capítulo se apoya en las presentaciones docentes del curso, 20250909_intro.pptx (historia, arquitectura y filosofía Unix) y 20250911_fundamentos.pptx (modelo de procesos, permisos, enlaces, variables de entorno, órdenes fundamentales y ergonomía), y en la bibliografía de referencia: la guía de bolsillo de Barrett [3], bibliografía básica de la asignatura; Buffalo [5] y Allesina y Wilmes [1] para la organización reproducible del trabajo; y los manuales de GNU Bash [16], GNU Coreutils [18] y GNU tar [20] para el comportamiento exacto de cada orden. Las opciones y mensajes descritos corresponden a las versiones de GNU instaladas en un Linux actual; en macOS y BSD pueden diferir.

Anexo práctico · Ambientarse, inspeccionar y entregar de forma reproducible

Pregunta, producto y separación de materiales

¿Puede alguien que no ha visto su pantalla repetir exactamente lo que usted hizo, a partir solo de lo que entregó? Este laboratorio construye un procedimiento mínimo, explorar, registrar el entorno y empaquetar, que responde que sí.

El producto individual contiene: un registro de entorno con los siete campos mínimos de la sección [Reproducibilidad, empaquetado e integridad](#), una tabla de respuestas con su predicción y su resultado para cada pregunta, la salida de las órdenes de inspección guardada en `resultados/`, una nota de protección, un `README.md` con las órdenes ejecutadas y sus supuestos, una defensa breve y un paquete `.tar.gz` verificado con `sha256sum`.

Este anexo es autónomo: no comparte laboratorio con otro capítulo. Usa `data/` del capítulo como entrada de solo lectura, que no se modifica, y un directorio de entrega que usted crea. Los valores esperados de las preguntas de inspección no se imprimen aquí: los produce usted y los contrasta después con la herramienta que el generador copia en `herramienta/`.

Mapa de ruta y productos entregables

Bloque	Producto
1 preparar el espacio de trabajo	directorio de entrega creado sin tocar <code>data/</code>
2 explorar e inspeccionar	predicciones y resultados en <code>notas/respuestas.tsv</code> ; salidas en <code>resultados/</code>
3 registrar el entorno	<code>notas/entorno.tsv</code> con los siete campos rellenos
4 confirmar antes de escribir	<code>notas/proteccion.txt</code> con <code>pwd</code> , <code>ls -ld</code> y una frase
5 empaquetar y verificar	<code>.tar.gz</code> y verificador en <code>ENTREGA_OK</code>

La ruta es única, sin separación entre esencial y de estudio. Cada bloque produce evidencia comprobable. Antes de ejecutar cada orden, escriba lo que espera obtener: el contraste entre predicción y resultado es lo que se evalúa, no el resultado solo.

1 • Preparar el espacio de trabajo

Desde el directorio del capítulo, compruebe la versión de Bash y cree el directorio de entrega:

```
bash --version
./practice_assets/create_workspace.sh TC01_entrega
cd TC01_entrega
```

El generador crea `datos/`, `notas/`, `resultados/` y `herramienta/`. En `datos/` copia, no enlaza, los cuatro ficheros de `data/` del capítulo y su manifiesto, conservando el nombre con espacio. En `notas/` deja `respuestas.tsv` solo con su cabecera. En `herramienta/` deja el oráculo del capítulo. Si el directorio ya existe, el generador se niega a sobrescribirlo: elija otro nombre.

Comprueba. Prediga qué mostrará `ls -la` dentro de `TC01_entrega` antes de ejecutarlo. ¿Cuántas entradas espera, contando `.` y `..`?

2 • Explorar e inspeccionar sin alterar

Para cada pregunta, anote primero su predicción en `notas/respuestas.tsv` (columnas pregunta, orden, predicción, resultado) y solo después ejecute la orden y complete el resultado. Las preguntas:

1. Cuántas líneas tiene cada uno de los cuatro ficheros de `datos/`. Para el más corto, cuente además a mano con `cat` y compare.
2. Qué tipo identifica `file` para `tc01_sample.log`, y si coincide con lo que sugiere la extensión.
3. Cuáles son las cinco primeras y las tres últimas líneas de `tc01_sample.fasta`.
- 4.Cuál es la ruta absoluta de `datos/`, obtenida con `pwd` desde dentro de `TC01_entrega`.

Las órdenes que las responden, con su salida guardada en `resultados/`:

```
wc -l datos/tc01_mini.fasta datos/tc01_sample.fasta datos/tc01_sample.log "datos/tc01_notas campo.txt" > resultados/
cat datos/tc01_mini.fasta
```



```
file datos/tc01_sample.log > resultados/tipo_log.txt
head -n 5 datos/tc01_sample.fasta > resultados/cabecera_sample.txt
tail -n 3 datos/tc01_sample.fasta > resultados/cola_sample.txt
pwd
```

Observe que el fichero con espacio en el nombre va entre comillas: sin ellas, la Shell lo partiría en dos argumentos. Cuando tenga sus resultados, y solo entonces, contrástelos con la herramienta:

```
bash herramienta/chapter_results.sh line_count_sample
bash herramienta/chapter_results.sh file_type
```

Comprueba. Si su conteo de `tc01_sample.fasta` no coincide con la herramienta, no corrija el número: averigüe qué cuenta cada uno. `wc -l` cuenta saltos de línea; ¿cuenta el último si el fichero no termina en salto?

3 • Registrar el entorno

El registro de entorno tiene siete campos: sistema, versión de Bash, origen de los datos, fecha, órdenes ejecutadas, resultado esperado y limitaciones conocidas. La herramienta rellena los que puede observar y deja marcados los tres que solo usted puede escribir:

```
../practice_assets/registrar_entorno.sh notas/entorno.tsv
```

Abra `notas/entorno.tsv` con un editor y sustituya cada valor marcado PENDIENTE por su propio juicio: de dónde proceden los datos, qué resultado esperaba de cada orden y qué limitaciones conoce. Después valide:

```
../practice_assets/registrar_entorno.sh --validar notas/entorno.tsv
```

La validación rechaza un registro al que le falte cualquiera de los siete campos o que conserve alguno sin rellenar. Los tres campos de juicio no son observaciones automáticas: son la parte del registro que permite a otra persona decidir si su procedimiento se aplica a su caso.

4 • Confirmar antes de escribir

Antes de guardar nada más en `resultados/`, confirme dónde está y qué es exactamente el directorio de destino, y conserve ambas salidas:

```
{ pwd; ls -ld resultados/; } > notas/proteccion.txt
cat notas/proteccion.txt
```

Añada después al final de `notas/proteccion.txt`, con su editor, una frase que explique qué habría mostrado `ls -ld` si el directorio no existiera todavía. Este capítulo no invoca `rm` en ningún paso. El caso más peligroso, una variable vacía dentro de una orden destructiva, necesita variables, que el capítulo siguiente enseña; su anexo retoma este mismo protocolo sobre ese caso concreto.

5 • Empaquetar y verificar la entrega

Escriba en `README.md`, dentro de `TC01_entrega`, las órdenes que ejecutó y los supuestos que hizo, y en `notas/defensa.md` la defensa que se pide más abajo. Después vuelva al directorio del capítulo, empaquete, inspeccione el paquete sin extraerlo y verifique:

```
cd ..
tar -czf TC01_entrega.tar.gz TC01_entrega
tar -tzf TC01_entrega.tar.gz | head -n 20
sha256sum TC01_entrega.tar.gz
./practice_assets/check_entrega.sh TC01_entrega TC01_entrega.tar.gz
```

El verificador comprueba, con Bash y GNU Coreutils, que `datos/` no fue modificado respecto al manifiesto del capítulo, que el registro de entorno tiene los siete campos rellenos, que la tabla de respuestas tiene al menos cuatro filas con predicción y resultado, que la nota de protección contiene la salida de `pwd` y de `ls -ld`, que `resultados/` no está vacía, que el `README.md` y la defensa existen con la extensión mínima, y que el paquete se extrae íntegro en un directorio temporal y contiene el registro y las respuestas. Si falta algo lo enumera y termina con `ENTREGA_INCOMPLETA`; si todo está, termina con `ENTREGA_OK`. No puntúa: la nota la pone quien corrige.

Rúbrica de evaluación

La evaluación pondera la ejecución técnica, la calidad del registro y la fundamentación conceptual sobre 10 puntos: 3 de integridad técnica y 7 de razonamiento.

- **3,0 puntos, entrega técnica íntegra:** paquete que se extrae, estructura de carpetas requerida, datos de partida inalterados y verificador en `ENTREGA_OK`.
- **2,5 puntos, registro de entorno:** los siete campos completos; los tres de juicio (origen, resultado esperado, limitaciones) son específicos y permitirían una repetición independiente en otro equipo.
- **2,0 puntos, exploración y predicción:** cada respuesta lleva su predicción previa; el contraste entre conteo automático y manual está explicado; las rutas relativas y absolutas se distinguen; el nombre con espacio se maneja correctamente.
- **1,5 puntos, protocolo de seguridad e integridad:** la nota de protección justifica el protocolo previo a escribir en disco, y la entrega muestra que se entiende qué garantiza y qué no garantiza una suma de comprobación.
- **1,0 punto, defensa:** justificación concisa, clara y ejemplificada con la propia entrega.

Obtener `ENTREGA_OK` es un requisito previo, pero no compensa un registro sin rigor ni una defensa equivocada.

Lista de autocorrección

- | | |
|--|--|
| <ul style="list-style-type: none"> ■ puedo decir desde qué directorio se interpreta cada ruta relativa que usé; ■ no modifiqué ningún fichero de <code>data/</code> ni de <code>datos/</code>; ■ el nombre con espacio está siempre entrecomillado en mis órdenes; ■ cada fila de respuestas tiene predicción y resulta- | <ul style="list-style-type: none"> do, y anoté si coincidieron; ■ el registro de entorno tiene los siete campos rellenos, no solo los automáticos; ■ ninguna orden de mi entrega usa <code>rm</code>; ■ el verificador termina con <code>ENTREGA_OK</code>; ■ puedo explicar la diferencia entre «la suma coincide» y «el contenido es correcto». |
|--|--|

Defensa

En `notas/defensa.md`, en 100 a 150 palabras, explique qué campo de su registro de entorno sería imprescindible para que otra persona, en otra máquina, repitiera exactamente su exploración, y qué campo es solo información de contexto. Use un ejemplo concreto de su propia entrega, no una afirmación general.

Fuentes

Los datos de partida están en `data/`, son sintéticos y se redistribuyen con licencia CC0 y manifiesto propio. El protocolo de seguridad y la estructura de proyecto siguen a Buffalo [5, pp. 67–70]; las órdenes de inspección están documentadas en el manual de GNU Coreutils [18] y, de forma más breve, en la guía de bolsillo de Barrett [3].

Bash, automatización y procesamiento de texto

Edición 2

Pregunta del tema. Ha ejecutado a mano la misma orden sobre veinte ficheros y mañana llegan veinte más. ¿Cómo se convierte esa secuencia de órdenes en un procedimiento que otra persona pueda ejecutar, leer y corregir?

Producto final. Un script propio, con sus comprobaciones y su registro, que recorre un corpus de ficheros biológicos y produce una tabla de resultados reproducible.

Mapa del tema

2.1. De la orden repetida al procedimiento automático	21
2.2. Fundamentos de Bash: órdenes, opciones y variables	22
2.3. Caracteres especiales y comillas	24
2.4. Flujos estándar: stdin, stdout, stderr	24
2.5. Redirección: escribir, añadir, leer y separar errores	25
2.6. Tuberías: componer programas sin que se conozcan entre sí	25
2.7. De la orden interactiva al script	26
2.8. Variables, comillas y argumentos posicionales	27
2.9. Aplicación: proteger una operación destructiva	28
2.10. Condicionales	29
2.11. Bucles	30
2.12. Funciones	31
2.13. Arrays y expansión de llaves	32
2.14. Operadores nativos de cadena	33
2.15. Hito de consolidación: del programa robusto al texto biológico	33
2.16. El formato FASTA como primer registro multilínea	34
2.17. Buscar con grep	35
2.18. Expresiones regulares	36
2.19. Utilidades clásicas de texto: cut, sort, uniq y tr	38
2.20. Transformar con sed	39
2.21. Procesar texto estructurado con awk	40
2.22. Lo que viene después: de procesar texto a razonar sobre su coste	42
2.23. Método y errores frecuentes	42
2.24. Síntesis de referencia	43
2.25. Glosario esencial	43
2.26. Fuentes y límites de uso	44

Cómo usar este tema

Este capítulo es largo porque cubre dos bloques que el temario separa y que en la práctica van juntos. El primero, **programar la Shell**, va de la sección **De la orden repetida al procedimiento automático** al hito de la sección **Hito de consolidación: del programa robusto al texto biológico**: variables, comillas, flujos, condicionales, bucles y funciones. El segundo, **procesar texto**, va del hito al final: FASTA, grep, expresiones regulares, sed y awk. El hito es la frontera, y conviene detenerse ahí: hasta ese punto se aprende a escribir programas que no mienten; a partir de ahí, a leer datos que no están limpios.

Al terminar sabrá: escribir un script con sus opciones de seguridad y saber qué protegen y qué no; entrecomillar de forma que un nombre con espacio no rompa nada; encadenar programas por tuberías y detectar el fallo de una etapa intermedia; recorrer ficheros con bucles y arrays sin tropezar con los globs vacíos; y extraer campos de un FASTA, un GFF o un TSV con grep, sed y awk, sabiendo cuál de los tres conviene en cada caso.

Las marcas **ESENCIAL**, **ESTUDIO** y **REFERENCIA** señalan qué bloques forman la ruta esencial, cuáles amplían y cuáles son de consulta. La ruta esencial son los dos bloques nominales sin sus apartados avanzados. La ruta de estudio añade *lookaround*, dialectos, arrays avanzados y el ejemplo trabajado de GFF. Escriba su predicción antes de ejecutar cada bloque de terminal.

2.1 De la orden repetida al procedimiento automático

En el primer capítulo aprendimos los cimientos del entorno Unix: cómo comunicarnos con el sistema operativo a través de una terminal de texto, cómo orientarnos en la jerarquía del sistema de archivos y cómo ejecutar programas con seguridad. Ese conocimiento nos permite resolver con soltura cualquier tarea puntual: inspeccionar un archivo, mover un conjunto de datos o lanzar una herramienta bioinformática.

Sin embargo, la investigación biológica moderna rara vez se enfrenta a un fichero aislado. En proyectos reales de genómica, transcriptómica o metagenómica, los datos de secuenciación masiva se distribuyen en decenas o cientos de muestras de secuenciación (*FASTQ*), con miles de secuencias (*FASTA*) y extensas tablas de anotación genómica (*GFF* o *VCF*). Escribir órdenes una por una a mano sobre veinte ficheros no solo es tedioso: es una fuente frecuente de errores y, sobre todo, un procedimiento que nadie puede reproducir. La orden tecleada a mano no deja registro. En cuanto cierra la sesión desaparece, y con ella la respuesta a la única pregunta que importa seis meses después: qué parámetros usó exactamente y sobre qué ficheros.

Una regla práctica de la computación científica: **si va a repetir un procedimiento más de dos veces, automatízelo**. No por elegancia, sino porque el script es a la vez el procedimiento y su documentación. Este capítulo sigue el orden del programa oficial de la asignatura (*Módulo 2: Shell Scripting y Procesamiento de Texto*) y está diseñado para transformar al estudiante de un usuario que ejecuta órdenes reactivas a un científico computacional capaz de diseñar procedimientos automáticos, robustos y verificables:

- **Fundamentos de Bash y sintaxis segura:** anatomía de órdenes, variables, caracteres especiales, comillas y aritmética entera.
- **Flujos y composición:** los tres canales estándar de Unix (`stdin`, `stdout`, `stderr`), operadores de redirección y la composición modular mediante tuberías (`|`).
- **De la orden al script:** cómo estructurar la lógica previa en pseudocódigo, empaquetar instrucciones en un script ejecutable, depurar con `bash -n` y blindar su ejecución con `set -euo pipefail`.
- **Estructuras de control y modularidad:** condicionales (`if/elif/else`, `[[ ]]`, `(( ))`), bucles (`for`, `while`, `read`, `break`, `continue`), funciones con ámbito local y operadores nativos de manipulación de cadenas.
- **La tríada Unix de procesamiento de texto:** el formato biológico *FASTA*, búsqueda con `grep` y expresiones regulares extendidas (*ERE*), transformación de secuencias con `sed` y análisis estructurado por columnas con `awk`.

Al finalizar el capítulo, se abrirá formalmente el puente hacia la algoritmia y el análisis del coste computacional, tema central del capítulo TC-CH03.

2.2 Fundamentos de Bash: órdenes, opciones y variables

2.2.1 Anatomía de una orden

Toda orden en la Shell de Bash se descompone en tres componentes fundamentales: `nombre_de_orden` [`opciones`] [`argumentos`]. El nombre indica el ejecutable que se desea lanzar; las opciones modifican el comportamiento del programa; y los argumentos representan los datos o rutas sobre los que actuará.

Las opciones suelen presentarse en dos formatos: **forma corta** con un único guion (como `-l` o `-h`) y **forma larga** con dos guiones (como `--all` o `--human-readable`). Cuando varias opciones cortas no requieren argumento propio, pueden combinarse tras un único guion (`-lh` equivale exactamente a `-l -h`).

Una orden de Bash se descompone en nombre de la orden, opciones (forma corta con un guion o larga con dos guiones) y argumentos; el orden relativo de las opciones entre sí normalmente no importa, pero su posición respecto a los argumentos sí puede importar según la orden.

Varias opciones cortas sin argumento propio pueden combinarse tras un único guion: `wc -l -w` y `wc -lw` piden el mismo resultado a `wc`.

Se reproduce con `verification/chapter_results.sh combined_short_options`.

2.2.2 Variables: asignar y recuperar un valor

Una variable es un contenedor con nombre que almacena información en la memoria de la Shell durante la sesión. En Bash, la asignación de una variable se escribe con la sintaxis `NOMBRE=valor`, con una condición indispensable: **no debe haber ningún espacio alrededor del signo igual**. Si se introduce un espacio (`NOMBRE = valor`), Bash interpretará que `NOMBRE` es un comando y que `=` y `valor` son sus argumentos, resultando en un fallo inmediato.

Para recuperar el valor almacenado en una variable se antepone el símbolo `$`: `$NOMBRE`. Cuando el nombre de la variable está adyacente a otros caracteres de texto que podrían confundir al analizador sintáctico, es obligatorio y una excelente práctica de ingeniería delimitar el nombre entre llaves: `${NOMBRE}`.

`NOMBRE=valor` asigna sin error; `NOMBRE = valor` (con espacios) no asigna la variable y falla o hace otra cosa.

Se reproduce con `verification/chapter_results.sh variable_assignment_spacing`.

`${NOMBRE}_sufijo` recupera `NOMBRE` y añade el sufijo literal; `$NOMBRE_sufijo` intenta recuperar una variable distinta que normalmente no existe y se expande a vacío.

Se reproduce con `verification/chapter_results.sh brace_disambiguation`.

2.2.3 Sustitución de órdenes, subshells y aritmética

A menudo necesitamos capturar el resultado emitido por una orden para guardarlo en una variable o pasarlo como argumento a otro programa. La sintaxis moderna y anidable de **sustitución de órdenes** es `$(orden)`. Bash ejecuta `orden`, captura su salida estándar (`stdout`), elimina el salto de línea final y sustituye la expresión por ese contenido.

Por otro lado, los paréntesis simples `( ... )` ejecutan el bloque de órdenes en un entorno secundario aislado o **subshell**. Cualquier cambio en variables o en el directorio de trabajo dentro del subshell se descarta al finalizar, sin alterar la sesión principal.

Para realizar cálculos matemáticos enteros, Bash proporciona la **expansión aritmética** mediante la sintaxis `$(expresión)`. Es fundamental recordar que la aritmética nativa de Bash es estrictamente **entera**: la división trunca la parte decimal hacia cero, por lo que `$( 7 / 2 )` da como resultado 3 y no 3.5.

`$(orden)` se sustituye por el `stdout` de `orden` sin el salto de línea final; el resultado puede asignarse a una variable o usarse directamente como argumento de otra orden.

Se reproduce con `verification/chapter_results.sh command_substitution`.

`$( 7 / 2 )` vale 3, no 3.5: la aritmética de `$( )` es entera y trunca hacia cero, no redondea.

Se reproduce con `verification/chapter_results.sh arithmetic_expansion`.

2.2.4 Separadores de línea y control de flujo

Además del salto de línea convencional, Bash reconoce operadores especiales para encadenar múltiples órdenes en una única línea con semántica de control:

- `;` (punto y coma): ejecuta la primera orden y, cuando termina, ejecuta la siguiente de forma incondicional.
- `&&` (Y lógico): ejecuta la segunda orden **únicamente si la primera terminó con éxito** (código de salida 0).
- `||` (O lógico): ejecuta la segunda orden **únicamente si la primera falló** (código de salida distinto de 0).
- `&` (ampersand simple): lanza la orden en segundo plano (*background*) y devuelve el control a la Shell de inmediato.

`cmdA && cmdB` ejecuta `cmdB` solo si `cmdA` termina con estado 0; `cmdA || cmdB` ejecuta `cmdB` solo si `cmdA` termina con estado distinto de cero; `;` encadena sin mirar el resultado; `&` lanza en segundo plano.

Se reproduce con `verification/chapter_results.sh line_separators`.

2.3 Caracteres especiales y comillas

En la Shell de Bash, ciertos caracteres tienen un significado especial y se denominan **metacaracteres**. La Shell interpreta estos caracteres antes de entregar los argumentos al programa ejecutable. Dominar el uso de las comillas es la habilidad más importante para escribir código seguro y predecible.

Carácter	Significado en Bash	Prevención
\$	Expansión de variables y sustituciones	'\$VAR' o \ \$VAR
* ? []	Coincidencia de nombres de fichero (<i>globbing</i>)	Entrecomillar o \
~	Expansión de directorio personal (<i>home</i>)	Entrecomillar
#	Inicio de comentario	Entrecomillar
; &	Separadores de órdenes y tuberías	Entrecomillar
espacio	Separador de palabras y argumentos	Comillas dobles " . . . "
\	Carácter de escape: quita significado al siguiente	\\
'	Comillas simples: suprime toda expansión	Cerrar comilla
"	Comillas dobles: permite \$ y \$(. . .), pero bloquea división	Cerrar comilla

Los metacaracteres de esta tabla son el conjunto mínimo necesario para leer y escribir con seguridad cualquier script de este capítulo.

La regla general de las comillas en Bash es sencilla pero crucial:

1. **Comillas simples (' . . . ')**: proporcionan literalidad absoluta. Nada dentro de comillas simples se expande: ni variables ('\$VAR'), ni sustituciones ('\$ (pwd) '), ni comodines ('*.fasta'). Es el escudo más fuerte para preservar texto intacto.
2. **Barra invertida (\)**: escapa un único carácter inmediatamente posterior, anulando su significado especial.
3. **Comillas dobles**: permiten la expansión de variables y la sustitución de órdenes, pero **bloquean la división en palabras** (*word splitting*) y **la expansión de rutas** (*globbing*). Es el entrecomillado que quiere casi siempre.

Dentro de comillas simples ninguna expansión ocurre; el texto se pasa literalmente carácter a carácter, incluido el signo dólar o el asterisco.

Se reproduce con `verification/chapter_results.sh single_quote_literal`.

Una barra invertida antes de un único carácter le quita su significado especial para esa aparición; `\$VAR` imprime literalmente el signo dólar seguido del nombre.

Se reproduce con `verification/chapter_results.sh backslash_escape`.

Dentro de comillas dobles sí se expanden `$variable` y `$(orden)`, pero el resultado no se somete a división de palabras ni a expansión de *glob*.

Se reproduce con `verification/chapter_results.sh double_quote_expands_blocks_split`.

2.4 Flujos estándar: stdin, stdout, stderr

En la arquitectura de Unix, todo proceso en ejecución nace con tres canales de comunicación unidireccionales abiertos de forma predeterminada, identificados por un número entero o **descriptor de archivo**:

1. **Entrada estándar (stdin, descriptor 0)**: canal por el que el programa lee sus datos de entrada. Por defecto, está conectado al teclado.
2. **Salida estándar (stdout, descriptor 1)**: canal por el que el programa emite sus resultados normales de procesamiento. Por defecto, está conectado a la pantalla del terminal.
3. **Error estándar (stderr, descriptor 2)**: canal por el que el programa emite avisos, diagnósticos y mensajes de error. Aunque por defecto también se muestra en la pantalla, es un canal completamente independiente

de stdout.

Esta separación conceptual es un principio maestro de ingeniería: permite guardar o canalizar los datos biológicos válidos emitidos por stdout sin que queden contaminados por advertencias o diagnósticos de depuración emitidos por stderr.

stdin, stdout y stderr son tres flujos independientes de un proceso; separarlos permite conservar un resultado mientras los avisos se descartan o se inspeccionan por separado.

Se reproduce con `verification/chapter_results.sh separate_stderr`.

2.5 Redirección: escribir, añadir, leer y separar errores

Los operadores de redirección permiten alterar el destino o el origen de los flujos estándar, conectándolos a archivos en disco en lugar de la terminal.

Operador	Significado y comportamiento	Ejemplo
>	Redirige stdout a fichero, truncando previo	<code>orden > salida.txt</code>
>>	Redirige stdout a fichero, añadiendo al final	<code>orden >> salida.txt</code>
<	Redirige fichero hacia stdin	<code>orden < entrada.txt</code>
2>	Redirige solo stderr a fichero	<code>orden 2> errores.log</code>
2>>	Redirige stderr añadiendo al final	<code>orden 2>> errores.log</code>
&>	Redirige ambos (stdout y stderr)	<code>orden &> todo.log</code>

>, >>, <, 2>/2>> y &> son los operadores de redirección documentados en el manual de Bash para este capítulo.

Advertencia sobre la truncación inmediata. El operador > trunca y vacía el archivo de destino en el mismo instante en que la Shell interpreta la orden, **antes** de que el comando empiece a ejecutarse. Si un script intenta leer y escribir sobre el mismo archivo a la vez (`sort muestra.txt > muestra.txt`), el archivo quedará completamente vacío antes de que sort pueda leer una sola línea, perdiéndose los datos de forma irreversible.

`orden > fichero` trunca fichero antes de escribir, incluso si la orden falla y no produce salida; el contenido previo se pierde.

Se reproduce con `verification/chapter_results.sh truncate_on_redirect`.

2> redirige únicamente stderr; una orden ejecutada con salida y error redirigidos por separado escribe su resultado y sus diagnósticos en ficheros distintos.

Se reproduce con `verification/chapter_results.sh separate_stderr`.

>> añade al final del destino sin truncar el contenido previo, a diferencia de >; dos escrituras sucesivas conservan ambos bloques de datos.

Se reproduce con `verification/chapter_results.sh append_does_not_truncate`.

2.6 Tuberías: componer programas sin que se conozcan entre sí

El operador de tubería (`|`, *pipe*) es la encarnación más elegante de la filosofía Unix: conecta directamente la salida estándar (stdout) de un programa con la entrada estándar (stdin) del siguiente.

En `a | b`, el stdout de a se convierte en el stdin de b; ni a ni b necesitan saber que su entrada o salida proviene de una tubería.

2.6.1 El peligro del estado de salida en tuberías: pipefail

Por defecto, en una tubería compuesta (`cmd1 | cmd2 | cmd3`), la Shell asigna a la variable especial `$?` el código de salida del **último comando ejecutado** (`cmd3`). Si el primer comando (`cmd1`) falla catastróficamente (por ejemplo, porque el archivo de datos no existe), pero `cmd3` termina correctamente recibiendo una entrada vacía, la Shell reportará que la tubería tuvo éxito (`$? = 0`).

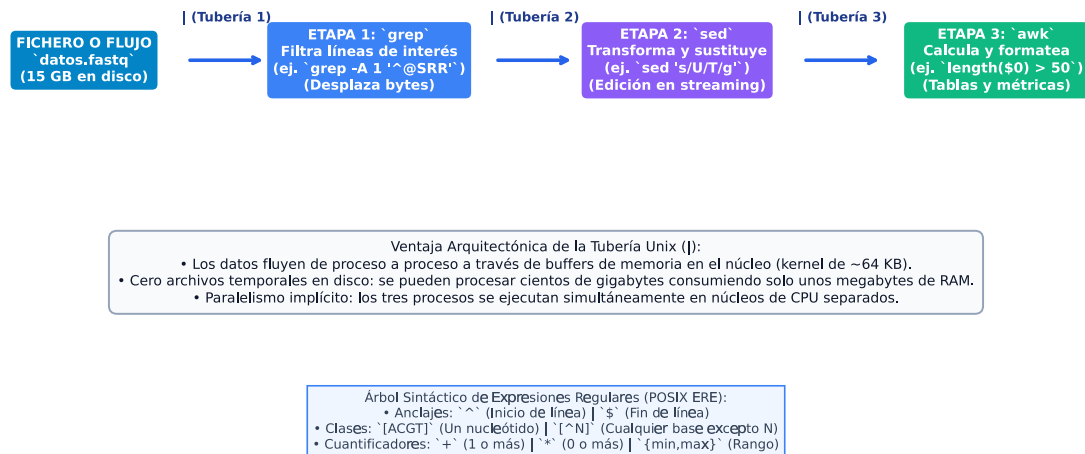


Figura 2.1: Flujograma del modelo de tuberías Unix (|) conectando grep, sed y awk en memoria, junto con el árbol sintáctico de expresiones regulares POSIX ERE. Figura original adaptada de las presentaciones docentes.

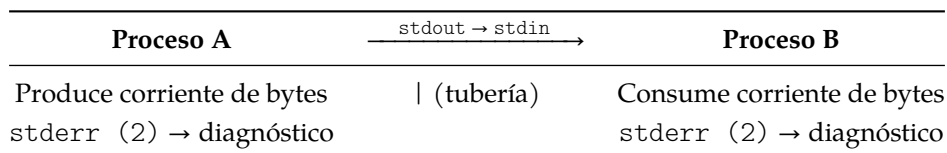


Figura 2.2: Esquema de una tubería Unix. Los programas no se conocen entre sí; solo el flujo de texto que cruza el pipe los acopla.

En bioinformática, este comportamiento por defecto puede arruinar un análisis completo sin dejar rastro de error. Para solucionar esto, activamos la opción `set -o pipefail`. Con `pipefail`, el estado de salida de la tubería será el del comando que haya fallado más a la derecha, de modo que un fallo en una etapa temprana ya no pasa desapercibido.

Atención. `pipefail` tiene un efecto secundario que conviene conocer antes de tropezar con él. Si una etapa posterior cierra la tubería antes de tiempo, como hace `... | head -n 5` en cuanto tiene sus cinco líneas, la etapa anterior recibe la señal `SIGPIPE` y termina con estado 141. Con `set -e` activo, el script muere aunque el resultado sea exactamente el que usted quería. Evite `head` en tuberías bajo `set -e`, o justifique con un comentario el `|| true` que añada.

Sin `set -o pipefail`, el estado de salida de una tubería de varias etapas refleja solo el del último comando; el fallo del primero pasa inadvertido.

Se reproduce con `verification/chapter_results.sh pipeline_exit_status`.

2.7 De la orden interactiva al script

2.7.1 De la idea al pseudocódigo

Escribir un script no consiste en teclear órdenes apresuradas en un archivo. El desarrollo profesional comienza siempre diseñando el procedimiento en **pseudocódigo**: una descripción lógica y estructurada en lenguaje natural, libre de particularidades de sintaxis, que descompone el problema biológico en pasos elementales:

```
Para cada archivo que termine en .fasta dentro de datos/corpus/:
  Comprobar si el archivo contiene datos regulares
  Contar el numero de registros (lineas que empiezan por >)
  Si tiene mas de 100 registros:
    Anotar el archivo en la lista de alta densidad
  Si no:
```



```
Anotar el archivo en la lista estandar
Fin del bucle
```

Escribir primero el procedimiento en pseudocódigo —lenguaje claro, sin sintaxis exacta— antes de traducirlo a Bash reduce errores de diseño y clarifica el flujo de datos.

2.7.2 El Shebang y las opciones de seguridad (`set -euo pipefail`)

Un script de Bash es un archivo de texto con permisos de ejecución (`chmod +x script.sh`). La primera línea debe contener el **shebang**: `#!/usr/bin/env bash`, que indica al sistema operativo qué intérprete debe ejecutar el archivo.

Para que los fallos sean visibles en vez de silenciosos, los scripts de este curso empiezan siempre por el mismo par de líneas [16]:

```
#!/usr/bin/env bash
set -euo pipefail
```

- `set -e`: termina el script si falla una orden simple. No actúa sobre la condición de un `if` o un `while`, ni sobre los miembros izquierdos de `&&` y `||`, ni sobre las etapas intermedias de una tubería mientras no active también `pipefail`. Es una red de seguridad amplia, no una garantía.
- `set -u`: termina con error si se intenta leer una variable no definida, evitando que una errata tipográfica se convierta silenciosamente en una cadena vacía.
- `set -o pipefail`: propaga los fallos ocurridos en cualquier eslabón de una tubería.

Límites y tratamiento explícito de excepciones.

Esta cabecera no demuestra que el script sea matemáticamente correcto ni sustituye la validación explícita de entradas y precondiciones. Cuando un comando puede devolver código no cero como comportamiento esperado (por ejemplo, cuando `grep` no encuentra coincidencias en una búsqueda condicional), el script debe gestionar ese estado explícitamente mediante una estructura condicional (`if`) o una alternativa controlada (`|| true`), en vez de delegar pasivamente la decisión en `set -e`.

Antes de ejecutar un script, podemos verificar si contiene errores de sintaxis sin lanzarlo realmente utilizando la opción de prueba en seco `bash -n script.sh`. Para seguir la traza completa de ejecución paso a paso, usamos `bash -x script.sh`.

`bash -n script.sh` comprueba la sintaxis sin ejecutar el script; detecta errores sintácticos, pero no de lógica ni de datos.

Se reproduce con `verification/chapter_results.sh syntax_check`.

`set -u` hace que el script termine con error al hacer referencia a una variable no definida, en vez de sustituirla silenciosamente por una cadena vacía.

Se reproduce con `verification/chapter_results.sh set_u_catches_typo`.

`set -o pipefail` hace que el estado de salida de una tubería sea distinto de cero si cualquiera de sus etapas falla.

Se reproduce con `verification/chapter_results.sh pipeline_exit_status`.

2.8 Variables, comillas y argumentos posicionales

Para que un script sea verdaderamente reutilizable, debe aceptar argumentos por línea de comandos. Dentro del script, estos argumentos se reciben en variables especiales denominadas **parámetros posicionales**:

- `$0`: nombre o ruta con la que se invocó el propio script.
- `$1`, `$2`, ..., `$9`: primer, segundo y sucesivos argumentos posicionales.

- `$#` : número total de argumentos recibidos.
- `"$@"` : expande todos los argumentos posicionales manteniendo cada uno como una palabra separada e individual, incluso si contienen espacios internos. Es la forma correcta de pasar los argumentos uno a uno; `"$*"` sirve para lo contrario, unirlos todos en una sola cadena.

Dentro de un script, `$0` es el nombre del propio script, `$1` es el primer argumento, `$#` es el número total de argumentos y `"$@"` expande todos los argumentos preservando sus límites.

Se reproduce con `verification/chapter_results.sh positional_args` .

Una variable sin comillas que representa una ruta se somete a división de palabras y expansión de *glob*; si contiene espacios o comodines, la orden recibe más o menos argumentos de los previstos.

Se reproduce con `verification/chapter_results.sh quoting_matters` .

2.9 Aplicación: proteger una operación destructiva

Una trampa clásica en scripting ocurre al combinar variables con operaciones de borrado (`rm -rf`). Consideremos el siguiente caso:

```
DIRECTORIO_TEMP=""
# ERROR PELIGROSO:
# rm -rf "$DIRECTORIO_TEMP"/*    <- Si la variable esta vacia, se expande a rm -rf /*
```

Aunque tengamos activado `set -u` , esta opción solo detecta variables **no declaradas**. Una variable declarada pero asignada a la cadena vacía (`VAR=""`) pasa `set -u` sin emitir ningún error. Al concatenarse con el comodín fuera de las comillas (`"$DIRECTORIO_TEMP"/*`), la expresión se expande a `/*` (la raíz del sistema de archivos), con consecuencias catastróficas. Si el asterisco estuviera dentro de las comillas (`"$DIR/*"`), no expandiría comodines sino que buscaría un nombre literal con asterisco.

Para evitar desastres, seguimos el protocolo preventivo de comprobación:

1. Validar con `[[ -n "$VAR" ]]` que la variable contenga una cadena no vacía.
2. Validar con `[[ -d "$VAR" ]]` que el directorio exista en disco.
3. Realizar siempre una inspección inocua antes de cualquier acción, para ver exactamente en qué se expande la ruta:

```
printf '<%s>\n' "$DIRECTORIO_TEMP"/*
```

`set -u` detecta una variable *no definida*, pero no detecta una variable definida y vacía (`VAR=""`); `rm -rf "$VAR/*"` pasa sin error y resulta destructivo sobre un ámbito indeseado.

Se reproduce con `verification/chapter_results.sh empty_var_passes_set_u` .

El protocolo de TC-CH01 se completa aquí con un paso adicional específico de scripts parametrizados: mostrar la expansión exacta con un comando inofensivo como `printf` antes de ejecutar cualquier acción destructiva.

Se reproduce con `verification/chapter_results.sh destructive_dry_run` .

2.9.1 Escribir el script: un editor mínimo

ESTUDIO Un script es un fichero de texto, así que necesita un editor dentro del terminal. Con dos basta para este curso.

`nano` es el más sencillo: se abre con `nano script.sh` , se escribe directamente y las órdenes aparecen abajo, donde el símbolo `^` significa la tecla Control. Guardar es Control-O y salir Control-X.

`vim` es más incómodo al principio y está en cualquier servidor, incluso en los que no tienen `nano` instalado, así que conviene conocer el mínimo para no quedarse atrapado: se abre con `vim script.sh` , arranca en modo orden, se pasa a modo inserción con `i` , se vuelve al modo orden con Escape, y desde ahí `:wq` guarda y sale, `:q!` sale descartando los cambios.

Concepto. Da igual cuál elija. Lo que no da igual es que el script quede guardado en el proyecto y no en el historial de la terminal: el fichero se puede leer, versionar y corregir; la orden tecleada, no.

2.9.2 Cuatro formas de ejecutar un script, y en qué se diferencian

- `bash script.sh`: lanza un intérprete nuevo que lee el fichero. No hace falta permiso de ejecución, y la línea `#!` se ignora.
- `./script.sh`: el sistema lee la línea `#!` para saber con qué intérprete ejecutarlo. Necesita permiso de ejecución.
- `script.sh`, sin ruta: solo funciona si el directorio está en `PATH`. El directorio actual no lo está, y por eso el punto y la barra del caso anterior no son un adorno.
- `source script.sh` o `. script.sh`: no lanza un proceso nuevo; ejecuta las órdenes en la sesión actual. Las variables que defina el script permanecen después. Es la forma de cargar un fichero de configuración, y una fuente de sorpresas si lo usa por costumbre: un `exit` dentro del script cierra su terminal.

2.10 Condicionales

2.10.1 `[ ]` frente a `[[ ]]`

Para evaluar condiciones en Bash disponemos del comando clásico `[ ]` (o `test`) y de la construcción moderna extendida `[[ ]]`. En scripts de Bash conviene **usar** `[[ ]]`, que evita errores de división de palabras y permite operadores lógicos avanzados como `&&`, `||` y coincidencia de patrones con expresiones regulares (`=~`). El corchete simple no es un resto del pasado: sigue siendo necesario cuando el script debe correr bajo `sh` POSIX, donde `[[ ]]` no existe. Si su cabecera dice `bash`, use el doble; si dice `sh`, no puede.

Aspecto	<code>[]</code> (clásico / POSIX)	<code>[[]]</code> (Bash moderno)
Comillas en variables	Obligatorias siempre (<code>"\$var"</code>)	Muy recomendadas por estilo
División de palabras	Sí, falla si la variable contiene espacios	No se produce dentro de <code>[[]]</code>
Operadores lógicos	<code>-a</code> (AND), <code>-o</code> (OR)	<code>&&</code> (AND), <code> </code> (OR)
Coincidencia de patrones	No	Sí (<code>== *.txt</code> , <code>= ~[0-9]+\$</code>)

`[ ]` y `[[ ]]` difieren en la necesidad de comillas y en los operadores lógicos que aceptan (`-a/-o` frente a `&&/||`); `[[ ]]` es la forma estándar en Bash.

2.10.2 Álgebra booleana y operadores de prueba

Operador	Significado de la condición
<code>[[-f "\$fichero"]]</code>	Cierto si el fichero existe y es un fichero regular
<code>[[-d "\$dir"]]</code>	Cierto si la ruta existe y es un directorio
<code>[[-s "\$fichero"]]</code>	Cierto si el fichero existe y su tamaño es mayor que cero
<code>[[-r "\$fichero"]]</code>	Cierto si el fichero tiene permiso de lectura
<code>[[-z "\$cadena"]]</code>	Cierto si la cadena de texto está vacía (longitud 0)
<code>[[-n "\$cadena"]]</code>	Cierto si la cadena de texto no está vacía
<code>[["\$a" == "\$b"]]</code>	Comparación de igualdad textual
<code>[["\$a" != "\$b"]]</code>	Comparación de desigualdad textual
<code>((a == b))</code>	Comparación de igualdad numérica
<code>((a > b))</code>	Comparación de desigualdad numérica (mayor que)
<code>[[-e "\$ruta"]]</code>	Cierto si la ruta existe, sea del tipo que sea

Operador	Significado de la condición
<code>[[-L "\$ruta"]]</code>	Cierto si la ruta es un enlace simbólico
<code>[["\$a" -nt "\$b"]]</code>	Cierto si a se modificó más recientemente que b
<code>[["\$m" -eq "\$n"]]</code>	Igualdad entera dentro de corchete simple; sus parientes son <code>-ne</code> , <code>-gt</code> , <code>-ge</code> , <code>-lt</code> y <code>-le</code>

Atención. Los operadores enteros y los textuales no son intercambiables. `[[ "10" > "9" ]]` es *falso*, porque compara texto y en el orden alfabético el uno va antes que el nueve; `(( 10 > 9 ))` es cierto. Cuando compare tamaños, conteos o posiciones, use la forma aritmética. Y si necesita el «uno u otro pero no los dos», el o exclusivo, no existe operador propio: se escribe como `[[ $a != $b ]]` sobre dos condiciones booleanas ya evaluadas.

Los operadores de cadena, enteros, ficheros y lógicos de esta tabla son el conjunto mínimo necesario para evaluar cualquier condición en los scripts de este capítulo.

`[[ -f "$ruta" ]]` comprueba si `$ruta` es un fichero regular existente; `[[ -z "$var" ]]` comprueba si `var` está vacía; `(( a > b ))` compara a y b como números, no como texto.

Se reproduce con `verification/chapter_results.sh test_operators_string_file_numeric`.

2.10.3 if/elif/else

```
if [[ condicion1 ]]; then
    comando1
elif [[ condicion2 ]]; then
    comando2
else
    comando3
fi
```

`if/elif/else/fi` evalúa condiciones en orden y ejecuta el primer bloque cuya condición sea verdadera; `else` se ejecuta solo si ninguna lo fue.

`(( a > b ))` compara a y b como números; `[[ "$a" > "$b" ]]` los compara como cadenas de texto y puede dar un resultado distinto (`"9" > "10"` es cierto lexicográficamente aunque falso numéricamente).

Se reproduce con `verification/chapter_results.sh numeric_vs_lexicographic`.

2.11 Bucles

2.11.1 for: iterar un número conocido de veces

El bucle `for` permite repetir un bloque de instrucciones sobre una colección de elementos o mediante un índice numérico estilo C:

```
for i in 1 2 3; do echo "$i"; done

for ((i=1; i<=3; i+=1)); do echo "$i"; done
```

El bucle `for` sobre una lista y el bucle `for` con inicialización, condición y paso son dos sintaxis equivalentes en Bash; la segunda es más cómoda cuando se itera con un contador numérico.

Se reproduce con `verification/chapter_results.sh for_loop_both_styles`.

2.11.2 while y lectura línea a línea

El bucle `while` repite instrucciones mientras una condición sea cierta. Combinado con el comando `read`, constituye el patrón más eficiente y seguro para procesar archivos de texto gigantescos línea a línea:

```
while IFS= read -r linea; do
    echo "Procesando: $linea"
done < archivo.txt
```

- IFS=: suprime el separador de campos por defecto para no recortar espacios en blanco al inicio o final de la línea.
- -r: desactiva la interpretación de la barra invertida como escape.

while IFS= read -r linea; do ... done < fichero procesa fichero línea a línea; IFS= evita recortar espacios en los extremos y -r evita que la barra invertida se trate como carácter de escape dentro de la línea leída.

Se reproduce con `verification/chapter_results.sh while_read_counts_lines`.

2.11.3 break y continue

Para modular el comportamiento de los bucles:

- break: interrumpe y finaliza el bucle de inmediato.
- continue: interrumpe la iteración actual y salta directamente a la siguiente.

Dentro de un for, break detiene el bucle antes de procesar los elementos restantes; continue omite el resto del cuerpo para el elemento actual pero sigue con el siguiente.

Se reproduce con `verification/chapter_results.sh break_and_continue`.

2.11.4 Dos formas de while que aparecen constantemente

La primera es el bucle sin condición, que se detiene desde dentro. Se usa cuando la condición de salida solo se conoce a mitad de la iteración:

```
while true; do
    read -r -p "Escriba «fin» para terminar: " respuesta
    [ "$respuesta" = fin ] && break
    echo "recibido: $respuesta"
done
```

La segunda reparte cada línea en varios campos a la vez. read corta por los separadores de IFS y asigna en orden; la última variable recoge todo lo que sobra, lo que resulta muy práctico cuando el último campo lleva espacios:

```
while read -r muestra condicion descripcion; do
    echo "$muestra ($condicion): $descripcion"
done < muestras.tsv
```

Atención. La opción -r no es opcional en la práctica. Sin ella, read interpreta la barra invertida como carácter de escape y le destroza cualquier ruta de Windows o cualquier secuencia que la contenga.

2.12 Funciones

Una función agrupa un conjunto de instrucciones bajo un nombre descriptivo para su reutilización:

```
saludar() {
    local nombre="$1"
    echo "Hola $nombre"
}
saludar "Ana"
resultado="$(saludar "Ana")"
```

Es fundamental distinguir dos conceptos:

1. **return N**: solo devuelve un **código de estado numérico** (de 0 a 255) accesible mediante `$?`. No devuelve texto.
2. **echo o printf**: emite datos a la salida estándar, los cuales pueden capturarse en variables mediante `$(funcion)`.

Una función de Bash se define con `nombre() { ... }` y se invoca como una orden; sus argumentos se leen con `$1, $2`, etc.

Se reproduce con `verification/chapter_results.sh function_define_and_call`.

`return N` termina una función con estado de salida `N` (accesible en `$?`); para devolver datos textuales se imprime con `echo/printf` y se captura con `$(...)`.

Se reproduce con `verification/chapter_results.sh function_return_vs_output`.

2.12.1 Buenas prácticas al escribir funciones

Bash admite dos sintaxis equivalentes, `nombre() { ... }` y `function nombre { ... }`. La primera es portable a `sh`; la segunda solo funciona en Bash. Elija una y manténgala en todo el script.

- Declare como `local` toda variable interna. Sin `local`, la variable es global y una función puede pisar el valor de otra sin que nada avise.
- Valide los argumentos en la primera línea del cuerpo, y termine con un mensaje claro si faltan: `[ $# -eq 2 ] || { echo "uso: ..." >&2; return 2; }`.
- Escriba en la salida estándar lo que la función *produce*, y en la de error lo que *informa*. Así quien la llame puede capturar el resultado con `$(funcion)` sin que se le mezclen los avisos.
- Use `return` para el estado, no para el resultado: `return` solo admite un número entre 0 y 255.
- Ponga a cada función un comentario de una línea que diga qué recibe y qué devuelve. Dentro de tres meses será usted quien lo lea.

2.13 Arrays y expansión de llaves

2.13.1 Arrays y recorrido seguro de ficheros

Al recorrer colecciones de archivos con comodines (`*.fasta`), si no hay ningún archivo coincidente, Bash deja por defecto el patrón literal sin expandir (`"*.fasta"`). Para evitar que el bucle intente procesar un fichero inexistente con ese nombre, activamos `shopt -s nullglob`:

```
shopt -s nullglob
ficheros=(datos/*.fasta)
if [[ ${#ficheros[@]} -eq 0 ]]; then
    echo "Aviso: no se encontraron archivos FASTA"
fi
```

Con `shopt -s nullglob`, un array construido como `f=(*.fasta)` queda vacío si no hay coincidencias en el directorio, en vez de contener el patrón literal sin expandir.

Se reproduce con `verification/chapter_results.sh nullglob_empty_dir`.

2.13.2 Expansión de llaves

La expansión de llaves genera combinaciones y secuencias de texto de forma puramente sintáctica antes de ejecutar la orden:

- Rangos numéricos o de caracteres: `{1..5}` genera `1 2 3 4 5`.

- Listas de alternativas: `muestra_{A,B,C}.txt` genera los tres nombres `muestra_A.txt`, `muestra_B.txt` y `muestra_C.txt`.
- Rangos con paso: `{1..10..2}` genera 1 3 5 7 9, y `{0..100..25}` sirve para recorrer umbrales sin escribirlos.

Atención. La expansión de llaves ocurre *antes* que la de comodines y no consulta el disco: `muestra_{A,B,C}.txt` genera los tres nombres existan o no. Si lo que quiere es «los ficheros que hay», use un comodín; si quiere «los nombres que deberían estar», use llaves y compruebe después cuáles faltan.

`{1..5}` se expande a 1 2 3 4 5 antes de que la Shell ejecute la orden; `{A,B}.txt` genera `A.txt` `B.txt` sin consultar el sistema de ficheros.

Se reproduce con `verification/chapter_results.sh brace_expansion_range_and_list`.

2.14 Operadores nativos de cadena

Bash incluye potentes operadores integrados de manipulación de cadenas que evitan lanzar procesos externos:

- **Extracción de subcadenas:** `${var:pos:long}` extrae `long` caracteres a partir de la posición `pos`.
- **Sustitución de patrones:** con una barra, `${var/antiguo/nuevo}`, sustituye la primera coincidencia; con dos barras, todas.
- **Eliminación de prefijos:** `${var#prefijo}` elimina el prefijo coincidente más corto; `${var##prefijo}` el más largo.
- **Eliminación de sufijos:** `${var%sufijo}` elimina el sufijo más corto (ideal para quitar extensiones de fichero: `${fichero%.fasta}`); `${var%%sufijo}` elimina el más largo, que es lo que hace falta con nombres como `muestra.trim.sorted.bam` cuando quiere quedarse solo con `muestra`.
- **Último carácter:** `${var: -1}` devuelve el último carácter. Los paréntesis no son decoración: sin ellos, los dos puntos seguidos de un signo menos significan otra cosa.

REFERENCIA Tres construcciones más que aparecerán en scripts ajenos. `$RANDOM` entrega un entero pseudoaleatorio distinto en cada lectura, útil para nombrar un temporal y desaconsejable para muestrear datos, porque no se puede fijar su semilla. `declare -i contador` marca una variable como entera, de modo que `contador=contador+1` funciona sin `$(( ))`. Y `${!muestra_*}` lista los nombres de todas las variables que empiezan por `muestra_`, que es la forma de recorrer un conjunto de parámetros declarados por separado.

`${var:0:3}` extrae los tres primeros caracteres de `var`; `${var/e/E}` sustituye la primera coincidencia y `${var//e/E}` sustituye todas.

Se reproduce con `verification/chapter_results.sh string_slice_and_substitute`.

`${var#prefijo}` elimina prefijo del inicio de la variable; `${var%sufijo}` elimina sufijo del final (útil para extensiones de fichero).

Se reproduce con `verification/chapter_results.sh string_prefix_suffix_removal`.

2.15 Hito de consolidación: del programa robusto al texto biológico

Hito 1 • Dominio de la construcción de programas robustos en Bash.

Hasta este punto del capítulo, hemos formalizado las cuatro capacidades fundamentales de la programación en Shell:

1. **Recepción segura de datos:** Manejo de argumentos posicionales con `"$@"` y protección contra separación de palabras y variables vacías.
2. **Estructuras de decisión:** Condicionales robustos con `[[ ... ]]` y evaluación lógica explícita.
3. **Control de flujo y repetición:** Bucles `for` sobre listas y archivos con `nullglob`, y bucles `while read` para flujos línea a línea.

4. **Modularidad:** Definición y llamada a funciones parametrizadas que devuelven resultados limpios.

A partir de la siguiente sección, aplicamos estas herramientas de automatización a nuestro objeto biológico principal: el procesamiento masivo de texto y secuencias genómicas. Para abordar cualquier fichero estructurado, distinguiremos tres operaciones complementarias:

- **Filtrar con grep:** Selecciona líneas enteras que coinciden con un patrón sin alterar su contenido.
- **Transformar con sed:** Modifica, sustituye o recorta el texto línea a línea de forma programática.
- **Procesar con awk:** Descompone cada línea en columnas/campos estructurados, acumulando estado y calculando estadísticas.

Comprobación de integración rápida.

Antes de continuar, prediga qué construcción utilizaría en cada caso: (a) ¿Cómo recorrer de forma segura todos los ficheros *.fasta de una carpeta sin fallar si no hay ninguno? → Con un array y la opción shopt -s nullglob. (b) ¿Cómo seleccionar exclusivamente las líneas de cabecera en un archivo genómico? → Con la expresión regular ^> en grep. (c) ¿Cómo garantizar que un pipeline detecte si falló la primera etapa de una tubería? → Con la directiva set -o pipefail.

2.16 El formato FASTA como primer registro multilínea

El formato **FASTA** es el estándar universal en biología computacional para almacenar secuencias de nucleótidos o aminoácidos. Su estructura es estrictamente multilínea:

1. Una **línea de cabecera** que comienza obligatoriamente con el símbolo mayor que (>), seguido de un identificador y una descripción opcional.
2. Una o más **líneas de secuencia** que contienen las letras del alfabeto biológico (A, C, G, T, N, etc.).

Atención. Su pariente cercano, el formato FASTQ que entrega el secuenciador, no se procesa igual. Sus cabeceras empiezan por arroba, cada registro ocupa exactamente cuatro líneas, y la cuarta guarda las calidades, donde la arroba es un carácter de calidad perfectamente válido. Buscar líneas que empiecen por arroba en un FASTQ cuenta cabeceras y líneas de calidad, y devuelve un número inflado. Para contar registros en un FASTQ se dividen las líneas entre cuatro; para contarlos en un FASTA se cuentan las cabeceras.

Concepto. Los ficheros de secuencias suelen llegar comprimidos, con extensión .gz, y descomprimirlos para mirarlos ocupa espacio y tiempo. Las utilidades zcat, zgrep y zless leen el fichero comprimido y escriben el contenido descomprimido en la salida, sin dejar copia en disco: zcat muestras.fasta.gz | grep -c '^>' cuenta secuencias sin descomprimir nada.

```
>seq1 Secuencia humana de prueba
ATGCGATCGATCGATC
GATCGATCGA
>seq2 Secuencia control con bases indeterminadas
ATGCGTNNNNNNCCGT
```

Diferencia entre contar líneas y contar registros. Un archivo FASTA puede tener sus secuencias repartidas en varias líneas. Por tanto, contar líneas con wc -l **no** indica cuántas secuencias hay. Para contar registros biológicos reales debemos contar cuántas líneas comienzan por >, usando grep -c '^>' fichero.fasta.

Un registro FASTA consiste en una línea de cabecera que empieza por > seguida de una o más líneas de secuencia hasta la siguiente cabecera o el final del fichero.

grep -c '^>' fichero.fasta cuenta el número de **registros** del fichero, no su número de líneas.

Se reproduce con verification/chapter_results.sh fasta_header_count.

```
ALGORITMO: Parser_FASTA_Multilinea_Streaming
ENTRADA:
```



```

- Flujo_Texto: Stream secuencial de líneas de entrada (stdin o archivo)
SALIDA:
- Stream_Registros: Tuplas emitidas (Cabecera, Secuencia_Completa, Longitud, Porcentaje_GC)
COMPLEJIDAD: Tiempo  $O(N)$  donde  $N$  es el total de caracteres, Espacio  $O(L_{\max})$  para el buffer

1. INICIALIZACION:
  Cabecera_Actual <- ""
  Buffer_Secuencia <- ""

2. BUCLE STREAMING:
  Para cada Línea en Flujo_Texto:
    Línea <- Eliminar_Espacios_Extremos(Línea)
    Si Línea es vacía entonces:
      Continuar al siguiente ciclo

    Si Línea comienza por '>' entonces:
      # Si tenemos un registro previo acumulado, lo procesamos y emitimos
      Si Cabecera_Actual != "" entonces:
        Longitud <- Longitud_Cadena(Buffer_Secuencia)
        Conteo_GC <- Contar_Ocurrencias(Buffer_Secuencia, {'G', 'C', 'g', 'c'})
        Si Longitud > 0 entonces:
          Porcentaje_GC <- (Conteo_GC / Longitud) * 100.0
        Sino:
          Porcentaje_GC <- INDEFINIDO
        AVISAR ("registro sin secuencia: ", Cabecera_Actual)
        EMITIR (Cabecera_Actual, Buffer_Secuencia, Longitud, Porcentaje_GC)

      # Inicializar nuevo registro
      Cabecera_Actual <- Línea
      Buffer_Secuencia <- ""
    Sino:
      # Acumular línea de nucleótidos/aminoácidos en el buffer
      Buffer_Secuencia <- Concatenar(Buffer_Secuencia, Línea)

3. TERMINACION (ULTIMO REGISTRO):
  Si Cabecera_Actual != "" y Buffer_Secuencia != "" entonces:
    Longitud <- Longitud_Cadena(Buffer_Secuencia)
    Conteo_GC <- Contar_Ocurrencias(Buffer_Secuencia, {'G', 'C', 'g', 'c'})
    Si Longitud > 0 entonces:
      Porcentaje_GC <- (Conteo_GC / Longitud) * 100.0
    Sino:
      Porcentaje_GC <- INDEFINIDO
    AVISAR ("registro sin secuencia: ", Cabecera_Actual)
    EMITIR (Cabecera_Actual, Buffer_Secuencia, Longitud, Porcentaje_GC)

```

2.17 Buscar con grep

grep (*Global Regular Expression Print*) es la herramienta habitual para buscar y filtrar patrones de texto en Unix.

Líneas coincidentes frente a ocurrencias totales. Si una línea de secuencia contiene tres repeticiones del motivo buscado, *grep -c* devolverá 1 (esa línea contiene al menos una coincidencia). Para contar las ocurrencias individuales exactas debemos extraer cada una con *grep -o* y contar con *wc -l*.

grep -c PATRÓN fichero cuenta **líneas** que contienen al menos una coincidencia, no el número total de coincidencias; una línea con dos coincidencias cuenta como una sola.

Se reproduce con `verification/chapter_results.sh grep_count_vs_occurrences`.

-n, *-i*, *-v*, *-c*, *-o*, *-q*, *-r/-R* y *-l* son las opciones de *grep* documentadas en el manual de GNU Grep para este capítulo. *egrep* es la forma histórica equivalente a *grep -E*.

2.17.1 Opciones avanzadas y control de contexto

Opción	Efecto	Ejemplo
-A n/-B n/-C n	líneas de contexto después/antes/ambos	grep -A2 '^>' f.fasta
-w	exige coincidencia de palabra completa	-w 'AT' no coincide con ATG
-x	exige que la línea entera coincida	grep -x 'ATGC' f.txt
-e PATRÓN (repetible)	añade un patrón más, en relación O	-e A -e T coincide con líneas que tengan A o T
-F	trata el patrón como cadena literal, no regex	un punto deja de ser comodín

La tabla de opciones extendidas de `grep` resume el control de contexto (-A/-B/-C), delimitación de palabra (-w), línea completa (-x), patrones múltiples (-e) y búsqueda literal rápida (-F).

`grep -A n/-B n/-C n` añaden a la salida las `n` líneas posteriores, anteriores o ambas alrededor de cada coincidencia.

Se reproduce con `verification/chapter_results.sh grep_context`.

`grep -w 'AT'` no coincide con una línea que contiene ATG, aunque sí coincide con una línea donde AT está rodeado de espacios o al final de línea.

Se reproduce con `verification/chapter_results.sh grep_word_boundary`.

`grep -x PATRÓN` solo cuenta como coincidencia una línea donde el patrón, anclado, cubre la línea completa.

Se reproduce con `verification/chapter_results.sh grep_exact_line`.

`grep -e PATRON1 -e PATRON2 fichero` es equivalente a `grep -E 'PATRON1|PATRON2' fichero`: ambas formas devuelven las líneas que coinciden con al menos uno de los dos patrones.

Se reproduce con `verification/chapter_results.sh grep_multiple_patterns`.

`grep -F` trata el patrón como cadena literal, no como expresión regular; un punto busca el carácter punto exacto y no actúa como comodín.

Se reproduce con `verification/chapter_results.sh grep_fixed_string`.

2.18 Expresiones regulares

2.18.1 Fundamento y sintaxis extendida (ERE)

Las expresiones regulares (*REGEX*) son un lenguaje formal para describir patrones en cadenas de texto, originadas en los trabajos sobre autómatas finitos del matemático Stephen Kleene (1951) e introducidas en Unix por Ken Thompson en el editor `ed`, hacia 1970, y en `grep`, en 1973.

Las expresiones regulares derivan del álgebra de autómatas finitos formalizada por Kleene (1951) e implementada en Unix por Thompson (1968, 1973).

Construcción	Significado	Ejemplo biológico
<code>^</code>	Ancla de inicio de línea	<code>^></code> (cabeceras FASTA)
<code>\$</code>	Ancla de fin de línea	<code>TAA\$</code> (codón de parada al final)
<code>.</code>	Cualquier carácter excepto salto de línea	<code>A.T</code> (A seguido de cualquier base y T)
<code>[]</code>	Clase de caracteres	<code>[AG]</code> (purinas)
<code>[^]</code>	Clase negada	<code>[^ACGT]</code> (bases no canónicas)
<code>*</code>	Cuantificador: 0 o más veces	<code>N*</code>
<code>+</code>	Cuantificador: 1 o más veces	<code>N+</code> (islas de indeterminación)
<code>?</code>	Cuantificador: 0 o 1 vez (opcional)	<code>AUG?</code>
<code>{n,m}</code>	Entre <code>n</code> y <code>m</code> repeticiones	<code>N{2,}</code> (dos o más bases N)

Construcción	Significado	Ejemplo biológico
()	Grupo de captura	(CG){2,} (repetición del dinucleótido CG)
	Alternancia (O lógico)	TAA TAG TGA (codones de parada)

Atención. Un patrón como (CG){2,} encuentra dinucleótidos CG contiguos, y eso no es una isla CpG. Una isla CpG se define por densidad sobre una ventana, típicamente varios centenares de bases con un contenido GC alto y una razón entre CpG observados y esperados por encima de un umbral. Ninguna expresión regular cuenta densidades. Es un buen ejemplo de la frontera del capítulo: la expresión regular reconoce patrones de texto, no conceptos biológicos, y confundir las dos cosas produce anotaciones que parecen correctas.

Ancas, clases, cuantificadores (incluido ? para cero o una aparición), grupos y alternancia son las cinco construcciones mínimas de ERE para describir patrones sobre secuencias de texto.

Sobre data/tc02_caso_regex.fasta, el patrón N{2,} coincide con la línea de secuencia de caso_dos y no con la de caso_uno.

Se reproduce con `verification/chapter_results.sh regex_positive_negative`.

2.18.2 Codicia, grupos de captura y aserciones de contexto (*Lookaround*)

Por defecto, los cuantificadores en REGEX son **voraces** (*greedy*): consumen la mayor cantidad posible de caracteres que satisfagan la condición. En motores compatibles con PCRE o lenguajes modernos, añadir ? tras un cuantificador (como .*?) lo convierte en **perezoso** (*lazy*), consumiendo el mínimo imprescindible.

Los **grupos de captura** con paréntesis (. . .) aíslan subcoincidencias para su reutilización mediante referencias hacia atrás (\1, \2).

Por último, en bioinformática es muy común buscar un motivo que deba estar precedido o seguido por una secuencia concreta, pero sin que esa secuencia sea consumida en el resultado. Para ello se emplean las construcciones de **ancho cero** (*lookaround*):

- **Positive Lookahead** (?=patrón): comprueba que lo siguiente coincida con patrón, sin incluirlo.
- **Positive Lookbehind** (?<=patrón): comprueba que lo precedente coincida con patrón, sin incluirlo.

Los cuantificadores son codiciosos por defecto en la mayoría de los motores de expresiones regulares, consumiendo la subcadena más larga posible.

Un grupo de captura (. . .) en una expresión regular aísla subcadenas para referencias hacia atrás o sustitución estructurada.

Sin escapar, el punto de una ERE coincide con cualquier carácter; \. coincide solo con el punto literal.

Se reproduce con `verification/chapter_results.sh regex_dot_escape`.

[[[:alpha:]]], [[[:digit:]]] y [[[:space:]]] son clases POSIX con nombre equivalentes a sus conjuntos manuales; un circunflejo al inicio de corchetes ([^0-9]) niega la clase.

Se reproduce con `verification/chapter_results.sh regex_posix_class_and_negation`.

2.18.3 Glob frente a regex; dialectos BRE, ERE y PCRE

Es crucial no confundir los comodines de la Shell (*globbing*) con las expresiones regulares:

- **Globbing**: lo interpreta la Shell para generar nombres de **archivos** antes de lanzar un comando (*.fasta).
- **Regex**: lo interpreta el **programa** (grep, sed, awk) para buscar patrones en el **texto**.

Un patrón de *glob* de Bash (*.fasta) y una expresión regular no son el mismo lenguaje: el *glob* lo interpreta la Shell sobre nombres de fichero, mientras que la expresión regular la interpreta el programa sobre el contenido de texto.

Dialecto	Dónde se usa	Características
BRE	grep básico, sed por defecto	+, ?, () y { } requieren barra: \+
ERE	grep -E, sed -E, awk	Sintaxis limpia y moderna sin escapar metacaracteres
PCRE	grep -P, utilidades avanzadas	Añade atajos (\d, \w), lookahead y no codicia

Este capítulo declara y usa siempre ERE; PCRE añade atajos de clase, cuantificadores no codiciosos y aserciones de contexto (*lookaround*); sed usa BRE por defecto y requiere -E para usar la misma sintaxis.

2.19 Utilidades clásicas de texto: cut, sort, uniq y tr

Unix cuenta con un repertorio de utilidades clásicas indispensables para procesar tablas y listas:

- **cut -f CAMPO**: extrae columnas de un fichero estructurado. Su delimitador por defecto ya es el tabulador, así que para un TSV basta `cut -f2`. Para otro separador se indica con `-d`, que admite **un solo carácter**: `cut -d ',' -f2`. Escribir `cut -d '\t' -f2` no funciona, porque ahí la barra y la `t` son dos caracteres; si necesita el tabulador explícito, la forma correcta es `cut -d $'\t' -f2`.
- **sort**: ordena líneas alfabéticamente (o numéricamente con `-n`).
- **uniq**: elimina líneas duplicadas consecutivas y cuenta frecuencias con `-c`. Compara solo líneas *adyacentes*: para eliminar duplicados que no están seguidos, ordene antes con `sort`.
- **tr**: traduce o elimina caracteres individuales. `tr 'a-z' 'A-Z'` pasa a mayúsculas; `tr -d '\r'` borra los retornos de carro que deja un fichero venido de Windows; `tr -s ' '` colapsa cada serie de espacios repetidos en uno solo. Las clases con nombre evitan depender del alfabeto: `tr '[:lower:]' '[:upper:]'` funciona igual en una máquina configurada en otro idioma.

ESTUDIO Dos utilidades más completan el repertorio mínimo. `find` recorre un árbol de directorios y selecciona por criterios en lugar de por comodín: `find datos -type f -name '*.fasta'` baja por todos los subdirectorios, cosa que `datos/*.fasta` no hace. Su compañera habitual es `xargs`, que convierte una lista de nombres en argumentos de otra orden; para que los espacios en los nombres no la rompan, la pareja se escribe `find ... -print0 | xargs -0 ...`. Y `diff fichero1 fichero2` muestra en qué líneas difieren dos ficheros, que es la forma rápida de comprobar si el resultado de hoy coincide con el de la semana pasada; `diff -q` se limita a decir si son iguales o no.

En un fichero separado por tabuladores, `cut -f2` extrae el segundo campo sin necesidad de indicar delimitador; con un delimitador que no aparece en la línea, `cut` devuelve la línea completa y no emite error.

Se reproduce con `verification/chapter_results.sh cut_field`.

Comprueba. La segunda mitad de esa afirmación es la peligrosa. Si se equivoca de delimitador, `cut` no protesta: le devuelve la línea entera como si fuera el campo pedido. Prediga qué imprime `cut -d ',' -f2` sobre una línea separada por tabuladores antes de ejecutarlo, y compruebe después que su tubería no estaba arrastrando ese error.

`uniq` solo elimina líneas duplicadas **adyacentes**; los duplicados no consecutivos requieren ordenar antes con `sort`.

Se reproduce con `verification/chapter_results.sh uniq_needs_sort`.

`tr 'a-z' 'A-Z'` traduce carácter a carácter de la primera clase a la segunda; no reconoce palabras ni límites de campo.

Se reproduce con `verification/chapter_results.sh tr_case`.

2.20 Transformar con sed

sed (*Stream Editor*) es un editor de flujo línea a línea. No abre una ventana interactiva; lee el texto, aplica un programa de transformación a cada línea y escribe el resultado en la salida estándar, dejando intacto el archivo original (salvo con la opción de edición en el lugar `-i .bak`).

La orden básica de sustitución es `sed 's/patrón/reemplazo/'` (cambia la primera coincidencia por línea) o `sed 's/patrón/reemplazo/g'` (con el modificador `g`, cambia todas). Si el patrón contiene barras (como en rutas de archivos), podemos usar cualquier otro delimitador: `sed 's#/usr/local#/opt#'`.

Podemos restringir las operaciones a líneas específicas mediante **direcciones**:

- **Por número o rango:** `sed '2,5s/A/T/g' fichero.txt`.
- **Última línea (\$):** `sed '$s/A/T/' fichero.txt` aplica la sustitución exclusivamente a la última línea del archivo. Aquí `$` actúa como dirección posicional de sed; dentro de una expresión regular (TAA\$) es un ancla de fin de línea, y en la Shell inicia una variable (\$VAR). Las comillas simples impiden que la Shell interprete el símbolo antes de entregarlo a sed.
- **Por coincidencia de patrón:** `sed '/^>/s/ /_/g'` (sustituye espacios solo en las cabeceras FASTA).
- **Eliminación:** `sed '/^$/d'` (elimina líneas vacías).
- **Inserción y adición:** `1i\TEXTO` (inserta antes de la línea 1) y `1a\TEXTO` (añade después).
- **Imprimir solo lo seleccionado:** `sed -n` apaga la impresión automática de cada línea, y entonces la orden `p` imprime únicamente las que usted elija. `sed -n '5p'` muestra la quinta línea; `sed -n '/^>/p'` muestra solo las cabeceras. Sin `-n`, esas mismas órdenes imprimirían las líneas seleccionadas *dos veces*, que es el desconcierto habitual de la primera vez.
- **Negar una selección:** el signo de admiración invierte la dirección. `sed -n '/^>/!p'` imprime todo *menos* las cabeceras, es decir, solo la secuencia; y `sed '1!d'` conserva la primera línea y borra el resto.

sed es un editor de flujo: lee su entrada línea a línea, aplica el programa de edición y escribe el resultado en stdout sin modificar el fichero de entrada a menos que se use `-i`.

`sed 's/patrón/reemplazo/'` sustituye únicamente la primera coincidencia de cada línea; las siguientes permanecen intactas.

Se reproduce con `verification/chapter_results.sh sed_first_match_only`.

`sed 's/patrón/reemplazo/g'` sustituye todas las coincidencias de cada línea gracias al indicador global `g`.

Se reproduce con `verification/chapter_results.sh sed_global_flag`.

sed acepta un delimitador distinto de `/` para la sustitución, evitando escapar las barras de una ruta.

Se reproduce con `verification/chapter_results.sh sed_custom_delimiter`.

`sed '2s/.../.../'` actúa solo en la línea 2; con un rango de líneas actúa en ese rango; con una dirección por patrón actúa en las líneas que coinciden; sin dirección se aplica a todas.

Se reproduce con `verification/chapter_results.sh sed_addressing`.

`sed '/^$/d'` elimina las líneas completamente vacías de la salida; las demás pasan sin cambios.

Se reproduce con `verification/chapter_results.sh sed_delete_blank_lines`.

`sed -i` sobrescribe el fichero de entrada sin guardar copia a menos que se especifique un sufijo de seguridad (`sed -i .bak`); usarlo sin sufijo hace irreversible cualquier error en el patrón.

Se reproduce con `verification/chapter_results.sh sed_inplace_backup`.

Las órdenes estructurales de sed para borrar (`d`), insertar (`i`), añadir (`a`) y cambiar líneas completas (`c`) permiten transformar el flujo de texto de forma programática.

Se reproduce con `verification/chapter_results.sh sed_delete_insert_append_change`.

2.20.1 La tríada Unix: roles complementarios

grep decide qué líneas pasan sin cambiarlas (filtra); sed transforma el contenido de cada línea que pasa (transforma); awk divide cada línea en campos y aplica lógica de programa (procesa). Encadenar los tres cubre la mayoría de las tareas de procesamiento de texto en bioinformática.

2.21 Procesar texto estructurado con awk

awk (creado por Aho, Weinberger y Kernighan) es un lenguaje de programación completo orientado al procesamiento de datos estructurados en filas (**registros**) y columnas (**campos**). Su estructura básica sigue el paradigma: patrón { acción }.

Variables especiales fundamentales:

- \$0: contenido completo de la línea actual.
- \$1, \$2, ..., \$NF: primer, segundo y último campo de la línea actual.
- NF: número total de campos en la línea actual (*Number of Fields*).
- NR: número de registro o línea global actual (*Number of Records*).
- FS: separador de campos (se define con -F ' , ' o -F '\t ').
- FNR: número de registro *dentro del fichero actual*. Con un solo fichero coincide con NR; con varios, NR sigue contando y FNR vuelve a empezar. De ahí el modismo FNR == 1 para hacer algo en la primera línea de cada fichero, por ejemplo saltarse una cabecera repetida.
- FILENAME: nombre del fichero que se está leyendo, imprescindible cuando la salida mezcla varios y hay que saber de dónde vino cada fila.
- OFS y ORS: separador de campos y de registros *de salida*. FS dice cómo se lee; OFS cómo se escribe. Para convertir un CSV en un TSV basta el programa de una línea que aparece bajo esta lista: la asignación aparentemente inútil del primer campo sobre sí mismo obliga a awk a reconstruir la línea con el nuevo separador.
- RS: separador de registros de entrada. Fijándolo al símbolo mayor que, un fichero FASTA se lee por secuencias en lugar de por líneas, que es la forma natural de tratar un formato multilínea.
- **Bloques BEGIN y END:** BEGIN { ... } se ejecuta una única vez antes de leer los datos; END { ... } se ejecuta una única vez tras procesar la última línea.

Y tres construcciones del lenguaje que hacen falta en cuanto el programa pasa de una línea. La condicional se escribe como en cualquier lenguaje, if (\$3 > 100) { ... } else { ... }, y puede ir dentro de la acción. split(cadena, destino, separador) parte una cadena en un array y devuelve cuántos trozos salieron, que es como se desmenuza el noveno campo de un GFF. Y index(cadena, subcadena) devuelve la posición de la primera aparición, o cero si no está, sin necesidad de expresión regular.

```
awk 'BEGIN { FS = ","; OFS = "\t" } { $1 = $1; print }' muestras.csv > muestras.tsv
```

Concepto. Cuando el programa awk pasa de tres o cuatro líneas, sáquelo del entrecomillado y guárdelo en un fichero con extensión .awk, que se ejecuta con awk -f programa.awk datos.tsv. Gana comentarios, sangrado y control de versiones, y deja de pelearse con las comillas de la Shell. Es el mismo salto que dio el capítulo al pasar de la orden suelta al script.

En awk, NR es el número de registro actual, NF es el número de campos y \$0 es la línea completa; un bloque END se ejecuta exactamente una vez tras procesar toda la entrada.

Se reproduce con verification/chapter_results.sh awk_nr_nf.

awk -F ' , ' cambia el separador de campos de espacios/tabuladores a coma para procesar ficheros CSV.

Se reproduce con verification/chapter_results.sh awk_custom_field_separator.

`awk '{suma += $1} END {print suma}'` acumula valores aritméticos de una columna a lo largo de las líneas y emite el total final en el bloque `END`.

Se reproduce con `verification/chapter_results.sh awk_arithmetic_accumulate`.

El operador `~` comprueba si un campo coincide con una expresión regular (`$1 ~ /^A/`); `!~` comprueba que no coincida.

Se reproduce con `verification/chapter_results.sh awk_match_operator`.

`length($0)` devuelve la longitud de la línea; `substr($0, pos, len)` extrae una subcadena a partir de la posición `pos`.

Se reproduce con `verification/chapter_results.sh awk_length_substr`.

`sub(patrón, reemplazo, $0)` sustituye la primera coincidencia en el registro; `gsub(patrón, reemplazo, $0)` sustituye todas.

Se reproduce con `verification/chapter_results.sh awk_sub_gsub`.

`awk -v nombre=valor '...'` declara una variable accesible dentro del programa `awk` con el valor asignado desde la Shell.

Se reproduce con `verification/chapter_results.sh awk_dash_v`.

Un array asociativo de `awk` (`count[$clave]++`) acumula frecuencias por clave sin necesidad de declarar previamente qué claves existirán.

Se reproduce con `verification/chapter_results.sh awk_assoc_array`.

2.21.1 Ejemplo trabajado: medir exones en un GFF

ESTUDIO El formato GFF anota regiones sobre un genoma. Cada línea tiene nueve campos separados por tabuladores: secuencia, origen, tipo, inicio, fin, puntuación, hebra, fase y un noveno campo con atributos en la forma `clave=valor` separados por punto y coma. Las líneas que empiezan por almohadilla son cabeceras. El fichero `data/tc02_annotacion.gff` contiene una anotación reducida con tres genes y sus exones.

La pregunta. Cuántos exones hay, cuánto suman y cuál es su longitud media.

La hipótesis. Las coordenadas de un GFF son cerradas y empiezan en 1, así que la longitud de una región es $fin - inicio + 1$, no $fin - inicio$. Con seis exones de unos trescientos o cuatrocientos pares de bases, la suma debería rondar los dos millares.

```
awk -F'\t' '$3 == "exon" { n++; suma += $5 - $4 + 1 }
END { printf "EXONES:%d LONGITUD_TOTAL:%d MEDIA:%.1f\n", n, suma, suma/n }' \
data/tc02_annotacion.gff
```

```
EXONES:6 LONGITUD_TOTAL:2186 MEDIA:364.3
```

La comprobación. Seis exones es lo que se cuenta a mano en el fichero, y la media cae dentro de lo previsto. Si hubiéramos olvidado el `+ 1` la suma habría sido 2180: un error de seis bases que ningún mensaje habría señalado y que en un genoma completo se convierte en miles.

El segundo paso. Ahora el nombre de cada gen, que vive dentro del noveno campo y hay que desmenuzar. Se parte primero por punto y coma para aislar el atributo `Name`, y después por el signo igual para quedarse con su valor:

```
awk -F'\t' '$3 == "gene" { split($9, campos, ";"); split(campos[2], nombre, "=")
printf "%s\t%d\n", nombre[2], $5 - $4 + 1 }' \
data/tc02_annotacion.gff
```

```
BRCA1x      2201
TP53x       901
MIRx        451
```

Atención. Este segundo programa da por hecho que Name es el segundo atributo del noveno campo. Es cierto en este fichero y falso en cuanto cambie de proveedor de anotación. Lo correcto en producción es recorrer los atributos y quedarse con el que se llama Name, no con el que ocupa la segunda posición. Que un programa funcione sobre su fichero de prueba no significa que funcione sobre el fichero siguiente, y esa distinción es la diferencia entre un análisis y un accidente.

Sobre la anotación reducida del capítulo, los seis exones suman 2186 bases con una media de 364,3, y las longitudes de los tres genes son 2201, 901 y 451 bases.

Se reproduce con `verification/chapter_results.sh awk_gff_exones`.

2.21.2 El puente de Shell a awk: conteo de nucleótidos

Para la secuencia AACG, el programa de conteo por bases imprime A:2 C:1 G:1 T:0, incluidas las bases ausentes con valor cero.

```
echo AACG | awk '{ for (i = 1; i <= length($0); i++) c[substr($0, i, 1)]++ }
                END { for (i = 1; i <= 4; i++) { s = substr("ACGT", i, 1)
                printf "%s:%d ", s, c[s] + 0 }
                print "" }'
```

Observe el + 0 del printf. Sin él, una base que no aparece en la secuencia no tiene entrada en el array y se imprimiría como cadena vacía en lugar de como cero. Es la diferencia entre una tabla que dice «no hay timinas» y otra que deja un hueco donde el lector no sabe si falta el dato o falta la base.

Se reproduce con `verification/chapter_results.sh bridge_counts`.

2.21.3 Instalar programas: dónde se trata

REFERENCIA Antes o después necesitará instalar una herramienta que no viene con el sistema, como BLAST. No se trata aquí porque el procedimiento depende de la distribución y del permiso que tenga en la máquina, y porque conviene verlo cuando la herramienta hace falta de verdad. El capítulo dedicado a la búsqueda por similitud lo aborda con BLAST como caso, incluidas las instalaciones sin privilegios de administrador y los entornos aislados. Si necesita adelantarlo, la guía de laboratorio de la asignatura tiene el procedimiento para los equipos del aula.

2.22 Lo que viene después: de procesar texto a razonar sobre su coste

A lo largo de este capítulo hemos dominado la construcción de tuberías, el uso de expresiones regulares y la automatización mediante scripts en Bash y awk. Hemos resuelto el problema de procesar miles de registros biológicos de manera reproducible y sin intervención manual.

Sin embargo, a medida que los conjuntos de datos en biomedicina crecen desde unos pocos megabytes hasta cientos de gigabytes, surge una pregunta crítica: ¿cuánto tiempo y memoria consume cada una de estas soluciones? ¿Por qué ordenar con `sort` antes de `uniq` es rápido para diez mil líneas pero lento para cien millones? Responder a estas preguntas requiere pasar de la mera automatización al **pensamiento algorítmico y al análisis de complejidad**, que es el núcleo fundamental que exploraremos en el capítulo TC-CH03 (Algoritmos y complejidad).

Este capítulo y TC-CH01 preparan explícitamente la transición a TC-CH03: una vez que se sabe automatizar y procesar texto, la pregunta abierta es cuánto cuesta cada construcción cuando el tamaño de los datos crece, que es el objeto de estudio de TC-CH03.

2.23 Método y errores frecuentes

Este capítulo recupera, sin volver a enseñarlo, el ciclo predecir/probar en pequeño/inspeccionar/contrastar/registrar introducido en TC-CH01; aquí se aplica a tuberías, patrones y scripts en vez de a órdenes sueltas.

Síntoma	Causa probable	Comprobación y corrección
grep -c da menos de lo esperado	cuenta líneas, no coincidencias	comparar con grep -o ... wc -l
uniq deja duplicados	no eran adyacentes	sort antes de uniq
[AG]+ no coincide	se usó BRE en vez de ERE	usar grep -E y declarar el dialecto
un bucle for no encuentra ficheros	no había coincidencias y falta nullglob	activar shopt -s nullglob y comprobar longitud del array
una tubería parece correcta pese a un fallo	solo se observó el estado del último eslabón	set -o pipefail y probar cada etapa por separado
sed no cambia todas las coincidencias	falta el indicador g	añadir g al final de s///
una función no «devuelve» el texto esperado	se confundió return (estado numérico) con capturar salida	usar \$(función) para texto, return solo para estado

Los síntomas de esta tabla corresponden a los errores de composición más frecuentes observados al automatizar procesamiento de texto con las herramientas de este capítulo.

2.24 Síntesis de referencia

Necesidad	Construcción principal	Pregunta de control
parametrizar	NOMBRE=valor, \$NOMBRE, \$(orden)	¿hay espacios alrededor del =?
citar con seguridad	comillas simples/dobles, \	¿quiero expansión o texto literal?
componer	, >, >>, 2>	¿qué recibe y produce cada etapa?
decidir	[[]], if/elif/else	¿comparo como texto o como número?
repetir	for, while, break/continue	¿sé de antemano cuántas veces?
reutilizar	funciones, return frente a \$(función)	¿necesito el estado o el texto?
buscar	grep -n -i -v -c -o -E -A -w -e -r -F	¿cuento líneas o coincidencias y con qué dialecto?
describir patrones	anclas, clases, cuantificadores, grupos	¿el caso negativo también falla como espero?
transformar	sed s///[g], direcciones, d/i/a/c, -i	¿sustituyo la primera coincidencia o todas?
procesar por registros	awk: NR/NF/FS, funciones, ~	¿qué separa un registro de un campo aquí?

La tabla de síntesis enlaza cada necesidad operativa del capítulo con su construcción principal y una pregunta de control verificable.

2.25 Glosario esencial

- **Metacaracter:** carácter con significado especial para la Shell antes de ejecutar la orden.
- **stdin/stdout/stderr:** entrada, salida y error estándar.
- **Tubería:** conexión del stdout de un proceso al stdin de otro.
- **Función:** bloque de órdenes con nombre, reutilizable, con argumentos propios.
- **Bucle:** estructura que repite órdenes mientras se cumple una condición o para una lista de elementos.
- **Expansión de llaves:** generación de texto literal a partir de { . . . , . . . } o {n . . m}, sin consultar el sistema de ficheros.

- **Glob:** patrón de Bash para nombres de fichero, interpretado por la Shell.
- **Regex (ERE):** lenguaje de patrones sobre el contenido de un texto, sintaxis extendida usada con `grep -E`.
- **Grupo de captura:** subpatrón entre paréntesis cuyo texto coincidente puede referenciarse después.
- **Cuantificador voraz (greedy):** cuantificador que coincide con la cadena más larga posible.
- **Aserción de ancho cero (lookaround):** comprobación de contexto que no consume caracteres en el resultado.
- **Stream editor (sed):** editor que procesa texto línea a línea sin modificar el origen salvo petición explícita.
- **Registro / Campo (awk):** fila y columna en el modelo de datos estructurados de awk.

2.26 Fuentes y límites de uso

Este capítulo se apoya en las presentaciones docentes de la asignatura sobre programación de la Shell y expresiones regulares, escritas por Álvaro Serrano Navarro. El comportamiento de las opciones de seguridad, las expansiones y los operadores de prueba está tomado del manual de referencia de Bash [16]; el de las utilidades de texto, de los manuales de GNU [18, 15, 19, 17]. La descripción del formato FASTA sigue la especificación del NCBI [23]. La organización de proyecto y el criterio de reproducibilidad siguen a Buffalo [5], y los patrones de scripting científico, a Allesina y Wilmes [1].

Los datos de `data/` son sintéticos y de elaboración propia, con licencia CC0 y manifiesto de sumas propio; están descritos en `data/README.md`. Todas las órdenes impresas se reproducen con `verification/` tal como están escritas.

Anexo práctico · Cinco programas sobre un corpus real

Pregunta, producto y separación de materiales

¿Puede convertir una exploración manual de veintidós ficheros en cinco programas seguros, explicables y comprobados, tres en Bash puro, uno que transforma con `sed` y otro que agrega con `awk`? Este laboratorio responde que sí sobre el corpus completo del capítulo.

El producto individual son los cinco guiones de `scripts/`, escritos por usted. Le acompañan la tabla de pruebas con su predicción y su resultado, las salidas guardadas en `resultados/`, un `README.md` con las órdenes y los supuestos, la defensa escrita y un paquete `.tar.gz` verificado.

El anexo es autónomo y usa el corpus de `data/corpus/`, veintidós ficheros sintéticos con licencia CC0, como entrada de solo lectura. Los valores esperados no se imprimen aquí: los produce usted y los contrasta con la herramienta que el generador deja en `herramienta/`. Las implementaciones de referencia están en la clave docente y no se distribuyen.

Mapa de ruta y productos entregables

Bloque	Producto
1 preparar y predecir	espacio creado y predicciones escritas
2 extremos de cada FASTA	<code>scripts/extremos.sh</code>
3 inventario de ficheros	<code>scripts/inventario.sh</code> y sus dos salidas
4 rama según el tamaño	<code>scripts/resumir.sh</code> con sus tres casos
5 normalizar cabeceras con <code>sed</code>	<code>scripts/normalizar.sh</code>
6 contar bases con <code>awk</code>	<code>scripts/contar_bases.sh</code>
7 empaquetar y verificar	<code>.tar.gz</code> y verificador en <code>ENTREGA_OK</code>

La ruta es única. Antes de ejecutar cada guion, escriba en `notas/pruebas.tsv` qué espera que salga. El contraste entre predicción y resultado es lo que se evalúa, no el resultado solo.

1 • Preparar el espacio de trabajo

```
./practice_assets/create_workspace.sh TC02_entrega
cd TC02_entrega
ls scripts/
```

El generador copia el corpus en `datos/corpus/`, deja `notas/pruebas.tsv` con la cabecera, resultados/vacía y cinco plantillas en `scripts/`. Las plantillas están vacías a propósito: llevan la línea del intérprete, las opciones de seguridad y un comentario con lo que debe hacer cada una. Si el directorio ya existe, el generador se niega a sobrescribirlo.

Antes de tocar nada, compruebe que el corpus llegó íntegro:

```
cd datos && sha256sum -c manifest.sha256 && cd ..
```

Comprueba. Prediga cuántos ficheros `.fasta` y cuántos `.txt` hay en el corpus antes de contarlos. Anótelos en `notas/pruebas.tsv`: será su primera fila.

2 • Extremos de cada FASTA

Tarea. Escriba `scripts/extremos.sh`, sin argumentos, que para cada fichero `.fasta` de `datos/corpus/` imprima su nombre y después la primera y la última línea de secuencia, sin contar la cabecera. Recorra los ficheros con un array y active `nullglob`, para que el caso sin coincidencias no le entregue el patrón literal.

El guion debe cumplir tres cosas: nombrar todos los ficheros `.fasta` que hay, emitir dos líneas de secuencia por cada uno y terminar con un mensaje de error, no en silencio, si el directorio no contiene ninguno.

3 • Inventario de ficheros

Tarea. Escriba `scripts/inventario.sh` que informe de cuántos ficheros `.txt` hay en `datos/corpus/`, escriba en `resultados/file_list.txt` la lista de los ficheros regulares del corpus, uno por línea y ordenada, y guarde en `resultados/summary.txt` las diez primeras líneas de esa lista.

Atención. No cuente analizando la salida de `ls`. Un nombre con espacio o un salto de línea rompen ese recuento, y el corpus tiene ficheros con espacios precisamente para que lo compruebe. Use `find` con `-type f`, que además distingue ficheros regulares de directorios y enlaces.

4 • Decidir según el tamaño del fichero

Tarea. Escriba `scripts/resumir.sh RUTA`, con un argumento, que muestre el fichero completo si tiene veinte líneas o menos, y que avise y muestre solo las diez primeras y las diez últimas si tiene más. Ante una ruta que no existe debe escribir un mensaje de error en la salida de error y terminar con estado distinto de cero, nunca con una salida vacía.

Pruebe las tres ramas, una por una, y anote las tres en su tabla:

```
bash -n scripts/resumir.sh
wc -l datos/corpus/*.fasta | sort -n | tail -n 4
```

Comprueba. La segunda orden le dice qué ficheros del corpus caen a cada lado del umbral. Elija uno de cada rama antes de probar, y prediga cuántas líneas imprimirá su guion en cada caso. La frontera exacta, veinte líneas, es donde se equivoican casi todos: compruebe si su condición usa `-le` o `-lt` y qué ocurre con un fichero de exactamente veinte.

5 • Normalizar cabeceras con sed

Tarea. Escriba `scripts/normalizar.sh` que recorra todos los `.fasta` de `datos/corpus/` y sustituya, con `sed`, el primer espacio de cada línea de cabecera por un guion bajo, dejando intactas las líneas de secuencia. El resultado concatenado va a la salida estándar, y los ficheros originales no se tocan.

Atención. Use `sed` con dirección, `sed '/^>/s/ /_/'`, no sobre todas las líneas. En este corpus las líneas de secuencia no tienen espacios, así que una sustitución sin dirección parecería funcionar; en un formato donde sí los hubiera, destrozaría los datos sin avisar. Declare siempre a qué líneas se aplica una sustitución, aunque en su fichero de prueba dé lo mismo.

6 • Contar bases con awk

Tarea. Escriba `scripts/contar_bases.sh` que, para cada `.fasta` del corpus, sume con `awk` la longitud de sus líneas de secuencia excluyendo las cabeceras, imprima el resultado por fichero y termine con el total acumulado.

Comprueba. Antes de fiarse del total, cuente a mano las bases de un solo fichero, el más corto, y compárelo con lo que dice su guion para ese fichero. Un total global no se puede comprobar de un vistazo; un fichero de seis líneas, sí. Si el testigo cuadra, el total es creíble; si no cuadra, el total está mal aunque parezca razonable.

7 • Comprobar, empaquetar y verificar

Cuando los cinco guiones estén escritos, compruébelos con la herramienta cuantas veces necesite. No le dará los valores esperados: le dirá si su resultado los cumple.

```
bash -n scripts/extremos.sh scripts/inventario.sh scripts/resumir.sh
bash herramienta/check_scripts.sh
```

Complete después `notas/pruebas.tsv` con las columnas de resultado, escriba `README.md` con las órdenes que ejecutó y sus supuestos, y `notas/defensa.md` con la defensa que se pide más abajo. Por último, vuelva al directorio del capítulo, empaquete y verifique:

```
cd ..
tar -czf TC02_entrega.tar.gz TC02_entrega
tar -tzf TC02_entrega.tar.gz | head -n 12
sha256sum TC02_entrega.tar.gz
./practice_assets/check_entrega.sh TC02_entrega TC02_entrega.tar.gz
```

El verificador comprueba que los cinco guiones cumplen lo pedido sobre el corpus, que el corpus no se ha modificado, que la tabla de pruebas tiene al menos cinco filas completas, que `resultados/` no está vacía, que el `README` y la defensa existen con la extensión mínima, y que el paquete se extrae con los guiones dentro. Si falta algo lo enumera y termina en `ENTREGA_INCOMPLETA`; si está todo, en `ENTREGA_OK`. No puntúa.

Rúbrica de evaluación

Sobre 10 puntos, tres de integridad técnica y siete de razonamiento.

- **3,0 puntos, entrega técnica íntegra:** los cinco guiones pasan `bash -n` y cumplen lo pedido, el corpus queda inalterado, el paquete se extrae y el verificador termina en `ENTREGA_OK`.
- **2,5 puntos, robustez y control de casos límite:** los guiones entrecomillan cada ruta, manejan el nombre con espacio, resuelven bien la frontera de las veinte líneas y terminan con estado distinto de cero ante una entrada ausente o un conjunto vacío.
- **1,5 puntos, transformación dirigida con sed:** la sustitución declara a qué líneas se aplica, no altera las de secuencia y no sobrescribe los originales.

- **1,5 puntos, agregación con awk y contraste:** las cabeceras quedan excluidas, el total es correcto y hay un testigo calculado a mano que lo respalda.
- **1,5 puntos, defensa:** explica el fallo explícito frente al silencioso con un ejemplo de la propia entrega.

Obtener ENTREGA_OK es requisito previo, no garantía de nota: un guion que acierta el número y no entrecomilla las rutas cumple el verificador y falla el segundo apartado.

Lista de autocorrección

- distingo *glob* de expresión regular y declaro `-E` cuando uso ERE;
- no analizo la salida de `ls` para contar ficheros;
- cada variable que representa una ruta va entrecomillada;
- compruebo el caso de cero entradas y el de fichero ausente;
- no escribo resultados encima de las entradas;
- `bash -n` pasa en los cinco guiones;
- cada sustitución de `sed` declara a qué líneas se aplica;
- probé las tres ramas de `resumir.sh`, incluida la frontera exacta;
- contrasté al menos un conteo con un testigo calculado a mano;
- el verificador termina en ENTREGA_OK.

Defensa

En `notas/defensa.md`, entre 150 y 200 palabras, explique por qué `resumir.sh` debe fallar de forma explícita ante un fichero inexistente en vez de no imprimir nada, y qué diferencia práctica supone eso para quien ejecute su guion dentro de una tubería sin leer el código. Apóyese en un caso concreto de su propia entrega.

Fuentes

El corpus de `data/corpus/` es sintético, de elaboración propia, con licencia CC0 y manifiesto de sumas propio; está descrito en `data/README.md`. Los patrones de scripting defensivo siguen el manual de Bash [16] y las recomendaciones de organización de proyecto de Buffalo [5].

Algoritmos, complejidad y estrategias de resolución

TC-CH03 – Técnicas Computacionales en Biología

Edición 2

Pregunta del tema. Un asistente automático escribe en segundos un procedimiento que compara cada lectura de secuenciación con cada posición del genoma. ¿Cómo se decide, antes de lanzarlo, si terminará esta tarde o dentro de nueve años?

Producto final. Una auditoría razonada de tres afirmaciones de coste erróneas, con la medición propia del escalado al duplicar la entrada y la comparación entre una recursión desbocada y su versión con tabla.

Mapa del tema

3.1. La receta, la despensa y la escala biológica	48
3.2. El coste de un algoritmo y el modelo de computación	49
3.3. Notación Asintótica: el lenguaje de la eficiencia	51
3.4. Las cinco grandes familias de diseño algorítmico	54
3.5. La operación peligrosa: la explosión combinatoria oculta	59
3.6. Actividad de depuración: localizar fallos en algoritmos ajenos	59
3.7. Síntesis y diagnóstico de problemas algorítmicos	60
3.8. Glosario esencial	61
3.9. Conexión con el curso y lo que viene después	61
3.10. Fuentes y límites de uso	62

Cómo usar este tema

Este capítulo no enseña a programar mejor: enseña a decidir, antes de ejecutar nada, si un procedimiento va a terminar. Se lee en tres tiempos. Primero se cuenta: cuántas operaciones hace un algoritmo sobre una entrada de tamaño n . Después se abstrae: qué parte de esa cuenta importa cuando n crece, que es de lo que habla la notación asintótica. Y por último se diseña: cinco familias de estrategias, cada una con su condición de aplicabilidad y su forma característica de fallar.

Al terminar sabrá: contar las operaciones elementales de un fragmento y expresar el resultado como una función de n ; distinguir una cota de un caso, que son cosas distintas y se confunden constantemente; predecir cómo cambia el tiempo al duplicar la entrada y comprobarlo midiendo; reconocer cuándo un algoritmo voraz da la respuesta correcta y cuándo no; y auditar una afirmación de coste ajena, venga de un compañero o de un asistente automático.

La ruta **esencial** son el modelo de coste, el conteo en los tres algoritmos, la notación asintótica con sus tres cotas y las cinco familias de diseño. La ruta de **estudio** añade la demostración del término dominante, la vuelta atrás y el anexo. Prediga siempre antes de medir: este capítulo trata precisamente de que la intuición sobre el coste es mala y hay que corregirla con cuentas.

3.1 La receta, la despensa y la escala biológica

ESENCIAL

En los dos primeros capítulos aprendimos a desenvolvernarnos en la terminal de Linux y a construir procedimientos automáticos con Bash, tuberías y expresiones regulares. Con esas herramientas podemos procesar archivos de datos, filtrar registros biológicos y encadenar programas. Sin embargo, en cuanto pasamos de pequeños ficheros de prueba a conjuntos de datos biomédicos a gran escala, surge de inmediato un obstáculo insalvable si solo aplicamos fuerza bruta: el **coste computacional**.

Consideremos un ejemplo real en genómica: el genoma humano contiene aproximadamente 3×10^9 pares de bases (3000 millones de nucleótidos). Si un secuenciador de última generación nos entrega 100 millones de lecturas cortas (FASTQ) y diseñamos un procedimiento ingenuo que compare cada lectura posición a posición contra todo el genoma desde el principio, el ordenador tendría que realizar del orden de $100 \times 10^6 \times 3 \times 10^9 = 3 \times 10^{17}$ operaciones. Incluso en un ordenador capaz de ejecutar mil millones de operaciones por segundo, que es un ritmo corriente para un procesador de sobremesa, ese cálculo ingenuo tardaría más de **9 años sin interrupción**.

El cálculo ilustra por qué la fuerza bruta es inviable a escala genómica. Las herramientas bioinformáticas reales no comparan cada lectura contra cada posición: construyen o reutilizan índices estructurados (como tablas de dispersión, árboles de sufijos o la transformada de Burrows-Wheeler, que estudiaremos en los siguientes capítulos) que descartan instantáneamente la inmensa mayoría del espacio de búsqueda. La diferencia entre un cálculo inviable que tardaría años y un análisis rápido no reside en comprar un ordenador más caro, sino en la **eficiencia matemática del algoritmo**.

Si una estructura de datos es la **despensa** —la forma en que se organizan y almacenan los ingredientes en la memoria—, un algoritmo es la **receta**: la secuencia ordenada de instrucciones que transforma los datos de entrada en el resultado biológico deseado. Este capítulo se centra en la receta apoyándose en lo que ya sabemos representar en Bash y pseudocódigo (TC-CH01 y TC-CH02). Las estructuras de datos avanzadas se formalizarán en los capítulos TC-CH04 y TC-CH05; aquí nos basta con saber que las entradas y salidas de un algoritmo son datos, independientemente de dónde y cómo se guarden.

Hoy en día, cualquier asistente de inteligencia artificial puede generar código en segundos, pero con frecuencia elige la estrategia equivocada: fuerza bruta donde existía una solución elegante, o un método voraz que parece resolver el problema pero no garantiza la respuesta óptima global. Reconocer el paradigma que tenemos delante, estimar su coste antes de lanzarlo y juzgar si escalará a nuestros datos es la competencia fundamental que proporciona este capítulo.

Un algoritmo es el procedimiento abstracto que resuelve un problema; un programa es su realización concreta en un lenguaje de programación. El mismo algoritmo puede escribirse como programas distintos en Bash o cualquier otro lenguaje de programación.

Un algoritmo es una secuencia finita de pasos bien definidos que, a partir de una entrada, produce una salida en un tiempo finito. «Finita» y «bien definidos» son las palabras importantes: un algoritmo no puede ser ambiguo ni eterno.

3.1.1 Niveles de representación de un algoritmo

Antes de lanzarse a teclear código en el ordenador, un algoritmo debe diseñarse y evaluarse conceptualmente. Para ello se utilizan cuatro niveles de representación, ordenados de menor a mayor rigor y precisión:

1. **Lenguaje natural:** describe la idea en lenguaje humano ordinario («recorrer la lista y guardar el número mayor»). Es muy intuitivo para comunicar la idea inicial, pero es propenso a ambigüedades e imprecisiones.
2. **Diagrama de flujo:** representación gráfica normalizada que utiliza cajas y flechas para ilustrar decisiones lógicas, bucles y flujos de datos.
3. **Pseudocódigo:** el punto dulce de la ingeniería. Es un lenguaje estructurado en texto, libre de las rigideces sintácticas de un lenguaje de programación concreto, pero con suficiente rigor lógico para analizar el coste y la corrección del algoritmo. Es la forma estándar en la que razonaremos los algoritmos a lo largo de este libro.
4. **Lenguaje de programación:** código ejecutable (Bash, C u otros lenguajes estructurados) que el sistema operativo puede ejecutar directamente.

Los cuatro niveles de representación de un algoritmo, de menor a mayor precisión, son: lenguaje natural, diagrama de flujo, pseudocódigo y lenguaje de programación.

Concepto. La célula ejecuta algoritmos, y conviene verlo antes de seguir. La replicación del ADN separa las dos hebras con la helicasa y copia cada una con la polimerasa: hay una entrada, la molécula original; una salida, dos moléculas idénticas; y una secuencia de pasos definidos que termina. Su equivalente informático es el problema de la copia: dada una cadena $s = (s_1, s_2, \dots, s_n)$, devolver una copia de s , con coste $O(n)$. Escribir un proceso biológico como entrada, salida y pasos es el primer ejercicio de pensamiento algorítmico, y es también la razón de que la biología se deje analizar así.

3.2 El coste de un algoritmo y el modelo de computación

Todo algoritmo, al ejecutarse en un ordenador real, consume dos **recursos fundamentales**:

- **Tiempo de ejecución:** medido en ciclos de procesamiento de la unidad central de procesamiento (CPU).
- **Espacio de memoria:** medido en la cantidad de memoria principal (RAM) adicional requerida para almacenar variables, tablas y estructuras auxiliares.

Un algoritmo consume dos recursos al ejecutarse: tiempo (ciclos de CPU) y espacio (memoria adicional que necesita).

3.2.1 ¿Por qué no medimos el coste en segundos?

Una tentación habitual en los primeros pasos computacionales es medir el tiempo que tarda un programa usando un cronómetro (con la orden `time`). Aunque la medición del tiempo de reloj (*wall-clock time*) es útil para comprobar el rendimiento práctico, **no sirve como medida teórica del algoritmo en sí**.

Si ejecutamos el mismo programa en un portátil antiguo, en un servidor de cómputo moderno, en una máquina con poca memoria o con el sistema operativo ocupado en otras tareas, el tiempo en segundos variará enormemente. Del mismo modo, el tiempo dependerá de si el código fue escrito en un lenguaje compilado de bajo nivel como C o en un lenguaje interpretado.

Para obtener una medida intrínseca y universal del algoritmo, la informática teórica utiliza el **conteo de operaciones elementales** en función del tamaño de los datos de entrada, habitualmente denotado por la variable n .

El coste de un algoritmo se mide contando operaciones elementales en función del tamaño de la entrada (n) en

vez de en segundos, porque el tiempo en segundos depende de la máquina, la carga del sistema y el lenguaje, no solo del algoritmo.

3.2.2 Definición del tamaño de entrada en problemas multivariados

En los ejemplos introductorios solemos resumir el tamaño de la entrada en una única variable n (como el número de elementos de una lista). Sin embargo, en biología computacional la entrada es habitualmente **multivariable**:

- En análisis de secuencias: M lecturas de longitud L cada una (la entrada total tiene tamaño $M \times L$).
- En alineamiento por pares: dos secuencias de longitudes n y m (la matriz de programación dinámica tiene orden $O(n \cdot m)$).
- En redes y vías metabólicas: un grafo con $|V|$ nodos (genes/metabolitos) y $|E|$ interacciones (aristas).

Antes de calcular cualquier complejidad, la primera pregunta obligatoria del científico es: *¿qué parámetros describen cuantitativamente el tamaño de los datos de entrada?*

3.2.3 El modelo RAM de operaciones elementales

Esta cuantificación se apoya en el modelo clásico de la máquina de acceso aleatorio (*Random Access Machine* o modelo RAM):

- La **memoria** se modela como un vector continuo de celdas con dirección propia donde leer o escribir datos toma tiempo constante ($O(1)$).
- La **CPU** ejecuta instrucciones elementales (sumas, restas, multiplicaciones, comparaciones lógicas, asignaciones y saltos de control) sobre palabras de tamaño fijo. Cada una de estas operaciones elementales se contabiliza con un coste unitario constante.

En el modelo RAM, la memoria es una sucesión de direcciones y la CPU ejecuta operaciones aritméticas, lógicas, de entrada/salida y de control sobre palabras de tamaño fijo; cada una de esas operaciones se cuenta como de coste constante.

3.2.4 Conteo de operaciones en tres algoritmos de ejemplo

Para comprender de forma práctica cómo se cuentan las operaciones elementales, analicemos tres algoritmos en Bash que resuelven exactamente la misma tarea: mostrar los N primeros múltiplos de 3. Los tres dan la misma salida. Lo que los diferencia es cuánto trabajo cuesta producirla.

```
# Algoritmo A: un bucle simple directo
for ((i = 1; i <= N; i++)); do
    echo $((i * 3))
done

# Algoritmo B: bucles anidados con sumas acumuladas
for ((i = 1; i <= N; i++)); do
    sum=0
    for ((j = 1; j <= i; j++)); do
        sum=$((sum + 3))
    done
    echo "$sum"
done

# Algoritmo C: un solo bucle con acumulador
sum=0
for ((i = 1; i <= N; i++)); do
    sum=$((sum + 3))
    echo "$sum"
done
```

Desglose del Algoritmo A (Lineal):

1. Inicialización: $i=1$ (1 operación).

2. En cada una de las N iteraciones:

- Comparación de control: $i \leq N$ (1 op).
- Multiplicación aritmética: $i * 3$ (1 op).
- Emisión por salida estándar: `echo` (1 op).
- Incremento y asignación: $i++$ (2 ops).

Total por vuelta: 5 operaciones.

El número total de operaciones elementales es $T_A(N) = 5N + 1$: cinco por vuelta y una de inicialización. Para $N = 10$ son 51, que es exactamente lo que cuenta el oráculo. El trabajo crece de forma **lineal** con N : al multiplicar N por 10, el tiempo se multiplica por 10.

El algoritmo A hace una operación de comparación, una de multiplicación y una de impresión por cada una de las N vueltas del bucle, más el establecimiento inicial de la variable: $5N + 1$ operaciones en total, que es $O(N)$.

Se reproduce con `verification/chapter_results.sh classify_single 10`.

Desglose del Algoritmo B (Cuadrático):

En el Algoritmo B, el bucle interior se ejecuta 1 vez en la primera iteración, 2 veces en la segunda, 3 en la tercera, y así sucesivamente hasta N veces en la última.

El número total de ejecuciones del cuerpo interior viene dado por la célebre suma de Gauss:

$$\sum_{i=1}^N i = 1 + 2 + 3 + \dots + N = \frac{N(N+1)}{2} = \frac{1}{2}N^2 + \frac{1}{2}N$$

El coste total de este algoritmo incluye un término en N^2 . A diferencia del Algoritmo A, si multiplicamos N por 10 (de $N = 100$ a $N = 1000$), el tiempo de ejecución se multiplica aproximadamente por **100**.

El algoritmo B ejecuta su bucle interior i veces en cada una de las N vueltas del bucle exterior: el total de vueltas del bucle interior es $1 + 2 + \dots + N = N(N+1)/2$, del orden de N^2 , $O(N^2)$.

Se reproduce con `verification/chapter_results.sh classify_nested 10`.

Desglose del Algoritmo C (lineal, con otra constante).

Dos operaciones de inicialización, `sum=0` e `i=1`, y seis por vuelta: comparación, suma, asignación de `sum`, emisión, incremento y asignación de `i`. En total $T_C(N) = 6N + 2$.

Concepto. Aquí está el punto del apartado. Los algoritmos A y C hacen trabajos distintos, cuentan operaciones distintas y tienen constantes distintas, $5N + 1$ frente a $6N + 2$, pero los dos pertenecen a $O(N)$: al duplicar N , los dos duplican su trabajo. El algoritmo B, en cambio, cambia de clase. Eso es exactamente lo que la notación asintótica conserva y lo que descarta. Distingue A y C de B, y no distingue A de C. Cuando alguien dice que dos algoritmos «son igual de eficientes» porque comparten notación, está diciendo algo más débil de lo que parece: para N pequeño, la constante decide, y la constante es justamente lo que la notación tira.

Comprueba. Prediga cuántas vueltas dará el bucle interior del Algoritmo B con $N = 10$ antes de ejecutarlo. La suma de Gauss da $\frac{10 \times 11}{2} = 55$. Compruébelo, y compruebe también los otros dos: con el mismo N , el Algoritmo A hace 51 operaciones y el C hace 62. Los tres números son del mismo orden de magnitud, y sin embargo uno de los tres crece de forma distinta. ¿Cuál, y cómo lo sabe sin volver a medir?

3.3 Notación Asintótica: el lenguaje de la eficiencia

ESENCIAL

Al analizar algoritmos, los detalles menores de implementación (como si un bucle hace $5N + 2$ o $3N + 7$ operaciones) carecen de importancia cuando los datos crecen hacia millones de elementos. Lo que determina si un pipeline bioinformático es viable o inviable es su **tasa de crecimiento asintótico**.

La **notación Big-O** (O) es el estándar matemático universal para describir cómo escala el tiempo o la memoria de un algoritmo cuando el tamaño de entrada $n \rightarrow \infty$.

3.3.1 Las tres reglas de simplificación de la notación Big-O

1. **Conservar el término dominante:** si una función de coste tiene múltiples términos ($f(n) = 3n^2 + 500n + 10\,000$), para valores grandes de n el término cuadrático n^2 eclipsa totalmente a los demás.
2. **Descartar constantes multiplicativas:** $5n$ y $100n$ pertenecen ambos a la clase $O(n)$, ya que las constantes dependen de la velocidad de la CPU o del compilador, no del comportamiento intrínseco del algoritmo.
3. **Describir el comportamiento asintótico:** evaluar el límite cuando n crece sin cota.

La notación O sigue tres reglas: conservar el término dominante, descartar las constantes multiplicativas y describir el comportamiento asintótico (la tendencia cuando n crece sin límite).

3.3.2 Escala de complejidades comunes

Notación	Nombre	Efecto al duplicar n	Ejemplo biológico
$O(1)$	Constante	No cambia	Acceso a base por índice
$O(\log n)$	Logarítmica	Suma una operación constante	Búsqueda binaria en tabla
$O(n)$	Lineal	Se duplica ($2\times$)	Conteo de GC en secuencia
$O(n \log n)$	Cuasilineal	Algo más del doble ($2.2\times$)	Ordenación con Mergesort
$O(n^2)$	Cuadrática	Se cuadruplica ($4\times$)	Comparación de pares (todas con todas)
$O(2^n)$	Exponencial	Se eleva al cuadrado	Ensamblaje ingenuo sin podar
$O(n!)$	Factorial	Explosión combinatoria	Alineamiento múltiple por fuerza bruta

Con $n = 1\,000\,000$, un algoritmo $O(n)$ hace un millón de pasos, uno $O(n^2)$ hace un billón, y uno $O(2^n)$ es directamente irrealizable.

El punto de cruce: constante oculta frente a tasa de crecimiento.

Un error conceptual habitual es suponer que un algoritmo $O(n)$ es siempre más rápido que uno $O(n^2)$ para cualquier tamaño de entrada n . La notación asintótica describe la tendencia cuando $n \rightarrow \infty$, no el tiempo absoluto sobre entradas pequeñas.

Consideremos dos algoritmos con tiempos de trabajo abstractos $T_A(n) = 50n$ (lineal con constante alta) y $T_B(n) = n^2$ (cuadrático con constante baja):

- Para $n = 20$: $T_A(20) = 50 \times 20 = 1000$, mientras que $T_B(20) = 20^2 = 400$. Para entradas pequeñas, el algoritmo cuadrático realiza menos trabajo.
- Para $n = 50$: $T_A(50) = 2500$ y $T_B(50) = 2500$ (punto de cruce).
- Para $n = 1000$: $T_A(1000) = 50\,000$, mientras que $T_B(1000) = 1\,000\,000$. A partir del punto de cruce, la tasa cuadrática domina y su desventaja crece sin límite.

Por esta razón, las librerías científicas optimizadas combinan con frecuencia algoritmos cuadráticos sencillos para casos diminutos y conmutan a algoritmos asintóticamente superiores cuando los datos crecen (estrategia híbrida que estudiaremos en TC-CH06).

Comprueba. Ordene de menor a mayor coste, para n grande, estas ocho funciones: n , $n/2$, $\log n$, $n \log n$, n^2 , 2^n , n^3 y $n!$. Dos de ellas pertenecen a la misma clase asintótica. ¿Cuáles, y por qué? Si duda, calcule el cociente entre las dos cuando n tiende a infinito: si sale una constante, están en la misma clase.

3.3.3 Cotas superior (O), inferior (Ω) y ajustada (Θ)

En el análisis de algoritmos formal se distinguen tres cotas asintóticas fundamentales:

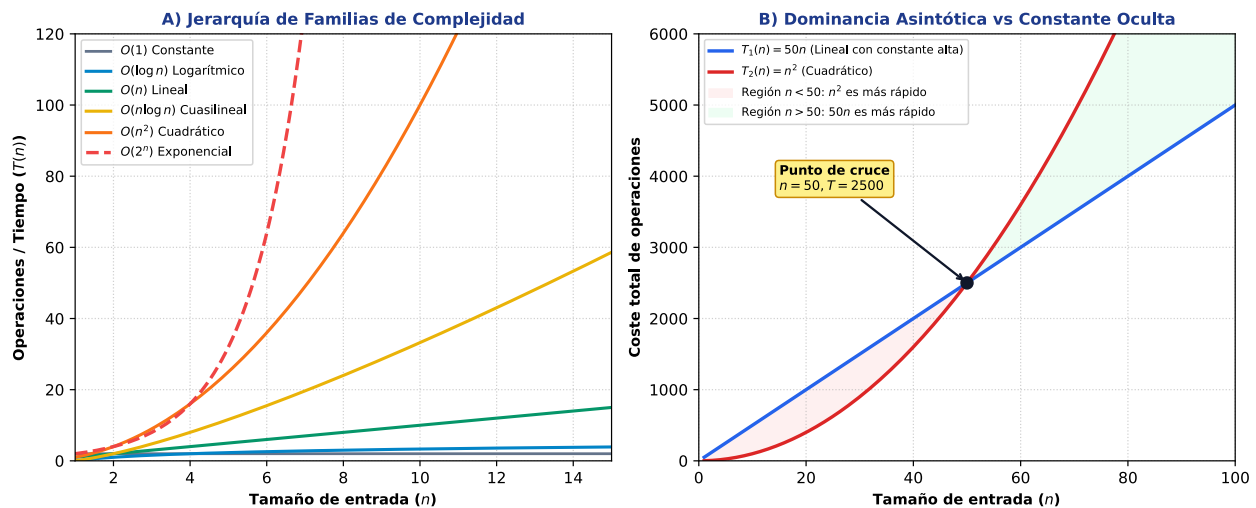


Figura 3.1: A) Jerarquía de crecimiento de las principales familias asintóticas. B) Detalle analítico del punto de cruce en $n = 50$ entre $T_1(n) = 50n$ y $T_2(n) = n^2$, ilustrando por qué la constante oculta domina para entradas pequeñas ($n < 50$) pero el orden de complejidad domina para problemas a escala genómica ($n \gg 50$). Figura original adaptada de las presentaciones docentes.

- **Cota superior $O(g(n))$ (Big-O):** garantiza que el algoritmo **no tardará más** que un múltiplo constante de $g(n)$ para entradas suficientemente grandes. Es la cota más utilizada en la práctica porque ofrece una garantía de peor caso.
- **Cota inferior $\Omega(g(n))$ (Big-Omega):** garantiza que el algoritmo tardará **al menos** un múltiplo de $g(n)$.
- **Cota ajustada $\Theta(g(n))$ (Big-Theta):** indica que el algoritmo tarda **exactamente** tanto como $g(n)$, estando acotado superior e inferiormente por la misma clase asintótica.

Cota asintótica no es sinónimo de caso analizado.

La notación $O(g(n))$ expresa una cota superior matemática sobre la función que se esté evaluando. Esa función puede ser el coste en el peor caso, en el caso promedio o en el mejor caso, según se declare explícitamente.

En ingeniería computacional se analiza prioritariamente el peor caso porque proporciona una garantía sobre el tiempo máximo, pero la notación Big-O y el «peor caso» son conceptos independientes: primero se fija el escenario o caso a estudiar (Sección 3.3.4) y posteriormente se acota asintóticamente su función de coste con O , Ω o Θ .

$O(g(n))$ representa una cota superior asintótica (la función no crece más rápido que $g(n)$ salvo constante); $\Omega(g(n))$ representa una cota inferior (crece al menos como $g(n)$); y $\Theta(g(n))$ representa una cota ajustada exacta. La notación de cota es independiente de si se analiza el peor caso, el caso promedio o el mejor caso.

3.3.4 Peor caso, mejor caso y caso promedio

El rendimiento de un algoritmo puede variar no solo según el tamaño n de la entrada, sino según la **disposición de los datos**:

- **Mejor caso:** la configuración más favorable de los datos (por ejemplo, buscar un elemento y que esté en la primera posición: $O(1)$).
- **Peor caso:** la configuración más desfavorable (el elemento buscado está al final o no existe: $O(n)$). Es la métrica estándar de garantía de ingeniería.
- **Caso promedio:** el comportamiento esperado sobre entradas aleatorias o realistas.

Se distinguen peor caso, caso medio y mejor caso de un algoritmo; el análisis de coste se aplica tanto al tiempo como al espacio adicional que necesita.

3.3.5 Clasificación de patrones elementales

Un bucle simple sobre una entrada de tamaño n es $O(n)$; un bucle anidado sobre la misma entrada es $O(n^2)$; un bucle que reduce n a la mitad en cada paso hasta llegar a 1 hace del orden de $\log_2 n$ pasos, $O(\log n)$.

Se reproduce con `verification/chapter_results.sh classify_single 10.`
`, classify_nested` y `classify_halving`.

3.3.6 Demostración matemática del término dominante

ESTUDIO

Una duda frecuente al aprender notación asintótica es qué significa exactamente que el término $3n^2$ sea «dominante» en la expresión $f(n) = 3n^2 + 500n + 200$.

Analicemos la proporción que representa el término cuadrático $3n^2$ sobre el total:

- Para $n = 10$: $3(10^2) = 300$, frente a un total de $300 + 5000 + 200 = 5500$ (representa solo el 5.4%).
- Para $n = 1000$: $3(1000^2) = 3\,000\,000$, frente a un total de $3\,500\,200$ (representa el 85.7%).
- Para $n = 100\,000$: $3(100\,000^2) = 30\,000\,000\,000$, frente a $30\,050\,000\,200$ (representa el 99.83%).

A medida que n crece, el término de mayor exponente explica casi el 100% del coste computacional. Por ello, la regla formal descarta los términos de orden inferior ($500n + 200$) y posteriormente la constante multiplicativa (3), quedando formalmente como $O(n^2)$.

Para n suficientemente grande, el término dominante completo, $3n^2$ (con su constante), representa la práctica totalidad del valor de $3n^2 + 500n + 200$ —85,7 % en $n = 1000$, 99,8 % en $n = 100\,000$ —; la notación O correcta es $O(n^2)$, no « $O(3n^2 + 500n)$ », porque una vez identificado el término dominante completo se descartan tanto los términos de orden inferior como su constante multiplicativa.

Se reproduce con `verification/chapter_results.sh dominant_term_share 1000.`

3.4 Las cinco grandes familias de diseño algorítmico

ESENCIAL

Ante un problema computacional nuevo en bioinformática, los científicos no reinventan la rueda desde cero: aplican uno de los cinco grandes **paradigmas de diseño algorítmico**.

3.4.1 1. Fuerza bruta y búsqueda exhaustiva

La estrategia de **fuerza bruta** genera y comprueba sistemáticamente todas las soluciones posibles del espacio de búsqueda hasta encontrar la correcta o la óptima. Garantiza siempre encontrar la respuesta correcta si existe, pero su coste suele escalar de forma exponencial ($O(2^n)$) o factorial ($O(n!)$), volviéndose impracticable para entradas reales.

La fuerza bruta explora todas las soluciones posibles de un problema y garantiza encontrar la óptima, a costa de un coste típicamente $O(2^n)$ o $O(n!)$ que la hace inviable salvo para entradas pequeñas.

Caso resuelto: Búsqueda de sitios de corte de restricción.

Consideremos la tarea de localizar todas las dianas de corte de la enzima de restricción *EcoRI* (motivo GAATTC, de longitud $k = 6$) en una secuencia de ADN de longitud n . La fuerza bruta compara el motivo contra todas las $n - k + 1$ posiciones posibles de la secuencia.

```
funcion mapaDeRestriccion(secuencia, patron):
  posiciones <- []
  n <- longitud(secuencia)
  k <- longitud(patron)
  para i desde 0 hasta n-k:           # las n-k+1 posiciones posibles
    si secuencia[i .. i+k-1] == patron:
```

```

    agregar i a posiciones
devolver posiciones          # ninguna posicion se descarta a priori

```

Buscar por fuerza bruta las apariciones de una subcadena de longitud k en una secuencia de longitud n compara la subcadena contra las $n - k + 1$ posiciones posibles: $O(n \cdot k)$ en el peor caso, una por comparación de hasta k caracteres en cada posición.

Se reproduce con `verification/chapter_results.sh restriction_sites GAATTCGAATTC GAATTC`.

3.4.2 2. Algoritmos voraces (*Greedy*)

Un algoritmo **voraz** construye la solución paso a paso, tomando en cada etapa la decisión que parece **óptima localmente**, con la esperanza de alcanzar el óptimo global. Los algoritmos voraces son extraordinariamente rápidos (típicamente $O(n)$ o $O(n \log n)$), pero tienen una limitación crítica: **no siempre garantizan el óptimo global**.

Un algoritmo voraz toma en cada paso la decisión que parece mejor localmente; solo es correcto si se demuestra que las decisiones locales óptimas construyen la solución global óptima. «Parece óptimo» no es una demostración.

```

funcion cambioVoraz(objetivo, monedas_descendente):
    resultado <- []
    restante <- objetivo
    para cada moneda en monedas_descendente:    # de mayor a menor valor
        mientras moneda <= restante:
            agregar moneda a resultado
            restante <- restante - moneda      # decision local, nunca se revisa
    devolver resultado

```

El contraejemplo del cambio de monedas.

Supongamos que debemos devolver un cambio de **6 unidades** disponiendo de monedas con valores $\{1, 3, 4\}$.

- **Estrategia voraz:** Toma la moneda más grande posible (4), quedando 2 por cubrir. No cabe otra de 4 ni de 3, por lo que toma una de 1 (queda 1) y otra de 1 (queda 0). Resultado: **3 monedas** ($4 + 1 + 1$).
- **Solución óptima real:** Tomar dos monedas de 3. Resultado: **2 monedas** ($3 + 3$).

El algoritmo voraz falló porque la elección local «tomar la moneda de 4» bloqueó la combinación global óptima ($3 + 3$).

Con monedas de valor $\{1, 3, 4\}$ y un cambio de 6, el método voraz (tomar siempre la moneda más grande que quepa) usa 3 monedas ($4 + 1 + 1$), mientras que la solución óptima usa 2 ($3 + 3$): el voraz no alcanza el óptimo en este caso.

Se reproduce con `verification/chapter_results.sh greedy_change_1_3_4 6`.

```

ALGORITMO: Seleccion_Voraz_Fragmentos_Restriccion
ENTRADA:
- F: Lista de n fragmentos genomicos, donde cada fragmento i tiene (inicio_i, fin_i) con inicio_i < fin_i
SALIDA:
- S: Subconjunto disjunto maximo de fragmentos no solapantes
COMPLEJIDAD: Tiempo  $O(n \log n)$  por la ordenacion inicial, Espacio auxiliar  $O(n)$ 

1. INICIALIZACION:
# Criterio voraz optimo: ordenar los fragmentos por tiempo/posicion de finalizacion ascendente
F_ordenado <- Ordenar(F, clave=fin ascendente)
S <- Conjunto_Vacio()
ultimo_fin <- -INFINITO

2. BUCLE VORAZ:
Para cada fragmento f = (inicio, fin) en F_ordenado:
    Si f.inicio >= ultimo_fin entonces:

```

```
# El fragmento es compatible con los seleccionados previamente
Agregar f a S
ultimo_fin <- f.fin    # Decision irrevocable localmente optima

3. TERMINACION Y RETORNO:
Retornar S (Se demuestra que |S| alcanza el maximo teorico global)
```

Problema general frente a sistema de monedas.

Con las denominaciones del euro (1, 2, 5, 10, 20 y 50 céntimos, 1 y 2 euros) el método voraz produce siempre el óptimo, porque ese sistema es canónico. Devolver 77 céntimos, por ejemplo, sale bien tomando siempre la moneda más grande que quepa: 50, 20, 5 y 2, cuatro monedas, y no hay forma de hacerlo con menos. Pruebe usted mismo unos cuantos importes; no encontrará ninguno que falle.

Y sin embargo el mismo método falla con las denominaciones {1, 3, 4}, como veremos enseguida. La conclusión que importa es precisa: la corrección de un algoritmo voraz depende del **sistema de denominaciones** con el que trabaja. Es una propiedad del conjunto de monedas, no del importe concreto que haya que devolver. Por eso no basta con probar el algoritmo sobre unos cuantos casos: hay que demostrar que el sistema tiene la propiedad, o buscar el contraejemplo.

Un algoritmo se define para un problema general; una instancia es el caso concreto sobre el que opera (por ejemplo, qué denominaciones de moneda hay disponibles). El mismo algoritmo voraz de cambio de moneda es óptimo para las monedas del sistema euro pero no para {1, 3, 4}: la corrección de un voraz es una propiedad del sistema de denominaciones, no del importe concreto.

3.4.3 3. Divide y vencerás y recursión

El paradigma de **divide y vencerás** descompone un problema en subproblemas más pequeños del mismo tipo, resuelve los subproblemas de forma independiente mediante llamadas recursivas y finalmente **combina** sus soluciones para resolver el problema original.

Una solución recursiva resuelve un problema reduciéndolo a versiones más pequeñas de sí mismo; divide y vencerás parte el problema en subproblemas independientes, los resuelve recursivamente y combina sus resultados.

Anatomía de la recursión:

Toda función recursiva correcta debe poseer dos elementos indispensables:

1. **Caso base:** la condición de parada simple que devuelve un resultado directo sin realizar llamadas adicionales.
2. **Llamada recursiva:** la invocación a sí misma que reduce el tamaño del problema acercándose estrictamente al caso base.

```
fib(n):
  si n <= 1: devolver n
  devolver fib(n-1) + fib(n-2)
```

En el árbol de llamadas de `fib(5)` sin memoizar, `fib(2)` se calcula 3 veces de forma independiente; el número total de llamadas crece de forma exponencial, $O(2^n)$.

Se reproduce con `verification/chapter_results.sh fib_naive_calls 5`.

Peligro de desbordamiento de pila (*Stack Overflow*):

Cada llamada a una función consume memoria en la pila del sistema (*stack*) para almacenar variables locales y la dirección de retorno. Si falta el caso base, la recursión entrará en un bucle infinito que agotará la memoria de la pila, provocando el colapso inmediato del programa.

Toda solución recursiva requiere un caso base (la condición simple que detiene la recursión sin más llamadas) y

```

fib(4)
+-- fib(3)
|   +-- fib(2)
|   |   +-- fib(1)
|   |   |-- fib(0)
|   |-- fib(1)
|-- fib(2)      <- recalculado desde cero
   +-- fib(1)
   |-- fib(0)

```

Figura 3.2: Árbol de llamadas de `fib(4)`: `fib(2)` se calcula dos veces de forma independiente, cada vez repitiendo su propio subárbol completo. Figura original.

una llamada recursiva que reduzca el problema hacia él; sin un caso base alcanzable, cada llamada consume memoria de pila adicional sin límite hasta agotarla (desbordamiento de pila).

La iteración usa menos memoria que la recursión porque no acumula llamadas en la pila; la recursión es más natural para problemas de estructura repetitiva o divisible, como recorrer árboles y grafos o aplicar divide y vencerás, a costa de esa memoria adicional por cada llamada abierta.

3.4.4 4. Programación dinámica

La **programación dinámica** es una de las técnicas más potentes de la computación científica y la base de los algoritmos de alineamiento de secuencias biológicas (como Needleman-Wunsch y Smith-Waterman). Se aplica cuando un problema cumple dos propiedades:

1. **Subestructura óptima:** la solución óptima del problema global se construye a partir de las soluciones óptimas de sus subproblemas.
2. **Subproblemas solapados:** los mismos subproblemas aparecen y se necesitan una y otra vez durante el cálculo.

La idea maestra es la **memoización**: calcular la solución de cada subproblema exactamente una sola vez y guardarla en una tabla o matriz para reutilizarla de inmediato en tiempo constante ($O(1)$).

La programación dinámica se aplica cuando un problema tiene subestructura óptima y subproblemas solapados; memoizar (guardar el resultado de cada subproblema la primera vez que se calcula) evita recalcularlo, convirtiendo una recursión exponencial en un algoritmo eficiente.

Memoizando `fib`, cada valor de 0 a n se calcula exactamente una vez: el número de cálculos pasa de exponencial a $O(n)$.

Se reproduce con `verification/chapter_results.sh fib_memo_calls 10`.

```

ALGORITMO: Fibonacci_Comparativa_Complejidad
ENTRADA:
- n: Entero no negativo representando la posicion en la serie de Fibonacci
SALIDA:
- F_n: El n-esimo numero de Fibonacci

# VARIANTE 1: INGENUA RECURSIVA (inviabile: tiempo  $O(2^n)$ , espacio de pila  $O(n)$ )
Funcion Fibonacci_Ingenuo(n):
  Si n <= 0 entonces retornar 0
  Si n = 1 entonces retornar 1
  Retornar Fibonacci_Ingenuo(n - 1) + Fibonacci_Ingenuo(n - 2)

# VARIANTE 2: PROGRAMACION DINAMICA MEMOIZADA (Optima: Tiempo  $O(n)$ , Espacio  $O(1)$ )
ALGORITMO: Fibonacci_DP_Iterativo_Optimo(n):
  Si n <= 0 entonces retornar 0
  Si n = 1 entonces retornar 1

  prev2 <- 0    # F_{i-2}
  prev1 <- 1    # F_{i-1}

```

```

Para i desde 2 hasta n:
    actual <- prev1 + prev2    # F_i = F_{i-1} + F_{i-2}
    prev2 <- prev1
    prev1 <- actual

Retornar actual    # Calculado en exactamente (n-1) sumas elementales

```

Resolución del cambio de monedas con programación dinámica:

Definiendo $dp[c]$ como el número mínimo de monedas para formar el importe c , con $dp[0] = 0$:

$$dp[c] = 1 + \min_{\substack{m \in \{1,3,4\} \\ m \leq c}} dp[c - m]$$

```

funcion monedasMinimasDP(objetivo, monedas):
    dp[0] <- 0
    para c desde 1 hasta objetivo:
        dp[c] <- infinito
        para cada m en monedas:
            si m <= c:
                dp[c] <- min(dp[c], dp[c - m] + 1)
    devolver dp[objetivo]

```

El cambio de monedas con $\{1, 3, 4\}$ tiene subestructura óptima y subproblemas solapados (por ejemplo, $dp[3]$ se necesita para calcular $dp[4]$ y $dp[6]$); se resuelve con la relación de recurrencia $dp[c] = 1 + \min_{m \leq c} dp[c - m]$ con $dp[0] = 0$.

Sobre monedas $\{1, 3, 4\}$ y objetivo 6, la tabla de programación dinámica da $dp[0..6] = 0, 1, 2, 1, 1, 2$: el importe 6 se resuelve con 2 monedas ($dp[6] = 2$, correspondiente a $3 + 3$), el óptimo real, no las 3 monedas ($4 + 1 + 1$) que dio el algoritmo voraz sobre la misma instancia. La programación dinámica considera todas las combinaciones posibles a través de la tabla, sin descartar ninguna opción localmente antes de tiempo.

Se reproduce con `verification/chapter_results.sh coin_change_dp 6`.

Memoizar solo aporta cuando existen subproblemas solapados; en un algoritmo cuyos subproblemas son independientes y no se repiten (como la ordenación por mezcla), memoizar no ahorra cómputo y solo añade sobrecarga de memoria.

3.4.5 5. Vuelta atrás (*Backtracking*)

El paradigma de **vuelta atrás** (*backtracking*) es una optimización de la fuerza bruta para problemas de búsqueda combinatoria. En lugar de generar todas las combinaciones posibles y filtrarlas al final, el algoritmo construye soluciones parciales candidatas y **abandona inmediatamente** (*poda*) una rama en cuanto comprueba que no puede satisfacer las restricciones del problema.

```

backtracking(solucion_parcial, datos_restantes):
    si solucion_parcial es completa:
        si cumple los criterios: guardarla
        devolver
    para cada opcion posible en datos_restantes:
        agregar opcion a solucion_parcial
        backtracking(solucion_parcial, datos_restantes sin opcion)
        quitar opcion de solucion_parcial    # deshacer: el paso que no tiene la fuerza bruta

```

Backtracking explora el espacio de soluciones abandonando una rama en cuanto se comprueba que no puede llevar a una solución válida, a diferencia de la fuerza bruta, que completa cada combinación antes de descartarla; su implementación típica se apoya en una pila que guarda el camino parcial.

Caso resuelto: Generación de motivos de ADN sin pares AA.

Supongamos que deseamos generar todas las secuencias de longitud $L = 4$ sobre el alfabeto $\{A, T\}$ con la restricción biológica de que nunca aparezcan dos adeninas consecutivas (AA).

- **Fuerza bruta sin podar:** Exploraría un árbol completo de $2^0 + 2^1 + 2^2 + 2^3 + 2^4 = 31$ nodos para generar 16 secuencias y descartar las inválidas.
- **Backtracking con poda:** En cuanto añade una A y la siguiente candidata es otra A, aborta la rama inmediatamente. Solo visita **19 nodos** y devuelve directamente las **8 cadenas válidas**.

Para generar cadenas de longitud 4 sobre $\{A, T\}$ sin dos «A» consecutivas, backtracking visita 19 nodos del árbol de búsqueda, frente a los 31 nodos que tendría el árbol completo sin podar ($2^0 + 2^1 + \dots + 2^4$); encuentra exactamente 8 cadenas válidas.

Se reproduce con `verification/chapter_results.sh backtracking_count 4`.

La predicción de estructura secundaria de ARN por emparejamiento de bases y el alineamiento simultáneo de tres o más secuencias son dos problemas que se pueden plantear como backtracking con poda; ambos son desarrollos de capítulos posteriores y no se implementan aquí.

3.5 La operación peligrosa: la explosión combinatoria oculta

ESTUDIO

En bioinformática, una de las trampas más devastadoras ocurre cuando un algoritmo con complejidad exponencial $O(2^n)$ se programa y prueba con entradas de juguete ($n = 10$) durante la fase de desarrollo. El algoritmo parece funcionar de forma instantánea ($2^{10} = 1024$ operaciones). Sin embargo, al aplicarlo al conjunto de datos real de laboratorio con $n = 50$, el número de operaciones salta a $2^{50} \approx 1.12 \times 10^{15}$, provocando el bloqueo total del servidor, saturación de la memoria RAM y pérdida de tiempo de computación.

Protocolo preventivo en 3 pasos ante la escala de datos:

1. **Análisis asintótico previo:** deducir la cota Big-O formal del algoritmo antes de escribir la primera línea de código.
2. **Prueba de escalado con duplicación (2×):** ejecutar el algoritmo con entradas de tamaño creciente ($n = 10, 20, 30, 40$) midiendo el tiempo de ejecución. Si al duplicar n el tiempo se multiplica por 4, es cuadrático ($O(n^2)$); si se eleva exponencialmente, es $O(2^n)$.
3. **Validación de límites y guardia de tamaño:** establecer una comprobación preventiva en el código que rechace o alerte ante tamaños de entrada que excedan la capacidad computacional disponible.

3.6 Actividad de depuración: localizar fallos en algoritmos ajenos

ESTUDIO

Diagnosticar errores en algoritmos diseñados por otros o por herramientas de IA es una habilidad crítica. Analice el siguiente fragmento con tres errores conceptuales:

```
funcion buscarRepetidos(lista, n):
    para i desde 0 hasta n:          # Fallo 1
        para j desde 0 hasta n:      # Fallo 2
            si lista[i] == lista[j]: # Fallo 3
                devolver VERDADERO
    devolver FALSO
```

Tarea. Localice los tres fallos, corrija cada uno y clasifíquelo antes de leer el diagnóstico: ¿es un error de corrección, que hace que la función devuelva una respuesta equivocada, o una ineficiencia, que hace que tarde más de lo necesario? La distinción no es académica: uno invalida el resultado y el otro solo cuesta tiempo.

Diagnóstico.

1. **Desbordamiento de índice** (corrección). El bucle llega hasta n en lugar de $n - 1$ y accede a una posición que no existe. Según el lenguaje, esto aborta el programa o lee memoria ajena, que es peor porque no avisa.
2. **Comparaciones repetidas** (eficiencia). Con j recorriendo todo el rango, cada par se compara dos veces. Empezando en $i + 1$ se hace la mitad del trabajo y el resultado es el mismo: el algoritmo sigue siendo $O(n^2)$, pero con la constante dividida por dos.
3. **Falta la condición $i \neq j$** (corrección, y el importante). Tal como está, la función compara cada elemento consigo mismo, encuentra que son iguales y devuelve verdadero para *cualquier* lista no vacía. No es que dé un resultado inexacto: es que no mira los datos. Un fallo así pasa todas las pruebas que solo usan listas con repetidos.

Concepto. Fíjese en el orden de gravedad. El tercero produce siempre la respuesta correcta cuando la respuesta correcta es «sí», y por eso una batería de pruebas mal elegida lo aprueba. El segundo no afecta al resultado en absoluto. Al auditar código ajeno, empiece preguntando qué devuelve en el caso en que la respuesta debería ser «no»; después, cuánto cuesta.

Comprueba. Una vez corregidos los tres, queda la pregunta de diseño: ordenar primero la lista, con un coste de $O(n \log n)$, deja los duplicados adyacentes y permite detectarlos en una sola pasada. ¿Compensa? Depende de si espera encontrar el repetido pronto: la versión cuadrática puede terminar en la segunda comparación, y la que ordena paga siempre el coste completo antes de empezar a mirar.

3.7 Síntesis y diagnóstico de problemas algorítmicos

REFERENCIA

Técnica	Idea central	¿Óptimo?	Coste típico	Ejemplo en biología
Fuerza bruta	Probar todas las opciones	Sí	$O(2^n)$, $O(n!)$	Búsqueda exhaustiva de diana
Voraz (<i>Greedy</i>)	Elegir lo mejor local	No siempre	$O(n)$, $O(n \log n)$	Selección de experimentos
Divide y vencerás	Dividir y combinar	Depende	$O(n \log n)$	Mergesort en secuencias
Prog. Dinámica	Subproblemas solapados	Sí	$O(n^2)$, $O(n^3)$	Alineamiento de secuencias
Vuelta atrás	Podar ramas inviables	Sí	variable; en el peor caso, sin nada que podar, sigue siendo $O(2^n)$	Plegamiento de ARN

Las cinco técnicas de diseño se sintetizan por idea central, optimalidad garantizada, coste típico y un ejemplo biológico: fuerza bruta (probar todas las soluciones, siempre óptima, $O(2^n)/O(n!)$, búsqueda exhaustiva de subcadena); voraz (elegir lo mejor en cada paso, no siempre óptimo, $O(n)/O(n \log n)$, selección de actividades); divide y vencerás (partir y combinar, depende, coste variable, árbol filogenético); programación dinámica (subproblemas solapados más memoización, óptima, $O(n^2)/O(n^3)$, alineamiento de secuencias); backtracking (podar ramas inválidas pronto, óptima, coste variable pero mejor que fuerza bruta, ensamblaje combinatorio restringido).

3.7.1 Tabla de diagnóstico de síntomas computacionales

Síntoma observado	Causa probable	Comprobación y corrección
El programa se congela al aumentar N de 15 a 30	Algoritmo con complejidad exponencial $O(2^n)$	Comprobar si hay subproblemas solapados y aplicar memoización
Error <i>Stack overflow</i> / desbordamiento	Recursión sin caso base o llamada que no reduce N	Verificar que el caso base esté al principio y sea alcanzable
La solución voraz devuelve un resultado subóptimo	La instancia no cumple la propiedad voraz	Plantear la relación de recurrencia con Programación Dinámica
El tiempo se multiplica por 4 al duplicar N	Bucle anidado oculto ($O(n^2)$)	Ordenar previamente ($O(n \log n)$) o usar indexación hash ($O(1)$)

3.7.2 Tabla de síntesis operativa con preguntas de control

Necesidad	Paradigma recomendado	Pregunta de control
Probar todo el espacio de soluciones	Fuerza bruta	¿Es n suficientemente pequeño ($n \leq 15$)?
Solución rápida con elecciones locales	Algoritmo voraz (<i>Greedy</i>)	¿Está demostrado que la elección local lleva al óptimo global?
Subproblemas independientes	Divide y vencerás	¿Cómo se combinan las soluciones parciales?
Subproblemas repetidos y solapados	Programación dinámica	¿Tengo memoria suficiente para la tabla de memoización?
Espacio combinatorio con restricciones	Backtracking con poda	¿Puedo detectar y abortar ramas inválidas en etapas tempranas?

3.8 Glosario esencial

REFERENCIA

- **Algoritmo:** secuencia finita y bien definida de pasos para resolver un problema computacional.
- **Notación Big-O (O):** cota asintótica superior que describe el orden de crecimiento del coste de un algoritmo.
- **Modelo RAM:** abstracción de computador donde las operaciones elementales de CPU y memoria toman tiempo unitario constante.
- **Fuerza bruta:** exploración exhaustiva de todas las soluciones posibles.
- **Algoritmo voraz (*Greedy*):** método que selecciona la opción óptima local en cada paso.
- **Divide y vencerás:** técnica que divide un problema en partes independientes, las resuelve y combina los resultados.
- **Recursión:** mecanismo por el cual una función se invoca a sí misma sobre una instancia reducida.
- **Caso base:** condición de terminación en una función recursiva.
- **Programación dinámica:** técnica para problemas con subestructura óptima y subproblemas solapados mediante memoización.
- **Memoización:** almacenamiento de resultados intermedios en una tabla para evitar recálculos.
- **Backtracking:** exploración combinatoria con retroceso y poda temprana de ramas inválidas.

3.9 Conexión con el curso y lo que viene después

El modelo de coste de este capítulo se usa sin volver a enseñarse en los siguientes:

- En TC-CH04 y TC-CH05, formalizaremos las estructuras de datos fundamentales (lineales, árboles, tablas hash y grafos biológicos) analizando el coste temporal y espacial de cada operación de acceso y mutación.

- En **TC-CH06**, aplicaremos y compararemos sistemáticamente algoritmos de búsqueda y ordenación (lineal, binaria, QuickSort, Mergesort, Heapsort y Radix Sort), comprobando empíricamente cotas y constantes.
- En **TC-CH07** y **TC-CH08**, aplicaremos teoría de la información y modelos probabilísticos (cadenas de Markov y HMMs con el algoritmo de Viterbi).
- En **TC-CH09**, la programación dinámica resolverá el alineamiento de secuencias con Needleman–Wunsch y Smith–Waterman, y la heurística de BLAST permitirá explorar bases de datos a escala genómica.
- En **TC-CH10**, extenderemos el aprendizaje computacional hacia redes neuronales y representaciones aprendidas.

La formalización de complejidad asintótica y paradigmas de este capítulo fundamenta el análisis de estructuras en TC-CH04/05, la búsqueda y ordenación en TC-CH06, los modelos probabilísticos en TC-CH08, el alineamiento en TC-CH09 y el aprendizaje profundo en TC-CH10.

3.10 Fuentes y límites de uso

Este capítulo se apoya en la presentación docente de la asignatura sobre análisis de algoritmos, escrita por Álvaro Serrano Navarro. Las definiciones de las tres cotas asintóticas y la distinción entre cota y caso siguen a Cormen y colaboradores [8]; los ejemplos de aplicación biológica, a Compeau y Pevzner [7]; y el conteo de operaciones sobre código de Shell, a Allesina y Wilmes [1].

Todas las cuentas que aparecen en el texto se reproducen con `verification/chapter_results.sh` tal como están impresas, con sus argumentos. Cuando una cuenta del texto y una del oráculo discrepen, la del oráculo es la que manda: es la que se ejecuta.

Anexo práctico · Auditar el coste y sentir la diferencia entre exponencial y lineal

Pregunta, producto y separación de materiales

¿Puede un estudiante de biociencias auditar tres afirmaciones de coste erróneas producidas por una IA y medir, con sus propias manos en la terminal, la diferencia real entre una recursión desbocada y una solución memoizada? Este laboratorio responde afirmativamente sin requerir ningún dato externo: todo se genera y se mide con `verification/chapter_results.sh`.

El producto individual contiene: un veredicto razonado sobre las tres afirmaciones de coste auditadas, la tabla de mediciones de `fib` ingenuo frente a memoizado, la comprobación de escalado al duplicar la entrada, y las dos comprobaciones de control, todo en `notas/respuestas.tsv`.

El anexo es autónomo: no necesita datos externos ni red. Todo lo que se mide lo produce el oráculo del capítulo, que el generador copia en `herramienta/` para que las mediciones se hagan siempre con la misma herramienta.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 preparar el espacio	directorio creado y herramienta comprobada
2 auditar las tres afirmaciones	veredicto y justificación por fragmento
3 medir la recursión y la tabla	mediciones guardadas e interpretadas
4 duplicar la entrada	cociente observado frente al previsto
5 controles de robustez	dos entradas inválidas rechazadas
6 empaquetar y verificar	verificador en <code>ENTREGA_OK</code>

La ruta única de este anexo (no hay separación esencial/estudio) produce evidencia comprobable en cada bloque.

1 • Preparar el espacio de trabajo

```
bash --version
./practice_assets/create_workspace.sh TC03_entrega
cd TC03_entrega
```

El generador crea `notas/` con la tabla de respuestas vacía, solo su cabecera, `resultados/` para guardar las salidas y `herramienta/` con el oráculo del capítulo, de modo que las mediciones se hagan siempre con la misma herramienta. No trae ninguna respuesta escrita. Si el directorio ya existe, se niega a sobrescribirlo.

2 • Auditar tres afirmaciones de coste generadas por IA

Un asistente de IA clasifica tres fragmentos de código con errores: (a) un bucle anidado que compara cada elemento de una secuencia con todos los demás lo clasifica como « $O(n)$ »; (b) un bucle que reduce n a la mitad en cada paso hasta llegar a 1 lo clasifica como « $O(n)$ »; (c) un bucle simple que recorre una secuencia una sola vez lo clasifica como « $O(n^2)$ ».

El fragmento (a) —bucle anidado— es en realidad $O(n^2)$, no $O(n)$; el fragmento (b) —reducir n a la mitad— es en realidad $O(\log n)$, no $O(n)$; y el fragmento (c) —bucle simple— es en realidad $O(n)$, no $O(n^2)$: las tres clasificaciones propuestas están intercambiadas o infladas respecto al orden real.

```
bash herramienta/chapter_results.sh classify_nested 10 > resultados/frag_a.txt
bash herramienta/chapter_results.sh classify_halving 1000000 > resultados/frag_b.txt
bash herramienta/chapter_results.sh classify_single 10 > resultados/frag_c.txt
cat resultados/frag_a.txt resultados/frag_b.txt resultados/frag_c.txt
```

Registre cada fragmento en `notas/respuestas.tsv`, con su veredicto, el orden real que usted determine y la justificación. La justificación es la parte que se evalúa: decir que un bucle anidado es cuadrático no explica nada; decir cuántas veces se ejecuta el cuerpo interior, sí.

3 • Medir la programación dinámica: Fibonacci

```
bash herramienta/chapter_results.sh fib_naive_calls 5 > resultados/fib_ingenuo.txt
bash herramienta/chapter_results.sh fib_memo_calls 5 > resultados/fib_tabla.txt
bash herramienta/chapter_results.sh time_compare_verdict 15 > resultados/veredicto.txt
cat resultados/fib_ingenuo.txt resultados/fib_tabla.txt resultados/veredicto.txt
time bash herramienta/chapter_results.sh fib_naive_calls 18
```

Para el mismo valor de n , el tiempo real que tarda `fib_naive` es mayor que el que tarda la tabla ascendente de `fib_memo_calls`; la diferencia crece con n porque una crece exponencialmente y la otra linealmente.

Se reproduce con `verification/chapter_results.sh time_compare_verdict 20`.

Antes de ejecutar la última orden (`time ... fib_naive_calls 18`), **prediga por escrito** si tardará menos de un segundo o varios segundos, justificando el motivo según la tasa $O(2^n)$. Compare con la salida de `time`.

4 • Perturbación: duplicar el tamaño de la entrada (2×)

```
bash herramienta/chapter_results.sh classify_single 10 > resultados/lineal_n.txt
bash herramienta/chapter_results.sh classify_single 20 > resultados/lineal_2n.txt
bash herramienta/chapter_results.sh classify_nested 10 > resultados/cuadratico_n.txt
bash herramienta/chapter_results.sh classify_nested 20 > resultados/cuadratico_2n.txt
cat resultados/lineal_n.txt resultados/lineal_2n.txt resultados/cuadratico_n.txt resultados/cuadratico_2n.txt
```

Al duplicar n de 10 a 20, las operaciones del fragmento $O(n)$ crecen aproximadamente $\times 2$ (de 51 a 101), mientras que las del fragmento $O(n^2)$ crecen aproximadamente $\times 4$ (de 55 a 210): el cociente observado coincide con lo que predice cada clase de la notación O .

Se reproduce con `verification/chapter_results.sh classify_single 10`.

y `classify_nested` sobre $n = 10$ y $n = 20$.

5 • Controles de robustez y validación de límites

```
bash herramienta/chapter_results.sh control_reject_invalid -3 > resultados/control_negativo.txt
bash herramienta/chapter_results.sh control_reject_invalid abc > resultados/control_texto.txt
cat resultados/control_negativo.txt resultados/control_texto.txt
```

`fib_naive_calls` y `fib_memo_calls` rechazan explícitamente una entrada negativa o no numérica en vez de fallar de forma opaca en la aritmética de Bash o entrar en un comportamiento no definido.

Se reproduce con `verification/chapter_results.sh control_reject_invalid ATGX`.

6 • Verificación automática y empaquetado

```
cd ..
tar -czf TC03_entrega.tar.gz TC03_entrega
sha256sum TC03_entrega.tar.gz
./practice_assets/check_entrega.sh TC03_entrega TC03_entrega.tar.gz
```

El verificador comprueba que la tabla de respuestas tiene la cabecera correcta y al menos tres fragmentos con veredicto, orden real y una justificación escrita; que en ninguna fila coinciden el veredicto de la máquina y el orden real, porque los tres fragmentos del enunciado están mal clasificados; que `resultados/` contiene las mediciones de los cuatro bloques, incluidas las dos parejas de escalado; y que la defensa existe con la extensión pedida. Rechaza el espacio recién generado. Si falta algo lo enumera y termina en `ENTREGA_INCOMPLETA`; si está todo, en `ENTREGA_OK`. No puntúa.

Rúbrica de calificación ponderada

La evaluación pondera la verificación técnica, el análisis formal y la defensa conceptual sobre 10 puntos, tres de entrega y siete de razonamiento:

- **3,0 puntos, entrega técnica y verificador:** estructura de carpetas, formato de `notas/respuestas.tsv`, mediciones guardadas y verificador en `ENTREGA_OK`.
- **2,5 puntos • Auditoría analítica de fragmentos:** Identificación rigurosa del orden asintótico real de los tres fragmentos propuestos y justificación formal anclada en el número de iteraciones y ramificaciones.
- **2,5 puntos • Experimentación y contraste de perturbación (2×):** Medición comparativa de Fibonacci, interpretación del cociente de operaciones al duplicar n ($2\times$ para $O(n)$, $4\times$ para $O(n^2)$) y control de entradas inválidas.
- **2,0 puntos • Defensa conceptual y conexión biológica:** Justificación rigurosa (100–150 palabras) sobre el papel de los subproblemas solapados en la memoización frente a subproblemas disjuntos, apoyándose en los datos obtenidos.

Terminar en `ENTREGA_OK` acredita que la entrega está completa, no que el razonamiento sea correcto: una justificación que acierta el orden y no explica por qué pierde el segundo apartado entero.

Lista de autocorrección

- Las tres afirmaciones de coste auditadas tienen orden real y justificación razonada.
- Comparé mi predicción de tiempo de `fib_naive_calls 18` con la medida real de `time`.
- El cociente al duplicar n concuerda con la teoría: aproximadamente el doble en el fragmento lineal y el cuádruple en el cuadrático.

- Las entradas inválidas fueron rechazadas explícitamente con mensaje de control.
- Puedo explicar por qué memoizar transforma una complejidad exponencial en lineal.
- Guardé en resultados/ la salida de cada medición que cito.
- El verificador termina en ENTREGA_OK.

Defensa conceptual

En notas/defensa.md, entre 100 y 150 palabras, explique por qué la memoización acelera drásticamente el cálculo de `fib` pero no aportaría ninguna ventaja en un algoritmo cuyos subproblemas son estrictamente independientes y nunca se repiten (como el algoritmo de ordenación por mezcla *Mergesort*). Emplee al menos un dato numérico concreto obtenido en sus propias mediciones del Bloque 3.

Fuentes y reproducibilidad

Este anexo no usa datos externos: todo lo que mide lo genera el propio oráculo del capítulo, que es determinista y da el mismo resultado en cualquier máquina. Las definiciones de coste siguen a Cormen y colaboradores [8].

Representación en memoria y estructuras lineales

TC-CH04 – Técnicas Computacionales en Biología

Edición 2

Pregunta del tema. Dos programas guardan el mismo millón de lecturas y hacen las mismas operaciones sobre ellas. Uno tarda un segundo y el otro veinte minutos. ¿Qué decidió la diferencia, si el procesador es el mismo?

Producto final. Una pila y una cola escritas por usted con arrays de Bash, capaces de validar el anidamiento de una estructura secundaria de ARN donde un contador falla, y de procesar lecturas respetando el orden de llegada.

Mapa del tema

4.1. Por qué una estructura de datos decide el éxito de un programa	66
4.2. Representar un valor antes de organizar una colección	66
4.3. Modelos de almacenamiento en memoria: contiguo frente a enlazado	68
4.4. El array y los arrays dinámicos	70
4.5. La lista enlazada	71
4.6. La pila (<i>Stack</i>) — Disciplina LIFO	72
4.7. La cola (<i>Queue</i>) — Disciplina FIFO	75
4.8. La operación peligrosa: mutación concurrente e índices inválidos	76
4.9. Actividad de depuración: diagnóstico de errores en punteros	76
4.10. Síntesis operativa y matriz comparativa de costes	77
4.11. Glosario conceptual	78
4.12. Conexión con el curso y siguientes pasos	78
4.13. Fuentes y reproducibilidad	78

Cómo usar este tema

Este capítulo baja un nivel respecto al anterior. Allí contábamos operaciones sin preguntarnos dónde estaban los datos; aquí la respuesta a esa pregunta es justamente lo que decide el coste. Se lee en dos mitades. La primera es cómo la máquina guarda un número y por qué eso produce resultados que parecen errores y no lo son. La segunda son las estructuras lineales, array, lista, pila y cola, cada una con las operaciones que hace baratas y las que hace caras.

Al terminar sabrá: explicar por qué una resta puede hacerse con el circuito de la suma, y por qué comparar dos números decimales con el signo igual es mala idea; distinguir un tipo abstracto de su implementación, que es la distinción que permite cambiar una por otra sin tocar el resto del programa; elegir entre almacenamiento

contiguo y enlazado según qué operación domine; y reconocer los dos problemas que resuelve una pila y los que resuelve una cola.

La ruta **esencial** son la representación en memoria, el par contiguo frente a enlazado, y la pila y la cola con sus operaciones. La ruta de **estudio** añade el coste amortizado del array dinámico, la lista doblemente enlazada, el caso del ARN y el anexo.

4.1 Por qué una estructura de datos decide el éxito de un programa

ESENCIAL Resolver un problema algorítmico es solo el primer paso; hacerlo viable sobre conjuntos masivos de datos biomédicos es el verdadero reto de la bioinformática. Y esa viabilidad depende mucho más de **cómo se organizan y almacenan los datos en la memoria** que de la velocidad del procesador.

La misma operación básica —buscar un nucleótido o variante, insertar una nueva lectura de secuenciación, o recuperar el siguiente elemento a procesar— puede costar un tiempo constante ($O(1)$) o crecer linealmente con el tamaño de los datos ($O(n)$) según la estructura de datos elegida. Sobre un genoma humano de 3×10^9 pares de bases o un experimento de transcriptómica con millones de moléculas, esa diferencia es la frontera exacta entre una respuesta instantánea y un proceso que no termina en un tiempo útil.

La misma operación sobre una colección de datos —buscar, insertar, recuperar el siguiente— puede costar tiempo constante o crecer linealmente con el tamaño de la colección, según la estructura de datos elegida para guardarla.

Cualquier asistente de inteligencia artificial puede generar código para manipular listas o matrices en segundos, pero casi siempre elegirá la estructura «por defecto» del lenguaje sin considerar las consecuencias en consumo de memoria o latencia de acceso. El objetivo de este capítulo no es simplemente aprender la sintaxis de una lista o una pila, sino comprender los fundamentos físicos de la memoria, qué garantías ofrece cada estructura y a qué precio computacional, para elegir siempre con criterio científico.

4.2 Representar un valor antes de organizar una colección

Para estructurar grandes colecciones biológicas, es indispensable comprender primero cómo se codifica y almacena un único dato elemental en los circuitos del ordenador.

4.2.1 Dato, tipo de dato y estructura de datos

- Un **dato** es una unidad mínima de información o valor concreto (por ejemplo, la base nitrogenada 'A' o una temperatura de desnaturalización 95.0).
- Un **tipo de dato** define formalmente el conjunto de valores posibles que puede tomar una variable y el repertorio de operaciones válidas que tienen sentido sobre ellos (no tiene sentido multiplicar dos cadenas de ADN, pero sí concatenarlas).
- Una **estructura de datos** es una forma sistemática de organizar, relacionar y almacenar una colección de datos en la memoria del ordenador, junto con los algoritmos que permiten acceder a ellos y manipularlos de forma eficiente.

Un dato es un valor; un tipo de dato define qué valores son posibles y qué operaciones tienen sentido sobre ellos; una estructura de datos organiza una colección de datos en memoria junto con las operaciones que permite hacer sobre ella eficientemente.

Concepto. Hay un tipo de dato cuyo valor es una dirección de memoria: el *puntero*. No guarda un número ni una letra, guarda dónde vive otra cosa. Conviene verlo como un tipo más y no como un detalle interno, porque es el que hace posible el almacenamiento enlazado: un nodo puede estar en cualquier hueco libre de la memoria, por lejos que quede de su vecino, porque el vecino guarda su dirección. Todo lo que viene después en este capítulo depende de esa idea.

4.2.2 Codificación física en memoria: ceros y unos

En el soporte físico de la memoria RAM, toda la información se reduce a estados binarios: bits (0 y 1). Interpretar esos bits exige conocer su esquema de codificación:

1. **Enteros en base 2:** los números naturales se representan en base binaria pura (por ejemplo, el valor decimal 115 se codifica en 8 bits como 01110011).
2. **Enteros con signo (Complemento a dos):** en complemento a dos no se reserva un bit de signo independiente: para una palabra de w bits, el bit más significativo (b_{w-1}) aporta un peso negativo -2^{w-1} y los restantes bits aportan pesos positivos estándar. Así, en 8 bits, 11111111 representa $-128 + 127 = -1$, mientras que 01111111 representa $+127$. Este esquema genera el intervalo asimétrico $[-2^{w-1}, 2^{w-1} - 1]$ y permite que la suma y la resta utilicen el mismo circuito binario.
3. **Coma flotante (Floating Point):** los números reales se codifican siguiendo el estándar internacional IEEE 754 como el producto de un bit de signo, un significando (o mantisa) y una base elevada a un exponente ($v = (-1)^s \times M \times 2^E$). Ocupan habitualmente 32 bits (precisión simple) o 64 bits (doble precisión).
4. **Caracteres y texto:** se representan asignando un número entero único a cada símbolo gráfico. El estándar ASCII clásico emplea 7 bits (128 caracteres básicos del alfabeto inglés). El estándar Unicode (con esquemas como UTF-8 de longitud variable de 1 a 4 bytes) permite representar más de 140 000 símbolos universales, alfabetos globales y caracteres técnicos.

En complemento a dos, que es lo que usan los procesadores actuales, no hay un bit de signo aparte: el bit más significativo pesa -2^{w-1} y los demás pesan en positivo. De ahí sale el rango asimétrico de -128 a 127 en ocho bits, y de ahí que la resta pueda hacerse con el mismo circuito que la suma. Los números con decimales se representan como un coeficiente por una potencia de dos, con precisión limitada, y los caracteres asignando un número a cada símbolo.

4.2.3 Mecanismo del complemento a dos

El complemento a dos es uno de los logros más elegantes de la ingeniería computacional: permite a la CPU realizar restas utilizando exactamente los mismos circuitos lógicos de la suma.

Para obtener la representación de -115 en 8 bits:

1. Se parte del valor positivo $+115$ en binario: 01110011.
2. Se invierten todos los bits (operación *NOT*): 10001100.
3. Se suma 1 al resultado: 10001101.

La verificación matemática es inmediata al sumar $+115$ y -115 :

$$\begin{array}{rcl}
 & 01110011 & (+115) \\
 + & 10001101 & (-115) \\
 \hline
 = & 10000000 & (256 \equiv 0 \text{ (mód 256)})
 \end{array}$$

Al descartar el noveno bit (el bit de acarreo que excede la capacidad del registro de 8 bits), el resultado es 00000000, es decir, 0 exacto.

Representar -115 en complemento a dos de 8 bits: invertir los bits de 115 ($01110011 \rightarrow 10001100$) y sumar 1 (10001101). Sumar 01110011 (115) y 10001101 (el resultado) da 100000000 ; descartando el noveno bit (el acarreo que excede los 8 bits) queda 00000000 , cero, confirmando que la representación es correcta: $115 + (-115) = 0$.

Se reproduce con `verification/chapter_results.sh twos_complement 115 8`.

4.2.4 La trampa de la coma flotante: imprecisión binaria

En coma flotante, muchas fracciones decimales cotidianas (como $0.1 = 1/10$ o $0.2 = 2/10$) **no tienen una representación binaria finita**, de la misma forma que la fracción $1/3 = 0.3333 \dots$ no tiene fin en el sistema

decimal.

Al sumar $0.1 + 0.2$ en coma flotante de doble precisión (64 bits), el valor almacenado es:

$$0.1 + 0.2 = 0.3000000000000000444089209850062616169452667236328125$$

Por esta razón, una comparación directa de igualdad estricta ($0.1 + 0.2 == 0.3$) evaluará como **FALSA**. En computación científica y bioinformática, los valores decimales nunca deben compararse con igualdad estricta, sino mediante una tolerancia o épsilon: $|a - b| < \epsilon$.

En coma flotante de doble precisión, $0.1 + 0.2$ no da exactamente 0.3: el resultado es 0.30000000000000004, distinto de 0.3 en el último dígito representable, porque ni 0.1 ni 0.2 tienen representación binaria finita. Comparar directamente el resultado de una suma de decimales con `==` es, por esta razón, un error frecuente en código generado tanto a mano como por una IA.

Se reproduce con `verification/chapter_results.sh float_imprecision`.

4.2.5 Tipos de Datos Abstractos (TDA) frente a Implementación

Una distinción fundamental en informática es la separación entre el **qué** y el **cómo**:

- Un **Tipo de Dato Abstracto (TDA)** define el comportamiento conceptual externo: qué operaciones ofrece (por ejemplo, «insertar al final», «extraer el más antiguo») y qué reglas semánticas cumplen, sin especificar en absoluto cómo se almacenan los datos por dentro.
- La **Implementación** es la realización concreta en código y memoria (usando un vector contiguo o nodos enlazados).

El TDA establece el contrato funcional; la implementación determina el coste en tiempo y memoria de cada operación.

Un tipo de dato abstracto (TDA) especifica qué operaciones ofrece una estructura y qué significan, sin fijar cómo se implementa; la implementación es la realización concreta que determina el rendimiento. Dos implementaciones del mismo TDA ofrecen las mismas operaciones con costes distintos.

4.3 Modelos de almacenamiento en memoria: contiguo frente a enlazado

ESENCIAL

Existen dos estrategias físicas fundamentales para colocar una colección de elementos en la memoria RAM:

El almacenamiento contiguo coloca los elementos en direcciones consecutivas, separadas por el tamaño de cada elemento, así que la dirección de cualquier elemento se calcula directamente desde su índice; el almacenamiento enlazado coloca cada nodo en cualquier dirección libre de memoria, sin relación aritmética entre las direcciones de nodos sucesivos, y la única forma de encontrar el siguiente nodo es seguir el puntero que guarda.

4.3.1 Cálculo de dirección directa en almacenamiento contiguo

En un array, si la dirección base es 1000 y cada elemento ocupa 4 bytes, la dirección del elemento con índice i se calcula con una única multiplicación y una suma:

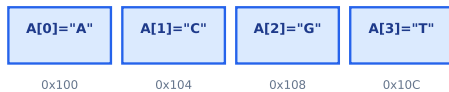
$$\text{Dirección}(e[i]) = \text{base} + i \times \text{tamaño}$$

Para el índice 7: $1000 + 7 \times 4 = 1028$. Para el índice 999: $1000 + 999 \times 4 = 4996$. Ambas operaciones toman exactamente el mismo tiempo: **coste constante** $O(1)$.

En cambio, en una lista enlazada no existe ninguna relación matemática entre las direcciones de los nodos. Para acceder al nodo 999 es estrictamente obligatorio recorrer los 998 nodos precedentes saltando de puntero en puntero: **coste lineal** $O(n)$.

En un array de enteros de 4 bytes que empieza en la dirección 1000, la dirección del elemento de índice i es $1000 + i \times 4$ sin recorrer nada: el elemento 7 está en 1028 y el elemento 999 está en 4996, ambos con una sola

A) Disposición en Memoria: Array vs Lista Enlazada

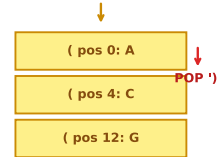
Array Contiguo (Acceso $O(1)$ por dirección base + $i \cdot \text{tamaño}$):Alta localidad espacial de caché | Redimensionar exige copiar $O(n)$

Lista Enlazada (Nodos dispersos en el Heap con punteros):

Inserción $O(1)$ con puntero | Acceso por índice $O(n)$ | Sobrecarga de punteros

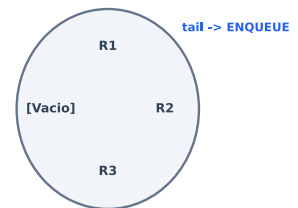
B) Pila (LIFO) en ARN y Cola Circular (FIFO)

Pila LIFO (Anidamiento ARN):



head -> DEQUEUE

Cola Circular FIFO (Buffer):



Pila: Backtracking y sintaxis de ARN | Cola: Streaming ordenado de lecturas

Figura 4.1: A) Comparación física en memoria: Array contiguo con cálculo aritmético directo $O(1)$ vs Lista Enlazada con nodos dispersos en Heap y punteros $O(n)$. B) Estructuras lineales especializadas: Pila LIFO (anidamiento de pares de bases de ARN) y Cola Circular FIFO (buffer de lecturas con punteros head/tail). Figura original adaptada de las presentaciones docentes.

Almacenamiento Contiguo (Array, base = 1000, elemento = 4 bytes):

Direccion: 1000 1004 1008 1012 1016

```

+-----+-----+-----+-----+-----+
| e[0] | e[1] | e[2] | e[3] | e[4] |
+-----+-----+-----+-----+

```

Formula de acceso: $\text{dir}(e[i]) = \text{base} + i \cdot \text{tamano} \rightarrow O(1)$ directo

Almacenamiento Enlazado (Nodos dispersos en memoria):

Dir 3040 Dir 1592 Dir 4801

```

+-----+-----+-----+-----+-----+
| val0 | 1592 | --> | val1 | 4801 | --> | val2 | null |
+-----+-----+-----+-----+

```

(Sin relacion aritmetica: para llegar a val2 hay que recorrer val0 y val1)

Figura 4.2: Comparación entre almacenamiento contiguo (array) y enlazado (lista). En el array, la dirección física se calcula inmediatamente en $O(1)$ mediante una fórmula aritmética. En la lista enlazada, las direcciones son arbitrarias y se deben seguir los punteros secuencialmente en $O(n)$. Figura original.

multiplicación y una suma. En una lista enlazada no existe esa fórmula: alcanzar el elemento i exige i saltos de puntero, uno por cada nodo intermedio.

Se reproduce con `verification/chapter_results.sh array_address_calc 1000 4 999`.

4.3.2 Costes fundamentales del modelo de memoria

- **Leer o escribir en una dirección:** tiempo constante $O(1)$.
- **Alojar un bloque contiguo de tamaño N :** tiempo $O(N)$, pues el sistema operativo debe localizar un segmento contiguo libre y reservarlo.
- **Inmutabilidad del bloque contiguo:** un bloque alojado no puede expandirse «en el mismo sitio» si la celda contigua ya está ocupada por otra variable. Para crecer, es obligatorio alojar un nuevo bloque más grande en otra zona de la memoria y copiar todos los datos existentes.
- **Liberar memoria (*free*):** tiempo constante $O(1)$.

En el almacenamiento contiguo, acceder al elemento i cuesta tiempo constante por cálculo directo de dirección, pero insertar en medio obliga a desplazar los elementos posteriores; en el almacenamiento enlazado, insertar o quitar es solo recolocar punteros sin mover datos, pero no hay acceso directo y hay que recorrer la cadena desde el principio.

Una estructura estática tiene tamaño fijado de antemano; una dinámica crece y mengua durante la ejecución. Las estructuras dinámicas son las más usadas en bioinformática porque el tamaño de los datos rara vez se conoce por adelantado.

Leer o escribir en una dirección de memoria cuesta $O(1)$; alojar un bloque contiguo de tamaño N cuesta $O(N)$ y, una vez alojado, no se expande en el mismo sitio: hace falta alojar un bloque mayor y copiar los datos; desalojar memoria cuesta $O(1)$.

Modelo docente de coste de memoria.

En este capítulo modelamos formalmente como $O(n)$ el coste de inicializar explícitamente n celdas contiguas de memoria o copiar n elementos durante el redimensionado de un array dinámico. Las solicitudes y liberaciones virtuales al sistema operativo pueden tener comportamientos dependientes del asignador y del lenguaje, pero la cota $O(n)$ del array dinámico se fundamenta invariablemente en la necesidad de copiar los n elementos preexistentes.

4.4 El array y los arrays dinámicos

ESENCIAL

El array es la estructura contigua por excelencia. Ofrece acceso directo $O(1)$ por índice, pero penaliza severamente las inserciones o eliminaciones intermedias ($O(n)$), ya que todos los elementos posteriores deben desplazarse una posición a la derecha o a la izquierda.

El acceso por índice a un array cuesta $O(1)$; la búsqueda de un valor cuesta $O(n)$ si no está ordenado, u $O(\log n)$ con búsqueda binaria si está ordenado; la inserción o el borrado en una posición intermedia cuesta $O(n)$ porque hay que desplazar los elementos posteriores.

4.4.1 Redimensionamiento geométrico y coste amortizado

En la práctica bioinformática no se conoce de antemano cuántas lecturas o variantes se van a procesar. Para evitar fijar un tamaño estático, los lenguajes modernos implementan **arrays dinámicos** (vectores con crecimiento automático).

¿Cómo crece un array dinámico eficientemente?

- Si incrementara su capacidad de 1 en 1 cada vez que se añade un elemento, insertar N elementos requeriría copiar $1 + 2 + 3 + \dots + N = O(N^2)$ elementos en total: una ineficiencia inasumible.
- En su lugar, utiliza una **estrategia de duplicación geométrica**: cuando el array se llena (capacidad C), se

aloja un nuevo bloque de tamaño $2C$ y se copian los elementos existentes.

Aunque la inserción puntual que dispara la duplicación cuesta $O(N)$, ocurre tan raramente que el coste total para insertar N elementos es:

$$N + \left(\frac{N}{2} + \frac{N}{4} + \dots + 4 + 2 + 1 \right) < N + N = 2N \Rightarrow O(N) \text{ en total}$$

Dividiendo el coste total entre las N operaciones, el coste medio por inserción es $\frac{2N}{N} = 2 = O(1)$ **amortizado**.

Un array dinámico que duplica su capacidad cada vez que se llena logra que añadir N elementos al final cueste en total $O(N)$ operaciones de copia, es decir $O(1)$ amortizado por elemento, frente a las $O(N^2)$ que costaría redimensionar de uno en uno.

Se reproduce con `verification/chapter_results.sh dynamic_array_growth 16`.

Para un patrón de acceso aleatorio frecuente sin inserciones ni borrados, un array (acceso $O(1)$) es preferible a una lista enlazada (acceso $O(n)$): sobre una colección de tamaño n , leer m posiciones aleatorias cuesta $O(m)$ con un array frente a $O(m \cdot n)$ en el peor caso con una lista.

Se reproduce con `verification/chapter_results.sh array_vs_list_access 1000`.

Caso límite: Acceso fuera de rango (*Out of Bounds*).

Un array de tamaño n tiene índices válidos estrictamente desde 0 hasta $n - 1$. Intentar acceder al índice n o a un índice negativo carece de sentido. En lenguajes de bajo nivel como C, este acceso no se comprueba, leyendo memoria arbitraria del sistema y generando graves fallos de seguridad o corrupción de datos. En entornos de scripting seguros o lenguajes modernos, se produce una excepción controlada o se devuelve un valor nulo.

Acceder al índice n (uno más allá del último válido, 0 a $n - 1$) o a un índice negativo en un array de n elementos no tiene elemento que devolver. En Bash, leer un índice fuera de rango de un array devuelve una cadena vacía en vez de fallar; en lenguajes sin comprobación de límites como C, el mismo acceso lee memoria ajena sin detectarlo. El comportamiento exacto ante un índice inválido depende del lenguaje y debe comprobarse, no asumirse.

Se reproduce con `verification/chapter_results.sh array_out_of_bounds 5`.

4.5 La lista enlazada

ESTUDIO

Una lista enlazada almacena sus elementos en **nodos** independientes dispersos en la memoria. Cada nodo contiene dos campos:

- **dato**: el valor biológico almacenado.
- **siguiente**: el puntero (dirección física) al nodo posterior en la secuencia.

```
insertarInicio(L, x):
    nuevo <- crearNodo(x)
    nuevo.siguiente <- L.cabeza
    L.cabeza <- nuevo

buscar(L, x):
    actual <- L.cabeza
    mientras actual != null:
        si actual.dato == x:
            devolver actual
        actual <- actual.siguiente
    devolver null
```

La inserción al inicio de una lista enlazada cuesta $O(1)$ porque solo se recolocan dos punteros; el acceso por posición y la búsqueda cuestan $O(n)$ porque no hay atajo al elemento i : hay que recorrer la cadena. El borrado de la cabeza cuesta $O(1)$.

4.5.1 El orden de asignación de punteros importa

Uno de los errores más comunes al programar listas enlazadas es alterar el orden en que se reasignan los punteros:

```
insertarInicio_ERROR(L, x):
    nuevo <- crearNodo(x)
    L.cabeza <- nuevo           # ERROR: se pierde la referencia a la lista
    nuevo.siguiete <- L.cabeza  # El nuevo nodo apunta a si mismo en bucle
```

Si se mueve `L.cabeza` antes de enlazar `nuevo.siguiete`, la dirección de la lista original se pierde irremediablemente en la memoria (*fuga de memoria* o *memory leak*), y el nuevo nodo queda atrapado apuntando a sí mismo en un ciclo infinito.

Si la cabeza se mueve al nodo nuevo antes de enlazar ese nodo con la cabeza anterior, el nodo nuevo acaba apuntándose a sí mismo. Queda un bucle de un solo nodo y el resto de la lista se pierde: nadie guarda ya su dirección.

Se reproduce con `verification/chapter_results.sh linked_list_pointer_bug`.

4.5.2 Variantes: Listas doblemente enlazadas y circulares

- **Lista doblemente enlazada** (*Doubly Linked List*): cada nodo almacena dos punteros: `siguiete` y `anterior`. Esto permite navegar en ambas direcciones y eliminar un nodo conocido en tiempo $O(1)$ sin tener que buscar quién lo precedía.
- **Lista circular**: el puntero `siguiete` del último nodo apunta de nuevo a la cabeza, formando un anillo continuo.

En una lista doblemente enlazada, cada nodo guarda también un puntero al nodo anterior, lo que permite recorrer hacia atrás y borrar un nodo ya localizado en $O(1)$ sin recorrer la lista de nuevo.

Insertar al final de una lista enlazada simple de n nodos sin puntero de cola cuesta $O(n)$ (recorrer hasta el último); manteniendo un puntero de cola actualizado, la misma inserción cuesta $O(1)$. Pero borrar el último nodo sigue costando $O(n)$ incluso con puntero de cola, porque no hay forma de retroceder desde el último nodo en una lista simplemente enlazada para recolocar el nuevo puntero de cola: solo una lista doblemente enlazada resuelve también el borrado del final en $O(1)$.

Se reproduce con `verification/chapter_results.sh linked_list_tail_insert 10`.

Buscar un valor ausente en una lista enlazada de n nodos recorre los n nodos completos sin encontrar coincidencia y termina en `null` cuando `actual` pasa a ser `null`, no por comparación con el valor buscado. El resultado de un recorrido completo sin coincidencia debe distinguirse explícitamente de un valor encontrado por accidente; devolver un valor por defecto ambiguo (como 0) en vez de `null` impide esa distinción.

Se reproduce con `verification/chapter_results.sh linked_list_search_miss 10`.

4.6 La pila (*Stack*) — Disciplina LIFO

ESENCIAL

Una pila es un TDA lineal restrictivo regido por la regla **LIFO** (*Last In, First Out*): el último elemento en entrar es estrictamente el primero en salir.

Ofrece tres operaciones fundamentales, todas en tiempo constante $O(1)$:

- `push(x)`: coloca el elemento x en la cima de la pila.
- `pop()`: retira y devuelve el elemento situado en la cima.
- `peek()` (o `top()`): consulta el elemento de la cima sin retirarlo.

La necesidad operativa de la pila: retroceso y desapilado.

Más allá de almacenar datos, la pila es la estructura computacional natural para cualquier proceso que deba **des-hacer acciones** o realizar **retroceso exploratorio** (*backtracking*). Cuando un algoritmo recorre una bifurcación y encuentra un camino inválido o un símbolo incompatible, desapilar (pop) devuelve inmediatamente el estado computacional al punto de decisión previo en tiempo $O(1)$, sin requerir búsquedas en la memoria.

Una pila ofrece tres operaciones, todas $O(1)$: push(x) (poner x en la cima), pop() (sacar y devolver el elemento de la cima) y peek() (mirar la cima sin sacarla). Hacer pop() sobre una pila vacía es un caso límite que debe detectarse, no ejecutarse.

4.6.1 Casos límite de la pila: Underflow y Overflow

- **Subdesbordamiento** (*Stack Underflow*): intentar ejecutar pop() o peek() cuando la pila está vacía. Un programa robusto debe comprobar esta condición antes de desapilar.
- **Desbordamiento** (*Stack Overflow*): intentar ejecutar push() sobre una pila implementada con capacidad estática que ya está llena. Es la misma causa que el colapso de memoria por recursión infinita, visto en el capítulo de algoritmos y coste.

Hacer pop() sobre una pila vacía es un subdesbordamiento (stack underflow); hacer push() sobre una pila de capacidad fija ya llena es un desbordamiento (stack overflow), el mismo nombre y la misma causa de fondo —falta de espacio donde crecer— que el desbordamiento de la pila de llamadas por recursión sin caso base, visto en el capítulo de algoritmos y coste.

Restringir una colección a un único punto de acceso (pila o cola) es una decisión de diseño, no una limitación técnica: al eliminar operaciones como leer o modificar un elemento intermedio se descarta de raíz esa clase de errores y el comportamiento de la estructura queda predecible sin necesidad de rastrear todos sus estados posibles.

Característica	Pila sobre Array Dinámico	Pila sobre Lista Enlazada
Rendimiento push/pop	$O(1)$ amortizado	$O(1)$ garantizado en cada paso
Localidad de caché	Excelente (elementos contiguos)	Pobre (nodos dispersos en RAM)
Sobrecarga de memoria	Baja (solo datos y un índice)	Alta (un puntero extra por nodo)

Una pila implementada sobre un array dinámico ofrece push/pop en $O(1)$ amortizado con excelente localidad de caché y poca sobrecarga por elemento, a cambio de memoria potencialmente desperdiciada y tamaño flexible solo mediante redimensionados costosos; sobre una lista enlazada ofrece $O(1)$ garantizado en cada operación individual y tamaño dinámico sin redimensionados, a cambio de un puntero adicional por nodo y peor localidad de caché.

4.6.2 Caso bioinformático: Validación de estructura secundaria de ARN

Las estructuras secundarias de ARN (horquillas, bucles y hélices) se anotan mediante la notación de corchetes y puntos (*dot-bracket notation*), donde los pares de bases emparejadas se representan con paréntesis. Conviene decirlo con precisión, porque aquí se confunden dos cosas distintas. En la notación de puntos y paréntesis, un punto es una base sin emparejar y cada par de paréntesis marca dos bases emparejadas entre sí. Cuando dos pares se cruzan, y por tanto no pueden anidarse uno dentro del otro, se recurre a una segunda clase de paréntesis, los corchetes: así se anotan los pseudonudos. Lo que validaremos en este capítulo es que **cada clase esté bien anidada consigo misma**, que es exactamente lo que sabe hacer una pila.

Un enfoque ingenuo para verificar si una cadena está bien balanceada es usar un **contador numérico** que sume +1 al abrir y reste -1 al cerrar. Sin embargo, sobre la cadena "([])", el contador evalúa +1, +2, +1, 0 (nunca negativo y termina en cero), por lo que **la acepta erróneamente**.

Alcance del validador y delimitación formal.

Bajo la gramática libre de contexto que modela este ejercicio, se aceptan exclusivamente pares correctamente anidados con delimitadores independientes () y []. La cadena "([])" se rechaza porque el cierre) no coincide

con la apertura [situada en la cima de la pila.

El veredicto formal de rechazo significa estrictamente: «*la cadena no pertenece al lenguaje anidado que valida este autómata de pila*». No debe interpretarse como «estructura biológicamente imposible», puesto que los pseudonudos reales existen en la naturaleza, pero su modelado estructural requiere gramáticas y formalismos más complejos que una pila elemental.

Una **pila** resuelve el problema de forma exacta:

1. Lee '(': apila '('.
2. Lee '[': apila '['.
3. Lee ')': consulta la cima. La cima contiene '[', que **no coincide** con ')'. La pila **rechaza inmediatamente**.

Símbolo	Contador	Pila (cima der.)	Veredicto Contador	Veredicto Pila
(	+1 = 1	(	Válido (> 0)	Apila (
[	+1 = 2	([	Válido (> 0)	Apila [
)	-1 = 1	(	Válido (> 0)	Rechaza:) no casa con [
]	-1 = 0	—	Termina en 0: acepta, y se equivoca	Ya rechazada

Figura 4.3: Traza comparativa sobre "([]) ". El contador acepta por error al no distinguir tipos de apertura; la pila valida estrictamente el emparejamiento por clases. Figura original.

Sobre la entrada ([)], un contador simple de aperturas y cierres la acepta como bien anidada (nunca negativo, termina en 0), pero una pila que compara cada cierre con el símbolo en la cima la rechaza correctamente, porque) no casa con el [que está en la cima cuando aparece.

Se reproduce con `verification/chapter_results.sh stack_validate '([)]'`.

```

ALGORITMO: Pila_Validacion_Estructura_Secundaria_ARN
ENTRADA:
- S: Cadena en notacion dot-bracket de longitud n sobre alfabeto {'(', ')', '[', ']', '.'}
SALIDA:
- Veredicto: {ESTRUCTURA_VALIDA, ERROR_SINTAXIS_DESBALANCEO, ERROR_TIPO_CIERRE_INCORRECTO}
COMPLEJIDAD: Tiempo O(n) un solo recorrido lineal, Espacio auxiliar O(n) para la pila

1. INICIALIZACION:
Pila <- Crear_Pila_Vacia()
Mapa_Pares <- {'(': ')', '[': ']'} # Mapeo de cierre a su apertura esperada

2. RECORRIDO LINEAL DE CARACTERES:
Para pos desde 0 hasta n - 1:
    simbolo <- S[pos]

    Si simbolo en {'(', '['} entonces:
        Pila.PUSH((simbolo, pos)) # Guardar tipo de apertura y su coordenada

    Sino si simbolo en {')', ']'} entonces:
        Si Pila.IS_EMPTY() entonces:
            Retornar ERROR_SINTAXIS_DESBALANCEO (Cierre sin apertura previa en pos)

        (cima_simbolo, cima_pos) <- Pila.POP()
        apertura_esperada <- Mapa_Pares[simbolo]

        Si cima_simbolo != apertura_esperada entonces:
            Retornar ERROR_TIPO_CIERRE_INCORRECTO (Cierre 'simbolo' no casa con 'cima_simbolo')

    Sino si simbolo == '.' entonces:
        Continuar # Base no emparejada en bucle o extremo terminal

3. COMPROBACION FINAL DE CIERRE:
Si NO Pila.IS_EMPTY() entonces:

```



```
Retornar ERROR_SINTAXIS_DESBALANCEO (Quedaron aperturas sin su correspondiente cierre)
```

```
4. RETORNO:
Retornar ESTRUCTURA_VALIDA
```

4.7 La cola (*Queue*) — Disciplina FIFO

ESENCIAL

Una cola es un TDA lineal regido por la disciplina **FIFO** (*First In, First Out*): el primer elemento que entra es el primero en ser procesado, como una cola de impresión o una tubería física.

Ofrece dos operaciones maestras en tiempo $O(1)$:

- `enqueue(x)`: inserta el elemento x al final (*rear* o *tail*) de la cola.
- `dequeue()`: retira y devuelve el elemento situado en el frente (*front* o *head*).

Y dos consultas que no la modifican, tan necesarias como las anteriores: *frente*, también llamada *peek*, que devuelve el elemento que saldría a continuación sin sacarlo, y *esta_vacia*, que responde si queda algo dentro. La segunda es la que evita el subdesbordamiento: se pregunta antes de sacar, no se saca y se comprueba después. La pila tiene exactamente las mismas dos consultas con otros nombres.

Una cola ofrece `enqueue(x)` (añadir x al final) y `dequeue()` (sacar y devolver el elemento del frente), ambas $O(1)$ con una implementación adecuada.

4.7.1 Colas circulares sobre arrays (Buffers de anillo)

Si una cola se implementa sobre un array estático simple avanzando el frente y el final, tras varios encolados y desencolados el puntero final llegará al tope del array, desaprovechando los huecos libres dejados al inicio.

Una **cola circular** (*ring buffer*) soluciona este problema mediante **aritmética modular**:

$$\text{nuevo_indice} = (\text{indice} + 1) \bmod M$$

Al llegar al extremo derecho del array de capacidad M , el puntero «da la vuelta» (*wrap-around*) y reutiliza las posiciones libres del principio sin necesidad de desplazar ningún dato existente.

Una cola circular reutiliza las posiciones liberadas al frente sin desplazar el resto de elementos; una cola de doble extremo (*deque*) permite insertar y extraer tanto por el frente como por el final.

En una cola circular sobre un array de tamaño $m = 5$, tras encolar 5 elementos (índices 0-4) y desencolar 2 (frente avanza a 2), encolar un sexto elemento lo coloca en el índice $(4 + 1) \bmod 5 = 0$ —el hueco liberado por el primer desencolado— gracias a la aritmética modular, sin desplazar ningún elemento existente ni fallar por desbordamiento del array subyacente.

Se reproduce con `verification/chapter_results.sh circular_queue_wrap 5`.

```
ALGORITMO: Cola_Circular_Buffer_Lecturas
ESTRUCTURA DE DATOS:
- Buffer: Array estatico de tamano fijo K
- head: Indice del primer elemento a desencolar (inicialmente 0)
- tail: Indice de la siguiente posicion libre de insercion (inicialmente 0)
- count: Numero de elementos activos en el buffer (inicialmente 0)
COMPLEJIDAD: Enqueue  $O(1)$  tiempo, Dequeue  $O(1)$  tiempo, Espacio total  $O(K)$ 

OPERACION ENQUEUE(Lectura_Fastq):
Si count == K entonces:
    Retornar ERROR_BUFFER_OVERFLOW (El buffer esta completamente saturado)

Buffer[tail] <- Lectura_Fastq
tail <- (tail + 1) mod K      # Aritmetica modular: da la vuelta al inicio si tail = K
count <- count + 1
Retornar EXITO
```

```

OPERACION DEQUEUE():
  Si count == 0 entonces:
    Retornar ERROR_BUFFER_UNDERFLOW (El buffer esta vacio)

  Elemento_Extraido <- Buffer[head]
  head <- (head + 1) mod K      # Avanza el frente en tiempo O(1) sin desplazar celdas
  count <- count - 1
  Retornar Elemento_Extraido

```

4.7.2 Anticipo: Colas de prioridad

Cuando el orden de procesamiento no depende del orden de llegada sino de una prioridad biológica (por ejemplo, procesar antes las lecturas con mayor calidad Phred o los alineamientos con mayor puntuación), se utiliza una **cola de prioridad**. Si se implementara con una lista enlazada no ordenada, extraer el máximo exigiría un escaneo lineal $O(n)$ en cada paso. Esta limitación motivará el estudio de las estructuras arborescentes del capítulo siguiente.

Extraer repetidamente el elemento de máxima prioridad de una colección guardada en una lista enlazada simple sin ordenar exige recorrerla entera cada vez para encontrar el máximo: $O(n)$ por extracción, no $O(1)$.

Se reproduce con `verification/chapter_results.sh priority_scan_cost 100`.

4.8 La operación peligrosa: mutación concurrente e índices inválidos

Una de las fuentes más críticas de fallos en bioinformática ocurre al **modificar o redimensionar una estructura de datos mientras se está iterando sobre ella**. En almacenamiento contiguo, insertar o borrar en un bucle desplaza los índices restantes, provocando que se omitan elementos o se procesen duplicados. En almacenamiento enlazado, reasignar punteros sin variables temporales destruye la cadena de acceso a los datos restantes.

Protocolo preventivo en 3 pasos:

1. **Inmutabilidad durante el recorrido:** si se debe filtrar una colección, construir una nueva estructura con los elementos seleccionados en lugar de mutar la original durante la iteración.
2. **Validación rigurosa de límites:** verificar siempre que los índices $0 \leq i < n$ antes de cualquier acceso por posición.
3. **Preservación de punteros temporales:** al manipular nodos enlazados, guardar siempre una referencia al nodo siguiente antes de desacoplar un elemento.

4.9 Actividad de depuración: diagnóstico de errores en punteros

ESTUDIO

Analice el siguiente fragmento para eliminar un nodo en una lista enlazada con tres errores comunes:

```

# Lista simplemente enlazada: cada nodo tiene solo .dato y .siguiente
eliminarNodo(L, x):
  actual <- L.cabeza
  mientras actual.dato != x:      # Fallo 1
    actual <- actual.siguiente    # Fallo 2
  L.cabeza <- actual.siguiente    # Fallo 3

```

Tarea. Corrija los tres fallos y diga cuál provoca un fallo inmediato y visible, y cuáles producen un resultado incorrecto en silencio. Fíjese en que la lista es simplemente enlazada: los nodos no guardan a su antecesor.

Diagnóstico.

1. El bucle lee `actual.dato` sin haber comprobado que `actual` exista. Con una lista vacía falla en la primera

vuelta; con un valor que no está en la lista, falla al llegar al final. Hay que comprobar antes de entrar y en cada vuelta que `actual` no sea nulo.

2. Avanza sin conservar el nodo previo, que es precisamente el que habrá que reenlazar. Sin él no se puede borrar nada que no sea la cabeza. Se arregla con una segunda variable que vaya un paso por detrás.
3. Reenlaza la cabeza siempre, encuentre el elemento donde lo encuentre. Si el nodo buscado está en medio, esta línea descabeza la lista y pierde todo lo anterior sin dar ningún error: la lista queda más corta y el programa sigue. Lo correcto es `previo.siguiente <- actual.siguiente`, y solo tocar la cabeza cuando el nodo encontrado sea la cabeza.

Atención. Los dos primeros fallos rompen el programa y se descubren pronto. El tercero devuelve una lista perfectamente válida, solo que equivocada, y puede sobrevivir meses en producción. Al revisar código sobre estructuras enlazadas, el caso que hay que probar primero no es el habitual: es borrar el primer elemento, borrar el último y borrar uno que no está.

4.10 Síntesis operativa y matriz comparativa de costes

REFERENCIA

Operación	Array	Lista Enlazada	Pila (LIFO)	Cola (FIFO)
Acceso por índice	$O(1)$ directo	$O(n)$ secuencial	—	—
Búsqueda no ordenada	$O(n)$	$O(n)$	—	—
Inserción al inicio	$O(n)$ (desplazamiento)	$O(1)$ (dos punteros)	—	—
Inserción al final	$O(1)$ amortizado	$O(1)$ (con cola)	push: $O(1)$	enqueue: $O(1)$
Borrado al inicio	$O(n)$ (desplazamiento)	$O(1)$	pop: $O(1)$	dequeue: $O(1)$
Localidad de caché	Óptima	Pobre	Variable	Variable

El almacenamiento contiguo (array) regala acceso por índice en $O(1)$ pero penaliza la inserción intermedia; el enlazado (lista, y con él pila y cola) regala inserción/borrado en $O(1)$ por los extremos pero penaliza el acceso. Elegir estructura es elegir qué operación se quiere que sea barata.

4.10.1 Tabla de diagnóstico de fallos en estructuras lineales

Síntoma observado	Causa probable	Comprobación y corrección
Bucle infinito al recorrer lista	Punteros enlazados en ciclo por mala inserción	Revisar que <code>nuevo.siguiente</code> se asigne antes que cabeza
El programa devuelve datos basura o falla	Acceso a índice fuera de límites ($i \geq n$)	Validar que $0 \leq i < n$ antes de consultar el array
Error al desencolar en buffer circular	Punteros no avanzan con módulo	Aplicar $(idx + 1) \bmod M$ en inserción y extracción
Condición $0.1 + 0.2 == 0.3$ da falsa	Imprecisión de coma flotante IEEE 754	Comparar mediante tolerancia $ a - b < \varepsilon$

4.10.2 Tabla de síntesis con preguntas de control

Necesidad del pipeline	Estructura recomendada	Pregunta de control
Acceso aleatorio masivo por posición	Array / Vector contiguo	¿El tamaño es fijo o se conoce de antemano?
Inserciones continuas al inicio/final	Lista enlazada con puntero de cola	¿Necesito recorrer hacia atrás (doble enlace)?
Validación de anidamientos / Deshacer	Pila (<i>Stack</i> LIFO)	¿He controlado la pila vacía antes de pop?
Procesamiento por orden de llegada	Cola (<i>Queue</i> FIFO circular)	¿El buffer de anillo gestiona el <i>wrap-around</i> ?

4.11 Glosario conceptual

REFERENCIA

- **Tipo de Dato Abstracto (TDA):** especificación funcional de una estructura sin fijar su implementación.
- **Almacenamiento contiguo:** disposición consecutiva en memoria con cálculo directo de dirección $O(1)$.
- **Almacenamiento enlazado:** disposición dispersa de nodos conectados mediante punteros explícitos.
- **Array dinámico:** vector contiguo con redimensionamiento geométrico ($O(1)$ amortizado).
- **Pila (Stack):** estructura LIFO con acceso exclusivo por la cima.
- **Cola (Queue):** estructura FIFO con inserción al final y extracción por el frente.
- **Buffer circular:** cola sobre array estático con punteros modulares.
- **Complemento a dos:** codificación binaria de enteros con signo.
- **IEEE 754:** estándar de representación de números en coma flotante.
- **Localidad de caché:** eficiencia de transferencia entre memoria RAM y CPU por contigüidad espacial.

4.12 Conexión con el curso y siguientes pasos

Las estructuras lineales estudiadas en este capítulo son los bloques constructivos sobre los que se desarrollan los temas posteriores:

- El capítulo siguiente da el salto a las estructuras no lineales: árboles binarios de búsqueda, montículos, tablas de dispersión y grafos de interacción biológica.
- El de búsqueda y ordenación compara la eficiencia de lo contiguo y lo enlazado sobre algoritmos concretos.
- Los de probabilidad y modelos de Markov representan vectores de probabilidad y matrices de transición.
- El de alineamiento usa matrices bidimensionales para programación dinámica, con Needleman–Wunsch y Smith–Waterman.
- El de aprendizaje automático estructura tensores y grafos de cómputo.

4.13 Fuentes y reproducibilidad

Este capítulo se apoya en la presentación docente de estructuras de datos de la asignatura, escrita por Álvaro Serrano Navarro. Los costes de las operaciones sobre cada estructura y el análisis amortizado del array dinámico siguen a Cormen y colaboradores [8]; los ejemplos de aplicación biológica, a Compeau y Pevzner [7]; y las implementaciones sobre arrays de Shell, a Allesina y Wilmes [1].

Todos los resultados numéricos se reproducen con `verification/chapter_results.sh` tal como están impresos.

Anexo práctico · Pila y cola con arrays Bash sobre estructura secundaria de ARN

Pregunta, producto y separación de materiales

¿Puede implementar, con nada más que arrays de Bash, una pila que valide el anidamiento de una estructura secundaria de ARN donde un contador falla, y una cola que procese lecturas respetando su orden de llegada? Este laboratorio responde que sí, y lo comprueba ejecutando lo que usted escriba.

El producto individual son dos guiones, `scripts/mi_pila.sh` y `scripts/mi_cola.sh`, con las cuatro operaciones de cada estructura. Le acompañan la tabla de predicciones y resultados, las salidas guardadas, la solución del ejercicio de simular una pila con dos colas y la defensa escrita.

El anexo es autónomo: no necesita datos externos ni red. El generador deja en `herramienta/` el oráculo del capítulo y un comprobador que ejecuta sus dos guiones contra el contrato. Las plantillas de `scripts/` vienen vacías: llevan la cabecera, el contrato en comentarios y nada más.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 preparar el espacio	directorio creado y plantillas a la vista
2 contar a mano una inserción	conteo manual contrastado
3 escribir la pila y la cola	<code>mi_pila.sh</code> y <code>mi_cola.sh</code>
4 validar el ARN con su pila	veredicto propio y contraste con el oráculo
5 simular una pila con dos colas	<code>notas/dos_colas.md</code> con los costes
6 empaquetar y verificar	verificador en <code>ENTREGA_OK</code>

La ruta es única. Escriba su predicción antes de ejecutar cada bloque: es la columna que se evalúa.

1 · Preparar el espacio de trabajo

```
bash --version
./practice_assets/create_workspace.sh TC04_entrega
cd TC04_entrega
ls scripts/
```

El generador crea `notas/` con la tabla de respuestas vacía, `resultados/`, `scripts/` con las dos plantillas y `herramienta/` con el oráculo y el comprobador. Ninguna plantilla trae código escrito: cada una lleva en comentarios el contrato que debe cumplir.

2 · Contar a mano una inserción en array

Antes de ejecutar nada, cuente a mano cuántos desplazamientos exige insertar un elemento en la posición 3 (contando desde 0) de un array de 6 elementos. Anote su predicción en `notas/respuestas.tsv` y compárela:

```
bash herramienta/chapter_results.sh array_insert_shifts 6 3
```

Insertar un elemento en la posición 3 de un array de 6 elementos exige desplazar los 3 elementos que van de la posición 3 a la 5, tantos desplazamientos como elementos hay desde la posición de inserción hasta el final.

Se reproduce con `verification/chapter_results.sh array_insert_shifts 6 3`.

2b • Decisión de diseño: array dinámico frente a lista enlazada

Tarea. Para cada uno de los dos escenarios biológicos siguientes, elija justificadamente entre un array dinámico o una lista enlazada simple, identificando la operación dominante y su coste asintótico:

1. **Escenario A:** Un analizador de lecturas indexadas realiza millones de accesos directos por posición numérica i y recorre secuencialmente la colección de principio a fin.
2. **Escenario B:** Un búfer de secuencias entrantes inserta y retira continuamente registros en la cabeza de la colección en tiempo constante, sin realizar nunca accesos por índice intermedio.

Anote su elección y justificación en `notas/respuestas.tsv`.

3 • Escribir la pila y la cola

Tarea. Implemente `scripts/mi_pila.sh` con `push`, `pop`, `peek` y `esta_vacia`, y `scripts/miCola.sh` con `enqueue`, `dequeue`, `frente` y `esta_vacia`, ambas sobre arrays de Bash. Cada guion lee una operación por línea de la entrada estándar y escribe una línea por operación, según el contrato que la plantilla lleva en sus comentarios.

Las cuatro operaciones de cada estructura no son una lista arbitraria: dos modifican y dos consultan. Sin `esta_vacia` no hay forma de evitar el subdesbordamiento, y sin `peek` o `frente` hay que sacar el elemento para mirarlo y volver a meterlo, que es una manera de perder el orden sin darse cuenta.

Compruebe su avance cuantas veces quiera:

```
bash herramienta/check_estructuras.sh
```

El comprobador no le da las respuestas del anexo: mete secuencias de operaciones en sus guiones y verifica el comportamiento. Que la pila devuelva los elementos en orden inverso al de entrada y la cola en el mismo orden. Que las dos consulten sin sacar. Que las dos avisen cuando están vacías en lugar de devolver una línea en blanco. Y que su pila permita detectar un cierre sin apertura, que es lo que necesitará en el bloque siguiente.

4 • Validar el ARN con su propia pila

Use ahora *su* pila para decidir si estas tres cadenas están bien anidadas: `() []`, `((() y ([[])`. Recorra la cadena carácter a carácter, apile cada apertura, y en cada cierre saque de la pila y compruebe que la apertura recuperada es de la misma clase. Anote su veredicto para las tres antes de ejecutar nada más.

Solo después, contraste con el oráculo del capítulo:

```
bash herramienta/chapter_results.sh stack_validate '([[]]' > resultados/arn_cruzada.txt
bash herramienta/chapter_results.sh stack_validate '() []' > resultados/arn_anidada.txt
bash herramienta/chapter_results.sh stack_validate '((()' > resultados/arn_incompleta.txt
cat resultados/arn_cruzada.txt resultados/arn_anidada.txt resultados/arn_incompleta.txt
```

Sobre una cadena bien anidada y sobre otra a la que le falta un cierre, la pila y el contador dan el mismo veredicto; sobre una cadena con las dos clases cruzadas, solo la pila rechaza.

Anote las tres en `notas/respuestas.tsv`, con su predicción, el resultado y la justificación. Para la tercera cadena, la justificación debe decir qué información conserva la pila que el contador tira, tal como se explicó en la sección [La pila \(Stack\) — Disciplina LIFO](#).

Su cola frente al oráculo

Haga lo mismo con la cola: procese tres lecturas con su implementación, anote el orden de salida y contraste después.

```
bash herramienta/chapter_results.sh queue_fifo_order "lectura1 lectura2 lectura3" > resultados/cola_fifo.txt
cat resultados/cola_fifo.txt
```

El orden en que la cola extrae las lecturas (DEQUEUEED) coincide exactamente con el orden en que se insertaron (ENQUEUEED): la disciplina FIFO no reordena nada.

Se reproduce con `verification/chapter_results.sh queue_fifo_order "lectura1 lectura2 lectura3"`.

5 • Simular una pila con dos colas

Tarea. Disponga de dos colas y de ninguna pila. Escriba en `notas/dos_colas.md` un procedimiento `push` y otro `pop` que, usando solo `enqueue` y `dequeue` sobre esas dos colas, se comporten como una pila. Indique el coste de cada una de las dos operaciones en su solución y explique por qué al menos una de las dos no puede ser $O(1)$.

Es un ejercicio de papel, no de código, y su interés está en la pregunta final: una estructura puede simular a otra, pero la simulación cuesta. Compare después su respuesta con los controles de subdesbordamiento del oráculo, que es lo que su propia implementación tuvo que resolver:

```
bash herramienta/chapter_results.sh stack_pop_empty > resultados/pila_vacia.txt
bash herramienta/chapter_results.sh queue_dequeue_empty > resultados/cola_vacia.txt
cat resultados/pila_vacia.txt resultados/cola_vacia.txt
```

Un `pop` sobre una pila vacía y un `dequeue` sobre una cola vacía se detectan explícitamente (REJECTED) en vez de ejecutarse como si hubiera un elemento.

Se reproduce con `verification/chapter_results.sh stack_pop_empty`.

y `queue_dequeue_empty`.

6 • Verificar la entrega

Escriba antes `notas/defensa.md` con la defensa que se pide más abajo. Después:

```
cd ..
tar -czf TC04_entrega.tar.gz TC04_entrega
sha256sum TC04_entrega.tar.gz
./practice_assets/check_entrega.sh TC04_entrega TC04_entrega.tar.gz
```

El verificador ejecuta sus dos guiones contra el contrato completo, comprueba que la tabla de respuestas tiene al menos cuatro filas con predicción, resultado y justificación, que `resultados/` recoge la validación del ARN y el orden de la cola, y que existen la simulación con dos colas y la defensa. Rechaza el espacio recién generado. Termina en `ENTREGA_OK` o enumera lo que falta. No puntúa.

Rúbrica de calificación ponderada

La evaluación pondera la verificación técnica, el análisis algorítmico y la defensa conceptual sobre 10 puntos, tres de entrega y siete de razonamiento:

- **3,0 puntos, entrega técnica y verificador:** los dos guiones cumplen el contrato completo, incluidos los casos de estructura vacía; el paquete se extrae y el verificador termina en `ENTREGA_OK`.
- **2,5 puntos • Decisión de diseño y predicción de desplazamientos:** Justificación formal de la elección de estructura para los Escenarios A y B (acceso aleatorio vs inserción en cabeza) y cálculo exacto de desplazamientos en array.

- **2,5 puntos, validación por clases y disciplina FIFO:** la pila propia valida las tres cadenas, la justificación de la cadena cruzada identifica qué información conserva la pila, y la simulación con dos colas razona el coste de cada operación.
- **2,0 puntos · Defensa conceptual y límites de la abstracción:** Justificación rigurosa (100–150 palabras) sobre por qué la pila reconoce el lenguaje de anidamiento y por qué el contador pierde el orden relativo, delimitando gramática formal de imposibilidad biológica.

Terminar en ENTREGA_OK acredita que las estructuras funcionan, no que se entienda por qué: una pila correcta con una justificación equivocada de la cadena cruzada pierde el tercer apartado entero.

Lista de autocorrección

- mi conteo manual de desplazamientos coincidió con el del oráculo, o anoté por qué no;
- mis dos guiones consultan sin sacar y avisan cuando la estructura está vacía;
- validé las tres cadenas con mi propia pila antes de mirar el oráculo;
- sé decir qué información conserva la pila que el contador descarta;
- el orden de salida de la cola coincide con el de entrada;
- mi simulación con dos colas indica el coste de push y de pop;
- guardé en resultados/ la salida de cada ejecución que cito;
- el verificador termina en ENTREGA_OK.

Defensa conceptual

En notas/defensa.md, entre 100 y 150 palabras, explique por qué un contador de aperturas y cierres no basta en general para validar un anidamiento de estructuras biológicas, y qué información adicional guarda una pila que el contador descarta. Use el resultado concreto de `( [ ) ]` de su propia entrega.

Fuentes y reproducibilidad

Este anexo no usa datos externos. El oráculo del capítulo es determinista y da el mismo resultado en cualquier máquina. Los costes de las operaciones sobre pila y cola siguen a Cormen y colaboradores [8].

Tablas hash, árboles y grafos

TC-CH05 – Técnicas Computacionales en Biología

Edición 2

Pregunta del tema. Ninguna estructura lineal consigue a la vez búsqueda barata e inserción barata. ¿Qué hay que cambiar para romper ese compromiso, y qué se paga a cambio?

Producto final. Una tabla de dispersión de codones y un recorrido de grafo escritos por usted, y un veredicto razonado sobre qué nodo de una red de interacción es realmente crítico, sostenido por el cálculo y no por la intuición del grado.

Mapa del tema

5.1. El límite de lo lineal	83
5.2. Árboles y estructuras jerárquicas	83
5.3. Árboles Binarios de Búsqueda (BST)	85
5.4. Montículos binarios (<i>Heaps</i>) y colas de prioridad	87
5.5. Tablas Hash y resolución de colisiones	88
5.6. Grafos y redes biológicas	90
5.7. La operación peligrosa: borrado en direccionamiento abierto y ciclos	93
5.8. Actividad de depuración: errores en BST y grafos	94
5.9. Síntesis operativa y tablas de diagnóstico	94

5.10. Glosario conceptual	95
5.11. Conexión con el curso y siguientes pasos	95
5.12. Fuentes y reproducibilidad	96

Cómo usar este tema

El capítulo anterior terminó en un callejón: con estructuras lineales, o la búsqueda es barata o la inserción lo es, pero no las dos. Este capítulo rompe ese compromiso tres veces, de tres maneras distintas, y cada una cobra su precio. El árbol de búsqueda ordena y paga con el rebalanceo. La tabla de dispersión reparte y paga con las colisiones y con la pérdida del orden. El grafo abandona la jerarquía y paga con que casi todas sus preguntas cuestan más.

Al terminar sabrá: recorrer un árbol de búsqueda y explicar los tres casos del borrado, que es donde está la dificultad real; construir una función de dispersión y decir por qué la suya reparte bien o mal; elegir entre encadenamiento y direccionamiento abierto sabiendo qué falla en cada uno; recorrer un grafo en anchura y en profundidad; y, sobre todo, distinguir una métrica de red de una conclusión biológica.

La ruta **esencial** son el árbol de búsqueda con su borrado, la tabla de dispersión con su función y sus colisiones, y los dos recorridos de grafo. La ruta de **estudio** añade montículos, rotaciones, caminos mínimos, métricas de red y el anexo.

5.1 El límite de lo lineal

ESENCIAL En las estructuras de datos lineales del capítulo anterior, todo intento de diseño se enfrenta a un compromiso inevitable: en un array ordenado, la búsqueda binaria es rápida ($O(\log n)$), pero insertar un nuevo dato exige desplazar elementos ($O(n)$); en un array sin ordenar o una lista enlazada, la inserción al final o al inicio es instantánea ($O(1)$), pero buscar un elemento obliga a un escaneo exhaustivo ($O(n)$). Ninguna estructura lineal puede ofrecer búsqueda e inserción rápidas simultáneamente.

En las estructuras lineales, un array ordenado logra búsqueda $O(\log n)$ pero inserción $O(n)$, y un array sin ordenar logra lo contrario: no hay estructura lineal que logre ambas operaciones baratas a la vez.

Las estructuras no lineales y jerárquicas rompen radicalmente este límite:

- Los **árboles binarios de búsqueda balanceados** organizan los datos en jerarquías que permiten buscar, insertar y eliminar elementos en tiempo logarítmico ($O(\log n)$) simultáneamente.
- Las **tablas hash** utilizan funciones matemáticas de dispersión para lograr acceso directo en tiempo constante promedio ($O(1)$), independiente del volumen de datos.
- Los **grafos** generalizan todas las estructuras anteriores para modelar redes biológicas complejas donde lo fundamental son las relaciones e interacciones entre entidades moleculares.

5.2 Árboles y estructuras jerárquicas

Un **árbol** es una estructura de datos no lineal que organiza los elementos en una jerarquía de niveles conectados por relaciones de paternidad/filiación.

5.2.1 Terminología fundamental y anatomía de un árbol

- **Nodo**: cada elemento individual del árbol que contiene un valor y enlaces a otros nodos.
- **Raíz (Root)**: el único nodo superior del árbol que no posee ningún padre.
- **Hoja (Leaf)**: cualquier nodo terminal que no tiene ningún hijo.
- **Subárbol (Subtree)**: cualquier nodo del árbol junto con todos los descendientes que cuelgan de él (que constituye a su vez un árbol completo).
- **Arista (Edge)**: el enlace directo que une un nodo padre con su nodo hijo.
- **Nodo interno**: cualquier nodo que posee al menos un hijo (en árboles generales incluye la raíz no solitaria; en árboles con raíz se distingue habitualmente entre raíz, nodos internos intermedios y hojas).

- **Profundidad de un nodo:** longitud del camino en número de aristas desde la raíz hasta dicho nodo (la raíz tiene profundidad 0).
- **Altura de un árbol (h):** longitud del camino más largo en número de aristas desde la raíz hasta la hoja más profunda (un árbol con un único nodo tiene altura $h = 0$ y 1 nivel de nodos; un árbol vacío se define convencionalmente con $h = -1$).

En un árbol, un nodo es cada elemento; la raíz es el nodo sin padre; una hoja es un nodo sin hijos; un subárbol es un nodo junto con todo lo que cuelga de él; y la altura es la longitud del camino más largo de la raíz a una hoja.

Atención. Contamos la altura en **aristas**, que es la convención habitual en análisis de algoritmos: un árbol de un solo nodo tiene altura 0. Otras fuentes, incluidas las diapositivas de la asignatura, cuentan nodos, con lo que sus valores salen una unidad mayores. Ninguna de las dos es incorrecta; lo que no se puede es mezclarlas dentro de un mismo cálculo, y en este capítulo se usa siempre la primera.

Una arista es la conexión entre un nodo padre y su hijo; un nodo interno es cualquier nodo que no es ni la raíz ni una hoja; la profundidad de un nodo es la longitud del camino en aristas desde la raíz hasta él (profundidad de la raíz: 0), mientras que la altura del árbol es la profundidad máxima de sus hojas. Cada nodo guarda un valor y tantos punteros como hijos tenga —dos en un árbol binario—, la misma idea de puntero de TC-CH04 extendida a más de una dirección por nodo.

5.2.2 Naturaleza recursiva de los árboles

Un árbol se define matemáticamente en términos de sí mismo: es una raíz de la cual cuelgan cero o más subárboles disjuntos. Esta naturaleza recursiva es la razón directa de que casi todos los algoritmos sobre árboles (inserción, búsqueda, recorridos) se implementen de forma natural mediante funciones recursivas con un caso base claro.

Un árbol se define recursivamente: un nodo raíz del que cuelgan cero o más subárboles, cada uno de ellos, a su vez, un árbol. Casi todo algoritmo sobre árboles se escribe de forma recursiva por esta razón.

5.2.3 Taxonomía de árboles binarios

En un **árbol binario**, cada nodo tiene como máximo dos hijos (denominados hijo izquierdo e hijo derecho). La clasificación formal distingue:

1. **Árbol Lleno (*Full*):** cada nodo tiene estrictamente 0 o 2 hijos (nunca exactamente 1).
2. **Árbol Completo (*Complete*):** todos los niveles están completamente llenos, excepto posiblemente el último, que se llena de forma compacta de izquierda a derecha. Esta propiedad permite almacenarlo en un array contiguo sin punteros, indexando los hijos del nodo i en $2i + 1$ y $2i + 2$.
3. **Árbol Perfecto (*Perfect*):** es un árbol lleno donde todas las hojas se encuentran exactamente al mismo nivel de profundidad.
4. **Árbol Degenerado o Sesgado (*Skewed*):** cada nodo padre tiene un único hijo, comportándose de forma idéntica a una lista enlazada.
5. **Árbol Balanceado:** la diferencia de altura entre el subárbol izquierdo y derecho de cualquier nodo está acotada.

Un árbol binario lleno tiene en cada nodo 0 o 2 hijos, nunca exactamente 1. Un árbol completo tiene todos los niveles llenos salvo, como mucho, el último, relleno de izquierda a derecha —forma que permite almacenarlo en un array indexando los hijos del nodo i en $2i + 1$ y $2i + 2$ —. Un árbol perfecto es un árbol lleno donde además todas las hojas están al mismo nivel. Un árbol degenerado o sesgado tiene un único hijo por nodo padre y se comporta como una lista enlazada. Ninguna de estas cuatro propiedades equivale a estar balanceado, aunque un árbol perfecto siempre lo está.

Un árbol perfecto de altura h contiene exactamente:

$$N = \sum_{k=0}^h 2^k = 2^{h+1} - 1 \text{ nodos}$$

Para una altura $h = 3$, el árbol tiene $2^4 - 1 = 15$ nodos.

Un árbol perfecto de altura h tiene exactamente $2^{h+1} - 1$ nodos: cada nivel k (de 0 a h) está completamente lleno con 2^k nodos, y $\sum_{k=0}^h 2^k = 2^{h+1} - 1$. Para $h = 3$, un árbol perfecto tiene $2^4 - 1 = 15$ nodos.

Se reproduce con `verification/chapter_results.sh perfect_tree_nodes 3`.

5.2.4 El poder del balanceo: la altura logarítmica

En un árbol balanceado de n nodos, cada nivel duplica la capacidad del anterior (1, 2, 4, 8, ...). Para almacenar n elementos solo se necesitan:

$$h \approx \log_2(n) \text{ niveles}$$

Dado que las operaciones de búsqueda y recorrido descienden por un único camino desde la raíz hasta una hoja, el número máximo de pasos es proporcional a la altura: **coste** $O(\log n)$.

En un árbol binario balanceado de n nodos, la altura es del orden de $\log_2(n)$: duplicar el número de nodos solo añade un nivel. Como las operaciones cuestan proporcionalmente a la altura, son $O(\log n)$.

5.3 Árboles Binarios de Búsqueda (BST)

Un **Árbol Binario de Búsqueda (BST)** impone una invariante de orden estricta: para cada nodo con clave k , todos los valores en su subárbol izquierdo son menores ($< k$) y todos los valores en su subárbol derecho son mayores ($> k$).

Un árbol binario de búsqueda mantiene, para todo nodo, los valores del subárbol izquierdo menores que él y los del derecho mayores; con esta invariante, buscar, insertar y borrar sobre un árbol balanceado cuestan $O(\log n)$.

```
ALGORITMO: BST_Insercion_y_Busqueda
ESTRUCTURA DE NODO: {clave, valor, izq: puntero, der: puntero}
INVARIANTE BST: Para todo nodo u: todo x en subarbol_izq(u) tiene x.clave < u.clave; todo y en subarbol_der(u) tiene y.clave > u.clave
COMPLEJIDAD: Tiempo  $O(h)$  donde  $h$  es la altura del árbol ( $O(\log n)$  en árbol balanceado,  $O(n)$  en peor caso degenerado)

FUNCION BST_Buscar(raiz, clave_buscada):
    Si raiz == NULL entonces:
        Retornar NO_ENCONTRADO
    Si clave_buscada == raiz.clave entonces:
        Retornar raiz.valor
    Sino si clave_buscada < raiz.clave entonces:
        Retornar BST_Buscar(raiz.izq, clave_buscada)
    Sino:
        Retornar BST_Buscar(raiz.der, clave_buscada)

FUNCION BST_Insertar(raiz, nueva_clave, nuevo_valor):
    Si raiz == NULL entonces:
        Retornar Crear_Nuevo_Nodo(nueva_clave, nuevo_valor)

    Si nueva_clave < raiz.clave entonces:
        raiz.izq <- BST_Insertar(raiz.izq, nueva_clave, nuevo_valor)
    Sino si nueva_clave > raiz.clave entonces:
        raiz.der <- BST_Insertar(raiz.der, nueva_clave, nuevo_valor)
    Sino:
        raiz.valor <- nuevo_valor # Actualizar valor asociado a clave existente

    Retornar raiz
```

5.3.1 Los tres casos del borrado en un BST

Eliminar un nodo en un BST requiere preservar la invariante de orden:

1. **Nodo hoja (sin hijos):** se desvincula directamente ($O(1)$ tras localizarlo).
2. **Nodo con un único hijo:** el padre del nodo a eliminar enlaza directamente con su nieto.

3. **Nodo con dos hijos:** se localiza su **sucesor in-order** (el nodo con menor valor en su subárbol derecho, hallado descendiendo hacia la izquierda). Se copia el valor del sucesor en el nodo a borrar y luego se elimina el sucesor original de su posición (que por construcción tiene a lo sumo un hijo derecho).

Para borrar un nodo de un BST: si es una hoja, se elimina directamente; si tiene un solo hijo, se sustituye por ese hijo; si tiene dos hijos, se busca su sucesor in-order (el valor más pequeño de su subárbol derecho, hallado descendiendo lo más a la izquierda posible), se copia su valor en el nodo a borrar, y se elimina el nodo sucesor de su posición original —que por construcción tiene como mucho un hijo derecho, cayendo siempre en uno de los dos casos anteriores—.

Sobre el árbol de referencia (raíz 5, hijos 3 y 8, hojas 1, 4, 7, 9), borrar la raíz —dos hijos— exige el sucesor in-order: el valor más pequeño del subárbol derecho (raíz 8) se halla descendiendo a la izquierda desde 8, que llega directamente a 7 (hoja, sin hijo izquierdo). Se sustituye 5 por 7 y se desengancha la hoja 7 original; el árbol resultante tiene raíz 7, in-order 1,3,4,7,8,9 y pre-order 7,3,1,4,8,9 —el árbol original con el 5 eliminado y la invariante de orden intacta—.

Se reproduce con `verification/chapter_results.sh bst_delete_two_children`.

5.3.2 El riesgo de degeneración y árboles autobalanceados

Si se insertan datos que ya vienen ordenados (10, 20, 30, 40, 50), un BST ingenuo inserta cada elemento sucesivamente en la rama derecha. El árbol **degenera en una lista enlazada de altura n** , degradando la búsqueda a $O(n)$.

Insertar 10, 20, 30, 40 y 50 en ese orden en un árbol de búsqueda sin rebalanceo produce un árbol degenerado de altura 4: una cadena de cinco nodos con cuatro aristas desde la raíz hasta la última hoja, frente a la altura 2 de un árbol balanceado con esos mismos cinco valores. Buscar el último valor insertado cuesta entonces $O(n)$ y no $O(\log n)$.

Se reproduce con `verification/chapter_results.sh bst_degenerate 5`.

Para garantizar la cota $O(\log n)$ en el peor caso se utilizan árboles autobalanceados mediante **rotaciones**:

Concepto. Antes de ver los tipos, conviene entender por qué una rotación está permitida. Una rotación cambia qué nodo es el padre y cuál el hijo, pero deja intacto el recorrido in-order del árbol, es decir, la secuencia ordenada de sus claves. Esa es exactamente la razón de que sea una operación válida sobre un árbol de búsqueda: la invariante de orden se conserva, mientras que la altura, que era el problema, se reduce. Rebalancear no reordena los datos, solo los recoloca.

- **Árboles AVL:** exigen que las alturas de los subárboles de cada nodo difieran como máximo en 1 ($|\text{balance}| \leq 1$). Imponen un balance estricto que reduce la altura media a costa de requerir rotaciones frecuentes en inserciones y borrados.
- **Árboles Rojo-Negro:** relajan ligeramente la exigencia mediante etiquetas de color (ningún nodo rojo tiene hijo rojo y todo camino raíz- hoja tiene el mismo número de nodos negros). Requieren menos rotaciones en escenarios con mutaciones frecuentes.

Criterio de diseño: Ambos garantizan altura $O(\log n)$; la elección no depende de una superioridad universal, sino del balance entre consultas y actualizaciones en la carga de trabajo real.

Un árbol AVL exige que las alturas de los dos subárboles de cada nodo difieran como mucho en 1 (balance casi perfecto, búsquedas muy rápidas, rotaciones frecuentes en inserción/borrado); un árbol rojo-negro relaja esa exigencia mediante un etiquetado de color (ningún nodo rojo tiene hijos rojos; todo camino raíz- hoja atraviesa el mismo número de nodos negros), suficiente para $O(\log n)$ sin balance perfecto, con menos rotaciones —inserción/borrado más baratos— a cambio de búsquedas algo más lentas que un AVL. La elección sigue el patrón de uso: AVL cuando las búsquedas dominan ampliamente, rojo-negro cuando inserciones y borrados son frecuentes.

5.3.3 Recorridos sobre árboles

1. **In-order (Izquierda → Raíz → Derecha):** en un BST produce los elementos en estricto orden ascendente

($O(n)$).

2. **Pre-order (Raíz → Izquierda → Derecha)**: procesa la raíz antes que sus descendientes; ideal para clonar o serializar árboles.
3. **Post-order (Izquierda → Derecha → Raíz)**: procesa los hijos antes que el padre; ideal para liberar memoria y evaluar expresiones.

El recorrido in-order (izquierda, raíz, derecha) de un BST devuelve sus elementos en orden ascendente; el pre-order (raíz, izquierda, derecha) sirve para copiar o serializar la estructura, no para ordenar; y el post-order (izquierda, derecha, raíz) es el orden para evaluar expresiones y para borrar el árbol sin dejar nodos huérfanos.

Para el BST con raíz 5, hijos 3 y 8, y hojas 1, 4, 7, 9, el pre-order es 5, 3, 1, 4, 8, 7, 9 (no ordenado) y el in-order es 1, 3, 4, 5, 7, 8, 9 (ascendente): confundir un pre-order con «la lista ordenada» es un error de tipo de recorrido, no de implementación.

Se reproduce con `verification/chapter_results.sh bst_traversals`.

5.4 Montículos binarios (*Heaps*) y colas de prioridad

Un **montículo binario** (*Heap*) es un árbol binario completo que cumple la propiedad de heap: en un **Max-Heap**, la clave de cada nodo es mayor o igual que la de sus hijos (el máximo absoluto reside siempre en la raíz).

Un heap es un árbol binario completo donde cada padre es mayor o igual que sus hijos (max-heap); consultar el máximo cuesta $O(1)$ —siempre está en la raíz—, e insertar o extraer el máximo cuesta $O(\log n)$ porque el elemento se reordena a lo largo de la altura, logarítmica, del árbol. Es la implementación eficiente de la cola de prioridad que TC-CH04 dejó pendiente.

Un heap se almacena de forma extraordinariamente compacta en un array contiguo sin necesidad de punteros:

- Hijo izquierdo de i : $2i + 1$.
- Hijo derecho de i : $2i + 2$.
- Padre de i : $\lfloor (i - 1)/2 \rfloor$.

```

heapInsertar(heap, valor):
  heap.append(valor)
  i <- heap.length - 1
  mientras i > 0 y heap[padre(i)] < heap[i]:
    intercambiar(heap[i], heap[padre(i)])
    i <- padre(i)                # Burbujeo ascendente (sift-up)

heapExtraerMax(heap):
  maximo <- heap[0]
  heap[0] <- heap.pop_last()
  i <- 0
  mientras heap[i] tenga un hijo mayor que heap[i]:
    j <- indice_del_hijo_mayor(i)
    intercambiar(heap[i], heap[j])
    i <- j                      # Hundimiento descendente (sift-down)
  devolver maximo

```

Insertando 5, 3, 8, 1, 9 en ese orden en un max-heap representado como array, el heap final es [9, 8, 5, 1, 3] —9 y 8 burbujearon hasta cerca de la raíz, 3 y 1 no necesitaron moverse—. Extraer el máximo de ese heap devuelve 9 y deja [8, 3, 5, 1]: el último elemento (3) ocupa la raíz vacía y se hunde una posición, hasta el nivel en que ya no tiene un hijo mayor que él.

Se reproduce con `verification/chapter_results.sh heap_insert_trace`.

y `heap_extract_max`.

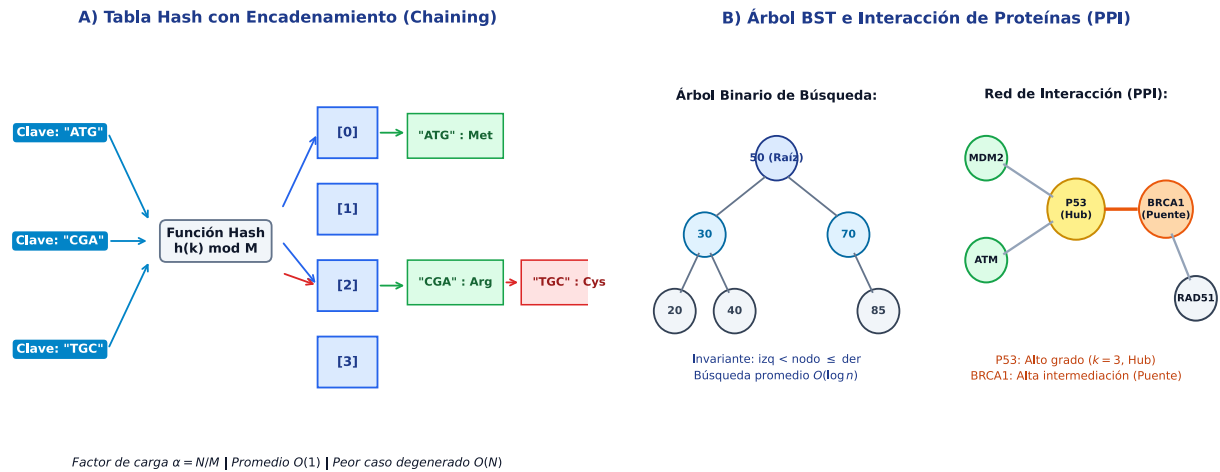


Figura 5.1: A) Tabla Hash con función de dispersión $h(k)$, factor de carga $\alpha = N/M$ y resolución de colisiones por encadenamiento (*chaining*). B) Árbol Binario de Búsqueda (BST) con invariante de orden $izq < nodo \leq der$, y Red de Interacción de Proteínas (PPI) diferenciando nodos de alto grado (*hubs*) frente a nodos puente (*bottlenecks* con alta intermediación). Figura original adaptada de las presentaciones docentes.

5.5 Tablas Hash y resolución de colisiones

Una **tabla hash** es una estructura asociativa que mapea pares *clave* $\rightarrow$ *valor* mediante una función matemática de hash $h(\text{clave})$ que transforma directamente una clave en un índice entero dentro del rango $[0, M - 1]$ de la tabla, logrando un acceso directo en tiempo constante promedio $O(1)$.

Una tabla hash usa una función de hash que transforma una clave directamente en un índice de una tabla, evitando recorrer la colección; la búsqueda por clave cuesta $O(1)$ de media.

5.5.1 De qué está hecha una función de dispersión

ESENCIAL Hasta aquí la función $h(k)$ ha sido una caja negra. Abrirla es sencillo y explica por qué las tablas de dispersión funcionan. Una función útil cumple tres condiciones:

- **Determinista:** la misma clave produce siempre el mismo índice. Sin esto no se podría recuperar nada de lo guardado, que es lo primero que uno esperaría de un almacén.
- **Uniforme:** reparte las claves por todo el rango disponible sin preferir unas casillas sobre otras. Es la condición que mantiene cortas las cadenas de colisión y, con ellas, el coste medio constante.
- **Barata:** calcularla debe costar mucho menos que recorrer la colección. Si no, no compensa.

La familia más común en la práctica es la *dispersión universal*:

$$h_{a,b}(k) = (ak + b) \bmod p \bmod M$$

donde p es un número primo mayor que cualquier clave posible, M es el tamaño de la tabla, y a y b se eligen al azar con $1 \leq a < p$ y $0 \leq b < p$. Elegir a y b al azar es lo que impide que un conjunto concreto de claves degrade la tabla de forma sistemática: ninguna distribución particular de datos, ni nadie que quisiera provocarlo a propósito, puede prepararse contra una función que no conoce de antemano.

Atención. Tomar simplemente el resto de la división, $h(k) = k \bmod M$, funciona en los ejemplos de clase y es lo que usamos aquí por claridad. Pero con M par y claves mayoritariamente pares, esa función llena solo la mitad de la tabla y duplica la longitud de todas las cadenas. La uniformidad no viene de regalo: es una propiedad que hay que buscar, y comprobar.

5.5.2 Colisiones y Factor de Carga (α)

Una **colisión** ocurre cuando dos claves distintas generan el mismo índice ($h(k_1) = h(k_2)$). El rendimiento de la tabla depende del **factor de carga**:

$$\alpha = \frac{N}{M} = \frac{\text{número de elementos insertados}}{\text{capacidad de la tabla}}$$

El $O(1)$ de una tabla hash es un promedio, no una garantía: depende de que la función de hash reparta bien las claves y de que el factor de carga (elementos entre tamaño de tabla) se mantenga bajo. Con una función que reparte mal o una tabla muy llena, la búsqueda degrada hacia $O(n)$ por el alargamiento de las cadenas de colisión.

Se reproduce con `verification/chapter_results.sh hash_load_factor 10 7`.

Cuando el factor de carga supera el umbral $\alpha > 0.7$, la tabla se redimensiona al doble ($2M$) y se ejecuta un **rehasheo completo** de todas las claves existentes.

Una tabla hash de tamaño 5 con encadenamiento, tras insertar 3 claves, tiene factor de carga $3/5=0.6$ (60%), por debajo del umbral 0.7, y no se redimensiona; al insertar una cuarta clave el factor de carga sube a $4/5=0.8$ (80%), supera el umbral y dispara un redimensionamiento a tamaño 10 seguido de un rehash completo de las 4 claves ya insertadas.

Se reproduce con `verification/chapter_results.sh hash_resize`.

5.5.3 Encadenamiento frente a Direccionamiento Abierto

- **Encadenamiento separado (*Chaining*)**: cada cubeta almacena una pequeña lista enlazada con los elementos colisionados.
- **Direccionamiento abierto (*Open Addressing*)**: todos los elementos residen en la propia tabla. Ante una colisión, se sondea secuencialmente:
 - *Sondeo lineal*: prueba índices consecutivos $+1, +2, \dots$ (sufre agrupamiento primario o *clustering*).
 - *Sondeo cuadrático*: incrementa el paso con potencias cuadráticas $+1^2, +2^2, +3^2, \dots$
 - *Doble Hashing*: calcula el salto mediante una segunda función de hash $h_2(k)$.

El direccionamiento abierto resuelve colisiones sondeando otras posiciones de la propia tabla en vez de encadenar: el sondeo lineal prueba el siguiente índice ($+1, +2, \dots$), el cuadrático incrementa cuadráticamente ($+1, +4, +9, \dots$), y el doble hashing calcula el incremento con una segunda función de hash. El sondeo lineal tiene el mejor comportamiento de caché pero sufre agrupamiento primario (las colisiones se acumulan en bloques consecutivos, aumentando la probabilidad de nuevas colisiones para claves no relacionadas); el doble hashing evita el agrupamiento a costa de más cálculo y peor caché; el cuadrático queda entre ambos. A diferencia del encadenamiento, no necesita memoria extra para listas, pero satura antes y su borrado es más delicado porque vaciar una casilla sin más rompería la cadena de sondeo de claves posteriores.

Insertando las claves 12, 3, 17 y 8, en ese orden, en una tabla de tamaño 5 con sondeo lineal y hash `clave%5`: 12 y 3 caen directamente en sus índices naturales (2 y 3, sin sondeo); 17 colisiona en el índice 2 (ocupado por 12), sondea 3 (ocupado por 3) y se coloca en 4 tras 2 sondeos; 8 colisiona en el índice 3 (ocupado por 3), sondea 4 (ocupado ahora por 17) y, al llegar al final de la tabla, da la vuelta (*wrap-around*) hasta el índice 0, libre, tras 2 sondeos. La clave 8, que no colisiona directamente con 12 ni con 17 en su hash original, termina desplazada por el efecto en cadena del agrupamiento: una colisión entre otras dos claves puede desplazar a una tercera que no compartía su índice natural con ninguna de ellas, algo que no ocurre con encadenamiento, donde cada cubeta es independiente de las demás.

Se reproduce con `verification/chapter_results.sh open_addressing_linear`.

5.5.4 Caso bioinformático: Traducción de codones de ARNm

El código genético se modela de forma natural como una tabla hash donde la clave es un triplete nucleotídico (codón) y el valor es el aminoácido correspondiente.

Para traducir la secuencia AUGAAUCUGCCU:

1. Con marco 0 (inicio 0, paso 3): se leen los codones no solapados AUG (Met), AAU (Asn), CUG (Leu) y CCU (Pro).
2. Con marco 1 (inicio 1, paso 3): empezando en la posición 1 y avanzando de tres en tres salen UGA, que es un **codón de parada prematuro**, AUC (isoleucina) y UGC (cisteína).
3. Con marco 2 (inicio 2, paso 3) la lectura vuelve a ser otra: GAA, UCU y GCC.

Tres marcos, tres proteínas incompatibles a partir de la misma secuencia de nucleótidos. Un desplazamiento de marco (*frameshift*) causado por una inserción o un borrado de una sola base no cambia una parte del mensaje: cambia el mensaje entero a partir de ese punto, y por eso suele ser mucho más grave que una sustitución.

Aclaración metodológica: Avanzar con paso 1 desde el origen generaría todos los tripletes solapados (k -mers con $k = 3$: AUG, UGA, GAA, ...), operación computacional útil para indexado genómico pero conceptualmente distinta de un marco de lectura biológico.

Sobre AUGAAUCUGCCU, el marco 0 da AUG-AAU-CUG-CCU, es decir metionina, asparagina, leucina y prolina; el marco 1 da UGA-AUC-UGC, una parada prematura seguida de isoleucina y cisteína; y el marco 2 da GAA-UCU-GCC, glutámico, serina y alanina.

Se reproduce con `verification/chapter_results.sh reading_frames AUGAAUCUGCCU`.

Se reproduce con `verification/chapter_results.sh translate_codons AUGAAUCUGCCU 3`.

Leer codones con un paso de 1 en vez de 3 produce lecturas solapadas y fuera de marco; sobre AUGAAUCUGCCU el segundo codón leído sería UGA, un codón de parada real, en vez del AAU (Asparagina) que da la lectura correcta con paso 3.

Se reproduce con `verification/chapter_results.sh translate_codons AUGAAUCUGCCU 1`.

Operación	Lista sin ordenar	Array ordenado	BST balanceado	Tabla Hash
Búsqueda	$O(n)$	$O(\log n)$	$O(\log n)$	$O(1)$ promedio
Inserción	$O(1)$	$O(n)$	$O(\log n)$	$O(1)$ promedio
Borrado	$O(n)$	$O(n)$	$O(\log n)$	$O(1)$ promedio
Recorrido ordenado	No soportado	$O(n)$	$O(n)$ (in-order)	No soportado

Si solo se necesita buscar/insertar por clave, la tabla hash es la mejor opción; pero si además hace falta recorrer los datos en orden o consultar rangos, la tabla hash no sirve —no guarda orden— y el árbol balanceado es la elección, porque combina $O(\log n)$ con recorrido in-order ordenado.

5.6 Grafos y redes biológicas

Un **grafo** $G = (V, E)$ es la estructura más general en computación, formada por un conjunto de vértices o nodos V y un conjunto de aristas E que conectan pares de vértices. Modela interacciones físicas proteína-proteína (PPI), redes de regulación génica, rutas metabólicas y grafos de ensamblaje genómico (de Bruijn).

Un grafo modela entidades (nodos) y sus relaciones (aristas); las listas y los árboles son casos particulares de grafo. Se usa cuando lo esencial del problema son las relaciones entre entidades, no una secuencia ni una jerarquía.

Un grafo $G=(V,E)$ puede ser dirigido o no dirigido según si sus aristas tienen un solo sentido, y ponderado si sus aristas llevan un peso numérico; es disperso si $|E|$ es mucho menor que $|V|^2$ y denso si se acerca a ese máximo. Las redes biológicas suelen ser dispersas.

5.6.1 Representación: Matriz de adyacencia frente a Lista de adyacencia

- **Matriz de adyacencia** ($|V| \times |V|$): tabla bidimensional donde $M[u][v]$ almacena la existencia o peso de la arista. Ofrece comprobación de adyacencia instantánea en tiempo constante $O(1)$, pero requiere memoria

cuadrática $O(|V|^2)$ independientemente de las aristas reales. Es la estructura ideal para **grafos densos** (por ejemplo, una matriz de correlación completa entre 200 muestras clínicas, donde la matriz es compacta y se consultan pares continuamente).

- **Lista de adyacencia:** cada nodo mantiene un vector con sus vecinos directos. Consume memoria proporcional a los datos reales $O(|V| + |E|)$, siendo la estructura óptima para **redes biológicas dispersas** (como redes de interacción proteína-proteína con millones de nodos y pocas conexiones por nodo, donde una matriz reservaría un volumen prohibitivo de memoria prácticamente vacía).

Una red dispersa de un millón de nodos representada con matriz de adyacencia ocuparía del orden de 10^{12} celdas, independientemente de cuántas aristas tenga realmente; representada con lista de adyacencia ocupa $O(V + E)$, proporcional a las aristas reales: para redes dispersas, la matriz es inviable en memoria y la lista es la elección correcta.

Se reproduce con `verification/chapter_results.sh matrix_vs_list_memory 1000 5000`.

5.6.2 Algoritmos de recorrido: BFS y DFS

- **Búsqueda en Anchura (BFS — *Breadth-First Search*):** explora el grafo concéntricamente nivel a nivel empleando una **cola FIFO**. Calcula el camino más corto en número de aristas. Coste: $O(V + E)$.
- **Búsqueda en Profundidad (DFS — *Depth-First Search*):** se adentra por una rama hasta el fondo antes de retroceder, empleando una **pila LIFO** o recursión. Detecta ciclos y componentes conexas. Coste: $O(V + E)$.

```
BFS(grafo, origen):
    visitados <- {origen}
    cola <- [origen]
    orden <- []
    mientras cola no este vacia:
        nodo <- cola.pop_front()
        orden.append(nodo)
        para cada vecino en grafo.vecinos(nodo):
            si vecino no esta en visitados:
                visitados.add(vecino)      # Marcar al encolar para evitar duplicados
                cola.append(vecino)
    devolver orden
```

BFS recorre un grafo por niveles usando una cola FIFO; DFS se adentra por una rama antes de retroceder, usando una pila o recursión. Sobre el mismo grafo, el orden de visita de BFS y de DFS es distinto; ambos cuestan $O(V + E)$.

Se reproduce con `verification/chapter_results.sh graph_bfs_dfs`.

5.6.3 Caminos mínimos en grafos ponderados

- **Algoritmo de Dijkstra:** calcula el camino de mínimo coste en grafos con pesos **no negativos** utilizando una cola de prioridad (Heap) para expandir siempre el nodo con menor distancia acumulada ($O((V + E) \log V)$).
- **Algoritmo de Bellman-Ford:** soporta aristas con pesos negativos relajando todas las aristas $V - 1$ veces ($O(V \cdot E)$).

BFS encuentra el camino con menos aristas, que solo coincide con el de menor coste si el grafo no tiene pesos (o todos valen lo mismo); con pesos, un camino de más aristas puede tener menor coste total, y BFS no lo detecta. El algoritmo adecuado con pesos no negativos es Dijkstra, que usa una cola de prioridad para expandir siempre el nodo de menor distancia acumulada.

Dijkstra solo funciona correctamente con pesos no negativos: su estrategia da por cerrado un nodo en cuanto lo expande, lo que puede dar resultados incorrectos si hay aristas de peso negativo. Para grafos con pesos negativos se usa Bellman-Ford, que relaja las aristas $V - 1$ veces.

5.6.4 Métricas de centralidad y análisis topológico

1. **Grado (k):** número de conexiones directas de un nodo. En redes dirigidas se desglosa en **grado de entrada** (*in-degree*) y **grado de salida** (*out-degree*). En redes de regulación génica, un gen con muchas salidas y pocas entradas es candidato a **factor de transcripción maestro**.
2. **Centralidad de intermediación (*Betweenness Centrality*):** proporción de caminos más cortos de toda la red que transitan obligatoriamente por un nodo dado.
3. **Centralidad de cercanía (*Closeness Centrality*):** inverso de la suma de distancias geodésicas desde el nodo hasta todos los demás.

ESTUDIO Las tres anteriores describen un nodo. Otras tres describen la red entera:

- **Grado medio**, que mide la densidad: cuántas interacciones tiene por término medio cada proteína.
- **Distancia media y diámetro**. El diámetro es la mayor de todas las distancias mínimas entre pares de nodos. Un diámetro corto indica que cualquier perturbación se propaga en pocos pasos, lo que importa al razonar sobre el efecto de un fármaco.
- **Coficiente de agrupamiento**, la proporción de vecinos de un nodo que además son vecinos entre sí. Un valor alto sugiere módulos funcionales, complejos que actúan juntos; uno bajo sugiere una red más parecida a una aleatoria.

Atención. Las tres son descripciones, no explicaciones, y vale para ellas lo mismo que para las anteriores: describen la topología observada, que depende tanto de la biología como de qué interacciones se han medido y cuáles no. Una proteína muy estudiada acumula aristas porque se ha buscado en ella, no necesariamente porque sea más central.

El grado de un nodo cuenta sus conexiones directas; la centralidad de intermediación cuenta la proporción de caminos más cortos que pasan por él; la centralidad de cercanía es el inverso de la suma de distancias a todos los demás nodos. Estas tres métricas pueden señalar nodos distintos como «el más importante» según qué relación midan.

```

ALGORITMO: Calculo_Centralidad_Grado_Red_Proteica
ENTRADA:
  - Grafo_PPI: Lista de adyacencia de la red de interaccion proteica con V proteinas y E interacciones
SALIDA:
  - Vector_Grados: Mapeo de cada proteina u a su grado deg(u)
  - Lista_Hubs: Proteinas cuyo grado supera el umbral k_hub (ej. media + 2 desviaciones estandar)
COMPLEJIDAD: Tiempo  $O(V + E)$  sobre listas de adyacencia, Espacio auxiliar  $O(V)$ 

1. INICIALIZACION:
  Para cada proteina v en V:
    Grado[v] <- 0

2. CONTEO DE ARISTAS:
  Para cada proteina u en V:
    Para cada vecino w en Grafo_PPI.Adyacentes(u):
      Grado[u] <- Grado[u] + 1  # Cuenta de interacciones directas

3. DETECCION DE PROTEINAS HUB:
  Grado_Medio <- Suma(Grado) / |V|
  Lista_Hubs <- Conjunto_Vacio()

  Para cada proteina u en V:
    Si Grado[u] >= 2 * Grado_Medio entonces:
      Agregar (u, Grado[u]) a Lista_Hubs

4. RETORNO:
  Retornar (Grado, Lista_Hubs)

```

En un grafo dirigido, el grado se desdobra en grado de entrada (aristas que llegan al nodo) y grado de salida (aristas que salen de él). En una red de regulación génica dirigida, el grado de salida de un gen mide a cuántos

otros regula y el grado de entrada mide cuántos reguladores lo controlan; un gen con grado de salida alto y grado de entrada bajo es candidato a regulador maestro, no a objetivo regulado, una distinción que el grado no dirigido no puede establecer.

5.6.5 El experimento del nodo puente

Considere una red compuesta por dos módulos triangulares ($\{1, 2, 3\}$ y $\{5, 6, 7\}$) unidos exclusivamente a través del nodo 4:

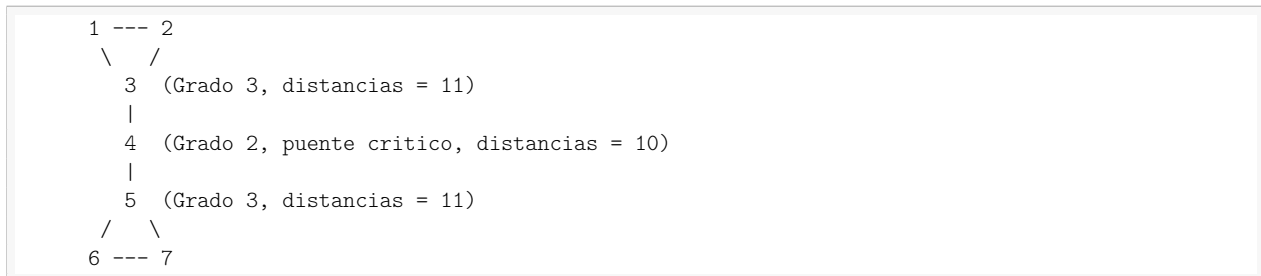


Figura 5.2: Estructura de dos comunidades triangulares unidas por el nodo puente 4. El nodo 4 tiene menor grado pero mayor centralidad de intermediación y cercanía. Figura original.

En un grafo de dos triángulos unidos solo por el nodo 4, eliminar el nodo 3 (grado 3, el más alto) separa la red en componentes de tamaño 2 y 4; eliminar el nodo 4 (grado 2, menor) la separa en dos componentes iguales de tamaño 3 y 3. Ambas eliminaciones fragmentan la red, pero la de mayor impacto estructural —partirla en dos mitades— es la del nodo de menor grado: el grado por sí solo no identifica qué nodo sostiene la conectividad entre las dos comunidades.

Se reproduce con `verification/chapter_results.sh bridge_node`.

En el grafo de dos triángulos unidos por el nodo 4, la suma de distancias del nodo 4 al resto de los seis nodos es 10 (distancias 2,2,1,1,2,2); la del nodo 3 es 11 (distancias 1,1,1,2,3,3). El nodo 4, pese a tener menor grado (2 frente a 3), tiene mayor centralidad de cercanía —menor suma de distancias— porque es el único paso obligado hacia el triángulo opuesto: grado y cercanía pueden coincidir en señalar al mismo nodo como importante, o no, según qué relación midan.

Se reproduce con `verification/chapter_results.sh closeness centrality`.

5.6.6 Cautela biológica: las métricas son hipótesis

Identificar una proteína con alta centralidad (*hub* o *bottleneck*) en una red de interacción sugiere que podría ser biológicamente esencial, pero **no constituye una prueba definitiva**. Requiere siempre validación experimental en el laboratorio húmedo (*knockout*, ensayos funcionales).

Que un nodo tenga alta centralidad en una red biológica lo hace candidato a ser funcionalmente importante, pero no lo demuestra: confirmar que una proteína es esencial exige evidencia biológica (experimentos, anotación funcional), no solo la topología de la red. Una lista de «nodos más centrales» no es, por sí sola, una lista de «elementos biológicamente más importantes».

5.7 La operación peligrosa: borrado en direccionamiento abierto y ciclos

Dos errores críticos en estructuras no lineales:

1. **Borrado ingenuo en direccionamiento abierto:** eliminar una clave dejando la celda como «vacía» rompe la cadena de sondeo de claves posteriores, haciéndolas ilocalizables. Se debe marcar la celda con un centinela especial (*tombstone* o «borrado»).
2. **Recorrido de grafos sin marcado de visitados:** en grafos con ciclos, omitir el conjunto de visitados provoca bucles infinitos y desbordamiento de pila.

Protocolo preventivo en 3 pasos:

1. Marcar siempre los nodos como visitados en el momento exacto de encolarlos en BFS o apilarlos en DFS.
2. Utilizar centinelas de borrado en tablas hash con direccionamiento abierto.
3. Monitorizar el factor de carga α antes de cada inserción masiva.

5.8 Actividad de depuración: errores en BST y grafos

Analice el siguiente código con tres errores conceptuales:

```

buscarGrafo(grafo, origen, destino):
    cola <- [origen]                # Fallo 1
    mientras cola no este vacia:
        actual <- cola.pop_front()
        si actual == destino: devolver True
        para cada vecino en grafo[actual]: # Fallo 2
            cola.append(vecino)          # Fallo 3
    devolver False

```

Tarea. Localice los tres fallos y clasifíquelos: cuál impide que el algoritmo termine, cuál lo rompe con un error visible y cuál devuelve un resultado incorrecto en silencio.

Diagnóstico.

1. No se comprueba que *origen* exista en el grafo. Si no está, la primera vuelta intenta consultar sus vecinos y falla, o peor, devuelve falso como si el destino fuera inalcanzable cuando en realidad la pregunta estaba mal planteada.
2. Se accede a *grafo[actual]* sin contemplar un nodo sin vecinos registrados. En muchos lenguajes eso lanza un error; en otros devuelve algo vacío y el fallo pasa desapercibido, que es lo peor de las dos opciones.
3. Se encola sin llevar un conjunto de visitados. Con un ciclo en el grafo, y las redes biológicas están llenas de ciclos, el bucle no termina nunca: los mismos nodos entran una y otra vez en la cola.

Concepto. El tercero es el que define el algoritmo. Un recorrido en anchura sin conjunto de visitados no es un recorrido en anchura lento: es un programa que no termina. La estructura auxiliar no es un detalle de eficiencia, es parte de la corrección.

5.9 Síntesis operativa y tablas de diagnóstico**5.9.1 Tabla de diagnóstico de síntomas en estructuras no lineales**

Síntoma observado	Causa probable	Comprobación y corrección
Búsqueda en BST tarda $O(n)$ en vez de $O(\log n)$	Datos insertados en orden (árbol degenerado)	Rebalancear mediante AVL o Árbol Rojo-Negro
Búsqueda en tabla hash se ralentiza	Factor de carga $\alpha > 0.7$ o mala función de hash	Duplicar capacidad y ejecutar rehash completo
Clave existente no se encuentra tras borrado	Rotura de cadena en direccionamiento abierto	Usar centinelas de borrado (<i>tombstones</i>)
Recorrido en grafo no termina (bucle infinito)	Falta de marcado de visitados ante ciclos	Registrar <code>visitados.add(nodo)</code> al encolar

5.9.2 Tabla de síntesis con preguntas de control

Requisito del sistema	Estructura recomendada	Pregunta de control
Búsqueda exacta por clave $O(1)$	Tabla Hash	¿El factor de carga se mantiene < 0.7 ?
Búsqueda rápida y recorrido ordenado	Árbol Binario de Búsqueda (AVL/Rojo-Negro)	¿Se prevé inserción de datos ordenados?
Acceso continuo al elemento prioritario	Montículo binario (<i>Heap</i>)	¿Se necesita un Max-Heap o un Min-Heap?
Modelado de interacciones moleculares	Grafo con listas de adyacencia	¿La red es dirigida o ponderada?

5.10 Glosario conceptual

- **BST (Binary Search Tree):** árbol con invariante $izq < k < der$.
- **Árbol AVL:** BST autobalanceado con diferencia de alturas ≤ 1 .
- **Heap (Montículo):** árbol completo con propiedad de orden padre/hijo.
- **Tabla Hash:** estructura asociativa directa mediante función de dispersión.
- **Factor de carga (α):** cociente N/M entre elementos y capacidad.
- **Encadenamiento:** resolución de colisiones mediante listas por cubeta.
- **Direccionamiento abierto:** resolución de colisiones mediante sondeo interno.
- **BFS (Breadth-First):** recorrido por niveles con cola FIFO.
- **DFS (Depth-First):** recorrido en profundidad con pila LIFO.
- **Betweenness:** centralidad por proporción de caminos más cortos.

5.11 Conexión con el curso y siguientes pasos

Distinción conceptual: Árbol Binario de Búsqueda frente a Árbol Filogenético.

Aunque un BST y un árbol filogenético comparten algoritmos generales de recorrido jerárquico, sus principios matemáticos y significados biológicos son completamente distintos:

- En un **BST**, la ramificación izquierda/derecha responde a un orden total de claves alfanuméricas ($izq < k < der$) para optimizar la búsqueda de datos.
- En un **árbol filogenético**, la topología representa una hipótesis de parentesco y divergencia evolutiva; las ramas cuantifican distancias o tiempos biológicos y los nodos internos modelan ancestros comunes hipotéticos, sin ninguna relación de orden de búsqueda.

Reutilizar una técnica de recorrido no autoriza a interpretar un árbol filogenético como una estructura indexada de búsqueda.

Continuidad con los temas posteriores.

Las estructuras no lineales de este capítulo sustentan la ordenación eficiente con montículos en TC-CH06, la cuantificación de información en TC-CH07, los modelos probabilísticos en TC-CH08, el alineamiento sobre grafos dirigidos acíclicos en TC-CH09 y el aprendizaje profundo en TC-CH10.

- En **TC-CH06**, utilizaremos montículos (*heaps*) para implementar el algoritmo de ordenación Heapsort y compararemos estructuras contiguas y enlazadas.
- En **TC-CH07** y **TC-CH08**, las tablas hash y matrices estructurarán el conteo de frecuencias, la entropía y los modelos ocultos de Markov (HMMs).
- En **TC-CH09**, el alineamiento de secuencias se formulará como la búsqueda del camino óptimo en grafos dirigidos acíclicos (DAGs) mediante programación dinámica.

- En **TC-CH10**, organizaremos representaciones matriciales y grafos computacionales para redes neuronales artificiales.

5.12 Fuentes y reproducibilidad

Este capítulo se apoya en la presentación docente de estructuras de datos de la asignatura, escrita por Álvaro Serrano Navarro. Los árboles de búsqueda, los montículos, la dispersión universal y los recorridos de grafo siguen a Cormen y colaboradores [8]; la traducción de codones y el uso de tablas para representar el código genético, a Compeau y Pevzner [7]; y el análisis de redes de interacción, a Allesina y Wilmes [1].

Todos los resultados se reproducen con `verification/chapter_results.sh` tal como están impresos, con sus argumentos.

Anexo práctico · Tabla hash de codones y recorrido de un grafo de interacción

Pregunta, producto y separación de materiales

¿Puede traducir una secuencia con una tabla hash de codones, recorrer un grafo pequeño con BFS y DFS, y decidir con datos —no con la intuición del grado— qué nodo es realmente crítico para la conectividad de una red? Este laboratorio responde que sí, sin necesitar ningún dato externo.

El producto individual son dos guiones escritos por usted: `scripts/mi_tabla_codones.sh`, una tabla de dispersión de codones con función propia que traduce en los tres marcos de lectura, y `scripts/mi_bfs.sh`, un recorrido en anchura sobre la red de interacción. Le acompañan la tabla de predicciones y resultados, las salidas guardadas y la defensa escrita.

El anexo es autónomo: usa la red sintética de `data/` como entrada de solo lectura y no necesita nada más. El generador deja en `herramienta/` el oráculo del capítulo y un comprobador que ejecuta sus guiones con entradas que no aparecen impresas en ningún sitio.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 preparar el espacio	directorio creado y datos verificados
2 escribir la tabla de codones	<code>mi_tabla_codones.sh</code> con función propia
3 escribir el recorrido en anchura	<code>mi_bfs.sh</code> sobre la red
4 grado frente a fragmentación	veredicto sobre el nodo crítico
5 búsquedas fallidas	dos ausencias detectadas explícitamente
6 empaquetar y verificar	verificador en <code>ENTREGA_OK</code>

La ruta es única. Escriba su predicción antes de ejecutar cada bloque.

1 · Preparar el espacio de trabajo

```
bash --version
./practice_assets/create_workspace.sh TC05_entrega
cd TC05_entrega
cd datos && sha256sum -c manifest.sha256 && cd ..
```

El generador crea `datos/` con la red de interacción y su manifiesto, `scripts/` con dos plantillas vacías, `notas/respuestas.tsv` con la cabecera, `resultados/` y `herramienta/` con el oráculo y el comprobador. Las plantillas llevan su contrato en comentarios y ningún código.

2 • Escribir la tabla de codones

Tarea. Implemente `scripts/mi_tabla_codones.sh`, que reciba una secuencia de ARN y la lea en los tres marcos, imprimiendo para cada uno los codones y los aminoácidos. Use una función de dispersión propia sobre el triplete, no una tabla asociativa del lenguaje indexada por el codón entero, y declare al final el tamaño de su tabla y cuántas colisiones produjo su función.

La parte interesante no es traducir: es la función de dispersión. Tiene que convertir tres letras en un índice, y ahí es donde se ve si reparte o amontona. Pruebe primero con algo ingenuo, como sumar los valores de las tres bases, y cuente las colisiones; después con una suma ponderada por la posición, y vuelva a contarlas. La diferencia entre las dos es el contenido de esta tarea.

Comprueba. Prediga, antes de ejecutar, qué le dará el marco 1 sobre la secuencia del capítulo. Es exactamente el punto donde una lectura descuidada se equivoca: el marco 1 no es «lo mismo desplazado una letra», es una secuencia de codones completamente distinta, y aquí lo va a ver con su propio programa.

3 • Escribir el recorrido en anchura

Tarea. Implemente `scripts/mi_bfs.sh`, que reciba un nodo de origen, lea la red de datos/`tc05_red_interaccion.tsv` y emita el orden de visita en anchura y cuántos nodos visitó. Recorra los vecinos de cada nodo en orden numérico creciente, para que su salida sea comparable con la de cualquier otra implementación correcta.

Recuerde el diagnóstico de la actividad de depuración del capítulo: sin conjunto de visitados, este programa no termina. La red tiene ciclos.

Compruebe los dos guiones cuantas veces necesite:

```
bash herramienta/check_scripts.sh
```

El comprobador usa una secuencia y unos orígenes que no aparecen en el enunciado, así que no basta con acertar el caso de ejemplo.

4 • Grado frente a fragmentación

Con su recorrido ya escrito, la pregunta interesante es cuál de los siete nodos es realmente crítico. Prediga primero cuál cree que lo es, mirando la red, y anótelos. Después mida:

```
bash herramienta/chapter_results.sh graph_all_degrees > resultados/grados.txt
bash herramienta/chapter_results.sh bridge_node > resultados/puente.txt
cat resultados/grados.txt resultados/puente.txt
```

Los nodos 3 y 5 tienen el grado más alto (3) del grafo puente, pero `bridge_node` confirma que eliminar el nodo 4 (grado 2) es lo que parte la red en dos mitades iguales: el grado más alto no identifica al nodo estructuralmente más crítico.

Se reproduce con `verification/chapter_results.sh graph_all_degrees`.

Anote en `notas/respuestas.tsv` qué nodo predijo, cuál resultó ser y por qué. La justificación debe explicar qué mide el grado y qué mide la fragmentación: son dos preguntas distintas sobre el mismo nodo, y en esta red dan respuestas distintas.

5 • Búsquedas fallidas

Una estructura de búsqueda tiene que saber decir «no está», y decirlo de forma distinguible de «está y vale cero». Compruébelo en las dos:

```
bash herramienta/chapter_results.sh hash_lookup_miss GGG > resultados/fallo_hash.txt
bash herramienta/chapter_results.sh bst_search_miss 5 99 > resultados/fallo_bst.txt
cat resultados/fallo_hash.txt resultados/fallo_bst.txt
```

Comprueba. Compruebe también qué hace *su* tabla de codones con un codón que no está en ella. Si devuelve una cadena vacía en lugar de un aviso, tiene el mismo problema que el capítulo describe: un resultado que parece un dato.

Buscar el codón GGG (ausente de la tabla) informa explícitamente MISS en vez de devolver un valor arbitrario; buscar el valor 99 en el BST de 5 nodos degenerado recorre toda la cadena (5 pasos) y termina en MISS, no en un acierto accidental.

Se reproduce con `verification/chapter_results.sh hash_lookup_miss GGG`.

6 • Verificar la entrega

Escriba antes `notas/defensa.md`. Después:

```
cd ..
tar -czf TC05_entrega.tar.gz TC05_entrega
sha256sum TC05_entrega.tar.gz
./practice_assets/check_entrega.sh TC05_entrega TC05_entrega.tar.gz
```

El verificador ejecuta sus dos guiones con entradas que no ha visto, comprueba que la red de partida no se ha modificado, que la tabla de respuestas tiene al menos cuatro filas con predicción, resultado y justificación, que `resultados/` recoge el experimento de grado y fragmentación, y que existe la defensa. Rechaza el espacio recién generado. No puntúa.

Rúbrica de calificación ponderada

La evaluación pondera la verificación técnica, el análisis de estructuras y la defensa metodológica sobre 10 puntos, tres de entrega y siete de razonamiento:

- **3,0 puntos • Entrega técnica y verificador (DELIVERY_OK):** Entrega íntegra del paquete comprimido `.tar.gz`, formato de `notas/respuestas.tsv` y superación de `check_practice_delivery.sh`.
- **2,5 puntos • Análisis formal de BST y resolución de colisiones en Hash:** Predicción de altura en árbol degenerado vs balanceado y comprobación explícita de búsquedas fallidas (MISS) en tabla hash y BST sin falsos positivos.
- **2,5 puntos • Topología de grafos, recorridos y centralidad:** Cálculo de grados frente a intermediación en el grafo puente y contraste sistemático entre órdenes de exploración BFS (cola FIFO) y DFS (pila LIFO).
- **2,0 puntos • Defensa conceptual y cautela biológica:** Justificación rigurosa (100–150 palabras) sobre por qué el grado local de un nodo no determina su criticidad funcional ni su papel como nodo puente, delimitando evidencia topológica de esencialidad biológica.

Terminar en `ENTREGA_OK` acredita que los guiones funcionan, no que el razonamiento sea correcto.

Lista de autocorrección

- Mi predicción de altura del BST con 8 valores coincidió con el resultado, o anoté por qué no.
- Las dos búsquedas fallidas terminaron en MISS explícito, verificado.
- La tabla de grados está completa y contrastada con el veredicto de `bridge_node`.
- Los órdenes de BFS y DFS están anotados y son distintos entre sí.
- `check_practice_delivery.sh` termina con `DELIVERY_OK`.

Defensa conceptual

En 100–150 palabras, explique por qué el grado de un nodo no basta para decidir cuál es más crítico para la conectividad de una red, usando el resultado concreto del grafo puente de su propia entrega (tamaños de componente al eliminar el nodo 3 frente al nodo 4).

Fuentes y reproducibilidad

Este anexo se apoya en los problemas de estructuras no lineales y redes del repositorio docente y se valida al 100% mediante `verification/chapter_results.sh`.

Búsqueda, ordenación y evaluación experimental

TC-CH06 – Técnicas Computacionales en Biología

Edición 2

Pregunta del tema. Ordenar un millón de registros cuesta un trabajo que no había que hacer para responder una sola pregunta. ¿Cuándo compensa pagarlo por adelantado, y cuándo es tirar el tiempo?

Producto final. Un conteo a mano de las comparaciones de la ordenación por selección, contrastado con la medición, y un veredicto razonado sobre el escalado real al duplicar la entrada.

Mapa del tema

6.1. Las dos operaciones más fundamentales	99
6.2. Algoritmos de Búsqueda	100
6.3. Algoritmos de Ordenación	102
6.4. Límite inferior teórico de la ordenación por comparación	105
6.5. Criterios de Selección y Evaluación Experimental	107
6.6. La operación peligrosa: recursión profunda en QuickSort	107
6.7. Actividad de depuración: fallos en partición y búsqueda	108
6.8. Síntesis operativa y tablas de diagnóstico	108
6.9. Glosario conceptual	109
6.10. Conexión con el curso y siguientes pasos	109
6.11. Fuentes y reproducibilidad	109

Cómo usar este tema

Buscar y ordenar son las dos operaciones que aparecen debajo de casi todo lo demás, y este capítulo las trata juntas porque están relacionadas: ordenar es caro, y solo compensa si después se va a buscar muchas veces. Esa es la decisión que el capítulo enseña a tomar con números en la mano.

Al terminar sabrá: contar las comparaciones de un algoritmo de ordenación y comprobar su cuenta midiendo; explicar por qué ningún algoritmo basado en comparaciones puede bajar de cierto límite, y por qué la ordenación por dígitos sí puede; decidir si conviene ordenar antes de buscar, según cuántas búsquedas vayan a hacerse; y reconocer qué significa que una ordenación sea estable, y cuándo eso importa con datos biológicos.

La ruta **esencial** son las dos búsquedas, las ordenaciones cuadráticas y eficientes, y el criterio de cuándo ordenar. La ruta de **estudio** añade la demostración del límite inferior, la ordenación por dígitos, la evaluación experimental y el anexo.

6.1 Las dos operaciones más fundamentales

ESENCIAL En cualquier pipeline de análisis biomédico —desde la identificación de variantes en secuenciación masiva (NGS) hasta la anotación funcional de transcritos—, dos operaciones se repiten millones de veces por segundo: **buscar** un elemento específico y **ordenar** una colección masiva de registros.

Ambas operaciones sirven como el banco de pruebas empírico definitivo para los paradigmas algorítmicos del capítulo de coste y diseño:

- La fuerza bruta se plasma en la **búsqueda lineal** y las ordenaciones elementales cuadráticas.
- El paradigma Divide y Vencerás da lugar a la **búsqueda binaria**, al **MergeSort** y al **QuickSort**.
- El uso de estructuras jerárquicas avanzadas (el montículo del capítulo de estructuras no lineales) origina el **HeapSort**.

Además, en este capítulo alcanzaremos uno de los resultados más profundos de la informática teórica: el **límite inferior de la ordenación por comparación** ($\Omega(n \log n)$), demostrando por qué ningún algoritmo basado en comparar pares de elementos, lo escriba una persona o una máquina, puede bajar de ese límite, que no es una conjetura sino un resultado demostrado, y cómo los algoritmos de distribución como **Radix Sort** logran sortearla mediante propiedades específicas de las secuencias biológicas.

Buscar y ordenar son las dos operaciones más frecuentes sobre cualquier colección de datos; sirven de banco de pruebas de los paradigmas de diseño ya vistos, fuerza bruta, divide y vencerás y uso de una estructura—aplicados al mismo problema con costes muy distintos.

6.2 Algoritmos de Búsqueda

6.2.1 Búsqueda lineal (Fuerza bruta)

Concepto. Antes de entrar en las dos búsquedas de este capítulo conviene situarlas. Las dos son *ciegas* o no informadas: no saben nada del problema salvo cómo comparar dos elementos. La lineal no supone nada; la binaria supone que los datos están ordenados. Hay otras dos familias que aquí no desarrollamos. Las búsquedas *informadas* usan una estimación del problema para decidir por dónde seguir, y de esa familia viene la heurística de semillas de BLAST que verá en el capítulo de alineamiento. Las búsquedas *con adversario*, como el minimax con poda, suponen que alguien juega en contra, y quedan fuera del curso. Saber que existen evita creer que toda búsqueda se reduce a recorrer o a partir por la mitad.

La **búsqueda lineal** examina secuencialmente cada elemento de la colección desde el índice 0 hasta encontrar el objetivo o agotar los n elementos. Su ventaja reside en su universalidad: no exige que los datos guarden ningún orden previo.

La búsqueda lineal recorre la colección comparando cada elemento con el buscado; su coste es $O(n)$ en el peor caso y no exige que la colección esté ordenada.

Análisis formal del caso promedio.

Si el elemento buscado está presente y tiene idéntica probabilidad ($1/n$) de ocupar cualquiera de las n posiciones, el número medio de comparaciones es la media aritmética de la serie $1, 2, 3, \dots, n$:

$$\text{Promedio} = \frac{1}{n} \sum_{i=1}^n i = \frac{n(n+1)}{2n} = \frac{n+1}{2}$$

Al descartar constantes en notación asintótica, el caso promedio sigue siendo $O(n)$. Para $n = 100$ elementos, el número medio de comparaciones es exactamente 50.5.

Si el elemento buscado está presente y tiene la misma probabilidad de estar en cualquiera de las n posiciones, el número promedio de comparaciones de la búsqueda lineal es $(1 + 2 + \dots + n)/n = (n+1)/2$ —la media de la serie de costes 1, 2, ..., n de encontrarlo en cada posición—. En notación O la constante $1/2$ se descarta y el caso promedio sigue siendo $O(n)$: promediar no cambia la clase de complejidad. La ventaja de la búsqueda lineal es no imponer precondition alguna sobre el orden de los datos; su desventaja es no escalar a colecciones grandes.

Simulando la búsqueda lineal del elemento situado en cada una de las $n = 100$ posiciones de una colección, la suma real de comparaciones necesarias es 5050 y el promedio es 50.5 —exactamente $(n+1)/2 = 50.5$ —, confirmando por ejecución directa, no solo por fórmula, el caso promedio de la búsqueda lineal.

Se reproduce con `verification/chapter_results.sh linear_search_average 100`.

6.2.2 Búsqueda binaria (Divide y vencerás)

Sobre una colección previamente ordenada, la **búsqueda binaria** compara el objetivo con el elemento central:

- Si coincide, termina con éxito.
- Si el objetivo es menor, descarta la mitad derecha.
- Si el objetivo es mayor, descarta la mitad izquierda.

La búsqueda binaria descarta la mitad de los datos restantes en cada comparación, logrando $O(\log n)$; a cambio exige que la colección esté ordenada de antemano.

Deducción de la relación de recurrencia.

Sea $T(n)$ el tiempo de ejecución sobre una entrada de tamaño n . En cada paso se realiza una comparación (c) y el tamaño se reduce a la mitad:

$$T(n) = T(n/2) + c$$

Tras k pasos sucesivos, el tamaño remanente es $n/2^k$. El proceso se detiene cuando queda 1 elemento ($n/2^k = 1 \Rightarrow 2^k = n$), lo que da estrictamente $k = \log_2(n)$ pasos.

El coste $O(\log n)$ de la búsqueda binaria se deriva de la recurrencia $T(n) = T(n/2) + c$ —cada paso cuesta una comparación constante c y reduce el tamaño del problema a la mitad—: tras k pasos el tamaño restante es $n/2^k$, y el algoritmo termina cuando $n/2^k = 1$, es decir, $k = \log_2(n)$.

Por ejemplo, sobre el array ordenado $[2, 3, 5, 7, 11, 13, 17, 19]$ de 8 elementos, buscar el valor 11 requiere exactamente $\log_2(8) = 3$ pasos:

1. Paso 1: medio en índice 3 ($A[3] = 7 < 11$), descarta mitad izquierda.
2. Paso 2: medio en índice 5 ($A[5] = 13 > 11$), descarta mitad derecha.
3. Paso 3: medio en índice 4 ($A[4] = 11$), objetivo localizado.

Sobre el array ordenado $2, 3, 5, 7, 11, 13, 17, 19$ (8 elementos), buscar 11 con búsqueda binaria tarda exactamente 3 pasos— $\log_2(8) = 3$, coincidiendo con la predicción de la recurrencia— y lo encuentra en el índice 4: paso 1 descarta la mitad izquierda comparando con 7, paso 2 descarta la mitad derecha comparando con 13, y paso 3 compara con 11 y lo halla.

Se reproduce con `verification/chapter_results.sh binary_search_trace 100 37`.

```

ALGORITMO: Busqueda_Binaria_Iterativa
ENTRADA:
- A: Array ordenado de n elementos A[0..n-1] con precondition A[0] <= A[1] <= ... <= A[n-1]
- objetivo: Valor buscado
SALIDA:
- Indice: Posicion k tal que A[k] == objetivo, o -1 si no existe en A
COMPLEJIDAD: Tiempo  $O(\log n)$  en el peor caso, Espacio auxiliar  $O(1)$ 

1. INICIALIZACION DE LIMITES:
   izq <- 0
   der <- n - 1

2. BUCLE DICOTOMICO:
   Mientras izq <= der:
       # Calculo seguro de la posicion central evitando desbordamiento de enteros
       medio <- izq + (der - izq) / 2

       Si A[medio] == objetivo entonces:
           Retornar medio # Elemento localizado con exito
       Sino si A[medio] < objetivo entonces:
           izq <- medio + 1 # Descartar subvector izquierdo A[izq..medio]
       Sino:
           der <- medio - 1 # Descartar subvector derecho A[medio..der]

3. TERMINACION:
   Retornar -1 # Rango agotado sin coincidencias

```

El error de aplicar búsqueda binaria sobre datos no ordenados.

Si los datos no están estrictamente ordenados, la búsqueda binaria descarta mitades asumiendo una invariante inexistente, perdiendo elementos presentes y generando falsos negativos críticos.

Aplicar búsqueda binaria sobre un array sin ordenar no garantiza encontrar un valor presente: las decisiones izquierda/derecha asumen un orden inexistente y pueden descartar la mitad que contiene el objetivo.

Se reproduce con `verification/chapter_results.sh binary_search_unsorted`.

6.2.3 La búsqueda con centinela

ESTUDIO Una optimización clásica de la búsqueda lineal consiste en escribir el valor buscado al final de la colección, como *centinela*. Así el bucle no necesita comprobar en cada vuelta si ha llegado al final: sabe que va a encontrar el valor, y solo al terminar mira si lo encontró en la posición del centinela, en cuyo caso no estaba. Se ahorra una comparación por vuelta, que es la mitad del trabajo del bucle.

Concepto. El coste sigue siendo $O(n)$. La optimización cambia la constante, no la clase. Es un buen recordatorio de que una mejora real de rendimiento puede no aparecer en la notación asintótica, y también de lo contrario: que dos algoritmos de la misma clase pueden tardar el doble uno que otro.

6.3 Algoritmos de Ordenación

Ordenar una secuencia es el prerequisite computacional que desbloquea la búsqueda binaria, la detección inmediata de duplicados en lecturas NGS y el cálculo de percentiles de expresión génica.

Ordenar permite la búsqueda binaria (de $O(n)$ a $O(\log n)$), convierte preguntas caras sobre datos sin ordenar —mediana, duplicados, percentiles— en triviales sobre datos ordenados, y es una precondition que muchos algoritmos posteriores asumen sin comprobar.

6.3.1 Algoritmos elementales cuadráticos ($O(n^2)$)**1. Bubble Sort (Ordenación por burbuja).**

Compara pares de elementos adyacentes y los intercambia si están desordenados. En cada pasada completa, el mayor elemento remanente «burbujea» hasta su posición final a la derecha. Con optimización de parada temprana (detectar si una pasada no realizó ningún intercambio), logra un mejor caso $O(n)$ sobre listas previamente ordenadas.

Bubble sort compara pares adyacentes y los intercambia si están desordenados, haciendo que el elemento mayor remanente burbujee a su sitio en cada pasada; con la optimización de detectar una pasada sin intercambios se detiene antes ante una entrada favorable. En una lista ya ordenada de tamaño n esa detección da un mejor caso $O(n)$ (una pasada, $n - 1$ comparaciones, cero intercambios); en una invertida se mantiene en $O(n^2)$ ($(n - 1) + (n - 2) + \dots + 1$ comparaciones en total repartidas en hasta n pasadas, la última solo para confirmar que ya no hay intercambios).

Bubble sort sobre $[5, 2, 8, 1, 3]$ produce pasada a pasada: $[2, 5, 1, 3, 8]$ tras la primera (el 8 llega al final), $[2, 1, 3, 5, 8]$ tras la segunda, y $[1, 2, 3, 5, 8]$ tras la tercera —ya ordenada—, confirmada por una cuarta pasada sin intercambios.

Se reproduce con `verification/chapter_results.sh bubble_sort_trace`.

2. Insertion Sort (Ordenación por inserción).

Construye la lista ordenada de forma incremental de izquierda a derecha. Para cada elemento, lo desplaza hacia la izquierda dentro del bloque ya ordenado hasta encontrar su posición correcta. Es muy adaptativo: sobre datos casi ordenados toma tiempo lineal $O(n)$, y tiene una sobrecarga de ejecución tan baja que es el método más rápido en la práctica para arrays pequeños, típicamente por debajo de unos 15 elementos.

Insertion sort desplaza cada elemento hacia la izquierda dentro de la parte ya ordenada hasta encontrar su hueco; en una lista ya ordenada de tamaño n no se desplaza ningún elemento —mejor caso $O(n)$ —, mientras que en una lista invertida cada elemento se desplaza hasta el principio con $(n - 1) + (n - 2) + \dots + 1$ desplazamientos.

en total —peor caso $O(n^2)$ —: el mismo comportamiento adaptativo que bubble sort pero con menos trabajo por comparación, lo que lo hace más rápido en la práctica pese a compartir la misma cota asintótica.

3. Selection Sort (Ordenación por selección).

Divide el array en una sección ordenada y otra no ordenada. En cada pasada, localiza el mínimo absoluto del bloque desordenado y lo intercambia con la primera posición libre. Realiza siempre exactamente $n(n-1)/2$ comparaciones, por lo que su coste es **estrictamente** $O(n^2)$ **en todos los casos** (no posee mecanismo de parada temprana). Su única ventaja es que realiza como máximo $n-1$ intercambios físicos en memoria, siendo preferible si la escritura es extremadamente costosa frente a la comparación.

Selection sort busca el mínimo de la parte sin ordenar en cada una de las n pasadas: $O(n^2)$ en todos los casos, no solo en el peor. Etiquetarlo como $O(n \log n)$ confunde un método simple con uno eficiente.

Se reproduce con `verification/chapter_results.sh selection_sort_comparisons 8`.

Selection sort realiza a lo sumo un intercambio por pasada, así que sobre una lista de tamaño n nunca hace más de $n-1$ intercambios en total, muy por debajo de sus $O(n^2)$ comparaciones; es preferible cuando la operación de escribir/intercambiar es costosa y la de comparar barata, a diferencia de bubble e insertion sort, que pueden intercambiar en casi cada comparación.

Selection sort sobre $[5, 2, 8, 1, 3]$ produce pasada a pasada: $[1, 2, 8, 5, 3]$ tras la primera (intercambia 5 con el mínimo 1), $[1, 2, 8, 5, 3]$ tras la segunda (2 ya está en su sitio, sin intercambio), y $[1, 2, 3, 5, 8]$ tras la tercera (intercambia 8 con el mínimo remanente 3), ya ordenada con 2 intercambios efectivos en total frente a los intercambios repartidos en 3 pasadas de bubble sort. A diferencia de bubble sort, selection sort ejecuta de todos modos una cuarta pasada ($i = 3$, comparando 5 y 8) aunque el array ya esté ordenado porque no tiene mecanismo de parada temprana, razón de que sea $O(n^2)$ en todos los casos y no solo en el peor.

Se reproduce con `verification/chapter_results.sh selection_sort_trace`.

6.3.2 Algoritmos eficientes ($O(n \log n)$)

1. MergeSort (Ordenación por mezcla).

Aplica Divide y Vencerás de forma canónica: divide la lista recursivamente en dos mitades hasta alcanzar sublistas de longitud 1 (que están trivialmente ordenadas), y luego las recombina mediante una rutina de mezcla lineal ($O(n)$). Su recurrencia $T(n) = 2T(n/2) + O(n)$ garantiza un rendimiento estricto de $O(n \log n)$ en todos los casos (peor, mejor y promedio). Es además un algoritmo **estable**, pero requiere un array auxiliar de memoria $O(n)$.

Merge sort divide en $\log n$ niveles y hace un trabajo de mezcla $O(n)$ en cada nivel: $\log n$ veces $O(n)$ da $O(n \log n)$ garantizado en todos los casos, a cambio de memoria extra para la mezcla.

Mezclar dos sublistas ya ordenadas de longitudes a y b compara siempre las cabezas de ambas, mueve la menor al resultado y avanza en esa sublista hasta vaciar una de las dos —momento en que el resto de la otra se copia sin más comparaciones por estar ya ordenada—; mezclar $[1, 3, 5]$ con $[2, 4, 6]$ produce $[1, 2, 3, 4, 5, 6]$ en exactamente 5 comparaciones, una menos que la suma de longitudes (6) porque el último elemento remanente no necesita comparación.

Se reproduce con `verification/chapter_results.sh merge_two_sorted_halves`.

2. QuickSort (Ordenación rápida por partición).

Selecciona un elemento como **pivote** y reordena el array en tiempo lineal de forma que todos los elementos menores que el pivote queden a su izquierda y los mayores a su derecha, ordenando luego recursivamente cada partición. En el caso promedio es extraordinariamente rápido ($O(n \log n)$) y opera *in-place* sin requerir memoria auxiliar para los datos. Sin embargo, si el pivote elegido es sistemáticamente el mínimo o máximo sobre datos ya ordenados, el árbol de recursión degenera a altura n , provocando un peor caso de $O(n^2)$.

Con un quicksort que elige siempre el primer elemento como pivote, ejecutarlo sobre una lista ya ordenada produce en cada partición un lado vacío y casi todo el resto del otro: n niveles de partición cada uno $O(n)$, total $O(n^2)$, el peor caso y no el $O(n \log n)$ promedio.

Se reproduce con `verification/chapter_results.sh quicksort_fixed_pivot_sorted 8`.

Para mitigar este riesgo, los motores modernos emplean la **mediana de tres** (tomar como pivote la mediana entre el primer, medio y último elemento) y hacen una transición a **Insertion Sort** cuando los subvectores tienen menos de 15 elementos.

Elegir la mediana de tres elementos (primero, medio, último) como pivote mitiga el peor caso de quicksort sobre datos ya ordenados sin el coste $O(n)$ de calcular la mediana real —que garantizaría peor caso $O(n \log n)$ pero penalizaría el promedio—; una segunda mitigación habitual e independiente del pivote es cambiar a insertion sort para subarrays pequeños, por debajo de unos 15 elementos, cuya menor sobrecarga por elemento lo hace más rápido que seguir recursando quicksort en entradas diminutas.

```
ALGORITMO: QuickSort_Particion_Hoare
ENTRADA:
  - A: Array de elementos mutables A[izq..der]
SALIDA:
  - Array A reordenado in-situ con todos los elementos <= pivote a la izquierda y >= pivote a la derecha
COMPLEJIDAD: Tiempo  $O(\text{der} - \text{izq} + 1)$  lineal por pasada, Espacio auxiliar  $O(1)$  in-place

FUNCION Particionar_Hoare(A, izq, der):
  pivote <- A[izq]          # Selección de pivote (o mediana de 3)
  i <- izq - 1
  j <- der + 1

  Bucle Infinito:
    Repetir i <- i + 1 hasta que A[i] >= pivote
    Repetir j <- j - 1 hasta que A[j] <= pivote

    Si i >= j entonces:
      Retornar j            # j es el punto de corte para la subdivision recursiva

    Intercambiar(A[i], A[j])

FUNCION QuickSort_Recurso(A, izq, der):
  Si izq < der entonces:
    corte <- Particionar(A, izq, der)
    QuickSort_Recurso(A, izq, corte)
    QuickSort_Recurso(A, corte + 1, der)
```

3. HeapSort (Ordenación por montículo).

Aprovecha el montículo de máximos del capítulo de estructuras no lineales. Transforma el array en un heap en tiempo $O(n)$ y luego extrae sucesivamente el elemento máximo colocándolo al final del array. Realiza n extracciones de coste $O(\log n)$, garantizando $O(n \log n)$ estricto *in-place*. Sin embargo, no es estable y su recorrido de memoria no contiguo presenta peor localidad espacial de caché que QuickSort.

Heapsort inserta los n elementos en un heap y los extrae uno a uno: como cada extracción del máximo cuesta $O(\log n)$, ordenar n elementos con un heap cuesta $O(n \log n)$.

6.3.3 Estabilidad y espacio en memoria

Un algoritmo de ordenación es **estable** si preserva estrictamente el orden de llegada relativo entre registros que comparten la misma clave. Por ejemplo, si se ordenan lecturas de secuenciación con idéntica calidad Phred ($Q = 90$), un algoritmo estable garantiza que la lectura que entró primero (L_1) permanezca antes que la que entró después (L_3).

Ordenando las lecturas $(L_1, 90)$, $(L_2, 85)$, $(L_3, 90)$ por puntuación decreciente (L_1 llegó antes que L_3 , ambas con puntuación 90), un método estable produce siempre $(L_1, 90)$, $(L_3, 90)$, $(L_2, 85)$ —conservando el orden de llegada en los empates—; un método inestable puede producir $(L_3, 90)$, $(L_1, 90)$, $(L_2, 85)$, con la misma puntuación correctamente ordenada pero sin garantía sobre el orden relativo de L_1 y L_3 . La estabilidad solo se manifiesta cuando hay claves repetidas.

Se reproduce con `verification/chapter_results.sh stability_demo`.

Algoritmo	Peor Caso	Caso Promedio	Memoria Extra	¿Estable?
Selection Sort	$O(n^2)$	$O(n^2)$	$O(1)$ in-place	No
Insertion Sort	$O(n^2)$	$O(n^2)$	$O(1)$ in-place	Sí
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(1)$ in-place	Sí
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n)$ auxiliar	Sí
QuickSort	$O(n^2)$	$O(n \log n)$	$O(\log n)$ prom. / $O(n)$ peor	No
HeapSort	$O(n \log n)$	$O(n \log n)$	$O(1)$ in-place	No

Nota metodológica sobre las implementaciones analizadas: En QuickSort la memoria de pila es $O(\log n)$ bajo particiones equilibradas y puede degenerar a $O(n)$ en el peor caso sin optimización; la técnica de recurrir sobre la partición menor (Sección 6.6) garantiza una profundidad máxima de pila $O(\log n)$. Entre los métodos de esta tabla, ninguno reúne simultáneamente cota estricta $O(n \log n)$, estabilidad y memoria auxiliar $O(1)$.

Merge sort es estable pero usa memoria extra; quicksort es in-place pero no estable; heapsort es in-place y tampoco estable. Ningún método eficiente combina las tres ventajas ($O(n \log n)$ garantizado, estable, in-place) a la vez.

6.4 Límite inferior teórico de la ordenación por comparación

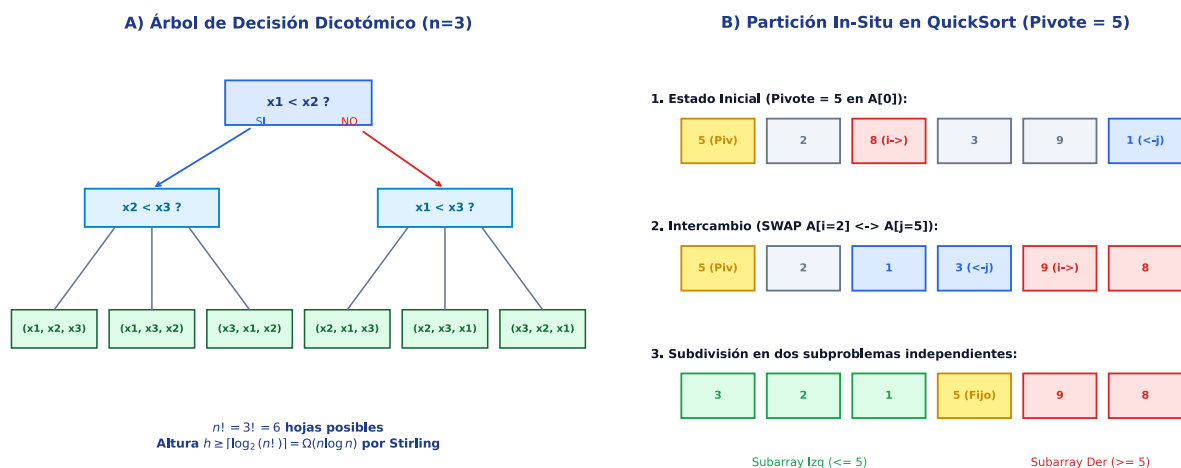


Figura 6.1: A) Árbol de decisión dicotómico para $n = 3$, ilustrando por qué ordenar por comparación exige una altura mínima de $\lceil \log_2(n!) \rceil = \Omega(n \log n)$ hojas posibles. B) Traza in-situ del esquema de partición de QuickSort (Hoare) con punteros convergentes i y j alrededor del pivote. Figura original adaptada de las presentaciones docentes.

¿Es posible diseñar un algoritmo de ordenación por comparación que tome tiempo lineal $O(n)$ en el peor caso? La respuesta matemática es un rotundo **NO**.

Demostración por Árbol de Decisión.

Cualquier algoritmo que ordene mediante comparaciones binarias ($a_i < a_j$) se puede modelar como un árbol de decisión donde:

- Cada nodo interno representa una comparación con dos salidas posibles (Sí / No).
- Cada hoja representa una de las permutaciones finales posibles del array.
- Para n elementos distintos, existen exactamente $n!$ permutaciones posibles, por lo que el árbol debe contener al menos $L \geq n!$ hojas.

Un árbol binario de altura h tiene como máximo 2^h hojas. Por tanto:

$$2^h \geq n! \implies h \geq \log_2(n!)$$

Para acotar ese logaritmo basta un argumento elemental, sin necesidad de la fórmula de Stirling. La suma tiene n términos crecientes; quedándonos solo con la mitad superior, cada uno de ellos vale al menos $\log_2(n/2)$:

$$\log_2(n!) = \sum_{i=1}^n \log_2(i) \geq \sum_{i=n/2}^n \log_2(n/2) = \frac{n}{2} \log_2(n/2) = \Omega(n \log n)$$

Dado que la altura del árbol representa el número de comparaciones en el peor caso, ningún algoritmo de comparación puede requerir menos de $\Omega(n \log n)$ pasos.

Ningún algoritmo que ordene comparando elementos puede requerir menos de $\Omega(n \log n)$ comparaciones en el peor caso: existen $n!$ permutaciones posibles y cada comparación binaria solo distingue dos casos, exigiendo un árbol de decisión de altura mínima $\lceil \log_2(n!) \rceil = \Omega(n \log n)$.

6.4.1 Ordenación no comparativa: Radix Sort para secuencias biológicas

El límite $\Omega(n \log n)$ aplica **exclusivamente a algoritmos basados en comparaciones**. Si las claves poseen estructura interna conocida —como secuencias de ADN de longitud fija k sobre el alfabeto $\Sigma = \{A, C, G, T\}$ —, se puede ordenar mediante **LSD Radix Sort** (ordenación por dígitos menos significativos).

Radix Sort distribuye las claves en cubetas según el símbolo en la posición i (desde el extremo derecho k hasta el 1) de forma estable. Realiza k pasadas de coste lineal $O(n)$:

$$\text{Coste total} = O(k \cdot n)$$

Para ordenar 10^9 lecturas de k -meros con $k = 8$, Radix Sort toma 8×10^9 operaciones elementales, superando ampliamente a cualquier método de comparación ($10^9 \times \log_2(10^9) \approx 30 \times 10^9$ operaciones).

El límite $\Omega(n \log n)$ aplica exclusivamente a algoritmos por comparación; Radix Sort ordena símbolo a símbolo sin comparar claves completas, alcanzando un coste asintótico $O(k \cdot n)$ con k la longitud fija de la clave. Para secuencias de ADN cortas y alfabeto acotado, realiza k pasadas lineales de distribución estable.

LSD radix sort ordena de derecha a izquierda: cada pasada distribuye las claves en cubetas según el símbolo en la posición actual y las recoge en orden de cubeta manteniendo el orden relativo dentro de cada una (distribución estable, imprescindible para que el resultado final quede ordenado). Sobre GAT, ACT, GAC, ACA ($k = 3$, alfabeto A, C, G, T), 3 pasadas —una por posición de última a primera— producen ACA, ACT, GAC, GAT, ordenadas alfabéticamente sin comparar nunca dos claves completas entre sí.

Se reproduce con `verification/chapter_results.sh radix_sort_dna`.

```
ALGORITMO: Radix_Sort_Secuencias_ADN
ENTRADA:
- Secuencias: Lista de n k-meros de longitud fija k sobre alfabeto Sigma = {'A', 'C', 'G', 'T'}
SALIDA:
- Secuencias_Ordenadas: Lista ordenada lexicograficamente en tiempo lineal
COMPLEJIDAD: Tiempo  $O(k * (n + |\Sigma|))$ , Espacio auxiliar  $O(n + |\Sigma|)$ 

1. BUCLE DE PASADAS POSICIONALES (De derecha a izquierda, d = k-1 descendiendo a 0):
  Para d desde k - 1 descendiendo hasta 0:
    # Inicializar cubetas estables para cada base del alfabeto
    Cubeta['A'] <- Lista_Vacia()
    Cubeta['C'] <- Lista_Vacia()
    Cubeta['G'] <- Lista_Vacia()
    Cubeta['T'] <- Lista_Vacia()

    # Fase 1: Distribucion estable en cubetas segun el caracter en posicion d
    Para cada secuencia S en Secuencias:
      base <- S[d]
      Cubeta[base].ENQUEUE(S)  # Preserva el orden relativo previo (estabilidad)

    # Fase 2: Recoleccion concatenada en orden canonico A -> C -> G -> T
```



```
Secuencias <- Concatenar(Cubeta['A'], Cubeta['C'], Cubeta['G'], Cubeta['T'])
```

2. RETORNO:
Retornar Secuencias

6.5 Criterios de Selección y Evaluación Experimental

No existe un algoritmo universalmente superior; la elección depende del patrón de acceso del pipeline biomédico:

No hay un algoritmo de ordenación universalmente mejor: la elección depende del tamaño de la entrada, de si se necesita garantía en el peor caso, y de si las claves tienen estructura (longitud fija) que un método no comparativo pueda aprovechar.

6.5.1 ¿Cuándo compensa ordenar antes de buscar?

Si sobre una colección desordenada de n elementos se van a realizar S búsquedas:

- **Opción A (Búsqueda lineal directa):** coste total = $S \cdot O(n)$.
- **Opción B (Ordenar primero y luego búsqueda binaria):** coste total = $O(n \log n) + S \cdot O(\log n)$.

Para una única búsqueda ($S = 1$), ordenar previamente es un desperdicio computacional ($n \log n + \log n > n$). Solo compensa ordenar cuando el coste del ordenamiento se **amortiza** entre un número suficiente de consultas:

$$S > \frac{n \log_2 n}{n - \log_2 n} \approx \log_2 n$$

Para una sola búsqueda sobre datos sin ordenar, ordenar primero y buscar con búsqueda binaria cuesta más que una búsqueda lineal directa; ordenar antes de buscar solo compensa cuando se va a buscar muchas veces sobre la misma colección, porque el coste de ordenar se amortiza entre las búsquedas.

Se reproduce con `verification/chapter_results.sh sort_once_vs_many_searches 1000 100`.

6.5.2 Evaluación experimental: escalado ante duplicación (2×)

Protocolo de benchmark y reproducibilidad experimental.

Para contrastar empíricamente las tasas teóricas de crecimiento, el protocolo debe controlar estrictamente las variables experimentales: fijar la semilla aleatoria (SEED=42), generar la misma distribución de claves, registrar el número exacto de comparaciones (métrica determinista e invariable frente a la máquina) y promediar múltiples ejecuciones para amortizar variaciones del planificador del sistema operativo.

Al duplicar el tamaño de la entrada ($n \rightarrow 2n$):

- En algoritmos $O(n^2)$ (Selection Sort), el tiempo se multiplica por $(2n)^2/n^2 = 4\times$.
- En algoritmos $O(n \log n)$ (MergeSort), el tiempo se multiplica suavemente por $\frac{2n \log(2n)}{n \log n} = 2 \left(1 + \frac{1}{\log n}\right) \approx 2.2\times$.

Duplicando el tamaño de la entrada, el tiempo real de selection sort escala aproximadamente $\times 4$ (compatible con $O(n^2)$) y el de merge sort escala de forma mucho más suave (compatible con $O(n \log n)$): la teoría predice la tendencia, la medición confirma que se cumple en esta máquina y con estos datos.

Se reproduce con `verification/chapter_results.sh time_compare_sorts 200`.

6.6 La operación peligrosa: recursión profunda en QuickSort

Un fallo crítico en entornos de producción ocurre cuando QuickSort procesa listas de gran tamaño que ya vienen casi ordenadas y el pivote se elige de forma ingenua (primer elemento). La profundidad del árbol de llamadas alcanza n , provocando un **desbordamiento de la pila de llamadas** (*Stack Overflow*) del sistema operativo.

Protocolo preventivo en 3 pasos:

1. Utilizar siempre la regla de pivote mediana-de-tres o pivote aleatorio.
2. Establecer un umbral de corte para delegar en la ordenación por inserción por debajo de unos 15 elementos. La cifra es un valor típico medido en bibliotecas reales, no una constante universal: depende de la máquina y del tipo de dato, y se ajusta midiendo.
3. Recurrir en primer lugar sobre la partición menor e iterar sobre la mayor para acotar el uso de pila a $O(\log n)$.

6.7 Actividad de depuración: fallos en partición y búsqueda

Existen dos esquemas de partición clásicos. El de **Hoare**, que aparece más arriba, hace converger dos punteros desde los extremos. El de **Lomuto**, que se depura a continuación, recorre el array con un solo puntero y coloca el pivote en su sitio al final. Los dos son correctos cuando están bien escritos; el de abajo no lo está.

Analice este código con tres fallos:

```
particionar(arr, bajo, alto):
    pivote <- arr[bajo]
    i <- bajo
    # Fallo 1: el limite del bucle no incluye el extremo derecho (alto)
    para j de bajo + 1 hasta alto - 1:
        # Fallo 2: comparacion incorrecta que invierte el orden del particionado
        si arr[j] > pivote:
            i <- i + 1
            intercambiar(arr[i], arr[j])
    # Fallo 3: se intercambia con la posicion original del pivote en vez de con i
    intercambiar(arr[bajo], arr[alto])
    devolver i
```

6.8 Síntesis operativa y tablas de diagnóstico**6.8.1 Tabla de diagnóstico de síntomas computacionales**

Síntoma observado	Causa probable	Comprobación y corrección
Búsqueda binaria no encuentra elemento presente	El array no estaba previamente ordenado	Asegurar ordenación estricta antes de invocar
QuickSort tarda $O(n^2)$ en datos reales	Peor caso por pivote extremo en datos ordenados	Implementar mediana-de-tres o pivote aleatorio
El orden relativo de registros idénticos cambia	Uso de un algoritmo inestable (QuickSort/HeapSort)	Cambiar a MergeSort o incluir índice como clave secundaria
Tiempo de ejecución se cuadruplica al duplicar n	Comportamiento cuadrático $O(n^2)$ activo	Migrar a un algoritmo $O(n \log n)$ o Radix Sort

6.8.2 Tabla de síntesis con preguntas de control

Requisito del pipeline	Algoritmo recomendado	Pregunta de control
Una sola búsqueda puntual	Búsqueda lineal $O(n)$	¿Se realizarán más búsquedas posteriores?
Búsquedas masivas frecuentes	Ordenar una vez + Búsqueda binaria	¿El volumen de búsquedas amortiza el ordenamiento?
Ordenación estable garantizada	MergeSort	¿Se dispone de memoria auxiliar $O(n)$?

Requisito del pipeline	Algoritmo recomendado	Pregunta de control
Máxima velocidad <i>in-place</i>	QuickSort (con mediana-de-tres)	¿Se ha prevenido el desbordamiento de pila?
Secuencias cortas de ADN (k -meros)	Radix Sort ($O(n \cdot k)$)	¿La longitud de las secuencias es constante?

6.9 Glosario conceptual

- **Búsqueda lineal:** escaneo secuencial exhaustivo en $O(n)$.
- **Búsqueda binaria:** descarte recursivo por mitades en $O(\log n)$.
- **Estabilidad:** preservación del orden relativo original ante claves iguales.
- **In-place:** algoritmo que opera sin requerir memoria auxiliar proporcional a n .
- **Pivote:** elemento de referencia para la partición en QuickSort.
- **MergeSort:** algoritmo Divide y Vencerás estable con coste $O(n \log n)$ garantizado.
- **HeapSort:** ordenación mediante montículo binario en $O(n \log n)$ *in-place*.
- **Stirling:** aproximación matemática $\ln(n!) \approx n \ln n - n$.
- **Radix Sort:** ordenación no comparativa por cubetas en $O(n \cdot k)$.
- **Amortización:** reparto del coste de una operación previa entre múltiples usos posteriores.

6.10 Conexión con el curso y siguientes pasos

La ordenación y la indexación sostienen el análisis de frecuencias e información del capítulo siguiente, los modelos probabilísticos del posterior, el filtrado por semillas de BLAST en el de alineamiento y el aprendizaje automático del último.

- El capítulo siguiente aplica conteo ordenado de frecuencias y perfiles de motivos con matrices de pesos.
- El de modelos probabilísticos usa ordenación de estados y caminos.
- El de alineamiento apoya en la ordenación su filtrado por semillas.
- El de aprendizaje automático ordena y particiona conjuntos de datos.

6.11 Fuentes y reproducibilidad

Este capítulo se apoya en las dos presentaciones docentes de algoritmos de la asignatura, escritas por Álvaro Serrano Navarro. La demostración del límite inferior, los esquemas de partición y el análisis de las ordenaciones siguen a Cormen y colaboradores [8]; la ordenación por dígitos aplicada a secuencias y el orden de magnitud de los datos de secuenciación, a Compeau y Pevzner [7]; y el protocolo de medición con semilla fija, a Allesina y Wilmes [1].

Todas las cuentas y mediciones se reproducen con `verification/chapter_results.sh` tal como están impresas.

Anexo práctico · Medir el escalado real de dos algoritmos de ordenación

Pregunta, producto y separación de materiales

¿Puede contar a mano las comparaciones de un algoritmo de ordenación, contrastarlas con la ejecución real, y comprobar que el escalado que predice la notación O se cumple —o no— al medir con `time`? Este laboratorio responde que sí, sin necesitar ningún dato externo.

El producto individual es la tabla de predicciones y resultados: el conteo manual de comparaciones de la ordenación por selección, la pasada a mano del esquema de Lomuto, el veredicto de escalado al duplicar

la entrada y las dos comprobaciones de casos límite, cada uno con su predicción escrita antes de medir. Le acompañan las salidas guardadas y la defensa.

El anexo es autónomo: no necesita datos externos ni red. Todo lo que se mide lo produce el oráculo del capítulo, que el generador copia en `herramienta/`.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 preparar el espacio	directorio creado
2 contar a mano la ordenación	conteo manual contrastado
3 una pasada de Lomuto a mano	posición final del pivote justificada
4 duplicar la entrada	cociente observado frente al previsto
5 casos límite	lista vacía y búsqueda sin resultado
6 empaquetar y verificar	verificador en <code>ENTREGA_OK</code>

La ruta es única. La regla del anexo es una sola: nada se ejecuta antes de estar predicho por escrito.

1 • Preparar el espacio de trabajo

```
bash --version
./practice_assets/create_workspace.sh TC06_entrega
cd TC06_entrega
```

El generador crea `notas/respuestas.tsv` con la cabecera y ninguna fila, `resultados/` y `herramienta/` con el oráculo del capítulo. No trae ninguna respuesta escrita.

2 • Contar a mano selection sort

Antes de ejecutar nada, aplique selection sort a mano sobre `[5, 2, 8, 1, 3]` y cuente cuántas comparaciones hace. Anote su predicción en `notas/respuestas.tsv` y compárela:

```
bash herramienta/chapter_results.sh selection_sort_manual > resultados/conteo_manual.txt
cat resultados/conteo_manual.txt
```

Ordenar `[5, 2, 8, 1, 3]` con selection sort exige 10 comparaciones ($4+3+2+1$, una por cada pasada sobre lo que queda de la parte desordenada), y produce `[1, 2, 3, 5, 8]`.

Se reproduce con `verification/chapter_results.sh selection_sort_manual`.

3 • Una pasada del esquema de Lomuto, a mano

Tarea. Aplique a mano una sola pasada de partición con el esquema de Lomuto sobre `[5, 2, 8, 1, 3]`, tomando como pivote el primer elemento. Escriba el estado del array después de cada intercambio y diga en qué posición acaba el pivote. Anótelos en `notas/respuestas.tsv` en una fila cuyo apartado se llame «lomuto».

Este ejercicio conecta con la actividad de depuración del capítulo: los tres fallos que allí se diagnostican son fallos de esta misma pasada. Hacerla a mano una vez es la forma más rápida de ver por qué el pivote tiene que acabar en la posición del último elemento menor que él, y no donde estaba.

Comprueba. Cuando termine, compruebe la propiedad que define una partición correcta: todo lo que queda a la izquierda del pivote es menor o igual que él, y todo lo que queda a la derecha es mayor. Si su resultado no la cumple, la pasada está mal aunque el array parezca «más ordenado» que antes.

4 • Duplicar la entrada

```
bash herramienta/chapter_results.sh time_compare_sorts 200 > resultados/tiempos.txt
bash herramienta/chapter_results.sh selection_sort_comparisons 100 > resultados/comparaciones_n.txt
bash herramienta/chapter_results.sh selection_sort_comparisons 200 > resultados/comparaciones_2n.txt
cat resultados/tiempos.txt resultados/comparaciones_n.txt resultados/comparaciones_2n.txt
```

Al duplicar N de 100 a 200, las comparaciones de selection sort crecen de 4950 a 19900, un factor de aproximadamente $4\times$, coherente con $O(n^2)$: duplicar la entrada cuadruplica el trabajo, no lo duplica.

Se reproduce con `verification/chapter_results.sh selection_sort_comparisons 8`.

sobre $n = 100$ y $n = 200$.

Calcule el cociente exacto ($19900/4950$) y anótelo junto al cociente teórico esperado ($2^2 = 4$ para $O(n^2)$) en `notas/respuestas.tsv`.

5 • Casos límite

```
bash herramienta/chapter_results.sh empty_and_single_sort > resultados/limite_vacia.txt
bash herramienta/chapter_results.sh binary_search_empty > resultados/limite_busqueda.txt
cat resultados/limite_vacia.txt resultados/limite_busqueda.txt
```

Selection sort y quicksort sobre una lista vacía o de un solo elemento hacen 0 comparaciones y no fallan; la búsqueda binaria sobre un array vacío informa «no encontrado» sin acceder a ningún índice fuera de rango.

Se reproduce con `verification/chapter_results.sh empty_and_single_sort`.

y `binary_search_empty`.

6 • Empaquetar y verificar

Escriba antes `notas/defensa.md`. Después:

```
cd ..
tar -czf TC06_entrega.tar.gz TC06_entrega
sha256sum TC06_entrega.tar.gz
./practice_assets/check_entrega.sh TC06_entrega TC06_entrega.tar.gz
```

El verificador comprueba que la tabla de respuestas tiene la cabecera correcta, al menos cuatro filas con predicción, resultado, veredicto y justificación, y una fila dedicada a la pasada de Lomuto; que la columna de veredicto es coherente con las otras dos; que `resultados/` recoge el par de medidas de escalado; y que existe la defensa. Rechaza el espacio recién generado. No puntúa.

Rúbrica de calificación ponderada

La evaluación pondera la verificación técnica, el análisis algorítmico y la defensa conceptual sobre 10 puntos, tres de entrega y siete de razonamiento:

- **3,0 puntos • Entrega técnica y verificador (DELIVERY_OK):** Entrega íntegra del paquete comprimido `.tar.gz`, formato de `notas/respuestas.tsv` y ejecución correcta de `check_practice_delivery.sh`.
- **2,5 puntos • Conteo analítico y estabilidad en ordenación:** Conteo manual de comparaciones en Selection Sort y análisis formal del impacto de la estabilidad en registros con claves duplicadas.
- **2,5 puntos • Protocolo de perturbación ($2\times$) y casos límite:** Cálculo exacto del cociente de operaciones al duplicar la entrada ($4950 \rightarrow 19900$, factor $4\times$ coherente con $O(n^2)$) y control de robustez sobre colecciones vacías y de un elemento.
- **2,0 puntos • Defensa conceptual y cruce asintótico:** Justificación rigurosa (100–150 palabras) sobre la compensación entre constantes ocultas y tasas de crecimiento asintótico, apoyándose en las evidencias empíricas obtenidas.

Terminar en ENTREGA_OK acredita que la entrega está completa, no que las predicciones fueran buenas: precisamente por eso se pide anotarlas antes.

Lista de autocorrección

- escribí cada predicción antes de ejecutar la orden correspondiente;
- mi conteo manual coincidió con el del oráculo, o anoté por qué no;
- mi pasada de Lomuto deja el pivote donde debe, y lo he comprobado con la propiedad de la partición;
- el cociente al duplicar la entrada está calculado y comparado con el previsto;
- los dos casos límite terminan sin fallo y sin acceder fuera de rango;
- sé explicar por qué un algoritmo con mejor notación puede tardar más en entradas pequeñas;
- guardé en resultados/ la salida de cada ejecución que cito;
- el verificador termina en ENTREGA_OK.

Defensa conceptual

En notas/defensa.md, entre 100 y 150 palabras, explique por qué un algoritmo con mejor notación asintótica, como la ordenación por mezcla frente a la de selección, puede tardar más en una entrada pequeña, y a partir de qué evidencia de su propia medición esperaría que la ventaja se note. Cite un número concreto de su tabla.

Fuentes y reproducibilidad

Este anexo no usa datos externos: todo lo que mide lo genera el oráculo del capítulo, que es determinista. El análisis de las ordenaciones y los esquemas de partición siguen a Cormen y colaboradores [8].

Medir sorpresa e incertidumbre: teoría de la información aplicada a secuencias

TC-CH07 – Técnicas Computacionales en Biología

Edición 2

Pregunta del tema. Dos mensajes de cuatro letras ocupan lo mismo en disco y no dicen lo mismo. ¿Cuánta incertidumbre había antes de leerlos, y cómo se mide eso con un número?

Producto final. Un guion propio que calcula entropía por ventanas sobre una secuencia, con su contrato declarado y su comportamiento ante entradas que no son ADN, contrastado con un oráculo que no le da los valores de antemano.

Mapa del tema

7.1. Una pregunta cuantitativa antes de empezar	113
7.2. Del suceso a la sorpresa	113
7.3. De muchas sorpresas a una incertidumbre media	114
7.4. Ejemplos que pueden comprobarse	116
7.5. De una secuencia a una distribución empírica	116
7.6. Lo que la composición no puede mostrar	119
7.7. Cuando hay datos y también hay un modelo	120
7.8. Divergencia KL: cuantificar el desacuerdo	121
7.9. Información mutua: pasar de diversidad a dependencia	123
7.10. Una lectura unificada de las cinco cantidades	125
7.11. La escala total y el supuesto iid	125
7.12. Implementar sin ocultar el razonamiento	125
7.13. Límites de inferencia biológica	127
7.14. Mapa de decisión para resolver problemas	128
7.15. Síntesis y transición a modelos probabilísticos	128

7.1 Una pregunta cuantitativa antes de empezar

Imagine dos mensajes de cuatro letras: AAAA y ACGT. Tienen la misma longitud y ocupan el mismo número de caracteres en un archivo de texto, pero no producen la misma impresión. Si una fuente casi siempre emite A, el primer mensaje era previsible; si las cuatro bases eran igualmente probables, cada posición del segundo mensaje resulta menos anticipable. La longitud por sí sola no responde a la pregunta que abre este capítulo: *¿cuánta incertidumbre había antes de observar el mensaje?*

La teoría de la información permite formular esa pregunta sin decidir primero si el mensaje es útil, verdadero, funcional o biológicamente importante. Esta separación es una virtud y también un límite. Una medida matemática puede describir regularidades de los símbolos; no convierte automáticamente una región de baja entropía en un gen, un sitio regulador o una isla CpG. La interpretación biológica necesita evidencia adicional y un modelo adecuado al fenómeno.

Al terminar podrá distinguir tres objetos que suelen confundirse: una *distribución* declarada, una *secuencia* observada y una *medida* calculada a partir de ambas. También podrá explicar qué hipótesis hacen posible un cálculo, comprobar ejemplos manuales con Bash y detener una interpretación cuando la evidencia no permite ir más lejos.

7.2 Del suceso a la sorpresa

7.2.1 La probabilidad pertenece a un modelo

Antes de asignar información a una letra necesitamos declarar probabilidades. En una moneda ideal, por ejemplo, el modelo asigna $p(\text{cara}) = 1/2$. En una moneda sesgada podría asignar $p(\text{cara}) = 3/4$. En una secuencia, una tabla como $p(A) = p(C) = p(G) = p(T) = 1/4$ no está escrita en la molécula: es una hipótesis, una estimación o una referencia elegida por quien analiza.

Esta observación evita una confusión frecuente. La probabilidad no es lo mismo que la frecuencia, aunque podamos estimarla mediante una frecuencia. Si aparecen 30 adeninas en 100 posiciones, la frecuencia observada es $30/100 = 0,30$. Usarla como estimación de $p(A)$ añade el supuesto de que esas observaciones representan la fuente que queremos estudiar. Cambiar el conjunto de datos, la región, la hebra o el mecanismo de muestreo puede cambiar la estimación.

7.2.2 Autoinformación

Buscamos una medida que aumente cuando disminuye la probabilidad. Además, si dos sucesos son independientes, queremos que la información conjunta sea la suma de las informaciones: observar dos caras independientes de probabilidad $1/2$ debería aportar el doble que observar una. El logaritmo satisface precisamente esta propiedad porque transforma productos en sumas.

$$I(x) = -\log_2 p(x) = \log_2 \left(\frac{1}{p(x)} \right). \quad (7.1)$$

La autoinformación de un resultado de probabilidad $p(x) > 0$ es $-\log_2 p(x)$ y se mide en bits cuando el logaritmo tiene base dos. El nombre *bit* no significa aquí que el archivo ocupe exactamente un bit: designa la unidad de la medida. Con logaritmos naturales la unidad sería el *nat*. No se deben mezclar resultados obtenidos con bases distintas sin convertirlos.

Conexión con la teoría computacional y árboles de decisión.

La relación entre $\log_2(1/p(x))$ y la unidad «bit» conecta formalmente con el límite inferior por árbol de decisión analizado en TC-CH06: si un suceso elemental tiene probabilidad $p(x) = 1/M$, hacen falta exactamente $\log_2 M$ preguntas binarias equilibradas (comparaciones dicotómicas) para aislar dicho suceso sin ambigüedad. Cada bit cuantifica la resolución de una decisión binaria ideal que divide por dos la incertidumbre del observador.

Tres valores construyen intuición. Si $p(x) = 1$, entonces $I(x) = 0$ bits: el resultado seguro no resuelve incertidumbre. Si $p(x) = 1/2$, aporta 1 bit. Si $p(x) = 1/8$, aporta 3 bits, pues harían falta tres decisiones binarias equilibradas para distinguir uno entre ocho resultados equiprobables.

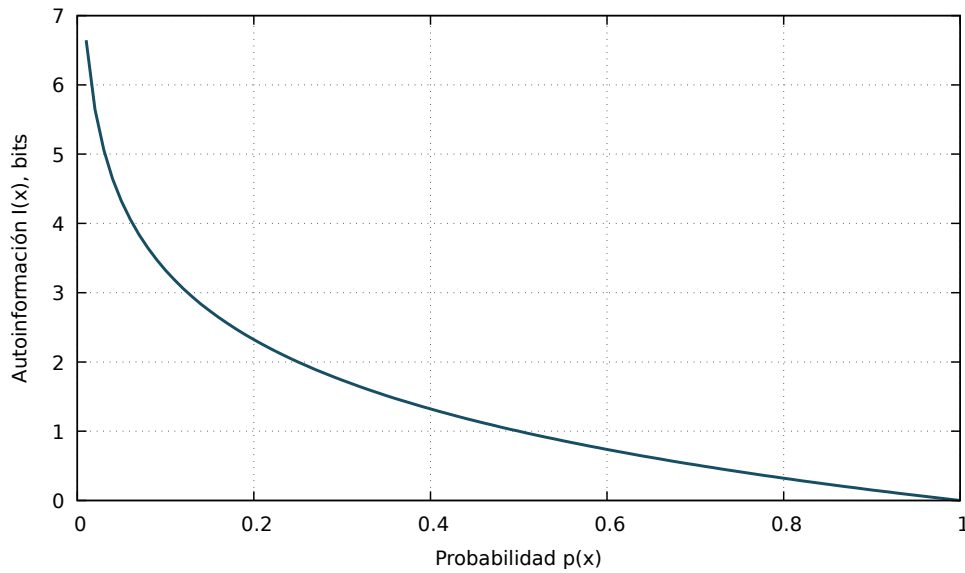


Figura 7.1: Autoinformación frente a probabilidad. En el dominio representado, $-\log_2 p(x)$ crece rápidamente al disminuir $p(x)$. Activo original TC07-F01, generado por `make_figures.sh selfinfo` desde TC07-F01-selfinfo.tsv.

El activo TC07-F01 se obtiene de cien probabilidades entre 0,01 y 1 mediante la función $-\log_2 p$ y muestra su crecimiento cuando p se aproxima a cero.

7.2.3 Por qué se suman las sorpresas independientes

Si x y y son independientes, su probabilidad conjunta es $p(x, y) = p(x)p(y)$. Sustituir ese producto en la definición y usar la regla del logaritmo da la aditividad.

$$I(x, y) = -\log_2[p(x)p(y)] = I(x) + I(y). \quad (7.2)$$

La autoinformación de dos resultados independientes es aditiva. La palabra *independientes* no es decorativa. Si conocer x cambia la probabilidad de y , el producto $p(x)p(y)$ deja de describir el suceso conjunto. Este punto regresará al estudiar información mutua.

Ejemplo resuelto: moneda sesgada.

En una moneda con $p(C) = 0,75$ y $p(X) = 0,25$, observar cara aporta $-\log_2(0,75) \approx 0,415$ bits; observar cruz aporta 2 bits. La cruz aporta más información cuando ocurre, pero ocurre menos. Para describir la incertidumbre media antes del lanzamiento tendremos que ponderar cada sorpresa por su probabilidad. Ese promedio conduce a la entropía.

Con probabilidades 0,75 y 0,25, las autoinformaciones de cara y cruz son aproximadamente 0,415037 y 2 bits, respectivamente.

7.3 De muchas sorpresas a una incertidumbre media

Sea X una variable aleatoria discreta que puede tomar valores en un alfabeto $\mathcal{X}$. Antes de observarla existe un abanico de resultados con sus probabilidades. Después de observar x , la sorpresa es $I(x)$. La media ponderada de todas esas sorpresas es la entropía.

La entropía de Shannon es la autoinformación esperada bajo una distribución declarada.

$$H(X) = \mathbb{E}[I(X)] = - \sum_{x \in \mathcal{X}} p(x) \log_2 p(x). \quad (7.3)$$

Para una variable discreta, la entropía de Shannon es $-\sum_x p(x) \log_2 p(x)$, con el convenio $0 \log 0 = 0$. El convenio

para $p(x) = 0$ no afirma que $\log 0$ exista. Afirma que el límite de $-p \log p$ cuando p se acerca a cero es cero: un resultado que el modelo declara imposible no contribuye al promedio bajo ese mismo modelo.

7.3.1 Leer la fórmula sin convertirla en un ritual

Cada término tiene una función distinta. $-\log_2 p(x)$ mide la sorpresa de un resultado; $p(x)$ indica con qué frecuencia esperamos pagar esa sorpresa; la suma promedia sobre resultados incompatibles. Olvidar el factor $p(x)$ sumaría sorpresas sin ponderarlas. Olvidar el signo produciría números negativos. Usar conteos sin normalizarlos sustituiría probabilidades por cantidades que no suman uno.

La entropía es una propiedad de una distribución, no una etiqueta metafísica de una cadena. Cuando decimos «entropía de una secuencia», normalmente abreviamos «entropía de la distribución empírica obtenida al contar símbolos en esa secuencia». La abreviatura es cómoda, pero conviene recordar la operación.

7.3.2 Extremos y escala

$$0 \leq H(X) \leq \log_2 K, \quad (7.4)$$

Para una distribución sobre K resultados posibles, la entropía está entre 0 y $\log_2 K$ bits; el máximo se alcanza con la distribución uniforme. El mínimo corresponde a una fuente degenerada: un resultado tiene probabilidad uno. El máximo corresponde a K alternativas equiprobables. En un alfabeto de cuatro nucleótidos canónicos, la cota superior de orden cero es $\log_2 4 = 2$ bits por símbolo. La cota depende del alfabeto: si se admite también N , ya no es correcto seguir usando 2 como máximo teórico del alfabeto de cinco símbolos.

Que una distribución tenga más entropía no significa que sea «mejor». Una muestra contaminada puede aumentar la diversidad aparente; una región funcional puede ser conservada y mostrar menor variabilidad; una fuente aleatoria puede tener entropía alta sin poseer una organización biológica interesante. La medida responde a una pregunta cuantitativa concreta, no a una escala universal de calidad.

```

ALGORITMO: Entropia_Shannon_Secuencia
ENTRADA:
- S: Cadena biologica de longitud L sobre alfabeto discreto Sigma (ej. {'A', 'C', 'G', 'T'})
SALIDA:
- H: Entropia de Shannon en bits (unidades base 2)
COMPLEJIDAD: Tiempo  $O(L + |\Sigma|)$ , Espacio auxiliar  $O(|\Sigma|)$ 

1. INICIALIZACION Y CONTEO:
  Si  $L == 0$  entonces:
    Retornar ERROR_SECUENCIA_VACIA

  Para cada simbolo c en Sigma:
    Conteo[c] <- 0

  Para i desde 0 hasta L - 1:
    simbolo <- S[i]
    Conteo[simbolo] <- Conteo[simbolo] + 1

2. CALCULO DE LA ENTROPIA DE SHANNON:
  H <- 0.0
  Para cada c en Sigma:
    Si Conteo[c] > 0 entonces:
      p <- Conteo[c] / L          # Distribucion de probabilidad empirica
      H <- H - p * (log(p) / log(2)) # Base 2; si p=0 el termino se ignora por  $\lim_{p \rightarrow 0} p \log(p) = 0$ 

3. RETORNO:
  Retornar H   #  $0.0 \leq H \leq \log_2(|\Sigma|)$ 

```

7.4 Ejemplos que pueden comprobarse

7.4.1 Cuatro símbolos equiprobables

Si $p(A) = p(C) = p(G) = p(T) = 1/4$, cada símbolo aporta $-\log_2(1/4) = 2$ bits. El promedio de cuatro contribuciones iguales sigue siendo 2.

$$H = -4 \left(\frac{1}{4} \right) \log_2 \left(\frac{1}{4} \right) = 2 \text{ bits/símbolo.} \quad (7.5)$$

Una distribución uniforme de cuatro símbolos tiene entropía de 2 bits por símbolo. La unidad «por símbolo» recuerda que hemos calculado un promedio. Bajo un modelo independiente e idénticamente distribuido, 12 posiciones tendrían una entropía conjunta de $12 \times 2 = 24$ bits. Esa multiplicación ya incorpora un modelo sobre la relación entre posiciones; no se deduce solo de ver una cadena de longitud 12.

Bajo un modelo iid uniforme de cuatro símbolos, doce posiciones tienen entropía conjunta de 24 bits.

7.4.2 Una moneda sesgada

La moneda del apartado anterior tiene una cara frecuente y una cruz rara. Su entropía pondera ambas sorpresas:

$$H = -0,75 \log_2(0,75) - 0,25 \log_2(0,25) \approx 0,811278 \text{ bits/lanzamiento.} \quad (7.6)$$

Una variable binaria con probabilidades 0,75 y 0,25 tiene entropía aproximada de 0,811278 bits. El resultado es menor que el máximo binario de 1 bit. No porque la cruz deje de ser sorprendente, sino porque la fuente suele emitir el resultado previsible.

7.4.3 AAAA frente a ACGT

Para convertir una cadena en distribución empírica contamos sus símbolos y dividimos por la longitud. En AAAA, $\hat{p}(A) = 1$ y las demás frecuencias son cero: la entropía de orden cero es 0. En ACGT, las cuatro frecuencias son 1/4: la entropía es 2 bits por símbolo.

$$\begin{array}{|c|c|c|c|} \hline A & A & A & A \\ \hline \end{array} \quad \hat{H}_0 = 0 \quad \begin{array}{|c|c|c|c|} \hline A & C & G & T \\ \hline \end{array} \quad \hat{H}_0 = 2$$

Figura 7.2: Dos cadenas de igual longitud producen distribuciones empíricas distintas. Composición del capítulo TC07-G01; los valores se reproducen con el caso `sequences` del oráculo Bash.

Las distribuciones empíricas de AAAA y ACGT tienen entropías de orden cero de 0 y 2 bits por símbolo, respectivamente.

Este ejemplo separa longitud y composición, pero todavía no dice nada sobre el orden. Para ver ese límite necesitamos un par construido con los mismos conteos.

7.5 De una secuencia a una distribución empírica

7.5.1 Contar, normalizar y agregar

Sea una secuencia de longitud n y n_x el número de apariciones del símbolo x . La estimación más directa sustituye la probabilidad desconocida por su frecuencia relativa.

$$\hat{p}(x) = \frac{n_x}{n}, \quad \hat{H}_0 = - \sum_{x:n_x>0} \hat{p}(x) \log_2 \hat{p}(x). \quad (7.7)$$

El estimador empírico de orden cero se obtiene contando cada símbolo, normalizando por la longitud y aplicando la fórmula de Shannon a las frecuencias relativas no nulas. El subíndice cero indica que el estado utilizado para predecir el símbolo siguiente no incluye símbolos anteriores. No significa «entropía aproximada de mala calidad»: identifica el orden del modelo.

En Bash, la tubería natural refleja las tres operaciones. Bash organiza la entrada, la salida y los errores; *awk* recorre caracteres, mantiene un arreglo de conteos y evalúa logaritmos. No hace falta introducir otro lenguaje curricular.

7.5.2 Puente de Shell a *awk*: una transformación cada vez

Este capítulo *recupera* destrezas cuyos propietarios son TC-CH01, *Entorno Linux, Shell y reproducibilidad* (ejecutar un archivo y registrar su entorno); TC-CH02, *Bash, automatización y procesamiento de texto* (argumentos, tuberías, expansión y *awk*); y TC-CH03, *Algoritmos, complejidad y estrategias de resolución* (entrada, estado, traza y oráculo). Si no puede anticipar qué texto recibe cada lado de `|`, conviene recuperar TC-CH02 antes del script completo.

Este capítulo recupera destrezas propiedad de TC-CH01 (ejecutar un archivo y registrar su entorno), TC-CH02 (argumentos, tuberías, expansión y *awk*) y TC-CH03 (entrada, estado, traza y oráculo), sin volver a enseñarlas.

El siguiente puente solo cuenta. Bash guarda AACG en *sequence*; *printf* la convierte en una línea; la tubería entrega esa línea a *awk*; el primer bloque actualiza *count*; y *END* imprime una tabla en el orden fijo A, C, G, T. Fijar `LC_ALL=C` estabiliza el punto decimal y la comparación de caracteres.

```
#!/usr/bin/env bash
set -euo pipefail
sequence=${1:-AACG}
printf '%s\n' "$sequence" |
LC_ALL=C awk '
{ for (i=1; i<=length($0); i++) count[substr($0,i,1)]++ }
END {
    print "symbol\tcount"
    for (i=1; i<=4; i++) { s=substr("ACGT",i,1); printf "%s\t%d\n",s,count[s]+0 }
}'
```

El programa `bridge_counts.sh AACG` imprime los conteos $A = 2$, $C = 1$, $G = 1$ y $T = 0$ en ese orden.

ESENCIAL El algoritmo completo añade contrato de entrada, normalización y entropía. Acepta una o más cadenas como argumentos separados; exige mayúsculas y el alfabeto ACGT; una entrada vacía o inválida termina con estado no cero. Recorre siempre ACGT, por lo que la salida no depende del orden interno de los arreglos asociativos de *awk*.

```
#!/usr/bin/env bash
set -euo pipefail

if (( $# == 0 )); then
    printf 'usage: %s SEQUENCE [SEQUENCE ...]\n' "$0" >&2
    exit 64
fi

for sequence in "$@"; do
    if [[ -z "$sequence" ]]; then
        printf 'ERROR: empty sequence\n' >&2
        exit 65
    fi
    if [[ ! "$sequence" =~ ^[ACGT]+$ ]]; then
        printf 'ERROR: expected uppercase A C G or T: %s\n' "$sequence" >&2
        exit 66
    fi
    LC_ALL=C awk -v sequence="$sequence" '
function log2(x) { return log(x)/log(2) }
BEGIN {
    for (i=1; i<=length(sequence); i++) count[substr(sequence,i,1)]++
    for (i=1; i<=4; i++) {
        symbol=substr("ACGT",i,1)
        if (count[symbol]>0) {
            p=count[symbol]/length(sequence)
            h-=p*log2(p)
        }
    }
}
```

```

    }
    printf "%s\t%.6f\n", sequence, h
  }'
done

```

Una lectura cuidadosa del programa distingue estado y cálculo. `count` guarda los conteos; `n` guarda la longitud; el bucle final normaliza antes de acumular. Solo se recorren símbolos observados, así que nunca se evalúa $\log(0)$. El formato con seis decimales es una decisión de presentación, no una precisión experimental garantizada.

Traza manual.

Para AACG, los conteos son $n_A = 2$, $n_C = 1$, $n_G = 1$. Las frecuencias son $1/2$, $1/4$ y $1/4$. Las contribuciones son $0,5$, $0,5$ y $0,5$ bits, de modo que $\hat{H}_0 = 1,5$ bits por símbolo. La suma de contribuciones, no la media aritmética de tres valores, ya contiene la ponderación por probabilidad.

La distribución empírica de AACG es $(1/2, 1/4, 1/4, 0)$ sobre A, C, G, T y su entropía de orden cero es $1,5$ bits por símbolo.

7.5.3 Precondiciones que el cálculo debe declarar

Un script fiable no «limpia» silenciosamente cualquier entrada. Antes de contar hay que decidir al menos:

- si minúsculas y mayúsculas representan el mismo símbolo;
- si se eliminarán espacios y saltos de línea de un FASTA;
- si las cabeceras que empiezan por `>` se excluyen;
- si `N` pertenece al alfabeto, se ignora o provoca error;
- qué hacer con una secuencia vacía;
- si se analiza una sola secuencia o se agrupan varias.

Cada elección cambia el objeto estadístico. Ignorar `N` cambia el denominador; tratarla como quinto símbolo cambia el alfabeto; repartirla entre las cuatro bases inventa fracciones que no fueron observadas. Ninguna política es universalmente correcta, pero debe quedar escrita y ser coherente con la pregunta.

El contrato publicado de `entropy0.sh` preserva el orden de varios argumentos válidos y rechaza ausencia de argumentos, cadenas vacías y símbolos fuera de ACGT con un estado de salida distinto de cero.

REFERENCIA

7.5.4 Frecuencia observada no equivale a verdad poblacional

La entropía obtenida al insertar frecuencias observadas es un estimador (*plug-in*) y puede presentar sesgo apreciable cuando el número de observaciones es pequeño respecto del número de categorías.

Supongamos una fuente generadora ideal con $P(A) = P(C) = P(G) = P(T) = 1/4$, cuya entropía poblacional es exactamente 2 bits por símbolo. Las muestras cortas "AAAA" y "ACGT" son ambas realizaciones perfectamente posibles bajo esta misma fuente uniforme:

- Para la muestra "AAAA", la probabilidad de emisión bajo la fuente uniforme es $(1/4)^4 = 1/256$: un suceso poco frecuente, pero válido. Sin embargo, su estimación empírica directa (*plug-in*) produce $\hat{H}_0 = 0$ bits/símbolo.
- Para la muestra "ACGT", la estimación *plug-in* produce $\hat{H}_0 = 2$ bits/símbolo.

Una muestra finita con frecuencias observadas cero para algunas bases no redefine la fuente estocástica subyacente. En bioinformática rigurosa es imperativo reportar el tamaño de muestra n , el alfabeto considerado y el estimador utilizado antes de inferir propiedades poblacionales.

7.6 Lo que la composición no puede mostrar

Compare AAAACCCC con ACACACAC. Ambas cadenas contienen cuatro A y cuatro C. Su distribución empírica de símbolos es $(1/2, 1/2)$ y por tanto su entropía de orden cero es 1 bit por símbolo. Sin embargo, la primera agrupa símbolos y la segunda alterna con regularidad.

$$\text{AAAACCCC} \rightarrow \{A : 4, C : 4\} \leftarrow \text{ACACACAC}$$

Mismo histograma, distinto orden secuencial

Figura 7.3: La entropía de orden cero factoriza el orden: conserva conteos y descarta la posición de cada aparición. Figura original.

Dos secuencias con la misma composición tienen la misma entropía de orden cero aunque difieran en el orden de sus símbolos. No se trata de un fallo de implementación: es una consecuencia exacta de la entrada elegida por el modelo. La fórmula recibe $p(A)$ y $p(C)$, no recibe la lista de posiciones. Ningún algoritmo puede recuperar de esos dos números qué cadena los produjo.

La consecuencia didáctica es profunda. «Entropía alta» no equivale a «secuencia sin patrones», porque un patrón puede conservar una composición equilibrada. ACACACAC es perfectamente predecible después de conocer un símbolo, pero su entropía de orden cero alcanza el máximo del alfabeto binario. Para representar esa dependencia se necesitan pares, contextos o estados: precisamente la puerta de entrada a modelos de Markov y HMM en el capítulo siguiente.

La cadena alternante ACACACAC tiene un bit de entropía de orden cero por símbolo, pero entropía condicional de primer orden cero porque cada A va seguida de C y cada C va seguida de A.

7.6.1 Entropía por ventanas: localizar sin diagnosticar

Una sola distribución para una secuencia larga mezcla regiones potencialmente distintas. Una ventana de ancho w calcula la distribución empírica en posiciones $i, \dots, i + w - 1$ y desplaza el inicio. Así obtenemos un perfil local. Con la secuencia sintética AAAAACGTACGT y $w = 4$, la primera ventana AAAA tiene entropía 0; al entrar otros símbolos, la entropía aumenta; las ventanas ACGT alcanzan 2.

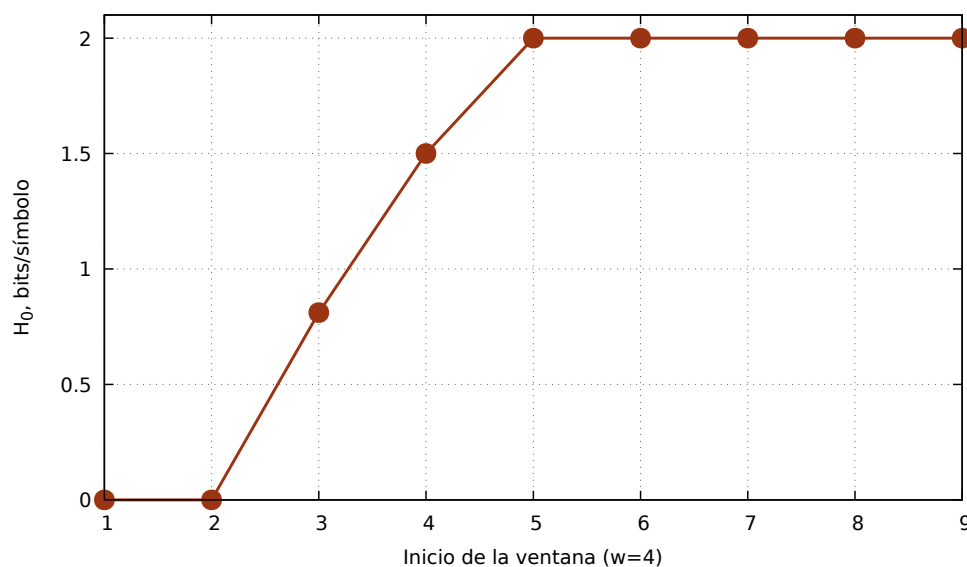


Figura 7.4: Perfil reproducible de AAAAACGTACGT con ancho cuatro. Activo original TC07-F03, generado por `make_figures.sh windows` desde TC07-F03-windows.tsv. Los puntos son ventanas discretas; la línea solo guía su lectura.

En el ejemplo sintético AAAAACGTACGT con ventanas de cuatro símbolos, el perfil de entropía de orden cero pasa de 0 en AAAA a 2 en las ventanas ACGT; la serie completa es 0, 0, 0,811278, 1,5, 2, 2, 2, 2 y 2.

El ancho controla un compromiso. Ventanas pequeñas localizan cambios con mayor precisión, pero sus frecuencias son inestables y solo pueden tomar pocos valores. Ventanas grandes estabilizan conteos, pero suavizan

fronteras. Además, ventanas solapadas no son observaciones independientes: comparten casi todos sus símbolos. Por ello, un perfil sirve para formular candidatos y comparar regiones bajo una regla declarada; no constituye por sí solo una prueba de función.

Al pasar de ancho cuatro a ancho ocho, el paso posible de las frecuencias baja de $1/4$ a $1/8$ y el solapamiento entre ventanas consecutivas sube de tres a siete símbolos. Bajo muestreo iid con $p = 1/2$, la varianza de la frecuencia estimada, $p(1 - p)/w$, baja de 0,0625 a 0,03125: se gana estabilidad de conteo a costa de localizar las fronteras en una región más ancha.

Un perfil de entropía por ventanas puede priorizar regiones candidatas bajo una regla declarada, pero no demuestra por sí solo función biológica ni satisface los campos de una anotación.

En una ventana de ancho w , las frecuencias cambian en pasos de $1/w$ y dos ventanas consecutivas desplazadas una posición comparten $w - 1$ símbolos; por ello su perfil no contiene observaciones independientes.

En particular, una ventana de baja entropía no identifica automáticamente un gen, una isla CpG o una región reguladora. Para una isla CpG se emplean propiedades como contenido GC y frecuencia observada/esperada del dinucleótido CpG, junto con una definición operativa y contexto genómico. La entropía puede añadir una descripción, pero no sustituye esas variables ni valida una anotación.

```
ALGORITMO: Perfil_Entropia_Ventana_Deslizante
ENTRADA:
- S: Cadena genómica de longitud L
- W: Ancho de ventana en nucleótidos (1 <= W <= L)
- Paso: Desplazamiento entre ventanas consecutivas (típicamente Paso = 1)
SALIDA:
- Perfil: Lista de tuplas (posicion_inicio, entropia_H)
COMPLEJIDAD: Tiempo  $O((L - W) / \text{Paso} + 1) * W$ 

1. INICIALIZACION:
  Perfil <- Lista_Vacia()
  pos <- 0

2. RECORRIDO DE VENTANAS DESLIZANTES:
  Mientras pos + W <= L:
    subcadena_ventana <- S[pos .. pos + W - 1]
    H_ventana <- Entropia_Shannon_Secuencia(subcadena_ventana)
    Perfil.APPEND((pos, H_ventana))
    pos <- pos + Paso

3. RETORNO:
  Retornar Perfil
```

La regla operativa histórica de una isla CpG combina una longitud superior a 200 pares de bases, contenido GC superior al 50% y cociente CpG observado/esperado superior a 0,6; por ello una entropía baja no satisface esa definición [13].

7.7 Cuando hay datos y también hay un modelo

Hasta ahora una sola distribución ha ocupado el centro. En la práctica suelen aparecer dos: P , que representa la fuente de datos o la distribución que queremos describir, y Q , un modelo usado para anticiparlos. Preguntar «¿cuánta incertidumbre tiene P ?» no es lo mismo que preguntar «¿qué sorpresa media sufro si codifico datos de P como si procedieran de Q ?» La segunda pregunta define la entropía cruzada.

$$H(P, Q) = - \sum_x p(x) \log_2 q(x). \quad (7.8)$$

La entropía cruzada $H(P, Q)$ es la sorpresa media bajo datos distribuidos como P cuando las probabilidades usadas para evaluarlos son las de Q . La asimetría de papeles se ve en la fórmula: $p(x)$ pondera qué observaciones ocurren; $q(x)$ determina la penalización asignada. Intercambiar P y Q cambia ambas funciones y puede cambiar el resultado.

Ejemplo resuelto.

Supongamos dos resultados. Los datos siguen $P = (0,5,0,5)$, mientras el modelo afirma $Q = (0,75,0,25)$. La entropía de los datos es 1 bit. Evaluarlos con Q produce

$$H(P, Q) = -\frac{1}{2} \log_2(0,75) - \frac{1}{2} \log_2(0,25) \approx 1,207519 \text{ bits.} \quad (7.9)$$

La diferencia, unos 0,207519 bits por observación, cuantifica el exceso medio asociado a usar ese modelo en vez de la distribución P .

Para $P = (0,5,0,5)$ y $Q = (0,75,0,25)$ se obtiene $H(P) = 1$, $H(P, Q) \approx 1,207519$ y un exceso $D_{\text{KL}}(P\|Q) \approx 0,207519$ bits.

Contrato numérico de distribuciones de probabilidad.

Las funciones de cálculo de entropía cruzada y divergencia aceptan valores decimales no negativos representados con punto (por ejemplo 0, 0.25 o 1.0), pertenecientes al intervalo cerrado $[0, 1]$ y cuya suma total equivalga a 1 dentro de una tolerancia numérica explícita ($|1 - \sum p_i| \leq 10^{-6}$). No se aceptan comas decimales ni signos negativos en este validador.

7.8 Divergencia KL: cuantificar el desacuerdo

La divergencia de Kullback–Leibler compara las mismas dos distribuciones y conserva la dirección «datos P frente a modelo Q ».

$$D_{\text{KL}}(P\|Q) = \sum_x p(x) \log_2 \frac{p(x)}{q(x)} = H(P, Q) - H(P). \quad (7.10)$$

Cuando Q asigna probabilidad positiva a todo resultado posible bajo P , la divergencia KL satisface $D_{\text{KL}}(P\|Q) = H(P, Q) - H(P)$. La identidad permite leer KL como una penalización media por desajuste. No dice que una muestra finita ocupe literalmente ese número de bits en un archivo. Conecta cantidades ideales bajo distribuciones y, para interpretaciones de codificación, requiere el marco y los límites apropiados.

7.8.1 No negatividad no significa distancia

La divergencia KL es no negativa y vale cero si y solo si las dos distribuciones coinciden en el soporte relevante. Que sea no negativa la hace útil como medida de desajuste, pero no basta para ser una distancia métrica. Una distancia debe ser simétrica y obedecer además la desigualdad triangular. KL falla ya en la primera condición.

Retomemos $P = (0,5,0,5)$ y $Q = (0,75,0,25)$. El cálculo anterior da $D_{\text{KL}}(P\|Q) \approx 0,207519$ bits. En la dirección inversa, $D_{\text{KL}}(Q\|P) \approx 0,188722$ bits.

$$\begin{array}{ccc} \boxed{P = (0,50,0,50)} & \xrightarrow[0,207519 \text{ bits}]{D_{\text{KL}}(P\|Q)} & \boxed{Q = (0,75,0,25)} \\ \boxed{P = (0,50,0,50)} & \xleftarrow[0,188722 \text{ bits}]{D_{\text{KL}}(Q\|P)} & \boxed{Q = (0,75,0,25)} \end{array}$$

Figura 7.5: La dirección forma parte de la pregunta. Figura original; valores reproducidos por `verification/chapter_results.sh crosskl`. Activo TC07-G02.

La divergencia de Kullback–Leibler no es, en general, simétrica. Por eso conviene evitar la expresión «distancia KL» salvo que se esté explicando con precisión un uso no métrico. Promediar ambas direcciones crea una divergencia simétrica, pero la simetría por sí sola tampoco demuestra la desigualdad triangular. El nombre correcto en este capítulo es *divergencia*.

7.8.2 El caso límite de un cero en el modelo

Si $p(x) > 0$ pero $q(x) = 0$, el modelo declara imposible un resultado que la fuente puede producir. La sorpresa $-\log_2 q(x)$ diverge y también lo hace KL.

$$p(x) > 0 \text{ y } q(x) = 0 \implies D_{\text{KL}}(P||Q) = +\infty. \quad (7.11)$$

Si P asigna masa positiva a un resultado al que Q asigna cero, la divergencia $D_{\text{KL}}(P||Q)$ es infinita. No debe «arreglarse» el problema omitiendo el término: hacerlo cambia la definición y oculta un conflicto esencial entre datos y modelo. Una alternativa práctica es suavizar Q con pseudoconteos, pero el parámetro de suavizado pasa a formar parte del análisis y debe comunicarse.

REFERENCIA

7.8.3 Pseudoconteos: útiles, no invisibles

Supongamos conteos $(A, C, G, T) = (3, 1, 0, 0)$. La estimación directa asigna cero a G y T. Si añadimos un pseudoconteo de Laplace $\alpha = 1$ a las cuatro categorías, el denominador pasa de 4 a 8 y obtenemos $Q = (4/8, 2/8, 1/8, 1/8)$. Su entropía es 1,75 bits por símbolo.

$$\tilde{p}(x) = \frac{n_x + \alpha}{n + K\alpha}, \quad (3, 1, 0, 0), \alpha = 1 \implies \tilde{H} = 1,75. \quad (7.12)$$

Añadir $\alpha = 1$ a los conteos $(3, 1, 0, 0)$ produce probabilidades $(0,5, 0,25, 0,125, 0,125)$ y entropía de 1,75 bits. El pseudoconteo impide ceros, pero también desplaza todas las probabilidades hacia la uniformidad. Con pocos datos, el desplazamiento puede ser grande. Por eso hay que comparar decisiones razonables de α y no presentar el valor suavizado como si fuera un conteo observado.

En el suavizado $\tilde{p}(x) = (n_x + \alpha)/(n + K\alpha)$ con $\alpha > 0$, aumentar α acerca cada probabilidad a $1/K$; el efecto relativo es mayor cuando el total observado n es pequeño.

Para los conteos $(3, 1, 0, 0)$, al usar $\alpha = 0,5, 1$ y 2 , la probabilidad suavizada de A baja de 0,583333 a 0,5 y 0,416667, mientras la entropía sube de 1,551098 a 1,75 y 1,887919 bits; una conclusión dependiente de ese cambio debe declararse sensible al pseudoconteo.

7.8.4 Un procedimiento de comparación reproducible

Para comparar una distribución empírica P y un modelo Q conviene seguir un orden fijo:

1. declarar el mismo alfabeto y el orden de sus categorías;
2. comprobar que cada distribución es no negativa y suma uno dentro de una tolerancia explícita;
3. inspeccionar si existe algún $p(x) > 0$ con $q(x) = 0$;
4. calcular primero $H(P)$ y $H(P, Q)$;
5. calcular KL de forma directa y comprobar la identidad $H(P, Q) - H(P)$;
6. informar siempre la dirección y la base del logaritmo.

Este doble cálculo es un oráculo interno. Si la suma directa de KL no coincide con la diferencia de entropías más allá del redondeo esperado, hay un error en el orden de categorías, en la normalización, en la base del logaritmo o en el manejo de ceros.

7.8.5 Programa guiado para P, Q y KL

ESTUDIO El archivo siguiente recibe cuatro números: dos probabilidades para P y dos para Q . awk valida números no negativos y normalización, detiene el caso de soporte incompatible y conserva en variables separadas $H(P)$, $H(P, Q)$ y KL. La fila CHECK calcula la misma divergencia por la identidad de la ecuación 7.10; la coincidencia de ambas filas es el oráculo. Aquí el andamiaje aún nombra cada cantidad antes de pedir autonomía.

```
#!/usr/bin/env bash
set -euo pipefail

if (( $# != 4 )); then
    printf 'usage: %s P0 P1 Q0 Q1\n' "$0" >&2
```



```

exit 64
fi

LC_ALL=C awk -v p0="$1" -v p1="$2" -v q0="$3" -v q1="$4" '
function log2(x) { return log(x)/log(2) }
function valid_number(x) { return x ~ /^[0-9]+([.][0-9]*)?|.[0-9]+$/ }
BEGIN {
    if (!valid_number(p0)||!valid_number(p1)||!valid_number(q0)||!valid_number(q1)) exit 65
    if (p0<0||p1<0||q0<0||q1<0) exit 66
    if ((p0+p1-1)^2>1e-12||((q0+q1-1)^2>1e-12)) exit 67
    if ((p0>0&&q0==0)||p1>0&&q1==0)) exit 68
    p[0]=p0; p[1]=p1; q[0]=q0; q[1]=q1
    for (i=0; i<2; i++) if (p[i]>0) {
        hp-=p[i]*log2(p[i]); hpq-=p[i]*log2(q[i]); d+=p[i]*log2(p[i]/q[i])
    }
    printf "H_P\t%.6f\nH_PQ\t%.6f\nKL_P_Q\t%.6f\nCHECK\t%.6f\n",hp, hpq, d, hpq-hp
}'

```

El programa `compare_pq.sh 0.5 0.5 0.75 0.25` devuelve 1,000000, 1,207519, 0,207519 y 0,207519 para $H(P)$, $H(P, Q)$, KL y su comprobación, respectivamente.

7.9 Información mutua: pasar de diversidad a dependencia

La entropía marginal de una variable no describe cómo se relaciona con otra. Podemos tener dos posiciones individualmente equilibradas y, sin embargo, perfectamente ligadas. La información mutua compara la distribución conjunta $p(x, y)$ con la que existiría si las variables fueran independientes, $p(x)p(y)$.

$$I(X; Y) = \sum_x \sum_y p(x, y) \log_2 \frac{p(x, y)}{p(x)p(y)}. \quad (7.13)$$

La información mutua es la divergencia KL entre la distribución conjunta P_{XY} y el producto de marginales $P_X P_Y$. Al sustituir $Q(x, y) = p(x)p(y)$ en la fórmula de KL aparece exactamente el sumando de la ecuación 7.13; la demostración enlazada en el registro nombra esta identidad. Responde a una pregunta precisa: ¿cuánto cambia la descripción de una variable al conocer la otra?

7.9.1 Tabla conjunta mínima: dependencia perfecta

Considere dos variables binarias con solo dos pares observables: (0, 0) y (1, 1), cada uno con probabilidad 1/2. Ambas marginales son uniformes, pero conocer X determina Y .

	$Y = 0$	$Y = 1$	$p_X(x)$
$X = 0$	1/2	0	1/2
$X = 1$	0	1/2	1/2
$p_Y(y)$	1/2	1/2	1

Tabla 7.1: Composición del capítulo TC07-G03: tabla conjunta de dependencia perfecta.

En cada celda no nula, el cociente es $(1/2)/[(1/2)(1/2)] = 2$ y su logaritmo es 1. Dos contribuciones de 1/2 suman 1 bit. Si, en cambio, las cuatro celdas valen 1/4, cada cociente es uno y la información mutua vale cero.

Una tabla binaria con $p(0, 0) = p(1, 1) = 1/2$ tiene 1 bit de información mutua; la tabla uniforme con cuatro celdas de 1/4 tiene 0 bits.

La implementación retira parte del andamiaje anterior: ya no recibe marginales, sino cuatro celdas y debe construir p_X y p_Y . El doble bucle calcula a la vez MI y KL frente al producto; sus dos filas deben coincidir.

El programa de información mutua retira el andamiaje de recibir marginales ya calculadas: a partir de cuatro celdas debe construir p_X y p_Y y calcular en el mismo recorrido MI y KL frente al producto de marginales, con resultados que deben coincidir entre ambos.

```
#!/usr/bin/env bash
set -euo pipefail

if (( $# != 4 )); then
    printf 'usage: %s P00 P01 P10 P11\n' "$0" >&2
    exit 64
fi

LC_ALL=C awk -v p00="$1" -v p01="$2" -v p10="$3" -v p11="$4" '
function log2(x) { return log(x)/log(2) }
function valid_number(x) { return x ~ /^[0-9]+(\.[0-9]*)?[0-9]+$/ }
BEGIN {
    if (!valid_number(p00) || !valid_number(p01) || !valid_number(p10) || !valid_number(p11)) exit 65
    p[0,0]=p00; p[0,1]=p01; p[1,0]=p10; p[1,1]=p11
    total=0
    for (x=0;x<2;x++) for (y=0;y<2;y++) { if (p[x,y]<0) exit 66; total+=p[x,y] }
    if ((total-1)^2>1e-12) exit 67
    for (x=0;x<2;x++) for (y=0;y<2;y++) { px[x]+=p[x,y]; py[y]+=p[x,y] }
    for (x=0;x<2;x++) for (y=0;y<2;y++) if (p[x,y]>0) {
        product=px[x]*py[y]
        contribution=p[x,y]*log2(p[x,y]/product)
        mi+=contribution; kl+=contribution
    }
    printf "MI\t%.6f\nKL_JOINT_PRODUCT\t%.6f\n",mi,kl
}'
```

El programa `mutual_info_2x2.sh 0.5 0 0 0.5` devuelve 1 bit tanto como información mutua como KL entre la conjunta y el producto de marginales.

Para variables discretas, la información mutua es no negativa y es cero si y solo si las variables son independientes. «Cero» es una afirmación sobre la distribución ideal. Una estimación finita puede ser positiva incluso si el mecanismo es independiente, especialmente cuando hay pocas observaciones respecto de las celdas posibles. Por eso un valor estimado requiere incertidumbre, controles y una comparación nula apropiada antes de convertirse en evidencia de asociación.

Si X e Y son binarias, uniformes e independientes, dos observaciones pueden producir por azar la tabla empírica diagonal $(0,0)$, $(1,1)$ —en cualquier orden— con probabilidad $1/8$; su estimador *plug-in* de información mutua vale 1 bit pese al mecanismo independiente.

Una información mutua empírica positiva necesita compararse con la variabilidad de muestreo o con un control nulo apropiado antes de interpretarse como evidencia de asociación.

7.9.2 Qué constituye una muestra para posiciones de secuencia

Para calcular información mutua entre dos posiciones no basta con elegir dos letras de una única cadena. Se necesita una colección de observaciones emparejadas: por ejemplo, columnas homólogas de un alineamiento de múltiples secuencias, o pares de posiciones obtenidos bajo un diseño de muestreo declarado. Cada secuencia aporta entonces un par (x_i, y_i) a la tabla conjunta.

Calcular información mutua entre dos posiciones exige una colección de observaciones emparejadas —por ejemplo columnas homólogas de un alineamiento múltiple o pares de posiciones bajo un diseño de muestreo declarado— y no dos letras tomadas de una única cadena.

Incluso con un alineamiento, la interpretación requiere prudencia. Secuencias muy relacionadas por ascendencia no forman necesariamente observaciones independientes; la calidad del alineamiento condiciona qué posiciones se comparan; huecos y símbolos ambiguos necesitan una política; múltiples pares de columnas crean un problema de multiplicidad. La información mutua puede señalar dependencia estadística, pero no demuestra contacto físico, coevolución directa ni causalidad.

La información mutua es simétrica, $I(X;Y) = I(Y;X)$; un único valor simétrico no identifica una dirección causal.

En alineamientos de proteínas, la información mutua cruda puede contener ruido de fondo y componentes filogenéticos, por lo que una asociación entre columnas no constituye por sí sola evidencia de contacto directo [10].

7.10 Una lectura unificada de las cinco cantidades

Las fórmulas se recuerdan mejor si se asocian a preguntas, no a nombres aislados.

Cantidad	Pregunta	Objeto que debe declararse
Autoinformación	¿Cuánto sorprende este resultado?	Una probabilidad $p(x)$
Entropía	¿Cuál es la sorpresa media de la fuente?	Una distribución P
Entropía cruzada	¿Cómo evalúa Q datos de P ?	Dos distribuciones y sus papeles
KL	¿Qué exceso medio introduce el desajuste?	Dirección $P\ Q$ y soporte
Información mutua	¿Cuánto se aparta la conjunta de la independencia?	Conjunta y marginales

Autoinformación, entropía, entropía cruzada, KL e información mutua responden a preguntas distintas y no son intercambiables.

Una estrategia útil ante cualquier problema es escribir primero la frase que se quiere cuantificar. Si la frase contiene «un resultado», piense en autoinformación. Si contiene «promedio de una fuente», piense en entropía. Si contiene «datos bajo un modelo», distinga entropía cruzada y KL. Si contiene «dependencia entre variables», construya la distribución conjunta antes de hablar de información mutua.

La frase que describe el problema orienta qué cantidad usar: «un resultado» sugiere autoinformación, «promedio de una fuente» sugiere entropía, «datos bajo un modelo» distingue entropía cruzada de KL, y «dependencia entre variables» exige construir la distribución conjunta antes de recurrir a información mutua.

7.11 La escala total y el supuesto iid

La entropía expresada en bits por símbolo es una tasa media bajo el modelo. Para una secuencia de n variables independientes e idénticamente distribuidas, la entropía conjunta es la suma de las entropías individuales.

$$H(X_1, \dots, X_n) = \sum_{i=1}^n H(X_i) = nH(X_1) \quad (\text{modelo iid}). \quad (7.14)$$

Bajo un modelo iid, una fuente de 1,5 bits por símbolo tiene entropía conjunta de 18 bits para 12 símbolos. La etiqueta iid es parte del resultado. En una secuencia con dependencia, la entropía conjunta puede ser menor que $nH(X_1)$ porque observar el contexto reduce incertidumbre. En ACACACAC, por ejemplo, conocer la regla de alternancia hace predecibles casi todas las posiciones aunque la composición marginal sea uniforme.

7.12 Implementar sin ocultar el razonamiento

ESENCIAL Bash es aquí un medio para convertir una derivación en una traza auditable. Un buen script debe permitir localizar dónde se originó cada número. La salida más útil durante el aprendizaje no es solo $H=1.5$, sino una tabla intermedia con símbolo, conteo, probabilidad y contribución.

```
#!/usr/bin/env bash
set -euo pipefail

if (( $# != 1 )); then
    printf 'usage: %s SEQUENCE\n' "$0" >&2
    exit 64
fi
sequence=$1
```

```

if [[ -z "$sequence" ]]; then
    printf 'ERROR: empty sequence\n' >&2
    exit 65
fi
if [[ ! "$sequence" =~ ^[ACGT]+$ ]]; then
    printf 'ERROR: expected uppercase A C G or T: %s\n' "$sequence" >&2
    exit 66
fi

LC_ALL=C awk -v sequence="$sequence" '
function log2(x) { return log(x)/log(2) }
BEGIN {
    n=length(sequence)
    for (i=1; i<=n; i++) count[substr(sequence,i,1)]++
    print "symbol\tcount\tprobability\tcontribution_bits"
    for (i=1; i<=4; i++) {
        symbol=substr("ACGT",i,1)
        if (count[symbol]>0) {
            p=count[symbol]/n
            contribution=-p*log2(p)
            h+=contribution
            printf "%s\t%d\t%.6f\t%.6f\n",symbol,count[symbol],p,contribution
        }
    }
    printf "TOTAL\t%d\t1.000000\t%.6f\n",n,h
}'

```

Para AACG, `entropy_trace.sh` imprime, en orden A,C,G, contribuciones de 0,5 bits y una fila total de 1,5 bits.

7.12.1 Por qué $\log(x)/\log(2)$

La función `log()` de `awk` calcula el logaritmo natural. La identidad de cambio de base permite obtener base dos mediante $\log(x)/\log(2)$. La razón de dos logaritmos en la misma base es exacta matemáticamente; el valor numérico está sujeto al redondeo de punto flotante.

No es adecuado comparar dos resultados decimales con igualdad textual si se han calculado por rutas numéricas distintas: se declara una tolerancia, por ejemplo 10^{-6} para los ejemplos de este capítulo.

Dos resultados decimales obtenidos por rutas numéricas distintas no se comparan por igualdad textual; se declara una tolerancia explícita, 10^{-6} para los ejemplos de este capítulo.

7.12.2 Controles mínimos

Antes de confiar en datos reales, conviene usar casos cuyo resultado se conoce:

Constante: AAAAA debe producir 0.

Uniforme: ACGT debe producir 2.

Misma composición: AAAACCCC y ACACACAC deben coincidir.

Vacía: debe fallar con un mensaje y código de salida no cero.

Inválida: ACNT debe seguir la política declarada, no desaparecer silenciosamente.

Normalización: la suma de probabilidades debe ser uno dentro de la tolerancia.

Un oráculo prueba una propiedad concreta, no «demuestra que el programa es correcto» en todos los casos. Los tres primeros casos detectan errores comunes de fórmula; los dos siguientes examinan contratos de entrada; la normalización detecta denominadores incoherentes. Hacen falta todos porque fallos distintos pueden producir el mismo número en un ejemplo afortunado.

7.12.3 Errores de implementación que cambian la ciencia

- **Contar la cabecera FASTA.** Las letras del identificador contaminan el alfabeto y el denominador.

- **Promediar entropías sin ponderar longitudes.** Dos secuencias de 10 y 1000 bases no representan el mismo número de observaciones.
- **Omitir categorías cero en KL cuando P es positivo.** Convierte una divergencia infinita en un número finito incorrecto.
- **Ordenar una tabla por separado.** Si las filas de P y Q dejan de corresponder al mismo símbolo, se comparan categorías distintas.
- **Redondear probabilidades antes de sumar.** La suma puede dejar de ser uno y el error se propaga al logaritmo.
- **Confundir bits por símbolo con bits totales.** El primero es un promedio; el segundo requiere longitud y un modelo conjunto.

Contar letras de una cabecera FASTA junto con la secuencia cambia los conteos y el denominador; en el ejemplo >idA seguido de AAAA, un conteo ingenuo ve siete letras en vez de cuatro símbolos de secuencia.

Promediar sin ponderación entropías de secuencias de longitudes 10 y 1000 con valores 0 y 2 produce 1 bit por símbolo, mientras ponderarlas por número de símbolos produce 1,980198; ambas operaciones responden a preguntas distintas.

Para $P = (0,9, 0,1)$ y $Q = (0,8, 0,2)$, KL es 0,052933 bits si las filas corresponden y 1,652933 bits si las filas de Q se intercambian; ordenar cada tabla por separado puede cambiar la conclusión.

Redondear tres probabilidades $1/3$ a 0,33 antes de sumar produce 0,99: el redondeo intermedio rompe la normalización y debe posponerse hasta presentar el resultado.

7.13 Límites de inferencia biológica

Una entropía de orden cero no es una anotación funcional: las anotaciones de Gene Ontology relacionan un producto génico y un término con una referencia y un código de evidencia. La medida puede priorizar regiones; no aporta por sí misma esos campos ni la evidencia experimental, filogenética o computacional que los códigos distinguen [14]. El límite es verificable: compare la salida del script, formada por cadena y número, con los componentes exigidos por una anotación.

Una entropía de orden cero escalar no determina una filogenia: cualquier permutación de una secuencia conserva el histograma y la entropía, pero el escalar no conserva patrones de caracteres por taxón ni una topología. Este límite se deriva de TC07-A004; no afirma que la teoría de la información sea inútil en filogenia, sino que esta representación concreta ha descartado la información necesaria para reconstruir una historia.

Evaluar exactitud diagnóstica clínica exige declarar una prueba índice, una condición diana, un estándar de referencia y una población objetivo; un valor aislado de entropía no constituye esa evaluación. STARD define esos elementos y pide medidas como sensibilidad y especificidad [4]. En este capítulo no se propone un uso clínico ni se estima ninguna de ellas.

Los otros dos límites ya tienen mecanismos propios: TC07-A037 demuestra que la simetría de MI no proporciona dirección causal, y TC07-A038 documenta por qué la MI cruda entre columnas no basta para inferir un contacto directo. Descomponerlos evita que una sola cita aparente justificar cuatro inferencias distintas.

7.13.1 Cuatro preguntas antes de interpretar

1. **¿Qué distribución se midió?** ¿Símbolos individuales, pares, ventanas, columnas de un alineamiento o categorías preprocesadas?
2. **¿Cuál fue la referencia?** En KL, cambiar Q cambia la pregunta y el valor.
3. **¿Qué incertidumbre tiene la estimación?** Tamaño de muestra, dependencia, sesgo y decisiones sobre ceros afectan al resultado.
4. **¿Qué alternativa explicativa existe?** Composición, calidad, ascendencia compartida, cobertura o preprocesamiento pueden producir una señal.

Una conclusión prudente conserva la escala de la evidencia. «Esta ventana tiene menor entropía de orden cero

que el fondo bajo el mismo alfabeto y ancho» es una afirmación reproducible. «Esta ventana es un gen» no se sigue de ella. «Estas dos columnas presentan información mutua estimada positiva» describe un cálculo. «Estos residuos contactan directamente» requiere evidencia estructural y análisis de dependencias indirectas.

Una conclusión prudente conserva la escala de la evidencia: una diferencia de entropía de orden cero entre una ventana y el fondo es una afirmación reproducible, pero no implica función biológica, y una información mutua estimada positiva describe un cálculo, no un contacto directo, que requeriría evidencia estructural adicional.

7.14 Mapa de decisión para resolver problemas

Ante un nuevo ejercicio, puede recorrerse la siguiente secuencia verbal:

1. *Defina la unidad observacional.* Una base, una ventana, una columna o un par de variables no son intercambiables.
2. *Declare el alfabeto y el soporte.* Enumere resultados posibles, incluso los no observados cuando el modelo los permita.
3. *Separe datos y modelo.* Use letras distintas para frecuencias empíricas y probabilidades de referencia.
4. *Formule la pregunta.* Sorpresa individual, incertidumbre media, desajuste o dependencia conducen a medidas distintas.
5. *Anticipe un caso límite.* Constante, uniforme, independencia o cero de soporte proporcionan un oráculo.
6. *Calcule con traza.* Conserve conteos, probabilidades, contribuciones y unidades antes de resumir.
7. *Perturbe una decisión.* Cambie ventana, pseudoconteo o tratamiento de ambiguos y observe la sensibilidad.
8. *Redacte el límite.* Termine diciendo qué información descartó el modelo y qué conclusión no está justificada.

El orden evita que una herramienta decida la pregunta a posteriori. También hace visible la diferencia entre un error de código y una limitación del modelo: si AAAACCCC y ACACACAC coinciden bajo orden cero, el programa puede estar funcionando exactamente como fue definido; lo que falta es contexto en la representación.

7.15 Síntesis y transición a modelos probabilísticos

La autoinformación convierte probabilidad en sorpresa; la entropía promedia esa sorpresa; la entropía cruzada evalúa datos de una fuente con probabilidades de otro modelo; KL aísla el exceso debido al desajuste; la información mutua compara una distribución conjunta con la independencia. Las cinco cantidades comparten logaritmos y probabilidades, pero responden a preguntas diferentes.

Los ejemplos han mostrado tanto potencia como pérdida. AAAA y ACGT separan concentración y uniformidad. AAAACCCC y ACACACAC demuestran que la composición no conserva el orden. Las ventanas localizan cambios, pero su ancho y solapamiento condicionan el perfil. KL exige dirección y atención al soporte. La información mutua exige verdaderas observaciones emparejadas y no demuestra causalidad.

El siguiente capítulo comienza justo donde termina la entropía de orden cero. Si el símbolo siguiente depende del anterior, una sola distribución marginal es insuficiente. Los modelos de Markov añadirán probabilidades de transición; los HMM añadirán estados no observados que generan símbolos observables. La pregunta ya no será solo «¿qué frecuencias tiene esta secuencia?», sino «¿qué mecanismo probabilístico, con qué dependencias y estados, podría haberla generado?».

Fuentes comentadas y rutas de ampliación

La formulación fundacional de entropía y tasa de información procede de las dos partes publicadas por Shannon [28, 29]. La divergencia dirigida entre distribuciones se apoya en el trabajo de Kullback y Leibler [22]. Para profundizar en propiedades, demostraciones y relaciones entre cantidades, una referencia sistemática es Cover y Thomas [9]. El sesgo y las dificultades de estimar entropía e información mutua a partir de muestras finitas se desarrollan en Paninski [25]. La implementación usa las funciones y reglas documentadas en el manual oficial de GNU awk [12]. Estas fuentes justifican conceptos y métodos; las figuras, ejemplos sintéticos y scripts del capítulo son originales y reproducibles.

Anexo práctico · Auditar información en secuencias

Pregunta, producto y separación de materiales

Dos secuencias pueden compartir composición y entropía de orden cero y, sin embargo, ordenar sus símbolos de manera distinta. Este laboratorio construye una cadena de evidencia para responder: *¿cómo detectar una diferencia local sin presentar la entropía como una anotación biológica?*

El producto individual contiene: el script `analysis_tc07.sh`; su directorio de entrega con resumen, dos perfiles, clasificación y controles; una predicción fechada anterior a la ejecución; una interpretación de 250–400 palabras; y una mini-defensa. El script recibe rutas como argumentos, usa Bash y GNU Awk, funciona sin red y rechaza destinos existentes.

El producto práctico exige script Bash, tablas TSV, controles, predicción previa, interpretación individual y defensa trazable.

El PDF del estudiante no contiene la tabla resuelta, la clasificación del par ni los valores de los perfiles. La clave docente permanece fuera de la distribución y solo se aplica después de la entrega. El analizador de `lib/` es una dependencia fijada, no la solución del script integrador que se evalúa.

El enunciado, el paquete estudiantil y la clave docente son objetos físicamente separados; la raíz distribuida no contiene ni enlaza los oráculos exactos.

Teoría que se recupera.

Este anexo se apoya en cuatro sitios del capítulo y en ninguno más: la fórmula de la entropía, la sección [De una secuencia a una distribución empírica](#) para pasar de conteos a frecuencias, la sección [Entropía por ventanas: localizar sin diagnosticar](#) para elegir la escala de la ventana, y la sección [Límites de inferencia biológica](#) para frenar la inferencia. No se remite a la divergencia ni a la información mutua porque ninguna decisión de esta práctica las utiliza.

Mapa de ruta y productos entregables

La ruta **esencial** conduce al producto completo evaluable. Estudio y referencia son ampliaciones posteriores y no se necesitan para superar el laboratorio. La carga se calcula desde un TSV auditable, no sumando tiempos a ojo.

secciones	ruta	producto	criterio principal
entorno + recuperación Bash	esencial	diagnósticos	trazabilidad, controles
predicción + ejecución guiada	esencial	predicción, resumen	interpretación, cálculo
función + perturbación	esencial	script, perfiles	cálculo
controles + auditoría	esencial	registro, nota adversarial	controles, interpretación
entrega + defensa	esencial	paquete, defensa	trazabilidad, defensa
caso nuevo	estudio	análisis transferido	cálculo
auditoría de implementación	referencia	informe técnico	controles

El mapa ejecutable reparte las secciones del anexo en ruta esencial, de estudio y de referencia; la ruta esencial cubre los cinco criterios de rúbrica.

1 · Entrar en la distribución y verificarla

Desde el directorio del capítulo, monte la raíz estudiantil y compruébela. La raíz es autónoma: contiene el andamiaje de análisis, la biblioteca de apoyo, los datos y el manifiesto que fija sus sumas. No necesita nada de fuera.

```
./practice_assets/create_workspace.sh TC07-student
cd TC07-student
bash check_environment.sh
sha256sum --check --strict manifest.sha256
```

El andamiaje `analysis_tc07.sh` llega con la parte que usted debe escribir marcada como pendiente. El verificador rechaza esa marca: mientras siga ahí, la entrega no se acepta.

La fábrica compila el anexo vigente y publica atómicamente una raíz estudiantil autónoma; el recorrido literal solo consume esa distribución y verifica todos sus hashes.

La prueba de distribución traslada únicamente la raíz entregable a un segundo temporal, impide enlaces de escape y busca nombres, hashes, fragmentos y valores de la clave antes de ejecutar el recorrido.

El diagnóstico comprueba Bash, GNU Awk, `cmp` y las herramientas de archivos requeridas, incluida una comparación funcional, sin depender de la red.

Todos los cálculos C5 invocan explícitamente `gawk`; un ejecutable distinto llamado `awk` no altera el análisis.

El manifiesto de la distribución protege el PDF vigente y los componentes inmutables, pero excluye la plantilla que debe editarse. La fábrica conserva además un manifiesto compuesto privado que fija datos, scripts, clave, parámetros y pruebas de cada artefacto C5.

El manifiesto estudiantil verifica datos, biblioteca, entorno, recuperación y comprobador público antes de calcular.

El manifiesto compuesto C5 enlaza cada reproducción con hashes de datos, scripts, clave docente, prueba y parámetros efectivos.

El FASTA principal distribuido tiene SHA-256:

```
35fac74d29076d48410f2db5ffcd66721d2bc60287ca4c777b9e89c017319953.
```

Los datos son secuencias sintéticas originales, no personales ni clínicas, distribuidas bajo CC0 1.0.

2 • Recuperar Bash antes de leer la plantilla

Este anexo puede distribuirse solo, por lo que no presupone memoria perfecta de capítulos anteriores. Antes de editar, responda: ¿qué diferencia hay entre `$1` y `$#`? ¿cómo se consulta el estado de un comando? ¿por qué se entrecomillan rutas? ¿qué hace una función? ¿por qué un temporal evita dejar una entrega a medias? Si alguna respuesta no es segura, consulte `help functions`, `help test`, `help trap` y `man mktemp`.

Ejecute el diagnóstico de destrezas. No modifica el espacio: usa un directorio temporal y lo elimina al terminar.

```
bash lib/bash_recovery.sh
```

La recuperación ejecutable comprueba funciones, argumentos, rutas, hashes y estados de salida antes de que la plantilla los utilice.

El diagnóstico de destrezas no modifica el espacio de entrega: trabaja en un directorio temporal propio y lo elimina al terminar.

Lea ahora el contrato de `analysis_tc07.sh`: el modo `classify` recibe dos valores y una tolerancia; el modo `analyze` recibe un FASTA y un directorio nuevo. La biblioteca se localiza respecto del propio script, por lo que el espacio puede moverse completo a otra ruta.

`analysis_tc07.sh` expone dos modos: `classify` recibe dos valores y una tolerancia, y `analyze` recibe un FASTA y un directorio nuevo; la biblioteca se localiza respecto del propio script, por lo que el espacio de trabajo puede moverse completo a otra ruta sin romperse.

3 • Conocer los datos y comprometer una predicción

El archivo principal tiene cinco registros sintéticos. Las líneas están envueltas y las cabeceras contienen letras del alfabeto para hacer visible un lector FASTA incorrecto. Antes de ejecutar nada, complete y guarde `prediccion.md` con fecha: longitud, composición esperada, posición relativa de H_0 entre sus cotas, y justificación. No se pide adivinar seis decimales.

registro	longitud	composición	posición de H_0	justificación
constant				
uniform				
blocks				
interleaved				
mixed				

Tarea. Sin IA, derive en `prediccion.md` la entropía de cuatro símbolos equiprobables usando EQ-TC07-ENTROPY. Firme esa predicción antes de ejecutar el resumen.

El cálculo ancla se predice sin IA y se reproduce con Bash antes de interpretar el registro equiprobable.

La evidencia ancla exige una derivación previa y un resultado posterior dentro de las cotas de entropía; la clave exacta no aparece aquí.

Tarea. Prediga qué extremo de la cota alcanza el registro constante y explique por qué aumentar su longitud sin introducir otro símbolo no cambia ese extremo.

El control constante debe alcanzar el extremo de mínima incertidumbre; el estudiante obtiene y registra el valor, no lo copia del PDF.

4 • Ejecución guiada y controles de representación

Solo después de guardar la predicción ejecute:

```
bash lib/tc07_analyze.sh summaries data/tc07_sequences.fasta > resultados/resumen.tsv
cat resultados/resumen.tsv
```

Reconstruya una fila: compruebe que los cuatro conteos suman la longitud, divida por el denominador correcto, verifique que las probabilidades suman uno dentro de 10^{-6} y aplique EQ-TC07-ENTROPY. Compare después con su predicción y anote si el desacuerdo estaba en datos, transformación o interpretación.

El resumen genera una fila por registro del FASTA efectivo; un lector independiente vuelve a derivar longitud y conteos A,C,G,T antes de comprobar normalización y entropía.

El resultado del registro mixto debe caer entre cero y dos bits por símbolo; se contrasta con los conteos derivados del FASTA efectivo, no con una fila prefabricada.

Dos lectores independientes excluyen cabeceras, concatenan líneas y preservan el orden; un FASTA válido generado durante la prueba impide resolver siempre el corpus canónico.

5 • Completar la decisión numérica

Abra el guion de análisis. Hay una única zona marcada como pendiente. La función debe imprimir solo una de las dos etiquetas previstas, ser simétrica al intercambiar los dos valores y rechazar tanto las entradas que no son números como las tolerancias negativas.

La semántica operativa es: comparar la diferencia absoluta con la tolerancia más una guarda de punto flotante de 10^{-12} multiplicada por la mayor escala entre uno, ambos valores y la tolerancia. La guarda absorbe el error binario de los decimales publicados sin aceptar un caso genuinamente exterior.

La clasificación es simétrica y coherente en igualdad, borde por ambos lados, interior y exterior de 10^{-6} ; las entradas no numéricas se rechazan.

No se publica qué etiqueta recibe el par del FASTA. Registre primero su predicción y compruébela después con el modo `classify`. El comprobador público usa relaciones metamórficas y no contiene la tabla docente resuelta.

Tarea. Prediga si `blocks` e `interleaved` serán distinguibles por H_0 . Justifique su predicción desde la composición, no desde el orden visual.

Dos registros con la misma tabla de conteos deben producir la misma entropía de orden cero, aunque su organización posicional difiera.

6 • Perturbar la escala sin sobrescribir

Para longitud n y ancho w , prediga el número de filas mediante $n - w + 1$. Calcule en papel la composición de la primera ventana para dos anchos. No use redirección `>` para guardar tablas: el modo `analyze` escribe todo en un directorio temporal nuevo y solo lo publica si termina completo.

Un perfil exhaustivo de una secuencia de longitud n con ventanas de ancho w contiene $n - w + 1$ filas cuando $1 \leq w \leq n$.

Cambiar el ancho reagrupa símbolos y puede cambiar la entropía de la primera ventana; los valores concretos quedan reservados a la ejecución.

Tarea. Compare anchos 4 y 8. Discuta localización, estabilidad y solapamiento sin afirmar que uno es universalmente mejor. Repita después con `interleaved` y explique qué información recupera el perfil.

El ancho modifica simultáneamente número de ventanas, escala espacial, solapamiento y perfil; la interpretación debe declarar esa elección.

Cuando haya completado la función, genere la entrega mediante el comando literal:

```
bash analysis_tc07.sh analyze data/tc07_sequences.fasta entrega_tc07
```

Esta orden solo funciona cuando la función esté escrita; con el andamiaje tal como llega, termina con error y no crea nada.

Si `entrega_tc07` ya existe, el comando debe fallar y conservarlo. Para repetir, elija otro nombre después de inspeccionar la ejecución anterior; no borre ni sobrescriba evidencia sin decidirlo explícitamente.

Resumen, perfiles y entrega integrada se escriben de forma atómica en destinos nuevos; todo intento de reutilizar el destino falla sin cambiar su hash.

7 • Probar la entrega, no el oráculo

Ejecute el comprobador público parametrizado sobre su propio script:

```
bash check_submission.sh ./analysis_tc07.sh
```

Este comprobador rechaza `TODO-TC07` y `PENDING`, ejecuta ambos modos y fija las etiquetas con valores sintéticos ajenos al FASTA. Prueba igualdad, interior, bordes, exteriores, inversiones y entradas inválidas. Un oráculo Bash/gawk, separado del candidato y sin filas congeladas, relee la ruta efectiva y deriva resumen y perfiles. En cada ejecución se elige una semilla nueva y se crean dos FASTA con longitudes, composición, envoltura, orden e identificadores auxiliares variables. Se imprimen semilla y hashes para repetir el desafío. La prueba docente elige otra semilla después de recibir el candidato.

La prueba contrasta los cinco productos con un oráculo sobre desafíos positivos elegidos en tiempo de ejecución, además de decidir etiquetas, controles, atomicidad y no sobrescritura.

La regresión conserva los mutantes `r3/r4` y rechaza además un candidato que solo admite los tres hashes positivos conocidos antes de `r5`.

El oráculo deriva resumen y ventanas del FASTA efectivo sin reutilizar el analizador ni tablas congeladas.

Público y docente generan dos entradas válidas por semilla, registran datos de replay y exigen que hash y productos sigan cada desafío.

La plantilla es sintácticamente válida y localiza su biblioteca en la copia limpia, pero no supera la prueba de entrega hasta completar el TODO y eliminar PENDING.

Los controles negativos rechazan un símbolo fuera del alfabeto y un registro FASTA sin secuencia mediante estado distinto de cero.

control	propiedad	fallo buscado
extremos conocidos	dentro de cotas	fórmula o base incorrecta
normalización	suma 1 ± 10^{-6}	denominador incoherente
FASTA inválido/vacío	estado no cero	filtrado silencioso
cabecera y orden	representación estable	contaminación/reordenación
tolerancia	simetría, bordes, exterior	punto flotante o igualdad textual
destino existente	fallo e igual hash	sobrescritura silenciosa
entrada alternativa	cambian hash y productos	ignorar la ruta recibida
entrega parametrizada	cero TODO/PENDING	probar el candidato equivocado

La batería cubre procedencia por entrada alternativa, normalización, cotas, cabecera, orden, inválidos, vacíos, tolerancia simétrica, coherencia entre productos, destino existente y candidato real.

El resumen declara ocho campos TSV estables: identificador, longitud, cuatro conteos, suma de probabilidades y H_0 en bits.

La prueba de fábrica solo emite PRACTICE_C5_R5_OK después de probar distribución, candidato, oráculo, desafíos runtime reproducibles, mutantes, entorno, controles, no sobrescritura y tolerancia.

El diagnóstico rechaza un cmp hostil antes del comprobador público.

El archivo `controls.log` no es un mero diagnóstico. Conserva el hash de entrada y un resumen PASS de igualdad, interior, bordes, exteriores, entradas numéricas, FASTA negativos, esquemas y condición de destino nuevo. Si falla un control, la entrega temporal no se publica.

La entrega registra resultados verificables de la batería interna en `controls.log`; el comprobador público exige cada control y el hash de la entrada efectiva.

8 • Auditar una conclusión engañosa

Considere: «los dos registros obtienen el mismo escalar; por tanto tienen la misma organización y función». Desmonte dato, representación, resultado y conclusión. Use el perfil para investigar orden local, pero no lo convierta en anotación.

eslabón	evidencia disponible	pregunta adversarial
datos	secuencias y metadatos	¿se excluyeron cabeceras e inválidos?
representación	histograma H_0	¿qué relaciones posicionales descarta?
resultado	escalares calculados	¿cambia la lectura con ventanas?
conclusión	comparación limitada	¿qué evidencia requeriría función?

Una coincidencia de entropía de orden cero no establece igual organización, función, anotación ni mecanismo biológico.

Un perfil por ventanas puede impugnar igualdad de organización local, pero no establece por sí solo función biológica.

Tarea. Redacte tres frases: resultado verificable; límite de representación; y siguiente comprobación proporcionada. No añada una promesa que los datos no sostienen.

9 • Entrega, IA y contingencia

Entregue script, carpeta generada, prediccion.md, interpretación, comando, versiones y hash. Ante una incidencia, conserve mensaje, estado y último control verde. Sin gnuplot, las tablas siguen siendo la evidencia evaluada.

La representación gráfica es opcional; el producto evaluado se completa íntegramente con tablas locales.

Predicción, primera interpretación y defensa son sin IA; una ayuda posterior de depuración exige registro, contraste local y explicación.

La ruta sin IA usa este PDF, help, manuales y pruebas locales. Con ayuda, toda sugerencia se contrasta con el candidato; ninguna fluidez sustituye controles.

10 • Mini-defensa y rúbrica

Tarea. Defienda por qué los controles extremos son necesarios pero insuficientes, qué cambia al modificar el ancho y qué evidencia pediría antes de hablar de función. Prediga en directo el efecto de una perturbación propuesta.

criterio	peso	evidencia observable
corrección computacional	30%	candidato, resumen, perfiles y tolerancia
controles y robustez	25%	negativos, bordes y no sobrescritura
interpretación	25%	resultado, supuesto, límite y siguiente prueba
trazabilidad	10%	predicción, versiones, hashes y comando
defensa individual	10%	explicación y perturbación en directo

La rúbrica asigna 30% a cálculo, 25% a controles, 25% a interpretación, 10% a trazabilidad y 10% a defensa.

La defensa examina controles, sensibilidad al ancho, límites y predicción ante una perturbación.

11 • Estudio, referencia y estado de los PDF

ESTUDIO Diseñe un sexto registro que conserve composición y cambie dependencia local. Prediga resumen y perfil; añada un control sin alterar los datos congelados.

REFERENCIA Audite una función de `lib/tc07_analyze.sh`: entrada, salida, error y operación anti-sobrescritura. Consulte la implementación tras comprometer sus predicciones; la solución queda fuera del paquete.

La fábrica compila `TC07-practice.pdf` desde la fuente vigente; ningún derivado local *superseded* entra en la distribución.

Cierre.

H_0 resume composición y la ventana añade escala espacial; ninguna demuestra función. Predicción, datos, candidato, controles e interpretación deben sostener la misma afirmación.

Bayes, cadenas de Markov y modelos ocultos

TC-CH08 – Técnicas Computacionales en Biología

Edición 2

Pregunta del tema. Dos regiones del genoma tienen exactamente las mismas proporciones de A, C, G y T. Una es una isla CpG y la otra no. ¿Qué hay que medir, si las frecuencias no bastan, y cómo se decide cuando la propiedad que interesa no está escrita en la secuencia?

Producto final. Una matriz de transición estimada por usted a partir de una secuencia, validada fila a fila, y una decodificación de Viterbi cuyo camino óptimo se contrasta con lo que diría mirar cada posición por separado.

Mapa del tema

8.1. De medir a decidir	135
8.2. Probabilidad condicional: la pieza que faltaba	136
8.3. Clasificar con Bayes	136
8.4. Cadenas de Markov: recuperar el orden	138
8.5. Del estado observado al oculto (HMM)	141
8.6. Trazar Viterbi	142
8.7. La operación peligrosa: matrices no normalizadas y ceros	145
8.8. Actividad de depuración: errores en Viterbi	145
8.9. Síntesis operativa y tablas de diagnóstico	145
8.10. Glosario conceptual	146
8.11. Conexión con el curso y siguientes pasos	146
8.12. Fuentes y reproducibilidad	146

Cómo usar este tema

El capítulo anterior medía. Este decide. La diferencia no es de grado: medir la entropía de una región no dice a qué clase pertenece, y para eso hace falta un modelo de cómo se generan las secuencias de cada clase. Aquí se construyen tres, cada vez menos ingenuos: uno que supone que las posiciones son independientes, otro que mira el símbolo anterior, y otro donde lo que interesa ni siquiera se observa.

Al terminar sabrá: aplicar la regla de Bayes para decidir entre dos hipótesis y explicar qué supone el adjetivo «ingenuo»; estimar una matriz de transición contando pares y comprobar que sus filas suman uno; contar cuántos parámetros necesita un modelo de orden creciente, y ver por qué eso lo hace inaplicable; y seguir una traza de Viterbi entendiendo por qué el camino óptimo global no es la suma de las mejores decisiones locales.

La ruta **esencial** son Bayes con su regla de decisión, las cadenas de Markov con su estimación y validación, y Viterbi con su traza. La ruta de **estudio** añade la frecuencia cero, el paso a logaritmos, la explosión de parámetros y el anexo.

8.1 De medir a decidir

ESENCIAL Medir el tiempo o el coste de un algoritmo (TC-CH03–TC-CH06) no responde cuánto orden o incertidumbre contiene una secuencia; contar frecuencias y calcular la entropía de Shannon (TC-CH07) sí cuantifica la diversidad estática de los símbolos, pero no dice nada sobre qué mecanismo biológico los generó ni sobre la dependencia de orden entre posiciones sucesivas.

Este capítulo da el salto metodológico fundamental: pasar de **medir** a **decidir** y **modelar**:

- **Clasificar:** Dada una secuencia desconocida de ADN, ¿pertenece a una región codificante de un gen o a un intrón no codificante? (**Clasificador Naïve-Bayes**).
- **Modelar transiciones:** ¿Cómo influye el nucleótido anterior en la probabilidad del siguiente para distinguir islas CpG de la heterocromatina? (**Cadenas de Markov**).
- **Decodificar estados biológicos ocultos:** Si la función genómica (promotor vs fondo intergénico) no viene etiquetada directamente en la molécula, ¿cómo reconstruir la secuencia más probable de estados que emitió los nucleótidos observados? (**Modelos Ocultos de Markov – HMM y Algoritmo de Viterbi**).

Contar frecuencias y calcular entropía describe una distribución estática de símbolos, pero no dice qué mecanismo genera esos símbolos ni si existe dependencia entre posiciones sucesivas. Bayes, Markov y los modelos ocultos añaden, sucesivamente: una regla de decisión entre clases; una hipótesis de generación con memoria de un símbolo; y un mecanismo generador cuyo estado no se observa directamente.

8.2 Probabilidad condicional: la pieza que faltaba

Para modelar dependencias se necesita la **probabilidad condicional**, $P(A | B)$ —la probabilidad de que ocurra el suceso A dado que ya se conoce con certeza la ocurrencia del suceso B —:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad \text{con } P(B) > 0$$

La probabilidad condicional $P(A | B)$ es la probabilidad de que ocurra el suceso A sabiendo que ya ha ocurrido el suceso B ; es en general distinta de la probabilidad no condicionada $P(A)$, porque incorpora la información de que B ya se sabe cierto.

Dos sucesos son **independientes** si y solo si conocer uno no altera en absoluto la probabilidad del otro: $P(A | B) = P(A)$. Si $P(A | B) \neq P(A)$, el suceso B aporta información predictiva sobre A , y asumir independencia destruye la estructura informativa real de la secuencia.

Dos sucesos A y B son independientes cuando $P(A | B) = P(A)$: saber que ocurrió B no cambia la probabilidad de A . Cuando $P(A | B) \neq P(A)$, B aporta información real sobre A y tratarlos como independientes pierde esa información.

Regla del producto y Regla de la cadena.

De la definición condicional se obtiene la **regla del producto**: $P(A \cap B) = P(A | B) \cdot P(B)$. Aplicada iterativamente sobre una secuencia de n símbolos $x_1, x_2, \dots, x_n$, se deriva la **regla de la cadena exacta**:

$$P(x_1, x_2, \dots, x_n) = P(x_1) \cdot P(x_2 | x_1) \cdot P(x_3 | x_1, x_2) \cdots P(x_n | x_1, \dots, x_{n-1})$$

La regla del producto, derivada directamente de la definición de probabilidad condicional, dice $P(A \cap B) = P(A | B) \cdot P(B)$. Aplicada en cadena a n sucesos da la regla de la cadena: $P(x_1, \dots, x_n) = P(x_1) \cdot P(x_2 | x_1) \cdots P(x_n | x_1, \dots, x_{n-1})$, el producto de cada símbolo condicionado a todos los anteriores.

8.3 Clasificar con Bayes

El objetivo de la clasificación en bioinformática es asignar una secuencia observada X a una de varias clases candidatas $C \in \{\text{Cod}, \text{NCod}\}$ (región codificante vs no codificante).

8.3.1 El Teorema de Bayes

El **teorema de Bayes** invierte la dirección de la inferencia:

$$P(C | X) = \frac{P(X | C) \cdot P(C)}{P(X)} = \frac{\text{Verosimilitud} \times \text{Probabilidad a Priori}}{\text{Evidencia Marginal}}$$

Donde:

- $P(C)$: Probabilidad a priori de la clase (proporción global en el genoma).
- $P(X | C)$: Verosimilitud de observar la secuencia X dentro de la clase C .
- $P(X) = \sum_C P(X | C)P(C)$: Evidencia normalizadora, idéntica para todas las clases que compiten.

El teorema de Bayes calcula la probabilidad a posteriori de una clase a partir de tres cantidades estimables: la verosimilitud $P(\text{datos} | \text{clase})$, la probabilidad a priori $P(\text{clase})$ y la evidencia $P(\text{datos})$, que actúa como normalizador y es idéntica para todas las clases que compiten.

8.3.2 El supuesto «ingenuo» (Naïve Bayes)

Calcular la verosimilitud exacta $P(x_1, \dots, x_n | C)$ requeriría estimar las probabilidades de las 4^n posibles secuencias completas, lo cual es inviable. El clasificador **Naïve-Bayes** asume la hipótesis simplificadora de que los nucleótidos de la secuencia son condicionalmente independientes entre sí dada la clase:

$$P(x_1, x_2, \dots, x_n | C) \approx \prod_{i=1}^n P(x_i | C)$$

Aunque ignora el orden de las bases, Naïve-Bayes es extremadamente eficaz en la práctica porque solo necesita ordenar correctamente las clases candidatas, no estimar probabilidades exactas.

El supuesto ingenuo de Naïve Bayes asume que los atributos son condicionalmente independientes dada la clase, lo que descompone la verosimilitud en el producto de las probabilidades de cada atributo; esta independencia casi nunca es exacta —ignora el orden de las bases—, pero el método funciona bien porque solo necesita ordenar correctamente las clases, no estimar probabilidades exactas.

8.3.3 Regla de decisión y ejemplos resueltos

Dado que el denominador $P(X)$ es constante para todas las clases, la regla de decisión consiste en seleccionar la clase con mayor producto de a priori por verosimilitud:

$$\hat{C} = \arg \max_C \left[P(C) \prod_{i=1}^n P(x_i | C) \right]$$

La regla de decisión de Naïve Bayes calcula para cada clase candidata el producto de la probabilidad a priori por las verosimilitudes de cada atributo —descartando el denominador del teorema de Bayes por ser idéntico para todas las clases— y elige la clase que obtiene el valor más alto.

Consideremos un modelo entrenado con los siguientes parámetros:

- **Clase Codificante (Cod):** $P(\text{Cod}) = 0.45$; $P(A | \text{Cod}) = 0.4$, $P(C | \text{Cod}) = 0.2$, $P(G | \text{Cod}) = 0.3$, $P(T | \text{Cod}) = 0.1$.
- **Clase No Codificante (NCod):** $P(\text{NCod}) = 0.55$; $P(A | \text{NCod}) = 0.3$, $P(C | \text{NCod}) = 0.3$, $P(G | \text{NCod}) = 0.2$, $P(T | \text{NCod}) = 0.2$.

Caso 1: Clasificación de 'AAGT'.

- $P(\text{Cod} | \text{AAGT}) \propto 0.45 \times 0.4 \times 0.4 \times 0.3 \times 0.1 = \mathbf{0.00216}$.
- $P(\text{NCod} | \text{AAGT}) \propto 0.55 \times 0.3 \times 0.3 \times 0.2 \times 0.2 = \mathbf{0.00198}$.

Gana la clase **Cod**, pero con un margen estrecho (0.00216 vs 0.00198), lo que refleja incertidumbre diagnóstica.

Con el modelo entrenado del ejemplo, con probabilidades condicionadas 0.4, 0.2, 0.3 y 0.1 para la clase codificante y previa 0.45, y 0.3, 0.3, 0.2 y 0.2 para la no codificante y previa 0.55, clasificar AAGT da $P(\text{Cod} | \text{AAGT})$ proporcional a 0.00216 y $P(\text{NCod} | \text{AAGT})$ proporcional a 0.00198: gana Cod, pero la decisión es ajustada y no una certeza.

Se reproduce con `verification/chapter_results.sh naive_bayes_classify AAGT`.

Caso 2: Clasificación de 'CGT'.

- $P(\text{Cod} | \text{CGT}) \propto 0.45 \times 0.2 \times 0.3 \times 0.1 = \mathbf{0.0027}$.
- $P(\text{NCod} | \text{CGT}) \propto 0.55 \times 0.3 \times 0.2 \times 0.2 = \mathbf{0.0066}$.

Gana **NCod** con una holgura contundente ($\approx 2.44\times$ superior a Cod). Comparar el margen entre clases distingue decisiones sólidas de empates técnicos.

Sobre el mismo modelo entrenado, clasificar CGT da $P(\text{Cod} | \text{CGT})$ proporcional a 0.0027 y $P(\text{NCod} | \text{CGT})$ proporcional a 0.0066: gana NCod con un margen relativo mucho más amplio (NCod casi 2.4 veces Cod) que en el ejemplo AAGT —una decisión holgada, no un empate técnico—. Comparar el margen entre clases, y no solo la clase ganadora, distingue una decisión sólida de una dudosa.

Se reproduce con `verification/chapter_results.sh naive_bayes_classify CGT`.

8.3.4 El problema de la frecuencia cero y el subdesbordamiento numérico

1. Suavizado de Laplace (+1).

Si un nucleótido nunca apareció en los datos de entrenamiento de una clase, su frecuencia estimada es 0. Dado que la verosimilitud es un producto, ese único cero anula completamente el valor de la clase ($0 \times \dots = 0$), aunque

todos los demás nucleótidos encajen a la perfección. Para solucionarlo, el **suavizado de Laplace** suma 1 a todos los conteos antes de normalizar:

$$P(x_i | C) = \frac{\text{Conteo}(x_i) + 1}{N_C + |\Sigma|}$$

donde $|\Sigma| = 4$ para el alfabeto de nucleótidos.

Si un nucleótido nunca apareció en el conjunto de entrenamiento de una clase, su probabilidad estimada es 0; como la verosimilitud es un producto, ese único factor cero anula por completo la clase, aunque el resto de atributos encajen perfectamente. El suavizado de Laplace lo corrige sumando 1 a cada conteo antes de calcular frecuencias, garantizando que ninguna probabilidad estimada sea exactamente 0.

Se reproduce con `verification/chapter_results.sh laplace_smoothing_demo`.

2. Cómputo en espacio logarítmico.

Al multiplicar cientos de probabilidades pequeñas ($P < 1$), el resultado cae por debajo de la precisión mínima de la coma flotante ($< 10^{-308}$), produciendo un **subdesbordamiento numérico** (*underflow*) a cero. La solución estándar consiste en operar en el dominio logarítmico, transformando los productos en sumas:

$$\log P(C | X) \propto \log P(C) + \sum_{i=1}^n \log P(x_i | C)$$

Dado que el logaritmo es una función estrictamente monótona creciente, la clase que maximiza la suma de logaritmos es idéntica a la que maximiza el producto.

Multiplicar muchas probabilidades pequeñas —una verosimilitud por símbolo de una secuencia larga— provoca un subdesbordamiento a cero en coma flotante, perdiendo la información necesaria para decidir. Sumar logaritmos en lugar de multiplicar probabilidades evita el subdesbordamiento —el logaritmo convierte productos en sumas— y, al ser una función creciente, produce exactamente la misma clase ganadora que el cálculo sin logaritmos.

Se reproduce con `verification/chapter_results.sh log_space_classify AAGT`.

8.4 Cadenas de Markov: recuperar el orden

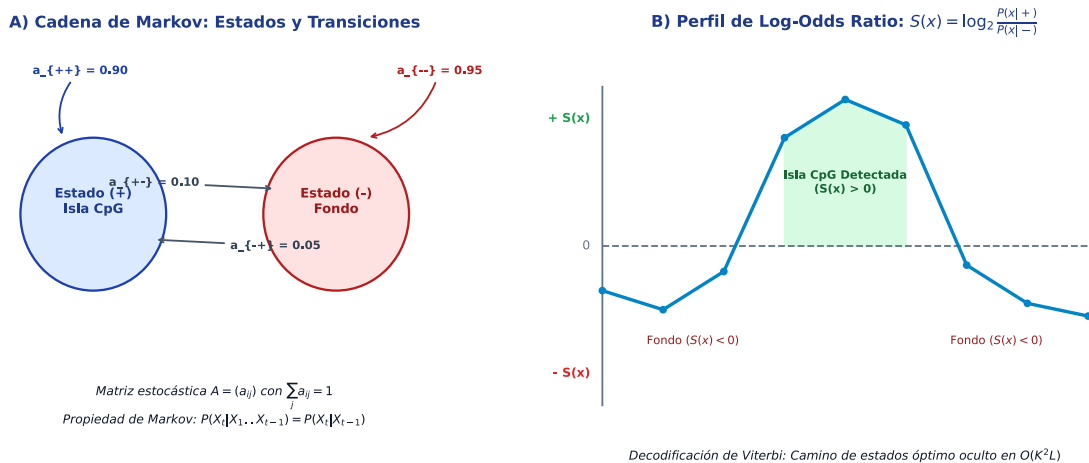


Figura 8.1: A) Modelo de Cadena de Markov discreta con estados y probabilidades de transición a_{ij} . B) Perfil de $\log\text{-odds ratio}$ $S(x) = \log_2 \frac{P(x|+)}{P(x|-)}$ para la discriminación estadística de islas CpG frente al fondo genómico neutro. Figura original adaptada de las presentaciones docentes.

8.4.1 La propiedad de Markov de orden 1

Para superar la ceguera de Naïve Bayes ante el orden de los nucleótidos, las **cadenas de Markov de orden 1** establecen que la probabilidad de un símbolo en la posición i depende exclusivamente del símbolo inmediatamente

anterior x_{i-1} :

$$P(x_i | x_1, x_2, \dots, x_{i-1}) = P(x_i | x_{i-1})$$

La propiedad de Markov de orden 1 establece que la probabilidad del símbolo en la posición i depende únicamente del símbolo en la posición $i - 1$, no de toda la secuencia precedente; se sitúa a medio camino entre Naïve Bayes (orden 0, ninguna dependencia) y un modelo que condicionara a todas las posiciones anteriores.

8.4.2 De una secuencia a una matriz de transición

ESENCIAL La matriz que vamos a validar no viene dada: se estima. El procedimiento es el mismo conteo y normalización del capítulo anterior, aplicado ahora a pares de posiciones y no a símbolos sueltos.

Recorra la secuencia contando cada par de posiciones adyacentes. Sobre ACGACGT, los seis pares son AC, CG, GA, AC, CG y GT. Agrupados por su primer símbolo:

Contexto	→A	→C	→G	→T	total
A	0	2	0	0	2
C	0	0	2	0	2
G	1	0	0	1	2
T	0	0	0	0	0

Cada fila se divide por su propio total, no por el total general. Así cada fila suma uno y se convierte en la distribución condicional de lo que viene después de ese contexto. De la fila A sale $P(C | A) = 1$; de la fila G salen $P(A | G) = P(T | G) = 0,5$.

Atención. La fila T está vacía porque en esta secuencia la T nunca aparece seguida de nada: es el último símbolo. Una fila sin observaciones no se puede normalizar, y dejarla a cero rompe la invariante que comprobaremos enseguida. Es el mismo problema de la frecuencia cero que ya vimos, y se resuelve igual: con suavizado, y declarándolo.

Comprueba. Con la matriz que acaba de construir, y tomando $P(A) = 1/7$, ¿cuánto vale la probabilidad de la secuencia ACG? ¿Y por qué no puede calcular la de ACGT sin suavizar antes?

8.4.3 Definición formal y validación de la matriz de transición

Una cadena de Markov discreta se define formalmente mediante tres elementos:

1. El conjunto de estados posibles $S = \{A, C, G, T\}$.
2. El vector de probabilidades iniciales $\pi = [P(A), P(C), P(G), P(T)]$, con $\sum \pi_i = 1$.
3. La **matriz de transición** A , donde cada elemento $a_{jk} = P(s_k | s_j)$ representa la probabilidad de transitar del estado s_j al s_k .

Invariante fundamental:

Cada fila de la matriz de transición debe sumar estrictamente 1:

$$\sum_k P(s_k | s_j) = 1, \quad \forall j$$

Una cadena de Markov se define por tres componentes: los estados posibles, la probabilidad inicial de cada estado y una matriz de transición con las probabilidades condicionales de pasar de cada estado al siguiente. Cada fila de la matriz de transición debe sumar exactamente 1, porque desde un estado dado la cadena pasa con certeza a alguno de los estados posibles.

La probabilidad conjunta de una secuencia completa bajo una cadena de Markov es:

$$P(x_1, x_2, \dots, x_n) = P(x_1) \prod_{i=2}^n P(x_i | x_{i-1})$$

La probabilidad de una secuencia completa bajo una cadena de Markov de orden 1 es la probabilidad inicial del primer símbolo multiplicada por el producto de cada transición sucesiva. A diferencia de Naïve Bayes, donde cada factor es un $P(x_i)$ independiente, aquí cada factor es condicional al símbolo anterior; esa es la diferencia formal entre ignorar el orden y modelarlo.

```

ALGORITMO: Cadena_Markov_Evaluacion_Probabilidad
ENTRADA:
- X: Secuencia observada de nucleotidos x_0, x_1, ..., x_{T-1} de longitud T
- pi: Vector de probabilidades iniciales pi[c] para cada c en Sigma
- A: Matriz estocastica de transicion A[origen, destino]
SALIDA:
- Log_Verosimilitud: log2(P(X | Modelo))
COMPLEJIDAD: Tiempo O(T), Espacio auxiliar O(1)

1. INICIALIZACION EN ESPACIO LOGARITMICO:
  Si pi[X[0]] <= 0 entonces:
    Retornar -INFINITO # Emision inicial imposible bajo el modelo

  Log_P <- log2(pi[X[0]])

2. PRODUCTO DE TRANSICIONES SUCESIVAS:
  Para t desde 1 hasta T - 1:
    previo <- X[t - 1]
    actual <- X[t]

    prob_transicion <- A[previo, actual]
    Si prob_transicion <= 0 entonces:
      Retornar -INFINITO # Transicion prohibida

  Log_P <- Log_P + log2(prob_transicion) # Suma de logaritmos previene underflow

3. RETORNO:
  Retornar Log_P

```

8.4.4 Trazas de cálculo y matrices inválidas

Consideremos el modelo de transición entre nucleótidos con $\pi(A) = 0.3$, $\pi(C) = 0.2$, $\pi(G) = 0.3$, $\pi(T) = 0.2$ y la matriz A:

$s_{i-1} \setminus s_i$	A	C	G	T
A	0.1	0.4	0.4	0.1
C	0.2	0.3	0.1	0.4
G	0.3	0.2	0.3	0.2
T	0.2	0.2	0.3	0.3

Traza 1: Probabilidad de 'ACG'.

$$P(ACG) = P(A) \cdot P(C | A) \cdot P(G | C) = 0.3 \times 0.4 \times 0.1 = \mathbf{0.012}$$

Con probabilidad inicial $P(A) = 0.3$ y la matriz de transición del ejemplo resuelto, la probabilidad de ACG es $P(A) \cdot P(C | A) \cdot P(G | C) = 0.3 \times 0.4 \times 0.1 = 0.012$.

Se reproduce con `verification/chapter_results.sh markov_sequence_prob ACG`.

Traza 2: Probabilidad de 'CGT'.

$$P(CGT) = P(C) \cdot P(G | C) \cdot P(T | G) = 0.2 \times 0.1 \times 0.2 = \mathbf{0.004}$$

Con probabilidad inicial $P(C) = 0.2$ y la misma matriz de transición, la probabilidad de CGT es $P(C) \cdot P(G | C) \cdot P(T | G) = 0.2 \times 0.1 \times 0.2 = 0.004$.

Se reproduce con `verification/chapter_results.sh markov_sequence_prob CGT`.

Detección de matrices corruptas.

Una fila como $[0.1, 0.5, 0.4, 0.2]$ suma $1.2 \neq 1.0$. No constituye una distribución probabilística válida y cualquier cálculo derivado de ella carece de validez matemática.

Una fila de matriz de transición $[0.1, 0.5, 0.4, 0.2]$ suma 1.2, no 1: no es una distribución de probabilidad válida y cualquier cálculo de probabilidad de secuencia que la use queda invalidado, por correcto que parezca el resultado. Validar que cada fila sume 1 debe hacerse antes de calcular, no tras obtener un resultado sospechoso.

Se reproduce con `verification/chapter_results.sh validate_transition_row`.

8.4.5 Explosión de parámetros y detección de islas CpG

Explosión de parámetros en orden superior.

Un modelo de Markov de orden k sobre un alfabeto de tamaño $|\Sigma|$ define $|\Sigma|^k$ contextos posibles y almacena $|\Sigma|^{k+1}$ probabilidades condicionales en su matriz. Puesto que las probabilidades de cada fila deben sumar obligatoriamente 1, cada contexto impone una ligadura matemática, dejando exactamente $|\Sigma|^k(|\Sigma| - 1)$ grados de libertad o parámetros libres independientes. Para ADN ($|\Sigma| = 4$) y $k = 3$, se tienen $4^3 = 64$ contextos, 256 probabilidades almacenadas y 192 parámetros libres independientes (más la distribución inicial). Con conjuntos de datos pequeños, la inmensa mayoría de los contextos no aparecen nunca, provocando sobreajuste o estimaciones nulas.

Un modelo de Markov de orden k sobre un alfabeto de 4 símbolos almacena 4^{k+1} probabilidades condicionales organizadas en 4^k contextos de 4 celdas cada uno, lo que supone $4^k \times 3$ grados de libertad independientes. Para $k = 3$ son 64 contextos, 256 probabilidades almacenadas y 192 parámetros libres; con conjuntos de entrenamiento reducidos, la mayoría de esos contextos apenas aparecerán, provocando sobreajuste o estimaciones nulas.

Se reproduce con `verification/chapter_results.sh markov_order_params 2`.

Detección de islas CpG.

Dos regiones con idéntica proporción de A, C, G y T resultan indistinguibles para Naïve Bayes (orden 0). Sin embargo, una cadena de Markov de orden 1 las discrimina directamente a través de la probabilidad $P(G | C)$, que es mucho más elevada en promotores e islas CpG desmetiladas que en el genoma general (donde la metilación de citosinas y posterior desaminación a timina provoca una depleción característica de dinucleótidos CG).

Dos regiones de ADN con idénticas frecuencias de A, C, G y T pueden diferir radicalmente en su proporción de transiciones C-seguida-de-G (islas CpG frente a ADN genómico general): un modelo de orden 0 (frecuencias sueltas como en Naïve Bayes) es ciego a esa diferencia porque nunca condiciona al símbolo anterior; un modelo de orden 1 la distingue directamente en la celda $P(G | C)$ de su matriz de transición, mucho más alta en una isla CpG que en el genoma general.

Se reproduce con `verification/chapter_results.sh cpG_vs_bulk_transitions`.

8.5 Del estado observado al oculto (HMM)

En muchas aplicaciones genómicas reales, la propiedad funcional de una base (si pertenece a un **Promotor** o a una región **No Promotora**) no está escrita explícitamente: es un **estado oculto**.

Un **Modelo Oculto de Markov (HMM)** desacopla el sistema en dos capas:

1. **Capa oculta:** Una secuencia de estados no observables $q_1, q_2, \dots, q_n$ que evoluciona como una cadena de Markov con probabilidades iniciales π y matriz de transición entre estados $A = (a_{ij})$.
2. **Capa observada:** La secuencia de nucleótidos visibles $x_1, x_2, \dots, x_n$, emitida por cada estado según una matriz de emisión $B = (b_j(x_k))$.

Un modelo oculto de Markov (HMM) separa dos capas: los estados, que no se observan directamente y evolucionan como una cadena de Markov (con probabilidad inicial y matriz de transición entre estados), y las

observaciones, los símbolos visibles que cada estado emite según su propia distribución de emisión. Un estado como «promotor» es una hipótesis del modelo, no una etiqueta visible en la molécula.

8.5.1 Los tres problemas clásicos de los HMM

La teoría de HMM plantea tres problemas fundamentales:

1. **Evaluación:** Calcular la probabilidad total de la secuencia observada $P(X | \lambda)$ mediante el **Algoritmo Forward** ($O(N^2T)$).
2. **Decodificación:** Determinar la secuencia óptima de estados ocultos $\hat{Q}$ que generó las observaciones mediante el **Algoritmo de Viterbi**.
3. **Aprendizaje:** Ajustar los parámetros del modelo a partir de secuencias no anotadas mediante el **Algoritmo de Baum-Welch** (EM).

Sobre un HMM se plantean tres preguntas distintas: evaluación (probabilidad total de una secuencia de observaciones bajo el modelo, resuelta con el algoritmo forward), decodificación (secuencia de estados ocultos más probable, resuelta con el algoritmo de Viterbi) y aprendizaje (estimar parámetros a partir de datos, p. ej. Baum-Welch). Este capítulo desarrolla la decodificación con parámetros dados; el aprendizaje no supervisado queda fuera de alcance.

8.5.2 Definición del modelo HMM para promotores

Definimos un HMM de dos estados para la identificación de promotores génicos: $S = \{P, NP\}$ (Promotor y No Promotor), con $\pi(P) = 0.5$ y $\pi(NP) = 0.5$.

- **Matriz de transición entre estados (A):** Favorece la persistencia dentro del mismo estado: $P(P | P) = 0.7$, $P(NP | P) = 0.3$; $P(NP | NP) = 0.8$, $P(P | NP) = 0.2$.
- **Matriz de emisión de símbolos (B):** El estado promotor emite preferentemente motivos ricos en A y T (caja TATA): $b_P(A) = 0.4$, $b_P(T) = 0.3$, $b_P(C) = 0.1$, $b_P(G) = 0.2$. El estado no promotor emite de forma uniforme: $b_{NP}(A) = b_{NP}(C) = b_{NP}(G) = b_{NP}(T) = 0.25$.

En el HMM de dos estados para identificar promotores (P=promotor, NP=no promotor), la matriz de transición favorece la persistencia en ambos estados ($P(P|P)=0.7$, $P(NP|NP)=0.8$) frente al cambio ($P(NP|P)=0.3$, $P(P|NP)=0.2$); la matriz de emisión sesga el estado P hacia A y T ($P(A|P)=0.4$, $P(T|P)=0.3$ frente a $P(C|P)=0.1$, $P(G|P)=0.2$), mientras que NP emite las cuatro bases de forma equiprobable (0.25 cada una).

8.6 Trazar Viterbi

8.6.1 El Algoritmo de Viterbi (Programación Dinámica)

El algoritmo de Viterbi encuentra la secuencia de estados ocultos globalmente óptima que maximiza $P(Q, X | \lambda)$ mediante programación dinámica sobre la variable $\delta_t(j)$, que representa la probabilidad del camino más probable que termina en el estado j en el instante t :

$$\delta_t(j) = \max_{1 \leq i \leq N} [\delta_{t-1}(i) \cdot a_{ij}] \cdot b_j(x_t)$$

Simultáneamente, se registra en una matriz de punteros $\psi_t(j)$ el estado predecesor i que maximizó dicha probabilidad:

$$\psi_t(j) = \arg \max_{1 \leq i \leq N} [\delta_{t-1}(i) \cdot a_{ij}]$$

El algoritmo de Viterbi es programación dinámica sobre $\delta_t(j) = b_j(x_t) \cdot \max_i [\delta_{t-1}(i) \cdot a_{ij}]$: cada celda guarda la mejor puntuación de un camino de estados que termina en el estado j en el instante t , junto con un puntero al estado predecesor que logró esa puntuación. El camino completo se reconstruye retrocediendo los punteros desde el mejor estado final. En la práctica se calcula en logaritmos por la misma razón que Naïve Bayes: evitar el subdesbordamiento numérico.

Convención de indexación y pseudocódigo del Algoritmo de Viterbi:

En este texto adoptamos la indexación estándar en computación $t = 0, \dots, T - 1$ para una secuencia observada de longitud T ($x_0, x_1, \dots, x_{T-1}$):

1. **Inicialización** ($t = 0$): Para cada estado j , calcular $\delta_0(j) = \pi_j \cdot b_j(x_0)$ y registrar puntero inicial $\psi_0(j) = \perp$ (sin predecesor).
2. **Inducción recursiva** ($t = 1 \dots T - 1$): Para cada instante t y cada estado destino j :

$$\delta_t(j) = \max_i [\delta_{t-1}(i) \cdot a_{ij}] \cdot b_j(x_t), \quad \psi_t(j) = \arg \max_i [\delta_{t-1}(i) \cdot a_{ij}]$$
3. **Terminación** ($t = T - 1$): Identificar el mejor estado final $q_{T-1}^* = \arg \max_j \delta_{T-1}(j)$.
4. **Trazado hacia atrás (Backtracking)**: Recuperar recursivamente los estados previos $q_t^* = \psi_{t+1}(q_{t+1}^*)$ desde $t = T - 2$ descendiendo hasta $t = 0$.

El pseudocódigo de Viterbi inicializa δ en $t = 0$ con la probabilidad inicial por la emisión del primer símbolo para cada estado, registra que allí no hay predecesor, avanza paso a paso eligiendo para cada estado el predecesor que maximiza la puntuación previa por la transición y multiplicando por la emisión actual, y finalmente reconstruye el camino completo retrocediendo por la cadena de punteros desde el estado final con mejor puntuación.

```

ALGORITMO: HMM_Algoritmo_Viterbi_Decodificacion
ENTRADA:
- X: Secuencia observada x_0, x_1, ..., x_{T-1} de longitud T
- Estados: Conjunto de K estados ocultos {s_1, s_2, ..., s_K}
- pi: Probabilidades iniciales pi[k]
- A: Matriz de transicion A[i, j] = P(s_j | s_i)
- E: Matriz de emision E[j, c] = P(c | s_j)
SALIDA:
- Q_optimo: Secuencia de estados ocultos q_0^*, q_1^*, ..., q_{T-1}^* mas probable
COMPLEJIDAD: Tiempo O(K^2 * T), Espacio DP O(K * T)

1. INICIALIZACION (t = 0 en escala logaritmica):
  Para cada estado k en Estados:
    delta[k, 0] <- log2(pi[k]) + log2(E[k, X[0]])
    psi[k, 0] <- NULO # Sin puntero previo

2. INDUCCION RECURSIVA (t = 1 hasta T - 1):
  Para t desde 1 hasta T - 1:
    Para cada estado destino j en Estados:
      mejor_score <- -INFINITO
      mejor_pred <- NULO

      Para cada estado origen i en Estados:
        score_candidato <- delta[i, t - 1] + log2(A[i, j])
        Si score_candidato > mejor_score entonces:
          mejor_score <- score_candidato
          mejor_pred <- i

      delta[j, t] <- mejor_score + log2(E[j, X[t]])
      psi[j, t] <- mejor_pred

3. TERMINACION Y RECUPERACION DEL MEJOR ESTADO FINAL:
  q_optimo[T - 1] <- argmax_k(delta[k, T - 1])

4. BACKTRACKING (Retroceso de punteros desde T-2 hasta 0):
  Para t desde T - 2 descendiendo hasta 0:
    q_optimo[t] <- psi[q_optimo[t + 1], t + 1]

5. RETORNO:
  Retornar q_optimo

```

Comprueba. Antes de decodificar estados que no se observan, calcule la probabilidad de la misma cadena bajo una cadena de Markov visible, con $P(A) = 0,2$, $P(T | A) = 0,3$, $P(G | T) = 0,5$ y $P(C | G) = 0,4$. Sale

$0,2 \times 0,3 \times 0,5 \times 0,4 = 0,012$. Guarde ese número. En lo que sigue, la misma cadena se analiza con estados ocultos, y la pregunta deja de ser cuán probable es la secuencia para pasar a ser qué la generó.

8.6.2 Traza completa sobre la secuencia 'ATGC'

Analicemos la decodificación de $X = \text{'ATGC'}$ con el modelo de promotores:

- $t = 0$ ($x_0 = A$): $\delta_0(P) = 0.5 \times 0.4 = 0.20$; $\delta_0(NP) = 0.5 \times 0.25 = 0.125$.
- $t = 1$ ($x_1 = T$): $\delta_1(P) = \max(0.20 \times 0.7, 0.125 \times 0.2) \times 0.3 = 0.14 \times 0.3 = 0.042$; $\delta_1(NP) = \max(0.20 \times 0.3, 0.125 \times 0.8) \times 0.25 = 0.10 \times 0.25 = 0.025$.
- $t = 2$ ($x_2 = G$): $\delta_2(P) = \max(0.042 \times 0.7, 0.025 \times 0.2) \times 0.2 = 0.0294 \times 0.2 = 0.00588$; $\delta_2(NP) = \max(0.042 \times 0.3, 0.025 \times 0.8) \times 0.25 = 0.020 \times 0.25 = 0.0050$.
- $t = 3$ ($x_3 = C$): $\delta_3(P) = \max(0.00588 \times 0.7, 0.0050 \times 0.2) \times 0.1 = 0.004116 \times 0.1 = \mathbf{0.000412}$; $\delta_3(NP) = \max(0.00588 \times 0.3, 0.0050 \times 0.8) \times 0.25 = 0.0040 \times 0.25 = \mathbf{0.001000}$.

El estado final óptimo en $t = 3$ es **NP** ($\delta_3(NP) = 0.001000 > \delta_3(P) = 0.000412$). Al retroceder los punteros de la matriz ψ , se obtiene el camino global óptimo:

$$\hat{Q} = [NP, NP, NP, NP]$$

Decodificando ATGC con el HMM de promotores, la tabla de Viterbi da $\delta_3(P) = 0.000412$ y $\delta_3(NP) = 0.001$: el camino óptimo completo es NP NP NP NP, con todos los punteros apuntando al mismo estado en cada paso (no ocurre ninguna transición de estado a lo largo del camino óptimo).

Se reproduce con `verification/chapter_results.sh viterbi_trace ATGC`.

8.6.3 El peligro del clasificador posicional ingenuo

Si un analista intentara clasificar la secuencia $X = \text{'ATGC'}$ nucleótido a nucleótido evaluando de forma aislada qué estado tiene mayor emisión en cada posición individual (ignorando las transiciones A):

- Posición 0 ('A'): $b_P(A) = 0.4 > b_{NP}(A) = 0.25 \Rightarrow \mathbf{P}$.
- Posición 1 ('T'): $b_P(T) = 0.3 > b_{NP}(T) = 0.25 \Rightarrow \mathbf{P}$.
- Posición 2 ('G'): $b_{NP}(G) = 0.25 > b_P(G) = 0.20 \Rightarrow \mathbf{NP}$.
- Posición 3 ('C'): $b_{NP}(C) = 0.25 > b_P(C) = 0.10 \Rightarrow \mathbf{NP}$.

El método ingenuo posicional predice erróneamente $[P, P, NP, NP]$.

Este resultado demuestra la diferencia conceptual entre optimización local y global:

- El clasificador por emisiones responde a una regla local posicional y predice $[P, P, NP, NP]$.
- El algoritmo de Viterbi maximiza la probabilidad conjunta de **toda la trayectoria** (decodificación MAP global) y produce $[NP, NP, NP, NP]$, evitando penalizaciones de cambio de estado.

Delimitación formal: Viterbi no garantiza que cada estado de su ruta sea el estado marginal más probable en esa posición individual; esa segunda cuestión requiere probabilidades posteriores ($P(q_t = S_i | X)$) calculadas mediante el algoritmo *Forward-Backward* (posterior decoding), el cual queda fuera del alcance de este curso.

Clasificar ATGC posición por posición usando solo la emisión más probable en cada instante (ignorando transiciones) da P P NP NP —A y T favorecen individualmente a P—, lo que difiere del camino de Viterbi completo NP NP NP NP. El camino más probable de Viterbi no equivale a elegir en cada posición el estado con mayor emisión: las transiciones acoplan las decisiones y una opción localmente atractiva puede no pertenecer al camino globalmente óptimo.

Se reproduce con `verification/chapter_results.sh naive_position_classify ATGC`.

8.7 La operación peligrosa: matrices no normalizadas y ceros

El fallo más destructivo en modelos probabilísticos ocurre cuando se introduce una matriz de transición donde una fila no suma exactamente 1 o cuando una probabilidad cero en una secuencia no suavizada cancela catastróficamente toda la cadena de decisión.

Protocolo preventivo en 3 pasos:

1. **Validación de invariantes:** Verificar con aserciones que cada fila de la matriz de transición y emisión sume 1.0 ± 10^{-6} .
2. **Aplicación universal de Laplace:** Aplicar siempre suavizado +1 en el entrenamiento con datos finitos.
3. **Espacio logarítmico estricto:** Ejecutar todas las recurrencias de Naïve Bayes, Markov y Viterbi con $\log(P)$ y sumas.

8.8 Actividad de depuración: errores en Viterbi

Analice el siguiente fragmento con tres errores conceptuales en la recurrencia de Viterbi:

```
viterbi(observaciones, estados, pi, A, B):
    # Fallo 1: inicializacion suma probabilidades en vez de multiplicarlas
    para cada estado s:
        delta[0][s] <- pi[s] + B[s][observaciones[0]]

    para t de 1 hasta longitud(observaciones) - 1:
        para cada estado j:
            # Fallo 2: se toma la suma de transiciones en vez del maximo
            delta[t][j] <- suma_sobre_i(delta[t-1][i] * A[i][j]) * B[j][observaciones[t]]
            # Fallo 3: se olvida registrar el puntero predecesor psi[t][j]

    # Sin matriz psi no es posible realizar el backtracking
    devolver reconstruir_camino(delta)
```

8.9 Síntesis operativa y tablas de diagnóstico

8.9.1 Tabla de diagnóstico de síntomas en modelos probabilísticos

Síntoma observado	Causa probable	Comprobación y corrección
Probabilidad a posteriori devuelve 0 en todas las clases	Cero no suavizado en una frecuencia observada	Aplicar suavizado de Laplace (+1) a los conteos
La probabilidad de una secuencia larga se convierte en 0	Subdesbordamiento numérico (<i>underflow</i>)	Cambiar la implementación a espacio logarítmico
La suma de probabilidades de estados en un instante supera 1	Matriz de transición corrupta (fila $\neq 1.0$)	Normalizar estrictamente cada fila antes de la inducción
Viterbi produce un camino distinto a la intuición posicional	Efecto de acoplamiento de las transiciones	Es el comportamiento correcto: Viterbi optimiza globalmente

8.9.2 Tabla de síntesis con preguntas de control

Requisito del pipeline	Modelo recomendado	Pregunta de control
Clasificación global de secuencias	Naïve Bayes	¿Se ha aplicado suavizado de Laplace y log-espacio?
Modelado de dinucleótidos (islas CpG)	Cadena de Markov (orden 1)	¿Suman 1 todas las filas de la matriz A ?
Segmentación y anotación de regiones funcionales	HMM (Viterbi)	¿Se realiza el backtracking sobre la matriz ψ ?

8.10 Glosario conceptual

- **Probabilidad condicional:** $P(A \mid B)$, probabilidad de A dado B .
- **Naïve Bayes:** clasificador basado en independencia condicional de atributos.
- **Suavizado de Laplace:** técnica de regularización (+1) contra frecuencias nulas.
- **Underflow:** pérdida de precisión por multiplicación de números cercanos a cero.
- **Cadena de Markov:** proceso estocástico con memoria de un paso ($P(x_i \mid x_{i-1})$).
- **Matriz de transición:** tabla estocástica donde cada fila suma 1.
- **HMM:** modelo de Markov con estados ocultos y emisiones observadas.
- **Algoritmo de Viterbi:** programación dinámica para decodificar el camino óptimo.
- **Backtracking:** recuperación hacia atrás de los estados óptimos mediante punteros ψ .
- **Caja TATA:** motivo consenso rico en A/T característico de promotores.

8.11 Conexión con el curso y siguientes pasos

El alineamiento por programación dinámica (TC-CH09) recupera la técnica de trazado hacia atrás (*backtracking*) de punteros vista en Viterbi, aplicada a comparar dos secuencias en lugar de decodificar estados ocultos de una sola; los HMMs en bioinformática de producción (predictores de genes, HMMER) son la extensión industrial de la maquinaria estudiada aquí; y las redes neuronales (TC-CH10) abordan el mismo problema de clasificación que Naïve Bayes aprendiendo representaciones más ricas. Estos tres desarrollos son propios de capítulos posteriores y no se desarrollan aquí.

8.12 Fuentes y reproducibilidad

Este capítulo se apoya en la presentación docente de bioinformática de la asignatura, escrita por Álvaro Serrano Navarro. Los modelos ocultos, el algoritmo de Viterbi y la remisión al algoritmo hacia delante y hacia atrás siguen a Durbin y colaboradores [11]; la definición de isla CpG por densidad y la depleción por metilación, a Gardiner-Garden y Frommer [13]; la clasificación de secuencias por composición, a Compeau y Pevzner [7]; y la nota sobre la aritmética en punto flotante de la implementación, al manual de GNU Awk [12].

Todos los resultados numéricos se reproducen con `verification/chapter_results.sh` tal como están impresos, con sus argumentos.

Anexo práctico · Clasificar, modelar y decodificar con probabilidad

Pregunta, producto y separación de materiales

¿Puede clasificar una secuencia con Naïve-Bayes a mano y contrastarlo con un script, validar una matriz de transición antes de confiar en ella, decodificar la ruta de estados ocultos más probable de un HMM con Viterbi, y distinguir esa ruta global de lo que parecería mejor posición a posición? Este laboratorio responde que sí, sin necesitar ningún dato externo.

El producto individual es una matriz de transición que usted estima a partir de una secuencia, en `notas/mi_matriz.tsv`, más la tabla de predicciones y resultados de los demás bloques. Sin la matriz, el resto del

anexo se queda en ejecutar órdenes ajenas.

El anexo es autónomo: no necesita datos externos ni red. El generador deja la secuencia de entrenamiento, el oráculo del capítulo y un comprobador que recalcula la matriz por su cuenta y la compara con la suya.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 preparar el espacio	directorio creado
2 clasificar con Bayes	clasificación manual contrastada
3 estimar la matriz de transición	notas/mi_matriz.tsv
4 validar y usar la matriz	filas comprobadas y probabilidad calculada
5 decodificar con Viterbi	ruta completa frente a la posicional
6 casos límite	tres controles verificados
7 empaquetar y verificar	verificador en ENTREGA_OK

La ruta es única. Escriba su predicción antes de ejecutar cada bloque.

1 • Preparar el espacio de trabajo

```
bash --version
./practice_assets/create_workspace.sh TC08_entrega
cd TC08_entrega
cat datos/secuencia_entrenamiento.txt
```

El generador crea `datos/` con la secuencia de entrenamiento y la de prueba, `notas/` con las dos tablas vacías, solo sus cabeceras, `resultados/` y `herramienta/` con el oráculo y el comprobador de la matriz.

2 • Clasificar con Naïve-Bayes

Antes de ejecutar nada, clasifique a mano la secuencia ACT con el modelo de la sección de clasificación con Bayes ($P(A | \text{Cod}) = 0.4$, $P(C | \text{Cod}) = 0.2$, $P(T | \text{Cod}) = 0.1$, $P(\text{Cod}) = 0.45$; $P(A | \text{NCod}) = 0.3$, $P(C | \text{NCod}) = 0.3$, $P(T | \text{NCod}) = 0.2$, $P(\text{NCod}) = 0.55$). Anote su predicción y compárela:

```
bash herramienta/chapter_results.sh naive_bayes_classify ACT
```

Clasificar ACT da $P(\text{Cod} | \text{ACT}) \propto 0.45 \cdot 0.4 \cdot 0.2 \cdot 0.1 = 0.0036$ y $P(\text{NCod} | \text{ACT}) \propto 0.55 \cdot 0.3 \cdot 0.3 \cdot 0.2 = 0.0099$: gana NCod, con una diferencia relativa amplia (NCod casi 2.75 veces Cod).

Se reproduce con `verification/chapter_results.sh naive_bayes_classify ACT`.

3 • Estimar su propia matriz de transición

Tarea. A partir de `datos/secuencia_entrenamiento.txt`, cuente los pares de posiciones adyacentes, agrúpelos por su primer símbolo y escriba la matriz de transición en `notas/mi_matriz.tsv`, con cuatro decimales y una fila por contexto observado. Cada fila debe sumar uno.

Puede contar a mano, con `grep -o` y `sort`, o con un programa de `awk`. Lo que se evalúa no es la herramienta sino el criterio: dividir cada fila por su propio total y no por el total general, y decidir qué hacer con un contexto que no aparece nunca seguido de nada.

Compruebe su matriz cuantas veces necesite:

```
bash herramienta/check_matriz.sh
```

El comprobador recalcula la matriz por su cuenta a partir de la misma secuencia y compara fila a fila, con tolerancia de media milésima. No le enseña la suya: le dice qué transición no cuadra.

Comprueba. Antes de mirar el resultado del comprobador, responda: ¿cuántas filas debe tener su matriz, si la secuencia tiene cuatro símbolos distintos? Cuento cuántos contextos aparecen seguidos de algo, que no son necesariamente cuatro.

4 • Validar y usar la cadena de Markov

La siguiente matriz de transición, propuesta por una IA, tiene un error:

```
bash herramienta/chapter_results.sh validate_transition_row
```

Anote qué fila falla, cuánto suma realmente, y por qué eso invalida cualquier cálculo posterior que la use. Después calcule la probabilidad de una secuencia nueva, GCAT, con la matriz válida de la sección de cadenas de Markov y probabilidad inicial $P(G) = 0,3$:

```
bash herramienta/chapter_results.sh markov_sequence_prob GCAT
```

Con probabilidad inicial $P(G)=0.3$ y la matriz de transición de la sección de cadenas de Markov, la probabilidad de GCAT es $P(G) \cdot P(C | G) \cdot P(A | C) \cdot P(T | A) = 0.3 \cdot 0.2 \cdot 0.2 \cdot 0.1 = 0.0012$.

Se reproduce con `verification/chapter_results.sh markov_sequence_prob GCAT`.

5 • Decodificar con Viterbi

Sobre el HMM de promotores presentado en teoría, decodifique la secuencia completa usada en la fuente original de este ejemplo, ATGCGTATATCGC —observe que contiene un fragmento TATA, que recuerda a una caja TATA real; antes de ejecutar, prediga si eso basta para que la ruta pase por el estado P—.

```
bash herramienta/chapter_results.sh viterbi_trace ATGCGTATATCGC > resultados/viterbi.txt
cat resultados/viterbi.txt
```

Decodificando ATGCGTATATCGC (13 símbolos) con el HMM de promotores, la ruta Viterbi completa es NP trece veces seguidas: ni siquiera el fragmento TATA que contiene basta para que la ruta óptima entre en el estado P, porque el coste acumulado de la transición más las emisiones de C/G intercaladas supera a lo que aportaría el sesgo de P hacia A/T. Reconocer un patrón «a ojo» (TATA) no sustituye ejecutar el modelo completo.

Se reproduce con `verification/chapter_results.sh viterbi_trace ATGCGTATATCGC`.

Perturbación: una secuencia con dos mitades

Decodifique ahora AAAACCCC —cuatro A seguidas de cuatro C— y localice en qué posición exacta cambia de estado la ruta óptima.

```
bash herramienta/chapter_results.sh viterbi_trace AAAACCCC
```

Decodificando AAAACCCC, la ruta Viterbi es PPPP,NNP,NNP,NNP: cambia de estado exactamente en la posición 4, donde empiezan las C. A diferencia de ATGCGTATATCGC, esta entrada sí produce un cambio de estado real, porque las cuatro A consecutivas acumulan suficiente ventaja para P antes de que las C fueren el cambio a NP.

Se reproduce con `verification/chapter_results.sh viterbi_trace AAAACCCC`.

Anote en `notas/respuestas.tsv` en qué posición ocurre el cambio y qué parámetro del modelo (transición o emisión) es responsable.

6 • Casos límite

```
bash herramienta/chapter_results.sh naive_bayes_classify ""
bash herramienta/chapter_results.sh viterbi_trace A
bash herramienta/chapter_results.sh laplace_smoothing_demo
```

Clasificar una secuencia vacía con Naïve-Bayes no falla: el producto vacío deja solo la comparación de las prioris, y gana NCod ($0.55 > 0.45$) por defecto. Decodificar una observación de un único símbolo con Viterbi no requiere ninguna transición: la ruta es el estado con mayor δ_0 , el mismo cálculo que la clasificación ingenua posición a posición para ese único instante. El suavizado de Laplace, ejecutado sin argumentos, reproduce el caso ancla de la sección de clasificación con Bayes de forma determinista. Los tres modos del oráculo que intervienen son el clasificador de Bayes, la traza de Viterbi y la demostración del suavizado.

7 • Empaquetar y verificar

Escriba antes `notas/defensa.md`. Después:

```
cd ..
tar -czf TC08_entrega.tar.gz TC08_entrega
sha256sum TC08_entrega.tar.gz
./practice_assets/check_entrega.sh TC08_entrega TC08_entrega.tar.gz
```

El verificador recalcula su matriz de transición y la compara fila a fila, comprueba que la tabla de respuestas tiene al menos cinco filas con justificación y una dedicada a Viterbi, que `resultados/` recoge la ruta decodificada y la comparación posicional, y que existe la defensa. Rechaza el espacio recién generado. No puntúa.

Rúbrica de calificación ponderada

La evaluación pondera la verificación técnica, el modelado probabilístico y la defensa conceptual sobre 10 puntos, tres de entrega y siete de razonamiento:

- **3,0 puntos, entrega técnica y verificador:** matriz de transición que cuadra con el recuento, tabla de respuestas en el formato pedido, paquete que se extrae y verificador en `ENTREGA_OK`.
- **2,5 puntos, clasificación de Bayes e invariantes de Markov:** Cálculo manual de verosimilitudes con log-probabilidades, suavizado de Laplace y comprobación estricta de que las filas de la matriz sumen 1.0.
- **2,5 puntos, Algoritmo de Viterbi y sensibilidad de parámetros:** Decodificación completa de la ruta MAP, identificación del punto de transición de estado en `AAAACCCC` y control de robustez ante entradas límite.
- **2,0 puntos, Defensa conceptual y acoplamiento global:** Justificación rigurosa (100–150 palabras) sobre el papel de las transiciones en Viterbi y por qué una caja TATA aislada no basta para transicionar a estado promotor sin el soporte de persistencia del modelo.

Terminar en `ENTREGA_OK` acredita que la matriz cuadra y la entrega está completa, no que el razonamiento sobre el modelo sea correcto.

Lista de autocorrección

- Mi clasificación manual de ACT coincidió con el script, o anoté por qué no.
- Detecté qué fila de la matriz de transición no sumaba 1 y por qué invalida los cálculos.
- Calculé la probabilidad de GCAT con la matriz corregida.
- Registré la ruta Viterbi completa de ATGCGTATATCGC y su contraste con TATA.
- Localicé la posición exacta de cambio de estado en `AAAACCCC`.
- Los tres controles de caso límite están anotados sin fallos.
- Puedo explicar por qué la ruta Viterbi puede diferir de clasificar posición a posición.

- mi matriz de transición sale del recuento de pares y cada fila suma uno;
- decidí y anoté qué hacer con el contexto que no aparece seguido de nada;
- guardé en resultados/ la salida de cada ejecución que cito;
- el verificador termina en ENTREGA_OK.

Defensa conceptual

En notas/defensa.md, entre 100 y 150 palabras, explique por qué la secuencia de prueba —que contiene un fragmento TATA— se decodifica enteramente como no promotor bajo este modelo, y qué tendría que cambiar en los parámetros (transición, emisión, o ambos) para que la ruta óptima sí pasara por el estado P en algún tramo.

Fuentes y reproducibilidad

La secuencia de entrenamiento es sintética y de elaboración propia. El modelo oculto, el algoritmo de Viterbi y su traza siguen a Durbin y colaboradores [11]. El oráculo del capítulo es determinista y da el mismo resultado en cualquier máquina.

Alineamiento, similitud y búsqueda en bases de datos

TC-CH09 – Técnicas Computacionales en Biología

Edición 2

Pregunta del tema. Dos secuencias de organismos distintos se parecen. ¿Cuánto de ese parecido significa algo y cuánto era esperable por azar, dado el tamaño de la base de datos en la que las buscó?

Producto final. Una matriz de programación dinámica rellena a mano y verificada contra el oráculo, un guion propio que recalcula la puntuación de un alineamiento columna a columna, y tres valores esperados que muestran qué cambia el tamaño de la base de datos y qué no.

Mapa del tema

9.1. De decodificar una secuencia a comparar dos	151
9.2. Alinear es optimizar	151
9.3. Needleman–Wunsch: el óptimo global	152
9.4. Smith–Waterman: el óptimo local	155
9.5. Huecos y matrices de sustitución	156
9.6. Bases de datos: identificadores, versiones y reproducibilidad	157
9.7. BLAST: una heurística, no una garantía	158
9.8. La operación peligrosa: parámetros invertidos y números sin versión	160
9.9. Actividad de depuración: errores en Needleman–Wunsch	160
9.10. Síntesis operativa y tablas de diagnóstico	160
9.11. Glosario conceptual	161
9.12. Conexión con el curso y siguientes pasos	161
9.13. Fuentes y reproducibilidad	161

Cómo usar este tema

Alinear dos secuencias es el problema más característico de la bioinformática, y también el más fácil de malinterpretar. El capítulo tiene dos mitades. La primera es exacta: dos algoritmos de programación dinámica que garantizan el óptimo, uno global y otro local, con sus matrices resueltas paso a paso. La segunda es aproximada: BLAST renuncia a la garantía para poder buscar en bases de datos enormes, y a cambio necesita una estadística que diga cuándo un parecido significa algo.

Al terminar sabrá: rellenar una matriz de alineamiento a mano y hacer el trazado hacia atrás; explicar en qué se diferencian el alineamiento global y el local, y cuándo se usa cada uno; verificar una puntuación por un camino distinto del que la produjo; interpretar un valor esperado sin confundirlo con una probabilidad de homología; y registrar una búsqueda de forma que otra persona pueda repetirla.

Concepto. Antes de entrar en el cómo, conviene decir para qué. Alinear secuencias sirve al menos para tres cosas distintas. Para **inferir función**: si una proteína nueva se parece a otra cuya función se conoce, esa similitud es la primera hipótesis. Para **reconstruir historia evolutiva**: las diferencias acumuladas entre secuencias homólogas de especies distintas permiten estimar cuánto hace que divergieron. Y para **identificar dianas terapéuticas**: encontrar una proteína esencial de un patógeno que no tenga equivalente humano es el punto de partida del diseño de fármacos, que el libro de Biología Computacional retoma en su último capítulo.

La ruta **esencial** son los dos algoritmos con sus trazas, la verificación ortogonal y el valor esperado de BLAST. La ruta de **estudio** añade huecos afines, matrices de sustitución, reproducibilidad de bases de datos y el anexo.

9.1 De decodificar una secuencia a comparar dos

ESENCIAL El capítulo anterior resolvió el modelado de una sola secuencia: dada una secuencia de ADN, clasificar su pertenencia a una clase (Naïve-Bayes) o reconstruir sus estados biológicos ocultos más probables (Viterbi).

Este capítulo cambia radicalmente la pregunta científica: dadas **dos** secuencias biológicas, ¿cómo cuantificar su parecido y cuál es la correspondencia óptima posición a posición que explica ese parecido?

La técnica computacional que resuelve este problema —la **programación dinámica con matrices de puntuación y trazado hacia atrás** (*backtracking*)— es exactamente la misma maquinaria conceptual utilizada en el algoritmo de Viterbi:

- En Viterbi, cada celda optimiza el camino de estados ocultos a lo largo del tiempo para una sola secuencia.
- En alineamiento, cada celda optimiza la correspondencia entre los prefijos de dos secuencias distintas.

El alineamiento por programación dinámica reutiliza la técnica de retroceso de punteros vista en el algoritmo de Viterbi del capítulo anterior, aplicada a comparar dos secuencias en vez de decodificar los estados ocultos de una sola: en ambos casos cada celda de una tabla guarda la mejor puntuación alcanzable y un puntero al origen de esa puntuación, y la solución completa se reconstruye retrocediendo esos punteros desde la celda final hasta el origen.

9.2 Alinear es optimizar

ESENCIAL

Un **alineamiento** entre dos secuencias $X = x_1x_2 \dots x_n$ e $Y = y_1y_2 \dots y_m$ propone una superposición columna a columna insertando caracteres de espaciado o **huecos** (*gaps*, denotados por $-$) donde una secuencia presenta un nucleótido o aminoácido sin contrapartida en la otra.

Biológicamente, las coincidencias y sustituciones reflejan mutaciones puntuales por reemplazo, mientras que los huecos representan eventos de inserción o delección (**indels**).

Para comparar objetivamente distintos alineamientos posibles, se requiere un **sistema de puntuación** explícito:

- Recompensa por coincidencia exacta (*match*): $+m$ ($m > 0$).
- Penalización por discrepancia (*mismatch*): $-s$ o $s < 0$.
- Penalización lineal por hueco (*gap*): $-g$ o $g < 0$.

Un alineamiento propone correspondencias entre símbolos de dos secuencias e inserta huecos para representar inserciones o delecciones (indels). Compararlo con otros alineamientos requiere un sistema de puntuación explícito (recompensa por coincidencia, penalización por sustitución, penalización por hueco); el alineamiento óptimo es óptimo relativo a ese sistema de puntuación concreto, no de forma universal e independiente de los parámetros elegidos y del objetivo del análisis.

Alineamiento Global frente a Alineamiento Local.

- **Alineamiento Global (Needleman–Wunsch)**: Alinea ambas secuencias de extremo a extremo ($1 \dots n$ contra $1 \dots m$). Es el método apropiado cuando las secuencias tienen longitudes similares y se espera homología a lo largo de toda su extensión.

- **Alineamiento Local (Smith–Waterman):** Busca la región o motivo subsecuencial de máxima similitud, ignorando los extremos flanqueantes no relacionados. Es indispensable para localizar dominios conservados dentro de proteínas multidominio o genes quiméricos.

El alineamiento global compara dos secuencias de extremo a extremo y es apropiado cuando su longitud y su homología son comparables en toda su extensión; el alineamiento local busca los mejores segmentos parciales y tolera flancos no relacionados, apropiado cuando solo una región interna de las secuencias es comparable.

Rigor conceptual: Identidad, Similitud y Homología.

En bioinformática es imperativo no confundir tres conceptos clave:

1. **Identidad:** Porcentaje de posiciones alineadas que contienen exactamente el mismo residuo químico.
2. **Similitud:** Porcentaje o puntuación que contabiliza residuos con propiedades fisicoquímicas equivalentes (p. ej., aminoácidos hidrofóbicos o cargados).
3. **Homología:** Hipótesis biológica cualitativa sobre la existencia de un ancestro evolutivo común.

La homología es una propiedad dicotómica (dos secuencias son homólogas o no lo son; *no existe la «homología al 80%»*). Una identidad elevada constituye evidencia en favor de la homología, pero no la demuestra automáticamente.

Identidad (fracción de posiciones alineadas con el mismo símbolo), similitud (que añade criterios biológicos o químicos entre símbolos distintos pero relacionados) y homología (hipótesis de origen evolutivo común) no son sinónimos. Una identidad o similitud alta es evidencia hacia la homología, pero no la demuestra por sí sola; la homología no es un porcentaje —un par es o no es homólogo— y lo que varía en grado es la evidencia disponible.

9.3 Needleman–Wunsch: el óptimo global

A) Matriz de Programación Dinámica (DP)

0	-2	-4	-6	-8
-2	+2	0	-2	-4
-4	0	0	+4	+2
-6	-2	-2	+2	+8

Recursión: $F(i, j) = \max\{F(i-1, j-1) + s, F(i-1, j) - d, F(i, j-1) - d\}$
 Smith-Waterman añade opción 0 (reinicio local)

B) Backtracking y Reconstrucción del Alineamiento

Decisiones de Backtracking:

Diagonal (↖) Alinea S1[i] con S2[j] (Match / Mismatch)

Arriba (↑) Huevo en Secuencia 2 (Delección en S2 / Inserción S1)

Izquierda (←) Huevo en Secuencia 1 (Inserción en S2 / Delección S1)

Alineamiento Óptimo Resultante (Score = +8):

S1: A C G T
 S2: A - G T

Figura 9.1: A) Matriz de Programación Dinámica evaluando los tres orígenes posibles (diagonal, arriba, izquierda) para alinear dos secuencias. B) Decisión de *backtracking* y alineamiento óptimo reconstruido de derecha a izquierda. Figura original adaptada de las presentaciones docentes.

El algoritmo de **Needleman–Wunsch** (1970) resuelve el problema del alineamiento global óptimo garantizado mediante programación dinámica.

9.3.1 Definición de la matriz $F(i, j)$ y condiciones de contorno

Sea $F(i, j)$ la puntuación del mejor alineamiento posible entre el prefijo $x_1 \dots x_i$ de longitud i y el prefijo $y_1 \dots y_j$ de longitud j .

Inicialización de fronteras:

Alinear un prefijo contra una secuencia vacía solo puede lograrse insertando i (o j) huecos consecutivos:

$$F(0,0) = 0, \quad F(i,0) = i \cdot g, \quad F(0,j) = j \cdot g$$

En Needleman–Wunsch, $F(i,j)$ es la puntuación óptima de alinear los primeros i símbolos de x con los primeros j símbolos de y . La primera fila y la primera columna acumulan penalizaciones de hueco porque alinear un prefijo contra la cadena vacía solo puede hacerse insertando huecos. El retroceso de punteros parte de $F(n,m)$, que contiene la puntuación óptima del alineamiento global completo, y termina en $F(0,0)$ reconstruyendo el alineamiento en orden inverso.

9.3.2 Ecuación de recurrencia

Cada celda interior $F(i,j)$ se calcula evaluando los tres orígenes posibles:

$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(x_i, y_j) & \text{(Diagonal: alinear } x_i \text{ con } y_j) \\ F(i-1,j) + g & \text{(Arriba: hueco en } Y) \\ F(i,j-1) + g & \text{(Izquierda: hueco en } X) \end{cases}$$

La recurrencia de Needleman–Wunsch es $F(i,j) = \max\{F(i-1,j-1) + s(x_i, y_j), F(i-1,j) + g, F(i,j-1) + g\}$, donde $s(x_i, y_j)$ es la recompensa por coincidencia o la penalización por sustitución según si x_i es igual a y_j , y g es la penalización por hueco.

ALGORITMO: Needleman_Wunsch_Alineamiento_Global

ENTRADA:

- X: Secuencia 1 de longitud n
- Y: Secuencia 2 de longitud m
- s(a, b): Matriz de puntuacion (match = +m_score, mismatch = -s_score)
- g: Penalizacion lineal por hueco (g < 0)

SALIDA:

- (Score_Optimo, X_alineado, Y_alineado)

COMPLEJIDAD: Tiempo $O(n * m)$, Espacio DP $O(n * m)$

1. INICIALIZACION DE FRONTERAS:

```
F[0, 0] <- 0
Para i desde 1 hasta n: F[i, 0] <- i * g
Para j desde 1 hasta m: F[0, j] <- j * g
```

2. LLENADO DE LA MATRIZ DE PROGRAMACION DINAMICA:

```
Para i desde 1 hasta n:
  Para j desde 1 hasta m:
    diag <- F[i - 1, j - 1] + s(X[i], Y[j])
    up <- F[i - 1, j] + g
    left <- F[i, j - 1] + g
    F[i, j] <- max(diag, up, left)
```

3. RECONSTRUCCION POR BACKTRACKING (Desde F[n, m] hasta F[0, 0]):

```
i <- n; j <- m
ali_X <- ""; ali_Y <- ""

Mientras i > 0 o j > 0:
  Si i > 0 y j > 0 y F[i, j] == F[i - 1, j - 1] + s(X[i], Y[j]) entonces:
    ali_X <- X[i] + ali_X; ali_Y <- Y[j] + ali_Y
    i <- i - 1; j <- j - 1
  Sino si i > 0 y F[i, j] == F[i - 1, j] + g entonces:
    ali_X <- X[i] + ali_X; ali_Y <- "-" + ali_Y
    i <- i - 1
  Sino:
    ali_X <- "-" + ali_X; ali_Y <- Y[j] + ali_Y
    j <- j - 1
```

4. RETORNO:

```
Retornar (F[n, m], ali_X, ali_Y)
```

9.3.3 Traza completa resuelta: alinear 'GAT' contra 'GC'

Consideremos los parámetros estándar: coincidencia $m = +1$, discrepancia $s = -1$, hueco $g = -2$.

Construcción de la matriz $F(i, j)$ para $X = \text{'GAT'}$ ($n = 3$) e $Y = \text{'GC'}$ ($m = 2$):

$F(i, j)$	$\epsilon (j = 0)$	$G (j = 1)$	$C (j = 2)$
$\epsilon (i = 0)$	0	-2	-4
$G (i = 1)$	-2	+1 ($\nwarrow$)	-1 ($\leftarrow$)
$A (i = 2)$	-4	-1 ($\uparrow$)	0 ($\nwarrow$)
$T (i = 3)$	-6	-3 ($\uparrow$)	-2 ($\nwarrow, \uparrow$)

Atención. Fíjese en $F(2, 2) = 0$, porque es lo que hace posible el empate de la celda final. Desde esa celda, bajar cuesta $0 - 2 = -2$, exactamente lo mismo que llegar en diagonal desde $F(2, 1)$. Si esa celda valiera otra cosa, el empate desaparecería y el trazado hacia atrás tendría un solo camino. Cuando una matriz de este tipo dice que hay empate y usted no lo encuentra, la celda mal calculada suele estar una posición antes.

Trazado hacia atrás (*Backtracking*):

Partiendo de la celda final $F(3, 2) = -2$:

1. En $F(3, 2) = -2$, se resuelve el empate eligiendo la diagonal $\nwarrow \rightarrow$ alinea T con C ($s = -1$). Se pasa a $F(2, 1)$.
2. En $F(2, 1) = -1$, el origen es arriba $\uparrow \rightarrow$ alinea A con hueco $-$ ($g = -2$). Se pasa a $F(1, 1)$.
3. En $F(1, 1) = +1$, el origen es diagonal $\nwarrow \rightarrow$ alinea G con G ($m = +1$). Se pasa a $F(0, 0)$.

Alineamiento global resultante:

X: G A T
Y: G - C

Alineando GAT con GC (coincidencia +1, sustitución -1, hueco -2), la puntuación óptima de Needleman-Wunsch es -2 con alineamiento GAT / G-C: G-G coincide, A se alinea con un hueco, T-C no coincide. La celda final empatiza entre dos orígenes (diagonal y arriba); el retroceso elige diagonal de forma sistemática y reproducible.

Se reproduce con `verification/chapter_results.sh needleman_wunsch GAT GC`.

Verificación ortogonal de la puntuación:

Calculando directamente sobre las columnas del alineamiento obtenido:

$$\text{Puntuación} = (+1 \text{ por G/G}) + (-2 \text{ por A/-}) + (-1 \text{ por T/C}) = 1 - 2 - 1 = -2$$

Recalcular la puntuación de GAT / G-C columna por columna sobre el alineamiento ya construido —sin repetir la matriz de programación dinámica— da de nuevo -2: una comprobación independiente que usa una propiedad distinta al procedimiento que produjo el resultado, no la misma matriz recalculada.

Se reproduce con `verification/chapter_results.sh recompute_alignment_score GAT G-C`.

Ejemplo de decisión de hueco vs discrepancia: 'ACGTACGT' vs 'ACGTTCGT'.

Al alinear estas dos secuencias de longitud 8 con un único nucleótido distinto en la posición 5:

$$\text{Score sin huecos} = 7 \times (+1) + 1 \times (-1) = 6$$

Si se introdujera un hueco para evitar el mismatch, se incurriría en al menos dos penalizaciones de hueco ($-2 + -2 = -4$), obteniendo una puntuación estrictamente inferior. La selección depende exclusivamente del sistema de puntuación.

Alineando ACGTACGT con ACGTTCGT (misma longitud, una posición distinta) con coincidencia +1, sustitución -1, hueco -2, el óptimo global no inserta ningún hueco: alinea símbolo a símbolo aceptando la única sustitución,

con puntuación $7(+1)+1(-1)=6$. Introducir un hueco para evitar esa sustitución costaría al menos -2 por esa posición más el desplazamiento del resto: siempre peor que aceptar la sustitución a coste -1. La elección la determina la puntuación, no una preferencia algorítmica por evitar huecos.

Se reproduce con `verification/chapter_results.sh needleman_wunsch GAT GC`.

Complejidad computacional:

El llenado de la matriz requiere evaluar $(n+1)(m+1)$ celdas a coste $O(1)$ por celda, resultando en una complejidad temporal de $O(n \cdot m)$ y memoria $O(n \cdot m)$.

Un empate entre dos o más orígenes en la recurrencia de Needleman–Wunsch significa que existe más de un alineamiento óptimo con la misma puntuación —información sobre el problema, no un error—; una implementación reproducible declara una regla determinista de desempate. Con penalización lineal por hueco, Needleman–Wunsch tiene complejidad $O(nm)$ en tiempo y memoria, ya que llena una tabla de $(n+1)$ por $(m+1)$ celdas en tiempo constante por celda.

9.4 Smith–Waterman: el óptimo local

El algoritmo de **Smith–Waterman** (1981) identifica el segmento subsecuencial de máxima similitud local mediante tres modificaciones fundamentales sobre Needleman–Wunsch:

1. **Fronteras en cero:** $H(i,0) = 0$ y $H(0,j) = 0$ (no se acumulan penalizaciones iniciales).
2. **Piso en cero en la recurrencia:** Si todas las opciones son negativas, la celda toma el valor 0, reiniciando el alineamiento local.
3. **Inicio y fin del *backtracking*:** Comienza en la celda con el valor máximo absoluto de toda la tabla ($\max_{i,j} H(i,j)$) y finaliza en cuanto encuentra una celda con valor 0.

El alineamiento local (Smith–Waterman) añade un cuarto origen —el cero— a la recurrencia de programación dinámica, que reinicia la puntuación en cuanto acumular penalizaciones deja de compensar; inicializa los bordes a cero en vez de acumular huecos; y retrocede desde la celda de máxima puntuación de toda la tabla hasta la primera celda con valor cero, no hasta el origen. El resultado puede cubrir solo una región interna de las secuencias, no su extensión completa.

9.4.1 Ecuación de recurrencia de Smith–Waterman

$$H(i,j) = \max \begin{cases} 0 \\ H(i-1,j-1) + s(x_i, y_j) \\ H(i-1,j) + g \\ H(i,j-1) + g \end{cases}$$

La recurrencia de Smith–Waterman es $H(i,j) = \max\{0, H(i-1,j-1) + s(x_i, y_j), H(i-1,j) + g, H(i,j-1) + g\}$: la única diferencia formal con Needleman–Wunsch es el término cero adicional, que impide que la puntuación baje de cero en ninguna celda.

9.4.2 Traza resuelta: ‘CCGAT’ frente a ‘TTGAT’

Con $m = +1, s = -1, g = -2$, el alineamiento local entre $X = \text{‘CCGAT’}$ e $Y = \text{‘TTGAT’}$ ignora por completo los flancos divergentes iniciales CC y TT (cuyas celdas se mantienen en 0) y acumula puntuación en el motivo idéntico común:

$$\begin{array}{c} X[3..5]: \text{ G A T} \\ Y[3..5]: \text{ G A T} \quad (\text{Puntuación máxima } H = 3) \end{array}$$

Alineando CCGAT con TTGAT (coincidencia +1, sustitución -1, hueco -2), Smith–Waterman recupera el alineamiento local GAT / GAT con puntuación 3, ignorando por completo los flancos CC y TT: un alineamiento global forzaría una correspondencia entre esos flancos no relacionados, reduciendo la puntuación total.

Se reproduce con `verification/chapter_results.sh smith_waterman CCGAT TTGAT`.

Un mismo par de secuencias puede presentar varios máximos locales con igual puntuación bajo Smith–Waterman; cada uno es un resultado válido y debe reportarse con sus coordenadas. Comparar alineamientos locales únicamente por porcentaje de identidad favorece alineamientos muy cortos (alta identidad sobre pocas posiciones dice menos que identidad moderada sobre muchas posiciones); la longitud y la cobertura deben leerse junto a la identidad, no en su lugar.

```
ALGORITMO: Smith_Waterman_Alineamiento_Local
ENTRADA:
- X: Secuencia 1 de longitud n, Y: Secuencia 2 de longitud m
- s(a, b): Matriz de similitud, g: Penalización de hueco (g < 0)
SALIDA:
- (Score_Maximo, Segmento_X_alineado, Segmento_Y_alineado)
COMPLEJIDAD: Tiempo O(n * m), Espacio DP O(n * m)

1. INICIALIZACION (Bordes a cero):
Para i desde 0 hasta n: H[i, 0] <- 0
Para j desde 0 hasta m: H[0, j] <- 0
max_score <- 0; (max_i, max_j) <- (0, 0)

2. LLENADO CON REINICIO EN CERO:
Para i desde 1 hasta n:
  Para j desde 1 hasta m:
    diag <- H[i - 1, j - 1] + s(X[i], Y[j])
    up <- H[i - 1, j] + g
    left <- H[i, j - 1] + g
    H[i, j] <- max(0, diag, up, left) # Termina 0 permite reiniciar alineamiento

    Si H[i, j] > max_score entonces:
      max_score <- H[i, j]
      (max_i, max_j) <- (i, j)

3. BACKTRACKING LOCAL (Desde max_score hasta primer 0):
i <- max_i; j <- max_j
ali_X <- ""; ali_Y <- ""

Mientras H[i, j] > 0:
  Si H[i, j] == H[i - 1, j - 1] + s(X[i], Y[j]) entonces:
    ali_X <- X[i] + ali_X; ali_Y <- Y[j] + ali_Y
    i <- i - 1; j <- j - 1
  Sino si H[i, j] == H[i - 1, j] + g entonces:
    ali_X <- X[i] + ali_X; ali_Y <- "-" + ali_Y
    i <- i - 1
  Sino:
    ali_X <- "-" + ali_X; ali_Y <- Y[j] + ali_Y
    j <- j - 1

4. RETORNO:
Retornar (max_score, ali_X, ali_Y)
```

9.5 Huecos y matrices de sustitución

ESTUDIO

9.5.1 Penalización de hueco afín (*Affine Gap Penalties*)

Biológicamente, la probabilidad de que una delección o inserción involucre varios nucleótidos consecutivos en un solo evento es mucho mayor que la ocurrencia de múltiples delecciones independientes de 1 base.

El **modelo de penalización afín** penaliza fuertemente la apertura del hueco (g_o) y penaliza con menor intensidad cada posición adicional de extensión (g_e):

$$g(k) = g_o + (k - 1) \cdot g_e, \quad \text{con } g_o < 0, g_e < 0, \text{ y } |g_o| > |g_e|$$

El coste afín por hueco es $g(k) = g_o + (k - 1)g_e$ para un hueco de k posiciones, con apertura g_o y extensión g_e ambas negativas y $|g_o| > |g_e|$. Modela que un indel de varias posiciones suele ser un único evento evolutivo, y hace que el modelo prefiera un hueco largo a varios huecos cortos que sumen la misma longitud total.

Cálculo de coste afín:

Con apertura $g_o = -5$ y extensión $g_e = -1$, un hueco de longitud $k = 3$ cuesta:

$$g(3) = -5 + (3 - 1) \cdot (-1) = -5 - 2 = -7$$

(frente a -15 , que resultaría de abrir 3 huecos independientes).

Con apertura -5 y extensión -1 , un hueco afín de longitud 3 cuesta $g(3) = -5 + 2 \cdot (-1) = -7$, no -15 (que resultaría de tratar erróneamente las tres posiciones como tres aperturas independientes). El propósito del coste afín es que un hueco largo resulte más barato que varios huecos cortos abiertos por separado que sumen la misma longitud total.

Se reproduce con `verification/chapter_results.sh affine_gap_cost 3 -10 -1`.

Validación de parámetros afines:

Configurar $g_o = -1$ y $g_e = -5$ ($|g_o| < |g_e|$) es un error grave: haría que abrir un hueco nuevo fuera más barato que extender uno existente, destruyendo el supuesto biológico del modelo.

Unos parámetros de hueco afín con $g_o = -1$, $g_e = -5$ (extensión más costosa que apertura) son inválidos: harían que abrir un hueco nuevo en cada posición fuera más barato que extender uno existente, lo contrario de lo que el coste afín pretende modelar. Validar $|g_o| > |g_e|$ antes de alinear evita construir un modelo que penalice indels largos justo al revés de lo previsto.

Se reproduce con `verification/chapter_results.sh validate_affine_gap_params -10 -1`.

9.5.2 Matrices de sustitución de aminoácidos: PAM y BLOSUM

Para alinear proteínas no se utiliza una penalización uniforme, sino **matrices de sustitución log-odds**:

- **Matrices PAM (Point Accepted Mutation, Margaret Dayhoff)**: Basadas en un modelo evolutivo explícito de mutaciones puntuales aceptadas sobre familias de proteínas muy cercanas. PAM1 equivale a un 1% de cambio; matrices como PAM250 modelan distancias evolutivas mayores mediante multiplicación matricial.
- **Matrices BLOSUM (Blocks Substitution Matrix, Henikoff & Henikoff)**: Construidas directamente a partir de bloques conservados de proteínas homólogas observadas experimentalmente. El número indica el porcentaje de identidad umbral usado para agrupar secuencias:
 - **BLOSUM62**: Matriz estándar de propósito general para divergencias medias.
 - **BLOSUM45 / BLOSUM80**: BLOSUM45 para secuencias divergentes (alineamientos distantes); BLOSUM80 para secuencias muy parecidas.

Las matrices de sustitución de proteínas (PAM, BLOSUM) asignan una puntuación distinta a cada par de aminoácidos, estimada a partir de las sustituciones observadas en proteínas emparentadas, en vez de una recompensa fija por coincidencia o sustitución. La elección de matriz y penalizaciones de hueco depende del tipo de secuencia, la divergencia esperada y el objetivo del análisis; las puntuaciones brutas de alineamientos calculados con esquemas distintos no son directamente comparables entre sí.

9.6 Bases de datos: identificadores, versiones y reproducibilidad

ESTUDIO

Una búsqueda de secuencias solo es científicamente reproducible si se registran los identificadores unívocos y sus versiones.

Estructura formal del formato FASTA:

La línea de cabecera debe comenzar estrictamente con el carácter > seguido inmediatamente por el identificador o número de acceso (sin espacios en blanco intermedios), y opcionalmente una descripción tras el primer espacio:

- **Válido:** >NC_000913.3 Escherichia coli str. K-12 complete genome
- **Inválido:** > descripcion sin accesion (espacio tras >, número de acceso vacío).
- **Inválido:** NC_000913.3 secuencia (falta el carácter >).

Las bases de datos biológicas identifican cada registro con un número de acceso, habitualmente versionado, porque un mismo identificador sin versión puede referirse a contenidos distintos en fechas diferentes. El resultado de una búsqueda solo es reproducible si registra la secuencia de consulta y su procedencia, el programa y su versión, la base de datos y su versión o fecha, y los parámetros empleados; la ausencia de cualquiera de esas piezas rompe la reproducibilidad, aunque la puntuación calculada sea correcta.

Una cabecera FASTA reproducible comienza con > seguido del identificador o número de acceso sin espacio previo, y opcionalmente una descripción tras el primer espacio. La cabecera >NC_000913.3 Escherichia coli str. K-12 chromosome, complete genome es válida con número de acceso NC_000913.3; una línea sin > inicial, o con > seguido inmediatamente de un espacio (número de acceso vacío), es inválida y debe rechazarse antes de procesar el fichero, no después.

Se reproduce con `verification/chapter_results.sh validate_fasta_header`.

El riesgo de la propagación circular de anotaciones:

En los repositorios biológicos públicos, una gran proporción de anotaciones funcionales proceden de inferencias computacionales automáticas basadas en similitud de secuencia y no de caracterizaciones bioquímicas directas. Si un registro anotado automáticamente con una función no confirmada se utiliza como referencia para anotar nuevas secuencias, el error se propaga en cascada.

Una anotación funcional asociada a un registro de base de datos no es un hecho demostrado: las descripciones se propagan entre registros por similitud de secuencia, heredando y a veces perpetuando errores de anotaciones previas sin nueva evidencia experimental. Atribuir función a partir de un alineamiento exige una cadena de evidencia completa (procedencia, programa, parámetros, acierto, taxonomía, curación, alternativas consideradas); la fuente primaria y la validación experimental tienen más peso que una descripción heredada del primer acierto.

9.7 BLAST: una heurística, no una garantía

Ejecutar el algoritmo exacto de Smith–Waterman sobre una base de datos genómica con miles de millones de nucleótidos requeriría días de cómputo exhaustivo ($O(N \cdot M)$).

BLAST (*Basic Local Alignment Search Tool*, Altschul et al., 1990) es una aproximación **heurística** en 3 fases:

1. **Sembrado** (*Seeding*): Encuentra coincidencias exactas de palabras cortas (W -meros; $W = 11$ en nucleótidos, $W = 3$ en proteínas).
2. **Extensión** (*Extension*): Extiende la alineación a partir de la semilla en ambas direcciones mientras la puntuación acumulada no decaiga por debajo de un umbral X .
3. **Evaluación estadística** (*HSP Evaluation*): Calcula la significación estadística del segmento de alta puntuación (*High-scoring Segment Pair*, HSP).

BLAST evita llenar una matriz de programación dinámica completa contra cada secuencia de una base de datos mediante una heurística en tres fases: busca palabras cortas exactas de la consulta en la base de datos (semillas), extiende cada coincidencia mientras la puntuación mejore, y evalúa estadísticamente los pares de segmentos de alta puntuación resultantes. Es mucho más rápido que la programación dinámica exhaustiva, pero no garantiza encontrar el óptimo que hallaría Smith–Waterman: una coincidencia real sin una semilla exacta suficientemente larga puede pasar desapercibida.

9.7.1 Estadística de Karlin–Altschul: puntuación bruta, Bit Score y E-value

Puntuación bruta (S), puntuación normalizada (*Bit Score*) y significación (E -value).

En BLAST se distinguen tres magnitudes estadísticas fundamentales:

- **Puntuación bruta (S):** suma de aciertos, desajustes y penalizaciones según la matriz empleada. No permite comparar búsquedas con matrices distintas.
- **Puntuación en bits (*Bit Score*, S'):** puntuación normalizada independiente del sistema de puntuación ($S' = \frac{\lambda S - \ln K}{\ln 2}$). Dos alineamientos con el mismo S' representan la misma calidad intrínseca aunque procedan de esquemas distintos.
- **Valor esperado (E -value):** número de alineamientos con puntuación $\geq S$ esperados puramente por azar en un espacio de búsqueda de tamaño $m \times n$:

$$E = K \cdot m \cdot n \cdot e^{-\lambda S} = m \cdot n \cdot 2^{-S'}$$

Regla de lectura inferencial: Un E -value bajo indica que el alineamiento es extremadamente improbable por puro azar, pero no equivale a una «probabilidad de homología». La homología es una hipótesis biológica de ancestría común que exige corroborar cobertura, dominios y coherencia taxonómica. donde K y λ son parámetros estadísticos calibrados según el sistema de puntuación, m es la longitud de la secuencia de consulta y n es el tamaño total de la base de datos en residuos.

El E -value de BLAST se estima como $E \approx K \cdot m \cdot n \cdot e^{-\lambda S}$, con K y λ parámetros estadísticos calibrados del sistema de puntuación, m la longitud de la consulta, n el tamaño de la base de datos y S la puntuación bruta del acierto. No es la probabilidad de que la secuencia sea homóloga ni de que su anotación sea correcta: es una estimación de aciertos esperados por azar para ese tamaño de búsqueda.

Análisis de sensibilidad y dependencia del tamaño n :

Consideremos $K = 0.1, \lambda = 0.5, m = 20, n = 10^6$:

- Para $S = 40$: $E = 0.1 \times 20 \times 10^6 \times e^{-0.5 \times 40} = 2 \times 10^6 \times e^{-20} \approx \mathbf{0.004122}$ (acierto muy significativo).
- Si se duplica la base de datos a $n = 2 \times 10^6$ manteniendo la misma puntuación $S = 40$: $E = 2 \times 0.004122 = \mathbf{0.008245}$. El mismo alineamiento pierde significación puramente porque el espacio de búsqueda es mayor.
- Sensibilidad exponencial ante S : Para $S = 30 \Rightarrow E \approx \mathbf{0.611805}$ (no significativo, esperable por azar). Para $S = 50 \Rightarrow E \approx \mathbf{0.000028}$ (altamente significativo).

Con $K = 0.1, \lambda = 0.5, m = 20, n = 1000000$ y $S = 40$, el E -value de BLAST es aproximadamente 0.004122: un acierto significativo, con menos de 0.005 coincidencias igual de buenas esperadas por azar en una búsqueda de ese tamaño.

Se reproduce con `verification/chapter_results.sh blast_evalue 0.1 0.5 20 1000000 40`.

Con $K = 0.1, \lambda = 0.5, m = 20, S = 40$ fijos, duplicar el tamaño de la base de datos de $n = 1000000$ a $n = 2000000$ duplica también el E -value (de 0.004122 a 0.008245): el mismo acierto se vuelve menos significativo puramente porque la base de datos creció, sin ningún cambio en la calidad del alineamiento en sí.

Se reproduce con `verification/chapter_results.sh blast_evalue 0.1 0.5 20 2000000 40`.

Con $K = 0.1, \lambda = 0.5, m = 20, n = 1000000$ fijos, una puntuación $S = 30$ da $E \approx 0.611805$ (no significativo, más de medio acierto igual de bueno esperado por azar), mientras que $S = 50$ da $E \approx 0.000028$ (altamente significativo): pequeñas diferencias de puntuación cerca del umbral de significación cambian el E -value en varios órdenes de magnitud.

Se reproduce con `verification/chapter_results.sh blast_evalue 0.1 0.5 20 1000000 30`.

Ficha técnica obligatoria para búsquedas reproducibles con BLAST:

Todo informe o pipeline científico que utilice BLAST debe registrar explícitamente: (1) identificador y hash de la consulta (*query*); (2) programa, tarea y versión exacta; (3) base de datos con fecha de congelación y versión/parche; (4) matriz de sustitución y penalizaciones afines (g_o, g_e); (5) tamaño de palabra (*word_size*)

y umbral de siembra; (6) política de enmascaramiento (*masking* duro frente a suave); (7) umbral de corte de significación (*E-value threshold*).

Enmascaramiento de baja complejidad (*Masking*):

Regiones ricas en repeticiones simples (p. ej., poli-A o regiones ricas en prolina) generan semillas masivas por sesgo composicional. El enmascaramiento (con DUST o SEG) sustituye estas regiones por N o X para evitar falsos positivos espurios.

Las regiones de composición sesgada o baja complejidad producen palabras semilla muy frecuentes que reflejan composición compartida, no parentesco específico. El enmascaramiento las excluye de la búsqueda de semillas reduciendo falsos positivos, pero puede ocultar una señal real si se aplica sin revisión; se conservan tanto los resultados con enmascaramiento como sin él como escenarios distintos en vez de elegir el que dé el E-value más favorable.

9.8 La operación peligrosa: parámetros invertidos y números sin versión

Los dos errores más dañinos en pipelines de alineamiento son:

1. **Penalizaciones de hueco afín invertidas** ($|g_o| < |g_e|$): Produce fragmentación patológica en cientos de huecos de 1 nucleótido.
2. **Referencias a bases de datos sin versión**: Almacenar únicamente el nombre del genoma sin fecha de congelación ni número de parche rompe la reproducibilidad científica.

Protocolo preventivo en 3 pasos:

1. **Aserción de apertura/extensión**: Comprobar $|g_o| > |g_e|$ antes de instanciar la matriz.
2. **Verificación sintáctica de FASTA**: Comprobar que la primera línea comience por > seguido inmediatamente de caracteres no vacíos.
3. **Congelación de metadatos**: Registrar siempre el número de versión de la base de datos y la semilla en cualquier informe de BLAST.

9.9 Actividad de depuración: errores en Needleman–Wunsch

Analice el siguiente fragmento con tres errores conceptuales en la inicialización y recurrencia de Needleman–Wunsch:

```
needleman_wunsch(seq1, seq2, match, mismatch, gap):
  # Fallo 1: inicialización de fronteras en 0 en lugar de penalizaciones acumuladas
  para i de 0 a len(seq1): F[i][0] <- 0
  para j de 0 a len(seq2): F[0][j] <- 0

  para i de 1 a len(seq1):
    para j de 1 a len(seq2):
      # Fallo 2: se suma gap a la diagonal en lugar de evaluar match/mismatch
      opc_diag <- F[i-1][j-1] + gap
      opc_arriba <- F[i-1][j] + gap
      opc_izq <- F[i][j-1] + gap
      F[i][j] <- max(opc_diag, opc_arriba, opc_izq)
      # Fallo 3: no se guardan punteros de dirección para el backtracking

  devolver F[len(seq1)][len(seq2)]
```

9.10 Síntesis operativa y tablas de diagnóstico

9.10.1 Tabla de diagnóstico de síntomas en alineamientos

Síntoma observado	Causa probable	Comprobación y corrección
Decenas de huecos de una sola base	abrir un hueco cuesta menos que extenderlo	invertir la relación: abrir debe costar bastante más
Smith–Waterman devuelve una tabla llena de ceros	Recompensa por match demasiado baja frente al mismatch	Asegurar que la puntuación esperada por residuo sea negativa
El <i>E</i> -value de un hit sube tras actualizar la base de datos	Aumento legítimo del tamaño <i>n</i> de la base	Es el comportamiento esperado de la estadística de Karlin
Cabecera FASTA ignorada o parseada como vacía	Espacio entre > y el ID (> ID)	Eliminar espacios en blanco tras >

9.10.2 Tabla de síntesis con preguntas de control

Tipo de problema	Herramienta recomendada	Pregunta de control
Homología global en secuencias de igual longitud	Needleman–Wunsch	¿Se acumulan penalizaciones de gap en las fronteras?
Detección de dominios y motivos locales	Smith–Waterman	¿Se incluye el piso en cero y se inicia en el máximo?
Búsqueda masiva en bases de datos genómicas	BLAST (heurística)	¿Se ha registrado el <i>E</i> -value, la versión y el enmascaramiento?

9.11 Glosario conceptual

- **Alineamiento global:** optimización de extremo a extremo (Needleman–Wunsch).
- **Alineamiento local:** búsqueda del motivo de máxima similitud (Smith–Waterman).
- **Indel:** evento mutacional de inserción o delección representado por un gap.
- **Hueco afín:** modelo con penalización diferenciada para apertura (g_o) y extensión (g_e).
- **Matriz PAM:** matriz de sustitución basada en mutaciones puntuales aceptadas (Dayhoff).
- **Matriz BLOSUM:** matriz de sustitución basada en bloques de proteínas observadas.
- **BLAST:** heurística de búsqueda rápida basada en semillas *W*-meros y extensión.
- **E-value:** número de aciertos esperados puramente por azar ($Kmne^{-\lambda S}$).
- **Homología:** relación evolutiva cualitativa derivada de un ancestro común.
- **Backtracking:** reconstrucción del alineamiento retrocediendo punteros desde el óptimo.

9.12 Conexión con el curso y siguientes pasos

El alineamiento por programación dinámica y la decodificación de Viterbi del capítulo anterior comparten dos ideas metodológicas: la programación dinámica con retroceso de punteros para reconstruir una solución completa a partir de una tabla de puntuaciones parciales, y la verificación independiente de un resultado mediante una vía distinta a la que lo produjo. Una puntuación de alineamiento o un *E*-value describe la similitud de secuencia bajo un modelo estadístico declarado; no demuestra homología, función compartida ni relevancia clínica sin una cadena de evidencia adicional. Las redes neuronales del capítulo siguiente abordan el mismo problema de comparación y clasificación de secuencias aprendiendo representaciones a partir de datos, en vez de partir de un esquema de puntuación fijo definido a mano.

9.13 Fuentes y reproducibilidad

Este capítulo se apoya en las presentaciones docentes de algoritmos y de bioinformática de la asignatura, escritas por Álvaro Serrano Navarro. El alineamiento global sigue a Needleman y Wunsch [24]; el local, a Smith y Waterman [30]; la heurística de semillas y la estadística de valores esperados, a Altschul y colaboradores [2]; los

huecos afines y las matrices de sustitución, a Durbin y colaboradores [11]; y la nota sobre la aritmética de la implementación, al manual de GNU Awk [12].

Todos los resultados numéricos se reproducen con `verification/chapter_results.sh` tal como están impresos, con sus argumentos.

Anexo práctico · Alinear, validar huecos y decidir con un E-value

Pregunta, producto y separación de materiales

¿Puede completar y retroceder a mano una matriz de alineamiento global y una local, calcular correctamente el coste de un hueco afín y detectar parámetros de hueco inválidos, interpretar un E-value de BLAST y cómo cambia con el tamaño de la base de datos, y validar el formato mínimo reproducible de una cabecera de base de datos? Este laboratorio responde que sí, sin necesitar ningún dato externo.

El producto individual son dos cosas que escribe usted: la matriz de programación dinámica de GAT frente a GC rellena a mano, en `notas/mi_matriz.tsv`, y un guion que recalcula la puntuación de un alineamiento columna a columna, en `scripts/verificar_puntuacion.sh`. Le acompañan la tabla de predicciones y resultados, las salidas guardadas y la defensa.

El anexo es autónomo: no necesita datos externos ni red. El generador deja el oráculo del capítulo y un comprobador que recalcula la matriz y prueba su guion con alineamientos que el enunciado no muestra.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 preparar el espacio	directorio creado
2 rellenar la matriz a mano	<code>notas/mi_matriz.tsv</code> completa
3 escribir la verificación ortogonal	<code>scripts/verificar_puntuacion.sh</code>
4 alineamiento local	motivo recuperado y contrastado
5 huecos afines	coste calculado y parámetros inválidos detectados
6 tres valores esperados	qué cambia el tamaño y qué cambia la puntuación
7 cabeceras y casos límite	formato reproducible validado
8 empaquetar y verificar	verificador en ENTREGA_OK

La ruta única de este anexo (no hay separación esencial/estudio) produce evidencia comprobable en cada bloque.

1 · Preparar el espacio de trabajo

```
bash --version
./practice_assets/create_workspace.sh TC09_entrega
cd TC09_entrega
cat notas/mi_matriz.tsv
```

El generador crea la rejilla vacía de la matriz, el guion de verificación con su contrato en comentarios y ningún código, la tabla de respuestas con la cabecera, la carpeta de resultados y la de herramientas, con el oráculo del capítulo y el comprobador dentro.

2 · Alineamiento global: Needleman–Wunsch

Tarea. Rellene a mano, en `notas/mi_matriz.tsv`, la matriz completa para alinear GAT con GC, con coincidencia +1, sustitución −1 y hueco −2. Son doce celdas y caben en un papel. Anote además qué celdas admiten más de un origen, es decir, dónde hay empate.

Es la misma matriz que el capítulo resuelve, y rellenarla usted tiene un objetivo concreto: comprobar el libro.

Una celda mal calculada en una matriz de este tamaño se detecta sola, porque rompe el empate que el texto anuncia en la celda final.

Compruebe su matriz cuantas veces quiera:

```
bash herramienta/check_trabajo.sh
```

Cuando la matriz cuadre, haga lo mismo con otro par distinto y contraste con el oráculo:

```
bash herramienta/chapter_results.sh needleman_wunsch ACT AGT 1 -1 -2
```

Alineando ACT con AGT (coincidencia +1, sustitución -1, hueco -2), la puntuación óptima de Needleman-Wunsch es 1: A-A coincide, C-G sustituye, T-T coincide, sin ningún hueco ($1 - 1 + 1 = 1$); el alineamiento es ACT / AGT sin desplazamiento alguno.

Se reproduce con `verification/chapter_results.sh needleman_wunsch ACT AGT 1 -1 -2`.

3 • Alineamiento local: Smith-Waterman

Alinee ahora, de forma local, AACGTAA con TTCGTTT —mismos parámetros que el bloque anterior—. Prediga primero qué motivo compartido debería recuperar el algoritmo antes de ejecutar:

```
bash herramienta/chapter_results.sh smith_waterman AACGTAA TTCGTTT 1 -1 -2
```

Alineando localmente AACGTAA con TTCGTTT (coincidencia +1, sustitución -1, hueco -2), Smith-Waterman recupera el motivo compartido CGT / CGT con puntuación 3, ignorando los flancos AA/TT al inicio y AA/TT al final de cada secuencia respectivamente.

Se reproduce con `verification/chapter_results.sh smith_waterman AACGTAA TTCGTTT 1 -1 -2`.

3 • Escribir la verificación ortogonal

Tarea. Implemente `scripts/verificar_puntuacion.sh`, que reciba dos cadenas alineadas de la misma longitud, con guiones donde haya huecos, y devuelva la puntuación sumada columna a columna. Debe rechazar dos cadenas de longitud distinta y una columna con hueco en las dos, avisando por la salida de error.

Esto es la verificación ortogonal del capítulo, convertida en producto. Su guion no repite la matriz: recorre el alineamiento ya construido y suma. Por eso sirve para detectar un error en la matriz, que un segundo cálculo con el mismo procedimiento no detectaría.

Comprueba. ¿Por qué una columna con hueco en las dos cadenas es imposible en un alineamiento bien construido? Piense qué significaría esa columna en términos de las dos secuencias originales. Su guion debe rechazarla, y usted debe saber decir por qué.

El comprobador prueba su guion con alineamientos que no aparecen en el enunciado. Cuando pase, contraste con el oráculo sobre el motivo local que acaba de recuperar:

```
bash herramienta/chapter_results.sh recompute_alignment_score CGT CGT 1 -1 -2
```

4 • Huecos afines

Calcule a mano el coste de un hueco afín de longitud 4 con apertura -6 y extensión -2, y después una fila propuesta por una IA con apertura -1 y extensión -5 —anote por qué esa segunda fila es inválida antes de ejecutar nada—:

```
bash herramienta/chapter_results.sh affine_gap_cost 4 -6 -2
bash herramienta/chapter_results.sh validate_affine_gap_params -1 -5
```

Un hueco afín de longitud 4 con apertura -6 y extensión -2 cuesta $g(4) = -6 + 3 \cdot (-2) = -12$. Los parámetros apertura -1 , extensión -5 son inválidos porque $|-1| < |-5|$: harían más barato abrir un hueco nuevo en cada posición que prolongar uno existente, lo contrario de lo que un modelo de hueco afín debe garantizar.

Se reproduce con `verification/chapter_results.sh affine_gap_cost 3 -10 -1`.

Se reproduce con `verification/chapter_results.sh validate_affine_gap_params -1 -5`.

5 • E-value y tamaño de la base de datos

Con $K = 0,08$, $\lambda = 0,4$, consulta $m = 25$ y score $S = 35$, calcule el E-value contra una base de $n = 2\,000\,000$ secuencias y decida si el hit es significativo. Después repita contra una base 5 veces mayor ($n = 10\,000\,000$) sin cambiar nada más, y anote qué cambia y por qué:

```
bash herramienta/chapter_results.sh blast_evalue 0.08 0.4 25 2000000 35 > resultados/evaluate_base.txt
bash herramienta/chapter_results.sh blast_evalue 0.08 0.4 25 10000000 35 > resultados/evaluate_base_mayor.txt
bash herramienta/chapter_results.sh blast_evalue 0.08 0.4 25 2000000 45 > resultados/evaluate_score_mayor.txt
cat resultados/evaluate_base.txt resultados/evaluate_base_mayor.txt resultados/evaluate_score_mayor.txt
```

Los tres cálculos varían una sola cosa cada vez. El segundo multiplica por cinco el tamaño de la base y deja el resto igual; el tercero sube la puntuación diez puntos y deja el tamaño original. Anote qué le hace cada cambio al valor esperado, y en qué se diferencian los dos efectos: uno es proporcional y el otro exponencial.

Con $K = 0.08$, $\lambda = 0.4$, $m = 25$, $S = 35$, el E-value contra $n = 2\,000\,000$ es ≈ 3.326115 -no significativo, más de tres coincidencias igual de buenas esperadas por azar-; contra una base 5 veces mayor, $n = 10\,000\,000$, el E-value crece proporcionalmente a ≈ 16.630574 , con el mismo score y el mismo hit: crecer la base de búsqueda nunca hace un hit más significativo, y puede volver no significativo lo que en una base pequeña parecía prometedor.

Se reproduce con `verification/chapter_results.sh blast_evalue 0.08 0.4 25 2000000 35`.

Se reproduce con `verification/chapter_results.sh blast_evalue 0.08 0.4 25 10000000 35`.

6 • Cabeceras FASTA

Valide las tres cabeceras siguientes y anote, para cada una, si es válida y por qué:

```
bash herramienta/chapter_results.sh validate_fasta_header ">seq_A"
bash herramienta/chapter_results.sh validate_fasta_header "> descripcion sin accesion"
bash herramienta/chapter_results.sh validate_fasta_header "seq_A sin corchete inicial"
```

`>seq_A` es una cabecera FASTA válida (accesión `seq_A`, sin descripción); `> descripcion sin accesion` es inválida porque el primer carácter tras `>` es un espacio (accesión vacía); `seq_A sin corchete inicial` es inválida porque no empieza por `>`. Las tres decisiones deben tomarse antes de procesar el fichero como si fuera una secuencia válida, no después de un error de más adelante en la tubería.

Se reproduce con `verification/chapter_results.sh validate_fasta_header >seq_A`.

7 • Controles de robustez

```
bash herramienta/chapter_results.sh needleman_wunsch "" "" 1 -1 -2
bash herramienta/chapter_results.sh smith_waterman A A 1 -1 -2
```

Alinear dos secuencias vacías con Needleman–Wunsch no falla: la única celda de la matriz es $F(0,0) = 0$, y el alineamiento es vacío frente a vacío -puntuación 0, sin coincidencias, sustituciones ni huecos-. Alinear un único

símbolo idéntico con Smith–Waterman (A frente a A) da el caso más simple posible: puntuación 1, un único origen diagonal, sin necesidad de ningún retroceso multi-paso.

Se reproduce con `verification/chapter_results.sh needleman_wunsch A A 1 -1 -2`.

Se reproduce con `verification/chapter_results.sh smith_waterman A T 1 -1 -2`.

8 · Verificar la entrega

Escriba antes `notas/defensa.md`. Después:

```
cd ..
tar -czf TC09_entrega.tar.gz TC09_entrega
sha256sum TC09_entrega.tar.gz
./practice_assets/check_entrega.sh TC09_entrega TC09_entrega.tar.gz
```

El verificador recalcula su matriz celda a celda, prueba su guion con alineamientos que no ha visto, comprueba que la tabla de respuestas tiene al menos cinco filas con justificación y una del valor esperado, y que resultados/recoge los tres valores esperados distintos del bloque de sensibilidad. Rechaza el espacio recién generado. No puntúa.

Rúbrica de calificación ponderada

La evaluación pondera la verificación técnica, el cálculo de alineamientos y la defensa conceptual sobre 10 puntos, tres de entrega y siete de razonamiento:

- **3,0 puntos, Entrega técnica y verificador (DELIVERY_OK):** Integridad del paquete comprimido `.tar.gz`, formato requerido de `notas/respuestas.tsv` y superación de `check_practice_delivery.sh`.
- **2,5 puntos, Matrices de programación dinámica (NW y SW):** Construcción manual de tablas de puntuación, condiciones de contorno y reconstrucción por retroceso (*backtracking*).
- **2,5 puntos, Modelo afín, estadística de Karlin–Altschul y reproducibilidad:** Cálculo de penalizaciones afines, análisis de sensibilidad del *E-value* ante cambios en la base de datos y validación estricta de cabeceras FASTA.
- **2,0 puntos, Defensa conceptual y cautela inferencial:** Justificación rigurosa (100–150 palabras) sobre por qué la significación estadística (*E-value*) y la similitud no equivalen automáticamente a homología ni a función biológica demostrada.

Terminar en `ENTREGA_OK` acredita que la matriz y el guion cuadran, no que la interpretación del valor esperado sea correcta.

Lista de autocorrección

- Mi matriz manual de ACT/AGT coincidió con el script.
- Recuperé el motivo CGT en el alineamiento local y comprobé su score.
- Calculé correctamente el coste del hueco afín de longitud 4 y detecté la fila inválida.
- Registré cómo cambia el *E-value* al multiplicar por 5 el tamaño de la base de datos.
- Clasifiqué correctamente las tres cabeceras FASTA como válidas o inválidas y por qué.
- Los dos controles de caso límite están anotados sin fallos.
- Puedo explicar por qué un *E-value* bajo no equivale a homología demostrada.
- el verificador termina en `ENTREGA_OK`.

Defensa conceptual

En 100–150 palabras, explique por qué duplicar el tamaño de una base de datos duplica el *E-value* de un hit con la misma puntuación, y qué tendría que cambiar en el score de ese hit para compensar exactamente ese

aumento y mantener el mismo E-value.

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas docentes de bioinformática (20251125_bioinfo_5.pptx) y se valida al 100% mediante `verification/chapter_results.sh`.

Redes neuronales para bioinformática

TC-CH10 – Técnicas Computacionales en Biología

Edición 2

Pregunta del tema. ¿Cómo puede un programa aprender por sí mismo a distinguir un enhancer activo de uno inactivo a partir de señales epigenéticas, y qué garantías tenemos de que lo que ha aprendido sirva en un hospital distinto del que produjo los datos?

Producto final. El cálculo a mano, verificado con un oráculo, de una pasada completa de entrenamiento de una red pequeña, incluido el paso hacia atrás hasta la capa oculta, más una auditoría del diseño de validación que evite la fuga de datos por paciente.

Mapa del tema

10.1. De modelos explícitos a representaciones aprendidas	166
10.2. El Perceptrón y la no linealidad	167
10.3. Entrenamiento, pérdida y retropropagación	168
10.4. Caso de estudio resuelto: predicción de enhancers	169
10.5. Sobreajuste y alta dimensionalidad en ciencias ómicas	173
10.6. Responsabilidad, ética y límites del aprendizaje automático	173
10.7. La operación peligrosa: fugas de datos y saturación	174
10.8. Actividad de depuración: errores en la retropropagación	174
10.9. Síntesis operativa y tablas de diagnóstico	175
10.10. Glosario conceptual	175
10.11. Cierre del libro: la caja de herramientas computacionales	175
10.12. Fuentes y reproducibilidad	176

Cómo usar este tema

Este capítulo cierra el libro y hace algo que ninguno de los anteriores necesitaba: además de explicar cómo funciona el método, dedica una sección entera a cuándo no hay que fiarse de él. Las dos partes son igual de importantes, y la segunda es la que distingue a quien sabe usar una red de quien sabe invocarla.

Al terminar sabrá: seguir a mano una pasada completa de entrenamiento de una red pequeña, hacia delante y hacia atrás; explicar por qué una red sin funciones de activación no lineales es equivalente a una regresión lineal; reconocer el régimen en que hay más variables que muestras y qué le hace al modelo; y auditar un diseño de validación buscando por dónde se filtra la información del conjunto de prueba.

La ruta **esencial** son el mapa del aprendizaje automático, la neurona y sus activaciones, la pérdida, el descenso de gradiente, el caso resuelto y el marco de responsabilidad. La ruta de **estudio** añade el sobreajuste con más variables que muestras, la saturación, la actividad de depuración y el anexo.

10.1 De modelos explícitos a representaciones aprendidas

ESENCIAL A lo largo de los capítulos anteriores hemos construido modelos con reglas, matrices o distribuciones explícitamente diseñadas por el analista:

- En **Naïve-Bayes** (TC-CH08), asumimos independencia condicional y estimamos frecuencias directas de nucleótidos.
- En **Cadenas y Modelos Ocultos de Markov** (TC-CH08), fijamos matrices de transición estocásticas para capturar dependencias de orden 1.

- En **Alineamiento de secuencias** (TC-CH09), definimos a mano matrices de sustitución (PAM, BLOSUM) y penalizaciones afines de hueco.

Sin embargo, cuando nos enfrentamos a problemas biomédicos complejos —como integrar simultáneamente múltiples pistas epigenéticas, perfiles de expresión génica masiva o predecir la actividad de regiones reguladoras distales (*enhancers*)—, las interacciones biológicas no lineales y combinatorias resultan difíciles de codificar a mano con reglas fijas.

Este capítulo cierra el libro introduciendo las **redes neuronales Artificiales**: arquitecturas parametrizadas capaces de **aprender representaciones jerárquicas y fronteras de decisión complejas directamente a partir de los datos**.

Las redes neuronales representan la transición desde modelos probabilísticos artesanales y reglas de puntuación fijas hacia arquitecturas parametrizadas que aprenden representaciones jerárquicas y fronteras de decisión no lineales directamente a partir de datos biológicos.

10.2 El Perceptrón y la no linealidad

10.2.1 Dónde estamos: un mapa del aprendizaje automático

ESENCIAL La **inteligencia artificial** es el campo. Dentro de ella, el **aprendizaje automático** construye modelos a partir de datos en vez de reglas escritas a mano. Dentro de este, el **aprendizaje profundo** usa redes neuronales con muchas capas. Y la **inteligencia artificial generativa** es la familia de modelos profundos que producen datos nuevos, texto, imágenes o secuencias, en lugar de clasificarlos.

Un modelo aprende en dos fases: **entrenamiento**, donde ve ejemplos y ajusta sus parámetros, e **inferencia**, donde predice sobre casos nuevos. Si los ejemplos llevan etiqueta, esta región es un enhancer y esta no, el aprendizaje es **supervisado**; si no la llevan y el modelo busca estructura por sí mismo, agrupando pacientes con perfiles parecidos, es **no supervisado**.

Hay otra distinción que conviene tener clara. Un modelo **discriminativo** aprende la frontera entre clases: dado este perfil, ¿enfermo o sano? Un modelo **generativo** aprende cómo son los datos de cada clase y puede fabricar ejemplos nuevos. El clasificador de Bayes del capítulo de probabilidad es generativo; las redes de este capítulo son discriminativas. Los grandes modelos de lenguaje son generativos: predicen la siguiente palabra calculando probabilidades.

Tarea	Modelos habituales	Cuándo
Clasificar pacientes a partir de expresión génica	máquinas de vectores soporte, bosques aleatorios, redes	supervisado, hay etiquetas
Reducir dimensiones y visualizar cohortes	análisis de componentes principales, t-SNE, UMAP	no supervisado, exploración
Predecir actividad reguladora desde secuencia y epigenética	redes neuronales	supervisado, interacciones no lineales

Concepto. Las redes neuronales no sustituyen a las otras familias: son una opción más. Conviene cuando hay muchos datos y las interacciones entre variables son complejas, y son mala elección cuando hay pocos ejemplos, que es justo el régimen habitual en biomedicina y del que se ocupa la sección de sobreajuste.

10.2.2 De la neurona biológica a la unidad de cómputo

El **perceptrón** [26] es la unidad básica del aprendizaje profundo, inspirado en el funcionamiento electroquímico de las neuronas biológicas:

1. **Dendritas (Entradas)**: Reciben el vector de características biológicas $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$ (p. ej., niveles de metilación o accesibilidad cromatínica).
2. **Soma (Combinación lineal)**: Realiza una suma ponderada de las entradas modulada por los pesos

sinápticos $\mathbf{w}$ y un sesgo basal (*bias*) b :

$$z = \sum_{i=1}^d w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

3. **Axón (Activación y Salida):** Transforma el potencial lineal z mediante una **función de activación no lineal** $f(z)$ para emitir la activación final $a = f(z)$.

El perceptrón artificial modela una neurona biológica calculando una combinación afín $z = \sum w_i x_i + b$ y aplicando una función de activación $a = f(z)$ para generar la salida.

10.2.3 La necesidad matemática de la no linealidad

Si utilizáramos funciones de activación lineales ($f(z) = c \cdot z$), una red neuronal con múltiples capas ocultas colapsaría matemáticamente a una única transformación afín equivalente:

$$\mathbf{y} = \mathbf{W}_2(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2 \mathbf{W}_1) \mathbf{x} + (\mathbf{W}_2 \mathbf{b}_1 + \mathbf{b}_2) = \mathbf{W}_{\text{eq}} \mathbf{x} + \mathbf{b}_{\text{eq}}$$

Una red puramente lineal no puede resolver problemas que no sean linealmente separables, y el caso canónico es la función XOR. Su equivalente biológico es un gen que se activa cuando está presente uno de dos factores, pero *no* cuando coinciden los dos, porque compiten por el mismo sitio de unión. Dibuje los cuatro casos en un plano y compruébelo: no hay ninguna recta que separe los dos que activan de los dos que no.

Las funciones de activación no lineales (Sigmoide, ReLU, Tanh) son matemáticamente indispensables porque una composición de capas puramente lineales colapsa a una única transformación lineal, incapaz de resolver problemas no linealmente separables como el XOR biológico.

Funciones de activación canónicas:

- **Sigmoide** (σ): $\sigma(z) = \frac{1}{1+e^{-z}}$, con codominio en $(0, 1)$. Es idónea para capas de salida en clasificación binaria porque su valor se interpreta como una probabilidad $P(y = 1 \mid \mathbf{x})$. Su derivada analítica es $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
- **Unidad lineal rectificadora (ReLU):** $\text{ReLU}(z) = \max(0, z)$. Es el estándar en capas ocultas porque su derivada vale 1 para $z > 0$, y ahí no hay saturación. Para $z < 0$ la derivada es 0: si una unidad deja de recibir entradas positivas, su gradiente es nulo y no vuelve a aprender. A eso se le llama unidad muerta, y es la contrapartida de su ventaja.
- **Tangente hiperbólica (tanh):** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, centrada en cero con codominio en $(-1, 1)$.

El peligro de la saturación del gradiente en la sigmoide:

Cuando el valor absoluto de la preactivación es elevado ($|z| \gg 0$), la función sigmoide se vuelve prácticamente plana ($\sigma(z) \approx 0$ o $\sigma(z) \approx 1$). En ese régimen, su derivada tiende a cero:

$$\lim_{|z| \rightarrow \infty} \sigma'(z) = \lim_{|z| \rightarrow \infty} \sigma(z)(1 - \sigma(z)) = 0$$

Esto provoca el fenómeno del **desvanecimiento del gradiente** (*vanishing gradient*), congelando el aprendizaje de los pesos en capas anteriores.

Cuando las entradas a una función sigmoide tienen gran magnitud absoluta ($|z| \gg 0$), la derivada $\sigma'(z)$ tiende a cero, provocando el desvanecimiento del gradiente y deteniendo el aprendizaje de los pesos en capas precedentes.

Se reproduce con `verification/chapter_results.sh saturation_control 10.0`.

10.3 Entrenamiento, pérdida y retropropagación

El entrenamiento de una red neuronal consiste en encontrar el conjunto de pesos $\mathbf{W}$ y sesgos $\mathbf{b}$ que minimicen la discrepancia entre las predicciones del modelo $\hat{y}$ y las etiquetas biológicas reales y .

10.3.1 Función de pérdida de Entropía Cruzada Binaria

Para tareas de clasificación binaria supervisada con $y \in \{0, 1\}$ y $\hat{y} = P(y = 1 | \mathbf{x}) \in (0, 1)$, la función de pérdida canónica es la **Entropía Cruzada Binaria** (*Binary Cross-Entropy*, BCE), fundamentada como el negativo de la log-verosimilitud de una distribución de Bernoulli:

$$P(y | \mathbf{x}) = \hat{y}^y (1 - \hat{y})^{1-y} \implies -\ln P(y | \mathbf{x}) = -[y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})]$$

$$\mathcal{L}(y, \hat{y}) = -[y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})]$$

Esta formulación conecta directamente con la divergencia KL y la entropía cruzada formalizadas en TC-CH07.

La función de pérdida de Entropía Cruzada Binaria mide la divergencia entre la etiqueta real $y \in \{0, 1\}$ y la probabilidad predicha $\hat{y}$ mediante $\mathcal{L}(y, \hat{y}) = -[y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})]$.

10.3.2 Optimización por Descenso de Gradiente

El algoritmo de **Descenso de Gradiente** ajusta iterativamente cada peso en la dirección opuesta al gradiente de la función de pérdida:

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial \mathcal{L}}{\partial w_{ij}}$$

donde $\eta > 0$ es la **tasa de aprendizaje** (*learning rate*).

El descenso de gradiente actualiza iterativamente los parámetros en la dirección opuesta al gradiente de la pérdida según $w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$, donde η es la tasa de aprendizaje.

Impacto de la tasa de aprendizaje η :

- Si η es excesivamente grande ($\eta = 10.0$), el algoritmo da pasos desproporcionados, sobrepasa el mínimo y diverge numéricamente.
- Si η es muy pequeña ($\eta = 10^{-4}$), cada paso es minúsculo y la convergencia se vuelve lentísima. El gradiente no es nulo: simplemente se avanza poco por iteración.

Atención. Conviene no confundir las dos cosas. Una meseta de gradiente casi nulo es un problema distinto y no se arregla cambiando la tasa de aprendizaje: procede de la saturación de las activaciones, cuando la sigmoide trabaja en sus extremos y su derivada tiende a cero, o de unidades muertas. Si el entrenamiento no avanza, la pregunta no es solo cuánto vale la tasa, sino cuánto valen los gradientes.

Una tasa de aprendizaje excesivamente grande provoca divergencia numérica u oscilaciones destructivas sobre la superficie de pérdida, mientras que una tasa excesivamente pequeña estanca la convergencia; un ajuste riguroso de hiperparámetros es indispensable para la estabilidad del entrenamiento.

Se reproduce con `verification/chapter_results.sh learning_rate_perturbation 0.1`.

10.3.3 El algoritmo de Retropropagación (*Backpropagation*)

El algoritmo de **Retropropagación** (Rumelhart, Hinton & Williams, 1986) calcula de forma eficiente las derivadas parciales $\frac{\partial \mathcal{L}}{\partial w_{ij}}$ aplicando la **regla de la cadena** del cálculo multivariable en sentido inverso, desde la capa de salida hacia la capa de entrada.

La retropropagación calcula las derivadas parciales exactas de la función de pérdida respecto a cada peso y sesgo aplicando la regla de la cadena del cálculo multivariable de forma recursiva desde la capa de salida hacia las capas ocultas.

10.4 Caso de estudio resuelto: predicción de enhancers

Consideremos una red neuronal feedforward con arquitectura $2 \rightarrow 2 \rightarrow 1$ diseñada para predecir si una región genómica es un **enhancer activo** ($y = 1$) a partir de dos biomarcadores epigenéticos continuos:

- $x_1 = 1.0$: Señal normalizada de enriquecimiento de la marca de histona activa H3K27ac.
- $x_2 = 0.5$: Accesibilidad cromatínica cuantificada por ATAC-seq.

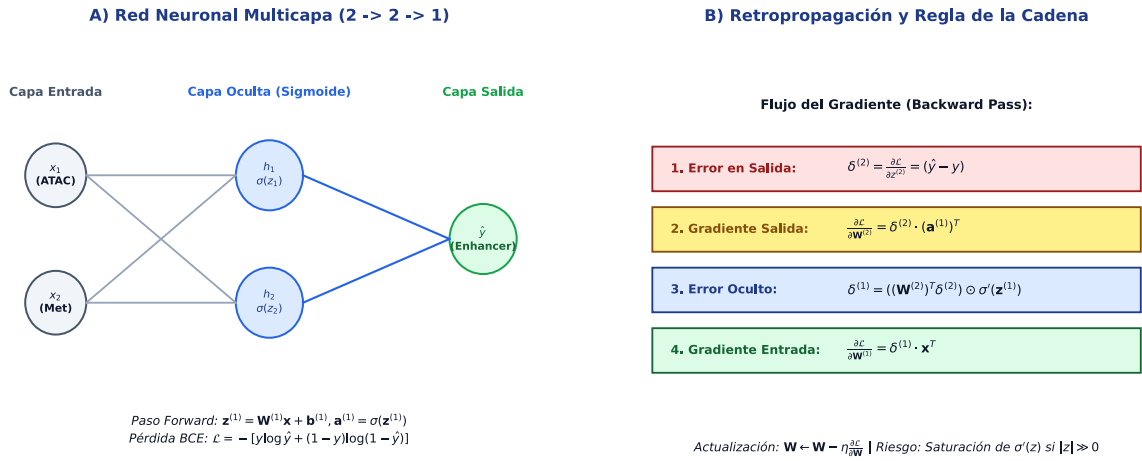


Figura 10.1: A) Arquitectura de Red Neuronal Multicapa (2 → 2 → 1) para clasificación epigenómica de enhancers activos. B) Flujo de retropropagación (*backpropagation*) y cálculo formal de gradientes mediante la regla de la cadena. Figura original adaptada de las presentaciones docentes.

Parámetros iniciales de la red:

- **Capa oculta (2 neuronas con activación sigmoide):**
 - Neurona h_1 : Pesos $w_{11}^{(1)} = 0.5$, $w_{12}^{(1)} = -0.3$; sesgo $b_1^{(1)} = 0.1$.
 - Neurona h_2 : Pesos $w_{21}^{(1)} = -0.2$, $w_{22}^{(1)} = 0.8$; sesgo $b_2^{(1)} = -0.2$.
- **Capa de salida (1 neurona con activación sigmoide):** Pesos $w_1^{(2)} = 0.7$, $w_2^{(2)} = -0.4$; sesgo $b^{(2)} = 0.2$.

10.4.1 Paso hacia adelante (*Forward Pass*)

1. Activaciones de la capa oculta:

- Preactivación de h_1 :

$$z_{h1} = w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + b_1^{(1)} = (0.5)(1.0) + (-0.3)(0.5) + 0.1 = 0.5 - 0.15 + 0.1 = \mathbf{0.45}$$

Activación de h_1 :

$$a_{h1} = \sigma(0.45) = \frac{1}{1 + e^{-0.45}} \approx \mathbf{0.610639}$$

- Preactivación de h_2 :

$$z_{h2} = w_{21}^{(1)}x_1 + w_{22}^{(1)}x_2 + b_2^{(1)} = (-0.2)(1.0) + (0.8)(0.5) - 0.2 = -0.2 + 0.4 - 0.2 = \mathbf{0.00}$$

Activación de h_2 :

$$a_{h2} = \sigma(0.00) = \frac{1}{1 + e^0} = \mathbf{0.500000}$$

2. Activación de la capa de salida:

- Preactivación de salida:

$$z_{\text{out}} = w_1^{(2)}a_{h1} + w_2^{(2)}a_{h2} + b^{(2)} = (0.7)(0.610639) + (-0.4)(0.500000) + 0.2 \approx \mathbf{0.427447}$$

- Probabilidad predicha final ($\hat{y}$):

$$\hat{y} = \sigma(0.427447) = \frac{1}{1 + e^{-0.427447}} \approx \mathbf{0.605264}$$

En la red 2-2-1 de predicción de enhancers con entradas epigenéticas $x_1 = 1.0$ (H3K27ac) y $x_2 = 0.5$ (ATAC-seq), el paso forward calcula las preactivaciones ocultas $z_{h1} = 0.45$ ($a_{h1} = 0.610639$) y $z_{h2} = 0.00$ ($a_{h2} = 0.500000$), produciendo una preactivación de salida $z_{out} = 0.427447$ y una predicción $\hat{y} = 0.605264$.

Se reproduce con `verification/chapter_results.sh forward_enhancer_pass 1.0 0.5`.

10.4.2 Evaluación de la función de pérdida

Dado que la muestra observada corresponde experimentalmente a un enhancer activo ($y = 1$):

$$\mathcal{L}(1, 0.605264) = -\ln(0.605264) \approx \mathbf{0.502091}$$

Para la etiqueta real $y = 1$ (enhancer activo) y la predicción $\hat{y} = 0.605264$, la pérdida de Entropía Cruzada Binaria evalúa a $-\ln(0.605264) = 0.502091$.

Se reproduce con `verification/chapter_results.sh bce_loss 1.0 0.605264`.

10.4.3 Paso hacia atrás (*Backward Pass*) y actualización de pesos

Para la combinación de pérdida de entropía cruzada con salida sigmoide, el error en el nodo de salida se simplifica elegantemente a:

$$\delta_{out} = \frac{\partial \mathcal{L}}{\partial z_{out}} = \hat{y} - y = 0.605264 - 1.0 = \mathbf{-0.394736}$$

Gradientes de los pesos de la capa de salida:

- $\frac{\partial \mathcal{L}}{\partial w_1^{(2)}} = \delta_{out} \cdot a_{h1} = (-0.394736)(0.610639) \approx \mathbf{-0.241041}$
- $\frac{\partial \mathcal{L}}{\partial w_2^{(2)}} = \delta_{out} \cdot a_{h2} = (-0.394736)(0.500000) \approx \mathbf{-0.197368}$

Actualización de pesos con tasa de aprendizaje $\eta = 0.1$:

- $w_1^{(2)} \leftarrow 0.7 - (0.1)(-0.241041) = 0.7 + 0.024104 = \mathbf{0.724104}$
- $w_2^{(2)} \leftarrow -0.4 - (0.1)(-0.197368) = -0.4 + 0.019737 = \mathbf{-0.380263}$

El peso $w_1^{(2)}$ conecta la neurona oculta h_1 con la salida, y aumenta porque aumentar la contribución de h_1 sube la probabilidad predicha, que es lo que hace falta cuando la etiqueta es 1.

Atención. Ese incremento no mide la importancia de H3K27ac. La neurona h_1 mezcla las dos entradas, así que su peso hacia la salida no se puede leer como «la red ha aprendido que esta marca importa». Atribuir importancia a una variable de entrada exige métodos específicos, y aun así son estimaciones y no explicaciones. Es el error de interpretación más frecuente con redes pequeñas, y la sección sobre la operación peligrosa vuelve sobre él.

En el paso backward para $y = 1$, el error de salida δ_{out} es -0.394736 , produciendo los gradientes $\frac{\partial \mathcal{L}}{\partial w_1^{(2)}} = -0.241041$ y $\frac{\partial \mathcal{L}}{\partial w_2^{(2)}} = -0.197368$; actualizando con $\eta = 0.1$ se obtiene el nuevo peso $w_1^{(2)} = 0.724104$.

Se reproduce con `verification/chapter_results.sh backward_enhancer_pass 1.0 0.5 1.0 0.1`.

Y ahora hacia atrás otra vez: la capa oculta.

Hasta aquí solo hemos actualizado la última capa, que es la parte fácil porque su error se conoce directamente. El interés de la retropropagación está en el paso siguiente: repartir ese error entre las neuronas ocultas, cada una según cuánto contribuyó.

El error que llega a cada neurona oculta es el de la salida, multiplicado por el peso que la conecta con ella y por la derivada de su propia activación:

$$\delta_h = (W^{(2)})^\top \delta_{out} \odot \sigma'(z_h)$$

Con los números del caso, $\delta_{out} = -0.394736$, $a_{h1} = 0.610639$ y $a_{h2} = 0.5$:

- $\delta_{h1} = 0.7 \cdot (-0.394736) \cdot 0.610639 \cdot 0.389361 = \mathbf{-0.065696}$

$$\blacksquare \delta_{h_2} = -0,4 \cdot (-0,394736) \cdot 0,5 \cdot 0,5 = 0,039474$$

Fíjese en los signos: los dos errores tienen signo contrario, porque los pesos que unen cada neurona con la salida también lo tienen. La red no ajusta las dos neuronas ocultas en la misma dirección.

Los gradientes de la primera capa son δ_{h_i} multiplicado por la entrada correspondiente, y con $\eta = 0,1$ los pesos quedan así:

$$\begin{aligned} \blacksquare w_{11} &= 0,506570, & w_{12} &= -0,296715 \\ \blacksquare w_{21} &= -0,203947, & w_{22} &= 0,798026 \\ \blacksquare b_1 &= 0,106570, & b_2 &= -0,203947 \end{aligned}$$

Continuando el backward hasta la capa oculta con los mismos parámetros, los errores retropropagados son $-0,065696$ y $0,039474$, y los pesos de la primera capa pasan a $0,506570$, $-0,296715$, $-0,203947$ y $0,798026$.

Se reproduce con `verification/chapter_results.sh backward_hidden 1.0 0.5 1.0 0.1`.

Comprueba. Compare la magnitud de los errores de la capa oculta con la de la capa de salida: son entre seis y diez veces menores. Multiplicar por pesos menores que uno y por derivadas de la sigmoide, que valen como mucho 0,25, encoge el error en cada capa que se retrocede. Con dos capas es irrelevante; con veinte, es el problema del gradiente que se desvanece, y explica por qué las redes profundas necesitaron ReLU y otras ideas para poder entrenarse.

```
ALGORITMO: Red_Neuronal_Forward_Backward_2x2x1
ENTRADA:
- x = [x1, x2]: Vector de características de entrada (H3K27ac y ATAC-seq, como en el caso resuelto)
- y: Etiqueta real binaria en {0, 1}
- eta: Tasa de aprendizaje (learning rate, ej. 0.1)
- W1 (2x2), b1 (2x1), W2 (1x2), b2 (1x1): Parametros actuales de la red
SALIDA:
- y_hat: Probabilidad predicha en [0, 1]
- Perdida_BCE: Perdida de entropia cruzada binaria
- W1_nuevo, b1_nuevo, W2_nuevo, b2_nuevo: Parametros actualizados por descenso de gradiente
COMPLEJIDAD: O(1) tiempo por iteracion, O(1) espacio

1. PASO HACIA ADELANTE (FORWARD PASS):
  z1 <- W1 * x + b1                # Preactivacion capa oculta (2x1)
  a1 <- [sigma(z1[0]), sigma(z1[1])] # Activacion sigmoide sigma(z) = 1 / (1 + exp(-z))

  z2 <- W2 * a1 + b2                # Preactivacion capa de salida (1x1)
  y_hat <- sigma(z2)                # Prediccion final

  Perdida_BCE <- - (y * log(y_hat) + (1 - y) * log(1 - y_hat))

2. PASO HACIA ATRAS (BACKWARD PASS):
  delta_salida <- y_hat - y          # Error en salida (gradiente BCE + Sigmoide)

  # Gradientes capa 2 (Salida)
  grad_W2 <- delta_salida * a1^T
  grad_b2 <- delta_salida

  # Propagacion del error hacia la capa oculta 1
  sigma_prime_z1 <- [a1[0] * (1 - a1[0]), a1[1] * (1 - a1[1])]
  delta_oculto <- (W2^T * delta_salida) (producto elemento a elemento) sigma_prime_z1

  # Gradientes capa 1 (Oculta)
  grad_W1 <- delta_oculto * x^T
  grad_b1 <- delta_oculto

3. ACTUALIZACION DE PARAMETROS (DESCENSO DE GRADIENTE):
  W2_nuevo <- W2 - eta * grad_W2
  b2_nuevo <- b2 - eta * grad_b2
  W1_nuevo <- W1 - eta * grad_W1
  b1_nuevo <- b1 - eta * grad_b1
```

4. RETORNO:

```
Retornar (y_hat, Perdida_BCE, W1_nuevo, b1_nuevo, W2_nuevo, b2_nuevo)
```

10.5 Sobreajuste y alta dimensionalidad en ciencias ómicas

En biomedicina computacional es ubicuo el régimen de **alta dimensionalidad** ($p \gg n$): disponemos de la expresión de $p = 20\,000$ genes o millones de variantes para una cohorte clínica de solo $n = 100$ pacientes.

En este escenario, una red neuronal profunda con miles de parámetros libres puede memorizar con facilidad el ruido muestral de entrenamiento, alcanzando un 100% de exactitud aparente pero fallando catastróficamente al predecir pacientes de un nuevo hospital (**sobreajuste** / *overfitting*).

Técnicas esenciales de regularización:

- **Regularización L_2 (Weight Decay):** Penaliza pesos grandes sumando $\frac{1}{2}\lambda\|\mathbf{w}\|^2$ a la pérdida.
- **Desconexión aleatoria (Dropout):** Desactiva aleatoriamente una fracción de neuronas (p. ej., 20–50%) en cada paso de entrenamiento para evitar coadaptaciones espurias.
- **Parada temprana (Early Stopping):** Monitoriza la pérdida en un conjunto de validación independiente y detiene el entrenamiento antes de que comience el sobreajuste.

En datos ómicos, la alta dimensionalidad ($p \gg n$, miles de variables de expresión génica frente a decenas de pacientes) crea un elevado riesgo de sobreajuste, donde el modelo memoriza el ruido de entrenamiento sin generalizar a nuevas cohortes clínicas.

10.6 Responsabilidad, ética y límites del aprendizaje automático

Un modelo que decide sobre personas exige algo más que exactitud. Esta sección recoge las limitaciones éticas y metodológicas que hay que auditar antes de llevar un modelo de caja negra a un contexto clínico, y forma parte de los resultados de aprendizaje de la asignatura tanto como el cálculo del gradiente.

1. Correlación estadística frente a causalidad biológica:

Una red neuronal detecta asociaciones estadísticas en la distribución de entrenamiento; **no demuestra causalidad biológica ni función mecanística**. Un biomarcador con alto peso predictivo puede ser un subproducto secundario de la enfermedad o un artefacto técnico (*efecto de lote* o *batch effect* del secuenciador).

2. Sesgo poblacional y demográfico:

Si un modelo de riesgo poligénico o de respuesta a fármacos se entrena mayoritariamente sobre genomas de ascendencia europea, su rendimiento se degrada drásticamente al aplicarse sobre poblaciones históricamente subrepresentadas, pudiendo inducir disparidades diagnósticas y daños clínicos.

Las predicciones de aprendizaje automático en bioinformática reflejan correlaciones estadísticas en los datos de entrenamiento y no establecen causalidad biológica ni sustituyen la validación experimental o el juicio clínico; los modelos deben auditarse frente a sesgos demográficos y aprendizaje de atajos espurios.

3. Protocolo de validación clínica en 3 niveles y prevención de fugas:

Toda aplicación biomédica de IA debe estructurarse en tres niveles metodológicos independientes:

1. **Entrenamiento y validación interna:** Ajuste de preprocesamiento, pesos e hiperparámetros mediante validación cruzada agrupada por paciente (*GroupKFold*). En un estudio con 1000 muestras de 100 pacientes, dividir filas al azar filtra la identidad del paciente entre entrenamiento y test (*data leakage*), haciendo que la red memorice al individuo en vez de la enfermedad.
2. **Conjunto de prueba congelado (Test Set):** Evaluación final única sobre datos retenidos desde el inicio, sin reajustar ningún hiperparámetro.

3. **Validación externa independiente:** Evaluación en una cohorte procedente de otro hospital o tecnología para comprobar la generalización real (*out-of-distribution*).

Métricas y supervisión: Reportar obligatoriamente sensibilidad, especificidad y AUC-ROC junto a intervalos de confianza, manteniendo la supervisión humana como principio innegociable.

La auditoría de riesgos de un modelo exige validación cruzada estratificada en cohortes independientes sin fugas de datos y evaluación mediante métricas clínicas completas (sensibilidad, especificidad, AUC-ROC) en lugar de la exactitud global agregada.

10.7 La operación peligrosa: fugas de datos y saturación

Los fallos más graves en proyectos bioinformáticos de aprendizaje automático son:

1. **Fuga de datos (*Data Leakage*):** Normalizar o seleccionar genes sobre la totalidad del dataset antes de dividir en entrenamiento y test, generando métricas optimistas falsas.
2. **Tasas de aprendizaje inadecuadas:** Divergencia silenciosa a valores NaN o estancamiento por saturación sigmoidea.

Protocolo preventivo en 3 pasos:

1. **Aislamiento del conjunto de prueba:** Congelar el test set antes de cualquier preprocesamiento, imputación o escalado.
2. **Aserción de gradientes finitos:** Comprobar tras cada época que ningún peso contenga valores infinitos o NaN.
3. **Diagnóstico de interpretabilidad:** Emplear mapas de saliencia o valores SHAP para formular hipótesis auditables sobre las variables relevantes, contrastándolas siempre con modelos de referencia (*baselines* lineales) y sin asumirlas como demostración causal o prueba de ausencia de atajos.

10.8 Actividad de depuración: errores en la retropropagación

Analice el siguiente fragmento, que tiene tres errores en la actualización de una red:

```
backward_step(x, y_true, y_hat, a_hidden, z_hidden, W_out, eta):
    delta_out <- y_hat - y_true

    W_out[1] <- W_out[1] - (eta * delta_out * a_hidden[1])    # Fallo 3

    delta_hidden <- W_out[1] * delta_out                      # Fallo 1
    a_hidden <- sigmoide(W_in * x + b_in)                     # Fallo 2
    devolver W_out
```

Tarea. Localice los tres y clasifíquelos: cuál produce un resultado numéricamente incorrecto desde la primera iteración, cuál lo desvía poco a poco, y cuál solo desperdicia trabajo. Compruebe cada uno contra el desarrollo del caso resuelto.

Diagnóstico.

1. **Falta la derivada de la activación.** El error que llega a una neurona oculta es el de la salida por el peso *y por* $\sigma'(z_h)$. Sin ese factor, el error retropropagado es varias veces mayor de lo que debe, y el signo puede ser el correcto mientras la magnitud no lo es. Es el error que da valores plausibles y equivocados.
2. **Se recalculan las activaciones con los pesos ya actualizados.** La retropropagación tiene que usar las activaciones de la pasada hacia delante que produjo esta pérdida, no unas nuevas. Recalcularlas mezcla dos estados de la red, y el efecto se acumula iteración tras iteración.
3. **Se actualiza el peso antes de usarlo para propagar.** La línea siguiente necesita W_{out} tal como estaba durante la pasada hacia delante. Al actualizarlo antes, propaga con un valor que ya no corresponde. Se arregla guardando el valor anterior, o reordenando las dos líneas.

Concepto. Los tres comparten una misma causa: la retropropagación es un cálculo sobre un estado congelado de la red. Todo lo que se actualiza durante el propio cálculo contamina el resto. Por eso las bibliotecas separan tan estrictamente el cálculo de los gradientes del paso del optimizador que los aplica.

Atención. Hay un par de cosas que a menudo se señalan como errores y no lo son. Escribir $\delta = y - \hat{y}$ en vez de $\hat{y} - y$ y después *sumar* el gradiente en lugar de restarlo da exactamente la misma actualización: los dos signos se compensan. Es confuso y conviene no escribirlo así, pero no es un fallo de cálculo. Al auditar código ajeno, compruebe el resultado antes de dar por buena una lista de defectos.

10.9 Síntesis operativa y tablas de diagnóstico

10.9.1 Tabla de diagnóstico de anomalías en entrenamiento

Síntoma observado	Causa probable	Comprobación y corrección
La pérdida devuelve NaN en las primeras épocas	Tasa de aprendizaje η excesivamente grande	Reducir η (p. ej., de 1.0 a 10^{-3})
La pérdida no desciende y los gradientes son cero	Saturación de activaciones sigmoideas ($ z \gg 0$)	Inicializar pesos con Xavier/He o usar ReLU
Casi perfecto al entrenar y azaroso en la prueba clínica	sobreajuste, con más variables que muestras	regularizar, abandonar unidades y validar fuera
El modelo predice la misma clase para todas las muestras	Desbalance extremo de clases o sesgo colapsado	Balancear pesos de clase en la pérdida o submuestrear

10.9.2 Tabla de síntesis con preguntas de control

Fase del proyecto	Técnica recomendada	Pregunta de control
Diseño de arquitectura	Capas densas con ReLU / salida Sigmoide	¿Se justifica la no linealidad frente a modelos lineales?
Optimización	Descenso de gradiente con BCE	¿Se verifica la estabilidad numérica de la pérdida?
Evaluación ética	Validación cruzada en cohortes externas	¿Se han descartado sesgos demográficos y fugas de datos?

10.10 Glosario conceptual

- **Perceptrón:** unidad de cómputo lineal con activación no lineal.
- **Sigmoide:** función $1/(1 + e^{-z})$ con codominio en $(0, 1)$.
- **ReLU:** rectificador lineal $\max(0, z)$ para redes profundas.
- **Entropía cruzada:** función de pérdida canónica para clasificación probabilística.
- **Backpropagation:** cálculo de gradientes mediante la regla de la cadena.
- **Learning rate (η):** tamaño de paso en el descenso de gradiente.
- **Vanishing gradient:** desvanecimiento de derivadas en regiones saturadas.
- **Overfitting:** memorización de ruido muestral en lugar de patrones reales.
- **Dropout:** regularización por desactivación estocástica de neuronas.
- **Data leakage:** filtración de información del test set al entrenamiento.

10.11 Cierre del libro: la caja de herramientas computacionales

Este capítulo final integra el conjunto de herramientas computacionales desarrollado a lo largo de todo el libro: desde la automatización reproducible en terminal Unix (TC-CH01–TC-CH02), el análisis asintótico de eficiencia

y estructuras de datos (TC-CH03–TC-CH06), la cuantificación de incertidumbre por teoría de la información (TC-CH07), y los modelos probabilísticos y de alineamiento (TC-CH08–TC-CH09), hasta el aprendizaje de representaciones mediante redes neuronales.

El estudiante dispone ahora de un criterio técnico riguroso: sabe cuándo basta un script de Bash con expresiones regulares, cuándo formular un modelo probabilístico transparente con cadenas de Markov o programación dinámica, y cuándo recurrir al aprendizaje automático supervisado, auditando siempre con responsabilidad ética y científica los límites de cada herramienta.

10.12 Fuentes y reproducibilidad

Este capítulo se apoya en la presentación docente de aprendizaje automático de la asignatura, escrita por Álvaro Serrano Navarro. El perceptrón sigue al artículo de Rosenblatt [26]; la retropropagación, al de Rumelhart y colaboradores [27]; y el tratamiento del sobreajuste, la regularización y las funciones de activación, al tratado de Goodfellow y colaboradores [21].

Todos los cálculos se reproducen con `verification/chapter_results.sh` tal como están impresos, con sus argumentos.

Anexo práctico · Entrenar, propagar gradientes y auditar un modelo

Pregunta, producto y separación de materiales

¿Puede calcular a mano el paso forward y backward de una red neuronal para predicción de enhancers epigenéticos, evaluar el impacto de perturbar la tasa de aprendizaje, verificar los regímenes de saturación del gradiente y formular una auditoría ética rigurosa frente a sesgos biomédicos? Este laboratorio responde que sí, de forma totalmente reproducible.

El producto individual es la tabla de cálculos hechos a mano y contrastados: la pasada hacia delante con su pérdida, el paso hacia atrás hasta la capa de salida y hasta la capa oculta, la perturbación de la tasa de aprendizaje y los dos casos límite. Cada fila lleva el valor que usted calculó, el que dio el oráculo y si coinciden. Le acompaña la auditoría escrita del diseño de validación.

El anexo es autónomo: no necesita datos externos ni red. El generador copia el oráculo del capítulo en la carpeta de herramientas y deja la tabla de respuestas con solo su cabecera.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 preparar el espacio	directorio creado
2 pasada hacia delante y pérdida	activaciones y pérdida a mano
3 hacia atrás: capa de salida	gradientes y pesos actualizados
4 hacia atrás: capa oculta	errores retropropagados y primera capa
5 perturbar la tasa de aprendizaje	sensibilidad al tamaño del paso
6 casos límite	entrada nula y saturación
7 auditar el diseño de validación	notas/auditoria.md
8 empaquetar y verificar	verificador en ENTREGA_OK

La ruta es única. Cada bloque se calcula primero a mano y se contrasta después: si no anota el valor antes de ejecutar, el bloque no cuenta.

1 · Preparar el espacio de trabajo

```
bash --version
./practice_assets/create_workspace.sh TC10_entrega
cd TC10_entrega
```

El generador crea `notas/respuestas.tsv` con la cabecera y ninguna fila, `resultados/` para guardar las salidas y `herramienta/` con el oráculo del capítulo. No trae ninguna respuesta escrita.

2 • Paso Forward y evaluación de la pérdida

Con las entradas $x_1 = 1.0$ (H3K27ac) y $x_2 = 0.5$ (ATAC-seq), y los parámetros iniciales de la red $2 \rightarrow 2 \rightarrow 1$ presentados en teoría, calcule a mano las activaciones de la capa oculta (a_{h1}, a_{h2}), la predicción final ($\hat{y}$) y la pérdida de entropía cruzada para $y = 1$.

```
bash herramienta/chapter_results.sh forward_enhancer_pass 1.0 0.5
```

El cálculo del paso forward sobre las entradas $x_1 = 1.0$ y $x_2 = 0.5$ reproduce de forma determinista la predicción $\hat{y} = 0.605264$.

Se reproduce con `verification/chapter_results.sh forward_enhancer_pass 1.0 0.5`.

```
bash herramienta/chapter_results.sh bce_loss 1.0 0.605264
```

El cálculo de la pérdida de entropía cruzada binaria para $y = 1.0$ e $\hat{y} = 0.605264$ produce un valor de pérdida $\mathcal{L} = 0.502091$.

Se reproduce con `verification/chapter_results.sh bce_loss 1.0 0.605264`.

3 • Paso Backward y actualización de pesos

Calcule a mano el error de salida $\delta_{\text{out}} = \hat{y} - y$, los gradientes $\frac{\partial \mathcal{L}}{\partial w_1^{(2)}}$ y $\frac{\partial \mathcal{L}}{\partial w_2^{(2)}}$, y los nuevos pesos con $\eta = 0.1$:

```
bash herramienta/chapter_results.sh backward_enhancer_pass 1.0 0.5 1.0 0.1 > resultados/backward_salida.txt
cat resultados/backward_salida.txt
```

El cálculo del paso backward produce los nuevos pesos actualizados de la capa de salida $w_1^{(2)} = 0.724104$ y $w_2^{(2)} = -0.380263$ con tasa de aprendizaje $\eta = 0.1$.

Se reproduce con `verification/chapter_results.sh backward_enhancer_pass 1.0 0.5 1.0 0.1`.

4 • Hacia atrás hasta la capa oculta

Aquí está el paso que da nombre a la retropropagación. Calcule a mano los dos errores que llegan a las neuronas ocultas, con la fórmula de la sección correspondiente del capítulo, y después los cuatro pesos actualizados de la primera capa. Anote los seis números antes de ejecutar nada.

```
bash herramienta/chapter_results.sh backward_hidden 1.0 0.5 1.0 0.1 > resultados/backward_oculta.txt
cat resultados/backward_oculta.txt
```

Comprueba. Compruebe dos cosas antes de dar por buenos sus números. Primera: los dos errores de la capa oculta deben tener signo contrario entre sí, porque los pesos que unen cada neurona con la salida lo tienen. Segunda: deben ser bastante menores en magnitud que el error de la salida. Si le salen del mismo orden, probablemente ha olvidado multiplicar por la derivada de la activación, que es el fallo número uno de la actividad de depuración del capítulo.

5 • Perturbación de la tasa de aprendizaje

Incremente la tasa de aprendizaje a $\eta = 0.5$ y registre cómo cambia la magnitud de la actualización del peso $w_1^{(2)}$:

```
bash herramienta/chapter_results.sh learning_rate_perturbation 0.5
```

Perturbar la tasa de aprendizaje a $\eta = 0.5$ incrementa la actualización proporcionalmente, produciendo un nuevo peso $w_1^{(2)} = 0.820521$.

Se reproduce con `verification/chapter_results.sh learning_rate_perturbation 0.5`.

6 • Casos límite

```
bash herramienta/chapter_results.sh zero_input_control
bash herramienta/chapter_results.sh saturation_control 10.0
```

El vector de entradas nulas ($x_1 = 0, x_2 = 0$) propaga exclusivamente los sesgos a través de la red, produciendo $z_{h1} = 0.1000$ y $z_{h2} = -0.2000$.

Se reproduce con `verification/chapter_results.sh zero_input_control`.

Con una preactivación saturada $z = 10.0$, la salida sigmoide es 0.999955 y el gradiente local evalúa a 0.00004540, demostrando cuantitativamente el fenómeno de desvanecimiento del gradiente en regiones saturadas.

Se reproduce con `verification/chapter_results.sh saturation_control 10.0`.

7 • Auditar el diseño de validación

Tarea. Considere un estudio clínico sintético con 1000 perfiles de expresión procedentes de 100 pacientes distribuidos en 2 hospitales. En `notas/respuestas.tsv`, identifique por qué una partición aleatoria simple a nivel de fila genera *data leakage* y describa cómo debe estructurarse la partición correcta agrupada por paciente y centro hospitalario, comparando el modelo propuesto con una regresión logística de referencia (*baseline*).

8 • Empaquetar y verificar

Escriba antes su auditoría en `notas/auditoria.md`. Después:

```
cd ..
tar -czf TC10_entrega.tar.gz TC10_entrega
sha256sum TC10_entrega.tar.gz
./practice_assets/check_entrega.sh TC10_entrega TC10_entrega.tar.gz
```

El verificador comprueba que la tabla de respuestas tiene al menos cinco filas con valor manual, valor del oráculo y veredicto, una de ellas del paso hasta la capa oculta; que la columna de veredicto es coherente con las otras dos; que el oráculo reproduce la pasada hacia delante del caso resuelto; que `resultados/` recoge el paso hasta la capa oculta; y que existe la auditoría. Rechaza el espacio recién generado. No puntúa.

Rúbrica de calificación ponderada

La evaluación pondera la verificación técnica, la derivación matemática y la auditoría metodológica sobre 10 puntos, tres de entrega y siete de razonamiento:

- **3,0 puntos, Entrega técnica y verificador (DELIVERY_OK):** Integridad del paquete comprimido `.tar.gz`, formato de `notas/respuestas.tsv` y superación de `check_practice_delivery.sh`.
- **2,5 puntos, Derivación de propagación directa y retropropagación:** Cálculo manual de activación oculta, salida $\hat{y}$, pérdida BCE y gradientes analíticos de pesos con precisión de 4 decimales.
- **2,5 puntos, Dinámica de optimización y casos límite:** Análisis empírico del efecto de la tasa de aprendizaje η , diagnóstico cuantitativo de saturación sigmoidea ($z = 10.0$) y propagación de sesgos con entradas nulas.

- **2,0 puntos, Auditoría ética y defensa metodológica:** Diseño riguroso de la partición clínica sin *data leakage*, comparación con un *baseline* interpretable y defensa conceptual (100–150 palabras) sobre sesgo poblacional y límites inferenciales de la IA.

Terminar en ENTREGA_OK acredita que los números cuadran, no que la auditoría del diseño de validación sea correcta, que es la parte que más pesa.

Lista de autocorrección

- anoté cada valor a mano antes de ejecutar la orden correspondiente;
- mi predicción de la salida coincidió con la del oráculo;
- evalué la pérdida de entropía cruzada para la etiqueta positiva;
- los dos errores de la capa oculta me salieron con signo contrario y menores que el de salida;
- registré qué le hace a la actualización elevar la tasa de aprendizaje;
- comprobé qué queda de la señal cuando la entrada es nula y cuando la activación satura;
- mi auditoría señala por dónde se filtraría la información entre entrenamiento y prueba;
- guardé en resultados/ la salida de cada ejecución que cito;
- el verificador termina en ENTREGA_OK.

Defensa conceptual

En notas/auditoria.md, en al menos 150 palabras, formule un protocolo de auditoría para un modelo de red neuronal entrenado para predecir respuesta a quimioterapia a partir de expresión génica, explicando qué precauciones deben tomarse respecto a sesgos poblacionales, fugas de datos y validación clínica independiente.

Fuentes y reproducibilidad

Este anexo no usa datos externos: todo lo que calcula lo reproduce el oráculo del capítulo, que es determinista. La retropropagación sigue a Rumelhart y colaboradores [27], y el tratamiento del sobreajuste y la regularización, a Goodfellow y colaboradores [21].

Bibliografía

- [1] Stefano Allesina and Madlen Wilmes. *Computing Skills for Biologists: A Toolbox*. Princeton University Press, 2019. Ejemplos de línea de comandos contrastados en el PDF local; no sostienen ninguna afirmación en solitario.
- [2] Stephen F. Altschul, Warren Gish, Webb Miller, Eugene W. Myers, and David J. Lipman. Basic local alignment search tool. *Journal of Molecular Biology*, 215:403–410, 1990.
- [3] Daniel J. Barrett. *Linux Pocket Guide: Essential Commands*. O'Reilly Media, 4 edition, 2024. Bibliografía básica de la asignatura según la presentación de introducción del curso.
- [4] Patrick M. Bossuyt, Johannes B. Reitsma, David E. Bruns, Constantine A. Gatsonis, Paul P. Glasziou, Les M. Irwig, Jeroen G. Lijmer, David Moher, Drummond Rennie, Henrica C. W. de Vet, Herbert Y. Kressel, Nader Rifai, Robert M. Golub, Douglas G. Altman, Lotty Hooft, Daniël A. Korevaar, and Jérémie F. Cohen. STARD 2015: An updated list of essential items for reporting diagnostic accuracy studies. *BMJ*, 351:h5527, 2015.
- [5] Vince Buffalo. *Bioinformatics Data Skills: Reproducible and Robust Research with Open Source Tools*. O'Reilly Media, 2015. Pasajes comprobados en las páginas físicas 51–54 y 67–70 del PDF local.
- [6] Canonical. *The Linux command line for beginners*, 2026. Documentación oficial. Consultada el 6 de agosto de 2026. <https://documentation.ubuntu.com/desktop/en/latest/tutorial/the-linux-command-line-for-beginners/>.
- [7] Phillip Compeau and Pavel Pevzner. *Bioinformatics Algorithms: An Active Learning Approach*. 3rd edition, 2018. Encuadre bioinformático de la programación dinámica como motor del alineamiento, citado hacia TC-CH09.
- [8] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009. Referencia canónica para la prueba del límite de la ordenación por comparación y la clasificación de paradigmas de diseño; no se reproduce, solo se cita.
- [9] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2 edition, 2006.
- [10] Simon D. Dunn, Lindi M. Wahl, and Gregory B. Gloor. Mutual information without the influence of phylogeny or entropy dramatically improves residue contact prediction. *Bioinformatics*, 24(3):333–340, 2008.
- [11] Richard Durbin, Sean R. Eddy, Anders Krogh, and Graeme Mitchison. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press, 1998. Referencia canónica para cadenas de Markov, HMM y el algoritmo de Viterbi aplicados a secuencias biológicas.
- [12] Free Software Foundation. *GNU Awk User's Guide*. GNU Project. GNU Awk User's Guide, nodos Numeric Functions, Arrays y Locales; versión web actualizada el 22-02-2026; consulta: 2026-08-05.
- [13] M. Gardiner-Garden and M. Frommer. CpG islands in vertebrate genomes. *Journal of Molecular Biology*, 196(2):261–282, 1987.
- [14] Gene Ontology Consortium. GO annotations and guide to evidence codes. <https://geneontology.org/docs/go-annotations/>; <https://geneontology.org/docs/guide-go-evidence-codes/>, 2026. Documentación oficial; consulta: 2026-08-05.
- [15] GNU Project. *GNU Grep 3.12 Manual*, 2025. Sintaxis BRE/ERE y opciones de grep. Consultado el 7 de agosto de 2026. <https://www.gnu.org/software/grep/manual/grep.html>.
- [16] GNU Project. *Bash Reference Manual*, 2026. Consultado el 6 de agosto de 2026. <https://www.gnu.org/software/bash/manual/bash.pdf>.
- [17] GNU Project. *GNU Awk User's Guide*, 2026. Consultado el 7 de agosto de 2026. https://www.gnu.org/software/gawk/manual/html_node/index.html.

- [18] GNU Project. *GNU Coreutils Manual*, 2026. Documentación de pwd, ls, cd, mkdir, cp, mv, head, tail, wc y file. Consultada el 6 de agosto de 2026. <https://www.gnu.org/software/coreutils/manual/coreutils.html>.
- [19] GNU Project. *GNU sed User's Guide*, 2026. Sintaxis de s///, direcciones y -i. Consultado el 7 de agosto de 2026. <https://www.gnu.org/software/sed/manual/sed.html>.
- [20] GNU Project. *GNU tar Manual*, 2026. Consultado el 6 de agosto de 2026. <https://www.gnu.org/software/tar/manual/tar.html>.
- [21] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. Texto de referencia para funciones de activacion, optimizacion basada en gradientes y retropropagacion.
- [22] Solomon Kullback and Richard A. Leibler. On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1):79–86, 1951.
- [23] National Center for Biotechnology Information. *FASTA Format for Nucleotide Sequences*, 2026. Documentación oficial. Consultada el 7 de agosto de 2026. <https://www.ncbi.nlm.nih.gov/genbank/fastafomat>.
- [24] Saul B. Needleman and Christian D. Wunsch. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, 48:443–453, 1970.
- [25] Liam Paninski. Estimation of entropy and mutual information. *Neural Computation*, 15(6):1191–1253, 2003.
- [26] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.
- [27] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.
- [28] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [29] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(4):623–656, 1948.
- [30] Temple F. Smith and Michael S. Waterman. Identification of common molecular subsequences. *Journal of Molecular Biology*, 147:195–197, 1981.